ClaudeForth

An ANS Forth System for the Motorola 6809

Target: the Minimalist Eurocard Board (MECB), 6809 version, configured with 16kB of ROM.

Author: Richard Rothwell

Date: October 17, 2026

Generated with: Claude Sonnet 5

Timeline

Design & code generation duration: 14 hours

Documentation generation duration: 4 hours

Duration addressing AI generated task list, debugging, assembler fixes, memory map resolution: 16 hours

Manual MAME testing and bug-fixing duration (guesstimate): 80 hours
Assembler tests: 7 hours
Via MAME, automated assembler unit tests, serial communications optimisations and ANS Forth Test Suite implementation: 6 weeks (~150 hours)
Global variable memory optimization: 3 days (~15 hours)

Table of Contents

1. Introduction 6
1.1 Overview 6
1.2 Inner-Interpreter Model: Subroutine Threading 6
1.3 Register Assignment 6
1.4 Cell Size and Character Set 6
1.5 The Three-Region Memory Split 6
1.6 Exception Handling 7
1.7 What This Document Covers 7
2. Memory Map 8
3. Glossary 12
3.1 System / Console I/O 12
3.2 Stack Manipulation — Data Stack 13
3.3 Stack Manipulation — Return Stack 14
3.4 Arithmetic — Single Cell 15
3.5 Arithmetic — Mixed & Double Precision 17
3.6 Logic, Shifts & Address Arithmetic 19
3.7 Comparison 20
3.8 Control Flow — Conditionals & Loops 21
3.9 Defining Words 24
3.10 Compiling Words 25
3.11 Memory 27
3.12 Strings & Parsing 29
3.13 Numeric Output 32
3.14 Base / Radix Control 33
3.15 Exception Handling 34
3.16 Comments 34
3.17 Environmental & System Queries 34
3.18 Tools 36
4. Forth Development Notes 37
4.1 Serial I/O and Handshaking 37
4.2 Forth Syntax and Stack Operations 39
4.3 Tools Word Set: Explanation and Examples 40
4.4 Dictionary Management with MARKER 41
4.5 The Search-Order Word Set Is Not Included 42
4.6 Exception Handling 42
5. Open Items Checklist 44

5.1 Known bugs — resolved since the original checklist....44
5.2 Structural duplication (identified, some resolved, some not)....56
5.3 Real, load-bearing gaps....56
5.4 Never resolved after being explicitly raised....57
5.5 Open design questions (not defects — deliberate unresolved choices)....58
5.6 What's solid (for reference, not action)....58
6. Build Instructions....59
6.1 Which Files to Assemble....59
6.2 Shadow Files....59
6.3 Assembler....59
6.4 Build Options....59
6.5 Assembler Unit Tests....60
6.6 Running a Single Test Group Manually....60
6.7 Running All Unit Test Groups: run all tests.sh....61
6.8 ANS Test Suite....61
6.9 Pasting Test Files by Hand....62
7. Architecture....63
7.1 (Outer) Interpreter....63
7.2 Dictionary Operations....64
8. Assembler Source....65
8.1 Memory Map, Constants & GLOBALS Layout....65
8.2 Hardware Vector Table....69
8.3 Init Code (COLDSTRT / WARM)....69
8.4 ACIA Interrupt Handler....71
8.5 COLD / ABORT / QUIT....77
8.6 Inner-Interpreter Support....79
8.7 Comma Family....82
8.8 HEADER....84
8.9 Defining Words....84
8.10 Outer Interpreter....91
8.11 QUERY / ACCEPT / EXPECT / KEY / KEY? / EMIT....96
8.12 CATCH / THROW....100
8.13 Control Flow....102
8.14 Compiling Words....111
8.15 Stack Manipulation....115
8.16 Arithmetic....119
8.17 Logic, Shifts & Address Arithmetic....131
8.18 Comparison....133

8.19 Memory..... 139
8.20 String Words..... 141
8.21 Numeric Output..... 154
8.22 Base / Radix Control..... 158
8.23 Output Formatting..... 159
8.24 Comment Words..... 160
8.25 Environmental Query / SOURCE / REFILL / EVALUATE..... 160
8.26 Tools Word Set..... 164
8.27 Constant-Value Words (TRUE, FALSE, 1, -1, 2, -2)..... 166
8.28 Forth Dictionary (ROM Base Dictionary Headers)..... 167
9. Alphabetical Index..... 186

1. Introduction

1.1 Overview

ClaudeForth is a 16-bit ANS Forth implementation targeting the Motorola 6809 microprocessor. It implements the Core word set and the Core Extension word set, together with the Double-Number word set (including its extensions), string handling (including the String word set's REPLACES and SUBSTITUTE), exception handling (CATCH and THROW), the Facility word KEY?, and the Tools words .S, WORDS and DUMP. This document describes the resulting architecture and the reasoning behind its major design choices, and provides a complete glossary of every word implemented.

1.2 Inner-Interpreter Model: Subroutine Threading

ClaudeForth uses subroutine-threaded code (STC) rather than indirect or direct threading. A colon definition compiles as a flat sequence of 6809 JSR instructions terminated by a compiled RTS. This means the 6809's own JSR/RTS mechanism is the inner interpreter — there is no separate NEXT dispatch loop for straight-line execution. This was chosen because the 6809's hardware stack (register S) is naturally suited to serving as both the CPU's call stack and Forth's return stack simultaneously, and because STC is generally the fastest threading model available on this processor, at the cost of slightly larger compiled code than indirect threading would produce (each call is 3 bytes — a JSR opcode plus a 16-bit address — rather than a 2-byte threaded pointer).

1.3 Register Assignment

U is the data (parameter) stack pointer, manipulated with PSHU/PULU. S is the return stack pointer, using the CPU's native hardware stack directly, so JSR/RTS work as the inner interpreter with no extra bookkeeping. X and Y are general-purpose working registers, used throughout for scanning, indexing, and address computation. D (the combined A:B accumulator) holds top-of-stack values in flight and is the primary vehicle for 16-bit arithmetic, matching the cell size described below.

1.4 Cell Size and Character Set

One cell is 16 bits (two bytes), matching the 6809's native D, X, Y, U, and S registers exactly. This gives a single-cell signed range of -32768 to 32767, unsigned 0 to 65535, and means a cell can always hold a full address across the 6809's entire 64K address space. Because the 6809 has no alignment restriction, ALIGN and ALIGNED are true no-ops. Characters are single bytes (values 0-255, as reported by ENVIRONMENT? MAX-CHAR), so CHARS and CHAR+ reduce to a no-op and a plain +1 respectively, while CELLS requires an actual multiply-by-two.

1.5 The Three-Region Memory Split

The single most distinctive architectural decision in ClaudeForth is that dictionary headers, compiled code, and user data are tracked as three independent, separately growing regions, each with its own allocation pointer (DPHERE, CODEHERE, and VARHERE respectively) rather than the single unified dictionary most Forth systems use. This departs from the ANS assumption of one HERE; in this system HERE reports CODEHERE specifically, and a parallel word family (V, / VC, / VALLOT / VHERE) exists to manage the variable region directly. The practical consequence is a genuine distinction between immutable data (CONSTANT values, compiled literals, and string literals, all living in code space) and mutable data (VARIABLE, VALUE, and BUFFER: contents, living in variable space) that most single-region Forths do not enforce architecturally.

The intent behind this split is to provide for a ROMable basic Forth system: BASECODE and BASEDICT (Section 2) hold the entire Core and Core Extension implementation as fixed, burned-in ROM content, needing no RAM at all to boot to a working interactive prompt. The RAM regions — APPCODE, APPDICT, and APPVARS — exist specifically to support interactive user application development on top of that fixed base: new words are compiled, tested, and revised freely in RAM without touching the ROM image underneath them. Once an application is complete, the same compiled RAM content (APPCODE and APPDICT together) is intended to be burned into an expanded ROM alongside the base system, turning a finished, debugged application into a second fixed, non-volatile layer rather than something that must be re-compiled from source into RAM on every power-up.

A feature under consideration, not yet designed or implemented, is the ability to strip the dictionary — discarding header (name, LINK, flags) information for some or all ROM or application words once development is finished, while leaving their code intact and callable. A stripped system would trade FIND-ability and interactive extensibility (WORDS, ', and ordinary text interpretation of a word by name) for a smaller ROM footprint and a correspondingly larger RAM variable area, since the dictionary and variable regions currently share the same fixed page-zero and BASEDICT budgets described in Section 2.

1.6 Exception Handling

CATCH and THROW work directly on the return stack (register S) rather than through a linked list of handler records. CATCH pushes a 12-byte frame on S holding the previous HANDLER value, the data-stack pointer (taken after the execution token has been popped) and the input-source state (SRCADDR, SRCLEN, SRCID and >IN), and then points HANDLER at that frame. A non-zero THROW sets S from HANDLER with a single TFR, discarding every intervening call level in one step; it then restores the input source, the data-stack pointer and the previous HANDLER, pushes the throw code and returns to CATCH's caller. With no active CATCH, THROW falls into ABORT. QUIT wraps INTERPRET in a CATCH, so every input line runs under exception control: on a caught error QUIT restores DPHERE, CODEHERE, VARHERE and LATEST to their values at the start of the failing input line (not the start of a multi-line definition, whose earlier lines and smudged header remain), empties the data stack, returns to interpretation state and prints CR and " ERROR n" with n in the current BASE. Code -1 (ABORT) is silent. A CATCH in user code restores none of the region pointers.

1.7 What This Document Covers

Section 2 gives the full memory map. Section 3 is the glossary of every word implemented, organized by topic, each with its stack-effect notation and any side effects. Section 4 covers five practical development topics: serial I/O configuration and handshaking (for both serial builds), a brief introduction to Forth syntax and stack operations, worked examples of the Tools word set, dictionary management with MARKER, and why the Search-Order word set is not included. Section 5 is a checklist of items that were open, resolved, or deliberately deferred during the build; the current checklist is maintained alongside the source. Section 6 covers build instructions: the files to assemble, the assembler, the serial and unit-test build options, and the scripts that run the assembler unit tests and the ANS Test Suite. Section 7 describes the outer interpreter and dictionary operations. Section 8 is the complete assembler source listing, including the ROM base dictionary (Section 8.28); long explanatory and bug-fix comments have been moved from that listing into companion shadow files (Section 6). Section 9 is an alphabetical index of every word with a page reference back into the glossary.

2. Memory Map

64K address space, addresses shown high to low. Three independent allocation pointers (DPHERE, CODEHERE, VARHERE) track the dictionary, code, and mutable-variable regions respectively; none of the three boundaries below is enforced by a runtime bounds check.

Address Range	Size/bytes	Region	Contents
$FFF0–$FFFF	16	VECTORS	Hardware vectors
$FFA4–$FFEB	72	INITCODE	COLDSTRT / WARM entry code (INITSIZE). Followed by 4 bytes of $FF padding, $FFEC–$FFEF, up to VECTORS
$DC44–$FE16 (polled) $DC44–$FF55 (interrupt)	8659 (polled) 8978 (interrupt)	BASECODE	ROM primitives and compiled system words — real sizes, confirmed by a real assembler run (BASECODESIZE) for the two serial builds (SERIALPOLL=1 and 0, Section 4.1); exactly contiguous with BASEDICT below, zero gap. $FF padding fills the space above it up to INITCODE
$D395–$DC43	2223	BASEDICT	ROM dictionary headers — real size, confirmed by a real assembler run (BASEDICTSIZE)
$C000–$C0FF	256	INOUT	Input/output device block, sits directly below USROMSTRT; 6850 ACIA registers live at INOUT+8, not the block base
$BD00–$BFFF	768	RSTACK	Return stack (S), grows downward — exactly contiguous with DSTACK below, no gap
$B900–$BCFF	1024	DSTACK	Data stack (U), grows downward — occupied bottom matches CODETOP exactly
$7000–$B8FF	18688	APPCODE + Transient	Application code (CODEHERE). Also hosts a floating Transient subregion between CODEHERE and CODEHERE+128, used by WORD/HOLD/PAD — see Transient Region, below
$2000–$6FFF	20480	APPDICT	Application dictionary (DPHERE)
$0240–$1FFF	7616	APPVARS	Mutable variable space (VARHERE) — APPVARSEND self-adjusts to APPDICT-1
$0200–$023F	64	SUBPOOL	REPLACES substitution table (SUBPOOLSZ = 64 bytes): a linked list of entries, each 5 bytes of header plus the name and the text, headed by the global SUBHEAD with the next free byte in SUBFREE. Holds three 8-character name and text pairs. Emptied by COLD and by CLR-REPLACES
$01C0–$01FF	64	OUTBUF	Output ring buffer (64 bytes; 63 usable). Used by the interrupt-driven build (SERIALPOLL=0) only
$0180–$01BF	64	INBUF	Input ring buffer (64 bytes; 63 usable). Used

Address Range	Size/bytes	Region	Contents
			by the interrupt-driven build only. Aligned to a 64-byte boundary
$0176–$017F	10	SERBUFCTL	Serial buffer indices and counters: INHEAD, INTAIL, RTSSTATE, OUTHEAD, OUTTAIL, OUTACTIVITY, and the receiver-error and backlog counts FECOUNT, OVRNCOUNT, PECOUNT, POLLREADYCNT
$0156–$0175	32	(unclaimed)	Not used for any purpose
$0106–$0155	80	TIBBUF	Terminal input buffer (80 bytes, TIBBUFL)
$0100–$0105	6	(unclaimed)	Not used for any purpose (formerly MVSCRATCH, whose cells moved onto the stacks)
$0000–$00FF	256	GLOBALS	Shared system globals — Direct Page = $00. 69 of the 256 bytes are used ($00–$44); the other 187 are free. Most former static temporaries now live in registers, stack frames or the data-stack frame of the word concerned (Section 5.1)

ROM Size Required

VECTORS, INITCODE, BASECODE, and BASEDICT are the four regions that must actually live in ROM; everything else in the table is RAM or the ACIA hardware itself, memory-mapped rather than stored. USROMSTRT ($C100) and USROMEND ($FFEF, one address before VECTORS' start) define the outer boundary these four must fall within, and all four are genuinely contained within it. A real assembler run (lwasm, with its symbol table) confirms the boundaries between them precisely: BASEDICT's real end lands exactly at BASECODE's start (BASEDICTEND = BASECODE = $DC44) — zero gap, zero overlap. BASECODE's real end depends on the serial build (Section 4.1): BASECODEEND = $FE17 for the polled build (SERIALPOLL=1, the default) and $FF56 for the interrupt-driven build (SERIALPOLL=0). Either way it falls below INITCODE's start ($FFA4), and the gap is explicitly padded with $FF bytes by the assembler (see BASEND's FILL directive) specifically to stop it from being silently absorbed into the .bin output — 397 bytes of padding in the polled build, 78 bytes in the interrupt-driven build. INITCODE's real end (INITEND = $FFEC; INITSIZE = 72 bytes) is likewise padded with 4 bytes of $FF up to VECTORS' start ($FFF0).

Using the real sizes confirmed by that assembler run (BASEDICTSIZE = 2223 bytes; BASECODESIZE = 8659 bytes in the polled build and 8978 bytes in the interrupt-driven build; INITSIZE = 72 bytes — all measured directly, not estimated), VECTORS + INITCODE + BASECODE + BASEDICT comes to 10970 bytes (16 + 72 + 8659 + 2223, about 10.7 KB) for the polled build and 11289 bytes (16 + 72 + 8978 + 2223, about 11.0 KB) for the interrupt-driven build, which is why the ROM part is a 16K×8 EPROM (a 27128 or equivalent) rather than an 8K×8 part — comfortable headroom for real usage.

The INOUT block shadows 256 bytes in the ROM block: a 16K×8 EPROM occupying the natural $C000–$FFFF footprint that size implies would include $C000–$C0FF, but INOUT claims that same 256-byte range for the ACIA and other memory-mapped I/O. This is why USROMSTRT starts at $C100, not $C000 — the bottom 256 bytes of the chip's raw address footprint are never available for ROM content at all, regardless of how much of the chip is otherwise unused.

ROM unused byte count: a 16K×8 EPROM (16384 bytes) minus INOUT's 256-byte shadow minus the real VECTORS + INITCODE + BASECODE + BASEDICT content leaves 5158 bytes genuinely unused in the polled build (16384 − 256 − 10970), about 31% of the chip's total capacity, and 4839 bytes in the interrupt-driven build (16384 − 256 − 11289), about 30%. Every figure here is a real assembler-measured value, not a manual count. BASECODE and BASEDICT are placed by hand, by the two EQU lines at the top of forth6809.asm: if the code grows they must be lowered together, and the assembler reports an invalid FILL length if BASECODE would overrun INITCODE. The interrupt-driven build is now the tight one, with 78 bytes left below INITCODE.

Transient Region

A floating scratch region within APPCODE, positioned relative to CODEHERE rather than at a fixed address — its actual location moves as compilation proceeds, since every new colon definition or interactively parsed word advances CODEHERE. It serves two purposes: WORD's own text-scanning buffer for the word currently being parsed, and the pictured numeric output buffer built by <# # #S HOLD HOLDS SIGN #>. Reused fresh on every WORD call and every <# ... #> conversion, since neither needs to persist past its own use. PAD sits directly above these two buffers (see the table below). Occupies PADOFFSET (128 bytes, PADMINSIZE) between CODEHERE and PAD: PAD = CODEHERE + 128 (see PADW). That 128-byte gap is shared by two independent, opposite-growing buffers — WORD's own buffer growing upward from CODEHERE toward PAD, and HOLD's buffer growing downward from PAD toward CODEHERE — sized so the two can never collide even in the worst case. PAD itself then extends a further PADMINSIZE (128) bytes upward from its own address, which comfortably exceeds ANS's /PAD requirement of 84 characters (3.3.3.6) — using the same constant for both roles was a deliberate design choice, not a coincidence.

Subregion	Size	Address (relative to CODEHERE)	Description
WORD parsing buffer	up to 91 bytes (1 count byte + up to WORDMAXCHARS = 90 characters)	CODEHERE, extending to at most CODEHERE+90. S", ." and ABORT" now parse with PARSE instead of WORD, so the 3 bytes they once reserved ahead of CODEHERE are spare	WORD's own scratch buffer for the word or string currently being parsed; grows upward from CODEHERE toward PAD, the traditional fig-Forth layout. Capped at WORDMAXCHARS specifically so it can never reach the HOLD buffer below, even in the worst case.
HOLD / pictured numeric output buffer	34 bytes (HOLDMINSIZE)	CODEHERE+94 to CODEHERE+127 (= PAD−34 to PAD−1)	Built by <# # #S HOLD HOLDS SIGN, one character at a time, growing downward from PAD toward CODEHERE; #> finalizes it, and <# restarts the buffer at PAD again next time.
PAD	at least PADMINSIZE (128) bytes, extending	CODEHERE+128 onward	General-purpose transient scratch area returned by PAD; also used by interpreted-mode S" for its own string storage. ANS (3.3.3.6) requires only 84 characters of

	upward; effectively unbounded until the next CODEHERE advance		guaranteed capacity; PADMINSIZE (128) exceeds that, and is the same constant used for the CODEHERE-to-PAD offset itself, by design.

Dictionary Entry Layout

Every word in BASEDICT (and every application word compiled into APPDICT) shares this same four-field layout, packed with no padding, in this order:

Code	Size	Name	Comments
LEN/FL	1 byte	Length + Flags	bit 7 = IMMEDIATE, bit 6 = SMUDGE, bits 4-0 = name length (0-31)
NAME	n bytes	Name Field	n raw ASCII characters, no terminator - n comes from LEN/FL's low bits
LINK	2 bytes	Link Field	address of the previous header in the chain, or 0 if this is the oldest entry
CFA	2 bytes	Code Field Address	address FIND returns - the word's own code, or a DODOES trampoline for a CREATEd word

3. Glossary

Words are grouped by topic below. Each entry gives the stack-effect notation, a short description, and any side effects beyond the stack transformation shown.

3.1 System / Console I/O

KEY

`( -- char )`

Blocking read of one character from the serial port.

Side effects: *Polled build (SERIALPOLL=1): spins on the ACIA receive-data-ready flag, and counts framing, overrun and parity errors (FECOUNT, OVRNCOUNT, PECOUNT) and characters already waiting (POLLREADYCNT). Interrupt-driven build (SERIALPOLL=0): spins with interrupts enabled on the input ring buffer, draining the output ring while RTS is high and lowering RTS once the ring has drained to INLOWATER.*

KEY?

`( -- flag )`

Non-blocking test for a waiting input character.

Side effects: *Read-only; never blocks. Polled build: tests the ACIA receive-data-ready flag. Interrupt-driven build: tests whether the input ring buffer is non-empty.*

EMIT

`( char -- )`

Transmit one character.

Side effects: *Polled build: waits for the ACIA transmit-data-empty flag, then writes the character directly. Interrupt-driven build: queues the character in the output ring (discarded if the ring is full) and enables the transmit interrupt if not already on, unless RTS is currently asserted high.*

ACCEPT

`( addr n -- n' )`

Line-edit up to n characters into addr; returns actual count.

Side effects: *Echoes input. Backspace and DEL erase the last character; LF is ignored; CR ends the line. Polled build only: transmits XON before reading the line and XOFF when the line is complete, pacing a host that honours XON/XOFF (Section 4.1).*

EXPECT

`( addr n -- )`

Obsolescent line input; same editing as ACCEPT.

Side effects: *Reports its count through SPAN rather than the stack.*

QUERY

`( -- )`

Refill the terminal input buffer and reset >IN.

Side effects: *Sets SRCADDR/SRCLEN/SRCID for the terminal source.*

TYPE

`( addr len -- )`

Output len characters starting at addr.

Side effects: *None.*

CR

`( -- )`

Output a newline (CR then LF).

Side effects: *None.*

SPACE

```
( -- )
```

Output one space character.

Side effects: *None.*

SPACES

```
( n -- )
```

Output n spaces.

Side effects: *No output if n <= 0.*

ABORT

```
( -- ) ( R: ... -- )
```

Throw -1. With no active CATCH, reset the data stack and enter QUIT. Never returns.

Side effects: *Silent: prints nothing. Reaches a full return-stack reset through THROW and QUIT.*

QUIT

```
( -- ) ( R: ... -- )
```

The outer interpreter's top-level loop. Never returns.

Side effects: *Resets the return stack and wraps each INTERPRET in CATCH. On a caught error (other than the silent -1 from ABORT) prints CR and " ERROR n", restores DPHERE, CODEHERE, VARHERE and LATEST to their values at the start of the failing input line, empties the data stack and returns to interpretation state.*

3.2 Stack Manipulation — Data Stack

DUP

```
( n -- n n )
```

Duplicate the top stack item.

Side effects: *None.*

DROP

```
( n -- )
```

Discard the top stack item.

Side effects: *None.*

SWAP

```
( n1 n2 -- n2 n1 )
```

Exchange the top two items.

Side effects: *None.*

OVER

```
( n1 n2 -- n1 n2 n1 )
```

Copy the second item to the top.

Side effects: *None.*

ROT

```
( n1 n2 n3 -- n2 n3 n1 )
```

Rotate the third item to the top.

Side effects: *None.*

?DUP

```
( n -- n n | 0 )
```

Duplicate top only if it is nonzero.

Side effects: *None.*

DEPTH

`( -- n )`
Push the number of items on the data stack.
Side effects: *Computed relative to SP0.*

2DUP
`( x1 x2 -- x1 x2 x1 x2 )`
Duplicate the top cell pair.
Side effects: *None.*

2DROP
`( x1 x2 -- )`
Discard the top cell pair.
Side effects: *None.*

2SWAP
`( x1 x2 x3 x4 -- x3 x4 x1 x2 )`
Exchange the top two cell pairs.
Side effects: *None.*

2OVER
`( x1 x2 x3 x4 -- x1 x2 x3 x4 x1 x2 )`
Copy the second cell pair to the top.
Side effects: *None.*

NIP
`( x1 x2 -- x2 )`
Discard the second item.
Side effects: *None.*

TUCK
`( x1 x2 -- x2 x1 x2 )`
Copy the top item under the second.
Side effects: *None.*

PICK
`( xu ... x0 u -- xu ... x0 xu )`
Copy the item u cells deep to the top.
Side effects: *No bounds check against actual stack depth.*

ROLL
`( xu ... x0 u -- xu-1 ... x0 xu )`
Remove the item u cells deep and move it to the top.
Side effects: *Cost scales with u; no bounds check.*

2ROT
`( x1..x6 -- x3 x4 x5 x6 x1 x2 )`
Rotate the third cell pair to the top.
Side effects: *None.*

3.3 Stack Manipulation — Return Stack

Because DO/LOOP (Section 3.8) stores its loop counters on the return stack, >R and R> — and their double-cell counterparts, 2>R and 2R> — must remain balanced within the body of any DO/LOOP they appear inside, not merely within the enclosing definition as a whole. This discipline is necessary to avoid corrupting the loop's own stack counters, and also the calling word's own subroutine return address,

since that address sits on the same return stack, just beneath the loop's own counters — an imbalance left active across a LOOP/+LOOP boundary can overwrite either.

>R

```
( x -- ) ( R: -- x )
```

Move top of data stack to the return stack.

Side effects: *Must be balanced by R> within the same definition.*

R>

```
( -- x ) ( R: x -- )
```

Move top of return stack to the data stack.

Side effects: *None.*

R@

```
( -- x ) ( R: x -- x )
```

Copy top of return stack to the data stack.

Side effects: *Non-destructive.*

2>R

```
( x1 x2 -- ) ( R: -- x1 x2 )
```

Move top cell pair to the return stack.

Side effects: *Must be balanced by 2R> in the same definition.*

2R>

```
( -- x1 x2 ) ( R: x1 x2 -- )
```

Move top cell pair back to the data stack.

Side effects: *None.*

2R@

```
( -- x1 x2 ) ( R: x1 x2 -- x1 x2 )
```

Copy top cell pair from the return stack.

Side effects: *Non-destructive.*

3.4 Arithmetic — Single Cell

+

```
( n1 n2 -- sum )
```

Add.

Side effects: *None.*

-

```
( n1 n2 -- diff )
```

Subtract n2 from n1.

Side effects: *None.*

*****

```
( n1 n2 -- prod )
```

Signed multiply, truncated to one cell.

Side effects: *None.*

/

```
( n1 n2 -- quot )
```

Signed symmetric division.

Side effects: *Throws -10 if n2 is zero.*

MOD
`( n1 n2 -- rem )`
Signed symmetric remainder.
Side effects: *Throws -10 if n2 is zero.*

/MOD
`( n1 n2 -- rem quot )`
Quotient and remainder together.
Side effects: *Throws -10 if n2 is zero.*

NEGATE
`( n -- -n )`
Two's-complement negate.
Side effects: *None.*

ABS
`( n -- |n| )`
Absolute value.
Side effects: *None.*

MIN
`( n1 n2 -- min )`
Signed minimum.
Side effects: *None.*

MAX
`( n1 n2 -- max )`
Signed maximum.
Side effects: *None.*

1+
`( n -- n+1 )`
Add one.
Side effects: *None.*

1-
`( n -- n-1 )`
Subtract one.
Side effects: *None.*

2+
`( n -- n+2 )`
Add two. Not an ANS-standard word.
Side effects: *None.*

2*
`( n -- n*2 )`
Arithmetic shift left one bit.
Side effects: *None.*

2/
`( n -- n/2 )`
Arithmetic shift right one bit, sign-preserving.
Side effects: *None.*

***/**

```
( n1 n2 n3 -- quot )
```

Multiply then divide via a double-cell intermediate.

Side effects: *Throws -10 if n3 is zero.*

***/MOD**

```
( n1 n2 n3 -- rem quot )
```

As */ but yields remainder and quotient.

Side effects: *Throws -10 if n3 is zero.*

1

```
( -- 1 )
```

Push the constant 1.

Side effects: *ROM words that push their constant directly, like -1, 2 and -2 below; they have no value cell.*

-1

```
( -- -1 )
```

Push the constant -1.

Side effects: *None.*

2

```
( -- 2 )
```

Push the constant 2.

Side effects: *None.*

-2

```
( -- -2 )
```

Push the constant -2.

Side effects: *None.*

3.5 Arithmetic — Mixed & Double Precision

UM*

```
( u1 u2 -- ud )
```

Unsigned single by single multiply, double result.

Side effects: *None.*

UM/MOD

```
( ud u1 -- u2 u3 )
```

Unsigned double by single divide.

Side effects: *Throws -10 if u1 is zero.*

M*

```
( n1 n2 -- d )
```

Signed single by single multiply, double result.

Side effects: *None.*

FM/MOD

```
( d n1 -- n2 n3 )
```

Floored double by single division.

Side effects: *Throws -10 if n1 is zero.*

SM/REM

```
( d n1 -- n2 n3 )
```

Symmetric double by single division.

Side effects: *Throws -10 if n1 is zero.*

D+

`( d1 d2 -- d3 )`

Double-cell add.

Side effects: *None.*

D-

`( d1 d2 -- d3 )`

Double-cell subtract.

Side effects: *None.*

DNEGATE

`( d1 -- d2 )`

Double-cell two's-complement negate.

Side effects: *None.*

DABS

`( d1 -- d2 )`

Double-cell absolute value.

Side effects: *None.*

M+

`( d1 n -- d2 )`

Add a single-cell value into a double.

Side effects: *None.*

S>D

`( n -- d )`

Sign-extend a single cell to double.

Side effects: *None.*

D>S

`( d -- n )`

Narrow a double to a single cell.

Side effects: *Implementation-defined if d does not fit; the high cell is dropped.*

DMAX

`( d1 d2 -- d3 )`

Double-cell signed maximum.

Side effects: *None.*

DMIN

`( d1 d2 -- d3 )`

Double-cell signed minimum.

Side effects: *None.*

D0=

`( d -- flag )`

True if d is zero.

Side effects: *None.*

D0<

`( d -- flag )`

True if d is negative.

Side effects: *None.*

D2*

`( d1 -- d2 )`

Shift a double left one bit (multiply by two).

Side effects: *Overflow wraps silently.*

D2/

`( d1 -- d2 )`

Arithmetic shift of a double right one bit; the sign is preserved.

Side effects: *None.*

M*/

`( d1 n1 +n2 -- d2 )`

Multiply d1 by n1 into a triple-cell product, then divide by n2.

Side effects: *Throws -10 if n2 is zero. The triple-cell intermediate cannot overflow, so the result is exact whenever it fits in a double. Division is symmetric (the quotient truncates toward zero), matching / and */. A negative n2 is accepted and handled by sign.*

3.6 Logic, Shifts & Address Arithmetic

AND

`( n1 n2 -- n1&n2 )`

Bitwise AND.

Side effects: *None.*

OR

`( n1 n2 -- n1|n2 )`

Bitwise OR.

Side effects: *None.*

XOR

`( n1 n2 -- n1^n2 )`

Bitwise exclusive OR.

Side effects: *None.*

INVERT

`( n -- ~n )`

Ones'-complement.

Side effects: *None.*

LSHIFT

`( x1 u -- x2 )`

Logical shift left u bits, zero-fill.

Side effects: *Cost scales with u; no early-out for large u.*

RSHIFT

`( x1 u -- x2 )`

Logical shift right u bits, zero-fill.

Side effects: *Cost scales with u; no early-out for large u.*

CELLS

`( n -- n*2 )`

Convert a cell count to a byte offset.

Side effects: *None.*

CELL+
`( addr -- addr+2 )`
Add the size of one cell.
Side effects: *None.*

CHARS
`( n -- n )`
Convert a character count to a byte offset. No-op on this system.
Side effects: *None.*

CHAR+
`( addr -- addr+1 )`
Add the size of one character.
Side effects: *None.*

ALIGN
`( -- )`
Align HERE to a cell boundary. No-op on the 6809.
Side effects: *None.*

ALIGNED
`( addr -- addr )`
Align a given address. No-op on the 6809.
Side effects: *None.*

3.7 Comparison

=
`( n1 n2 -- flag )`
True if equal.
Side effects: *None.*

<
`( n1 n2 -- flag )`
True if n1 signed less than n2.
Side effects: *None.*

>
`( n1 n2 -- flag )`
True if n1 signed greater than n2.
Side effects: *None.*

0=
`( n -- flag )`
True if n is zero.
Side effects: *None.*

0<
`( n -- flag )`
True if n is negative.
Side effects: *None.*

U<
`( u1 u2 -- flag )`
True if u1 unsigned less than u2.

Side effects: *None.*

<>

```
( n1 n2 -- flag )
```

True if not equal.

Side effects: *None.*

0<>

```
( n -- flag )
```

True if n is not zero.

Side effects: *None.*

0>

```
( n -- flag )
```

True if n is greater than zero.

Side effects: *None.*

U>

```
( u1 u2 -- flag )
```

True if u1 unsigned greater than u2.

Side effects: *None.*

WITHIN

```
( n1 n2 n3 -- flag )
```

True if n2 <= n1 < n3.

Side effects: *Computed via unsigned comparison of offsets, handles wraparound ranges correctly.*

D=

```
( d1 d2 -- flag )
```

Double-cell equal.

Side effects: *None.*

D<

```
( d1 d2 -- flag )
```

Double-cell signed less than.

Side effects: *Compares high cells (signed) first, low cells (unsigned) only if equal.*

DU<

```
( d1 d2 -- flag )
```

Double-cell unsigned less than.

Side effects: *Both tiers compared unsigned.*

TRUE

```
( -- flag )
```

Push the true flag, all bits set ($FFFF).

Side effects: *A ROM word that pushes the constant directly; it has no value cell.*

FALSE

```
( -- flag )
```

Push the false flag, zero.

Side effects: *A ROM word that pushes the constant directly; it has no value cell.*

3.8 Control Flow — Conditionals & Loops

IF

```
Compile: ( -- ) Run: ( flag -- )
```
Begin a conditional. IMMEDIATE, compile-only.
Side effects: *Compiles ZBRANCH and a forward-patch field.*

THEN
```
( -- )
```
Resolve an IF or ELSE forward branch. IMMEDIATE, compile-only.
Side effects: *Throws -22 if the control-flow tag does not match.*

ELSE
```
( -- )
```
Alternate branch of IF. IMMEDIATE, compile-only.
Side effects: *Throws -22 if the control-flow tag does not match.*

BEGIN
```
( -- )
```
Mark a loop target. IMMEDIATE, compile-only.
Side effects: *None.*

UNTIL
```
( flag -- )
```
Branch back to BEGIN while flag is false. IMMEDIATE, compile-only.
Side effects: *Throws -22 on a tag mismatch.*

AGAIN
```
( -- )
```
Unconditional branch back to BEGIN. IMMEDIATE, compile-only.
Side effects: *Throws -22 on a tag mismatch.*

WHILE
```
( flag -- )
```
Conditional exit within a BEGIN loop. IMMEDIATE, compile-only.
Side effects: *Leaves a second pending forward-patch field.*

REPEAT
```
( -- )
```
Close a BEGIN…WHILE loop. IMMEDIATE, compile-only.
Side effects: *Patches both the back-edge and WHILE's forward exit.*

RECURSE
```
( -- )
```
Compile a call to the word currently being defined. IMMEDIATE, compile-only.
Side effects: *Bypasses FIND's smudge check; reads LATEST's own CFA directly.*

DO
```
Compile: ( -- ) Run: ( n1|u1 n2|u2 -- ) ( R: -- loop-sys )
```
Begin a counted loop. IMMEDIATE, compile-only.
Side effects: *Builds a 4-cell loop frame on the return stack; runs at least once even if limit=index.*

?DO
```
Compile: ( -- ) Run: ( n1|u1 n2|u2 -- ) ( R: -- loop-sys )
```
As DO, but skips the loop entirely if limit=index. IMMEDIATE, compile-only.
Side effects: *Builds the same loop frame; adds a forward skip branch.*

LOOP

`( -- ) ( R: loop-sys -- | loop-sys )`
Increment index by 1; loop until index reaches limit. IMMEDIATE, compile-only.
Side effects: *Discards the loop frame on exit.*

+LOOP
`( n -- ) ( R: loop-sys -- | loop-sys )`
Increment index by n; exits when the step crosses limit. IMMEDIATE, compile-only.
Side effects: *Handles positive or negative step; discards the frame on exit.*

I
`( -- n ) ( R: loop-sys -- loop-sys )`
Fetch the innermost loop's index.
Side effects: *Non-destructive read of the current loop frame.*

J
`( -- n ) ( R: loop-sys1 loop-sys2 -- loop-sys1 loop-sys2 )`
Fetch the next-outer loop's index.
Side effects: *Only reaches one level of nesting out; no J2 equivalent exists.*

LEAVE
`( -- ) ( R: loop-sys -- loop-sys )`
Force the innermost loop to exit at its next LOOP/+LOOP.
Side effects: *Assumes it is textually inside the loop it affects; unverified across an intervening word call.*

UNLOOP
`( -- ) ( R: loop-sys -- )`
Discard the innermost loop frame (index, limit and LEAVE flag: 6 bytes) from the return stack, so that a following I or J sees the enclosing loop.
Side effects: Must be used before EXIT when leaving a loop early, as the standard requires. Run time only: it expects the DO frame directly beneath its own return address.

EXIT
`( -- )`
Return from the current definition immediately. IMMEDIATE, compile-only in the sense that it must be used inside a definition.
Side effects: Compiles a call to EXITUNLOOP followed by a count of the DO/?DO frames open on the control-flow stack (the count is not used at run time). EXIT does not discard loop frames itself: a loop must be left with UNLOOP before EXIT.

CASE
`( -- )`
Begin a multi-way selection. IMMEDIATE, compile-only.
Side effects: *None.*

OF
`( x n -- | x )`
Begin one CASE clause's test. IMMEDIATE, compile-only.
Side effects: *Drops the selector only on a matching clause.*

ENDOF
`( -- )`
End one CASE clause. IMMEDIATE, compile-only.
Side effects: *Throws -22 on a tag mismatch.*

ENDCASE
`( x -- )`

Close a CASE structure, discarding the selector.
Side effects: *Throws -22 if the control-flow stack is malformed.*

3.9 Defining Words

:
```
( "name" -- )
```
Begin a colon definition.
Side effects: *Header is smudged until ; snapshot control-flow depth for later checking.*

:NONAME
```
( -- xt )
```
Start an anonymous definition and leave its execution token.
Side effects: *No header is built and LATEST is left alone, so the word can never be found by name. The xt is the current CODEHERE. Enters compile state; finish with ; and then EXECUTE the xt, or store it (for example with IS).*

;
```
( -- )
```
End a colon definition. IMMEDIATE.
Side effects: *Unsmudges the header; throws -22 if a control structure is left open.*

CREATE
```
( "name" -- )
```
Build a header and a DOES>-ready trampoline.
Side effects: *PFA defaults to CODEHERE (immutable space).*

DOES>
```
( -- )
```
Attach runtime behavior to a CREATEd word. IMMEDIATE, compile-only.
Side effects: *Patches the trampoline's BEHAVIOR field at run time via a double return.*

VARIABLE
```
( "name" -- )
```
Create a one-cell mutable variable.
Side effects: *PFA reserved in VARHERE (mutable space); initialized to zero.*

CONSTANT
```
( x "name" -- )
```
Create a word that returns a fixed value.
Side effects: *Value stored in CODEHERE (immutable space).*

VALUE
```
( x "name" -- )
```
Create a reassignable named value.
Side effects: *PFA in VARHERE (mutable space); reassigned via TO.*

TO
```
( x "name" -- )  or  ( x1 x2 "name" -- )
```
Assign a new value to a VALUE or a 2VALUE. IMMEDIATE.
Side effects: *Behaves differently interpreting vs. compiling. TO checks whether the target is a 2VALUE and, if so, stores two cells with 2! instead of one with !.*

2VALUE
```
( x1 x2 "name" -- )
```
Create a reassignable named double value.

Side effects: *Both cells live in VARHERE (mutable space), x2 at the lower address as for 2CONSTANT, so name behaves as 2@ on its own cells. Reassigned with TO, which recognizes a 2VALUE and stores through 2!; inside a definition TO compiles the same store.*

2VARIABLE

```
( "name" -- )
```

Create a two-cell mutable variable.

Side effects: *PFA in VARHERE; both cells initialized to zero.*

2CONSTANT

```
( x1 x2 "name" -- )
```

Create a word returning a fixed double value.

Side effects: *Stores x2 at the lower address and x1 at the higher, matching 2@/2!.*

BUFFER:

```
( u "name" -- )
```

Create a word returning the address of u reserved, uninitialized bytes.

Side effects: *PFA in VARHERE.*

DEFER

```
( "name" -- )
```

Create a word whose action can be set later.

Side effects: *Default action throws -21 until IS/DEFER! is used.*

DEFER@

```
( xt-defer -- xt )
```

Fetch a deferred word's current target.

Side effects: *None.*

DEFER!

```
( xt xt-defer -- )
```

Set a deferred word's target.

Side effects: *None.*

IS

```
( xt "name" -- )
```

Set a deferred word's target by name. IMMEDIATE.

Side effects: *Behaves differently interpreting vs. compiling.*

ACTION-OF

```
( "name" -- xt )
```

Fetch a deferred word's current target by name. IMMEDIATE.

Side effects: *Behaves differently interpreting vs. compiling.*

MARKER

```
( "name" -- )
```

Create a word that, when executed, forgets itself and everything defined after it.

Side effects: *Restores all three HERE pointers and LATEST from a snapshot.*

3.10 Compiling Words

IMMEDIATE

```
( -- )
```

Mark the most recently defined word as IMMEDIATE.

Side effects: *Sets a header flag bit.*

STATE
`( -- addr )`
Variable: nonzero while compiling.
Side effects: *None.*

[
`( -- )`
Enter interpret state. IMMEDIATE.
Side effects: *None.*

]
`( -- )`
Enter compile state.
Side effects: *None.*

'
`( "name" -- xt )`
Look up a word's execution token.
Side effects: *Throws -13 if not found. Works identically interpreting or compiling.*

COMPILE,
`( xt -- )`
Compile a call to the given execution token.
Side effects: *None.*

LITERAL
`( x -- )`
Compile x as a literal. IMMEDIATE, compile-only.
Side effects: *None.*

2LITERAL
`( x1 x2 -- )`
Compile x1 x2 as a double-cell literal. IMMEDIATE, compile-only.
Side effects: *Executing the compiled literal pushes x1 then x2, so x2 is on top, matching 2CONSTANT and 2@.*

[']
`( "name" -- )`
Compile-time tick: compile a word's xt as a literal. IMMEDIATE, compile-only.
Side effects: *Throws -14 if used while interpreting; -13 if the name is not found.*

POSTPONE
`( "name" -- )`
Defer a word's compiling behavior into the current definition. IMMEDIATE, compile-only.
Side effects: *Throws -14 if used while interpreting; -13 if the name is not found.*

[COMPILE]
`( "name" -- )`
Obsolescent predecessor to POSTPONE: unconditionally compile a direct call to the found word, right now. IMMEDIATE, compile-only.
Side effects: *Throws -14 if used while interpreting; -13 if the name is not found. Unlike POSTPONE, makes no immediate/non-immediate distinction - prefer POSTPONE for a word whose own compilation semantics is not simply "compile a call to me".*

>BODY
`( xt -- pfa )`
Recover a CREATEd word's parameter field address.

Side effects: *Assumes the standard trampoline layout; fixed offset arithmetic.*

EXECUTE

`( xt -- )`

Execute the word with the given execution token.

Side effects: *None.*

SLITERAL

`( addr len -- )`

Compile a runtime string as a literal. IMMEDIATE, compile-only.

Side effects: *Throws -14 if used while interpreting.*

ABORT"

`Compile: ( "msg" -- ) Run: ( flag -- )`

Conditionally type a message and abort. IMMEDIATE, compile-only.

Side effects: On true flag: types the message, throws -2. The message is read with PARSE up to the closing quote, as for S", so an empty message is valid; over 255 characters throws -18.

3.11 Memory

@

`( addr -- x )`

Fetch a cell.

Side effects: *None.*

!

`( x addr -- )`

Store a cell.

Side effects: *None.*

C@

`( addr -- char )`

Fetch a byte, zero-extended.

Side effects: *None.*

C!

`( char addr -- )`

Store the low byte only.

Side effects: *None.*

+!

`( n addr -- )`

Add n into the cell at addr.

Side effects: *None.*

2@

`( addr -- x1 x2 )`

Fetch a double cell.

Side effects: *x2 is fetched from the address (and ends on top), x1 from address+2, matching the standard.*

2!

`( x1 x2 addr -- )`

Store a double cell.

Side effects: *x2 is stored at the address, x1 at address+2, matching the standard.*

,
`( n -- )`
Append one cell to the immutable/code region (CODEHERE).
Side effects: *Advances CODEHERE.*

C,
`( char -- )`
Append one byte to the immutable/code region.
Side effects: *Advances CODEHERE.*

ALLOT
`( n -- )`
Reserve (or release, if negative) bytes in the immutable/code region.
Side effects: *No bounds check against adjoining regions.*

HERE
`( -- addr )`
Current allocation point of the immutable/code region.
Side effects: *Reports CODEHERE.*

V,
`( n -- )`
Append one cell to the mutable/variable region (VARHERE).
Side effects: *Advances VARHERE.*

VC,
`( char -- )`
Append one byte to the mutable/variable region.
Side effects: *Advances VARHERE.*

VALLOT
`( n -- )`
Reserve (or release) bytes in the mutable/variable region.
Side effects: *No bounds check against adjoining regions.*

VHERE
`( -- addr )`
Current allocation point of the mutable/variable region.
Side effects: *Reports VARHERE.*

PAD
`( -- addr )`
A scratch buffer address, offset above CODEHERE by PADMINSIZE (128 bytes). Floats relative to CODEHERE, not a separately reserved region - its address changes as CODEHERE advances.
Side effects: Also used by interpreted S" (see S", below) for its own string storage, per ANS's allowance for non-standard words to use PAD as long as the use is documented. This means PAD and interpreted S" text share the same space - using PAD directly right after an interpreted S" call (or vice versa) will overwrite one with the other. PAD has at least 128 characters of capacity (ANS requires 84), reported by ENVIRONMENT? as /PAD.

UNUSED
`( -- u )`
Bytes remaining in the region HERE addresses.
Side effects: *Reports distance to the return/data-stack boundary.*

VUNUSED

`( -- u )`

Bytes remaining in the mutable/variable region.

Side effects: *Reports distance to APPCODE's start; not itself enforced.*

MOVE

`( src dst u -- )`

Copy u bytes, overlap-safe.

Side effects: *Chooses direction automatically based on address order.*

FILL

`( addr u char -- )`

Fill u bytes with char.

Side effects: *None.*

ERASE

`( addr u -- )`

Fill u bytes with zero.

Side effects: *None.*

BLANK

`( addr u -- )`

Fill u bytes with spaces (ASCII 32).

Side effects: *Equivalent to (addr u 32 FILL); a count of zero changes nothing.*

CMOVE

`( src dst u -- )`

Copy u bytes low address to high.

Side effects: *Copies from the lowest address upward, so it is safe when the destination is below the source or the regions do not overlap. With the destination above an overlapping source it repeats the first bytes: c-addr DUP CHAR+ 3 CMOVE turns "ABCD" into "AAAA", as in the standard's example. A count of zero changes nothing.*

CMOVE>

`( src dst u -- )`

Copy u bytes high address to low.

Side effects: *Copies from the highest address downward, so it is safe when the destination is above the source or the regions do not overlap. With the destination below an overlapping source it repeats the last bytes: c-addr DUP CHAR+ SWAP 3 CMOVE> turns "ABCD" into "DDDD", as in the standard's example. A count of zero changes nothing.*

3.12 Strings & Parsing

COUNT

`( c-addr -- addr len )`

Convert a counted string to address/length form.

Side effects: *None.*

WORD

`( char -- c-addr )`

Parse a delimited token into a counted string.

Side effects: WORDMAXCHARS-character cap (90) - well above the ANS minimum of 33. The cap is PADOFFSET (128) minus HOLDMINSIZE (34), minus the count byte, minus 3 spare bytes (S", ." and ABORT" once reserved them ahead of CODEHERE, but now parse with PARSE, so WORD never sees them). A longer token is cut at 90 characters; >IN is left after the 90th character, so the remainder is read as the next token. Scans directly into the CODEHERE-to-PAD gap rather than a separate fixed buffer, leaving room at the PAD end for the pictured numeric output buffer (HOLD and friends), which shares this same gap growing the opposite direction. Skips leading delimiters. The returned region isn't advanced past by

WORD itself - like PAD, it's expected to be overwritten by whatever gets compiled or allocated next, per standard transient-region semantics.

FIND

```
( c-addr -- c-addr 0 | xt 1 | xt -1 )
```

Look up a counted string in the dictionary, newest first.

Side effects: *Smudged entries are ignored. Returns xt and 1 if the word is immediate, xt and -1 if not; if not found, returns c-addr unchanged and 0.*

CHAR

```
( "name" -- char )
```

Parse the next word, leave its first character.

Side effects: *Inherits WORD's WORDMAXCHARS cap (90, irrelevant here since only the first character is kept).*

[CHAR]

```
( "name" -- )
```

Compile-time CHAR: compiles the character as a literal. IMMEDIATE, compile-only.

Side effects: *Throws -14 if used while interpreting.*

PARSE

```
( char "ccc<char>" -- addr len )
```

Parse a delimited token, no length cap.

Side effects: *Does not skip leading delimiters, unlike WORD.*

PARSE-NAME

```
( "<spaces>name<space>" -- addr len )
```

Parse a space-delimited token, no length cap.

Side effects: *Skips leading spaces.*

>NUMBER

```
( ud1 c-addr1 u1 -- ud2 c-addr2 u2 )
```

Convert the digits of a string, accumulating into ud1.

Side effects: *Uses the current BASE and stops at the first character that is not a valid digit; c-addr2 u2 is the unconverted remainder (u2 = 0 if the whole string was converted). Takes no sign or trailing dot; the interpreter's own number conversion handles those.*

CONVERT

```
( ud1 c-addr1 -- ud2 c-addr2 )
```

Obsolescent. Add the digits after c-addr1 to ud1 in the current BASE; c-addr2 is the first character that is not a digit.

Side effects: *Built on >NUMBER with no length limit. Use >NUMBER in new code.*

S"

```
Compile: ( "str" -- ) Run: ( -- addr len )
```

Parse and compile, or stage, a string literal.

Side effects: Compiling: appends inline to CODEHERE - the definition's own permanent storage, unaffected by anything below. Interpreting: copies into PAD (see PAD, above). The text is read with PARSE up to the closing quote, so an empty string (S" ") is valid; a string over 255 characters throws -18. PAD's interpreted-mode address is computed fresh from CODEHERE on every call, but only changes when something is compiled or allocated in between - a second S" call before an earlier string is actually used will still silently overwrite it. For any use needing more than one string to coexist (e.g. the two strings given to COMPARE), wrap each in its own colon definition (: name S" text" ;) instead - compiled S" never touches PAD at all, giving each string its own permanent storage.

S\"

```
Compile: ( "ccc<quote>" -- ) Run: ( -- addr len )
```

As S", but backslash escape sequences are translated and a backslash-quote does not end the string.

Side effects: *Escapes: \a 7, \b 8, \e 27, \f 12, \l 10, \m the pair 13 10, \n 10, \q and \" 34, \r 13, \t 9, \v 11, \z 0, \\ 92, and \ xhh (one or two hex digits, either case). Any other character after a backslash stands for itself. The text is translated at PAD and then compiled or returned exactly as S" does (see S", above), so interpreted S\" shares S"'s PAD caveat. A result over 255 characters throws -18; a string with no closing quote ends at the end of the line.*

C"

```
Compile: ( "ccc<quote>" -- ) Run: ( -- c-addr )
```

Compile a counted string; at run time return its address. IMMEDIATE, compile-only.

Side effects: *Throws -14 when interpreting. The string is stored inline after the compiled call, and execution skips over it. A string over 255 characters throws -18.*

."

```
( "str" -- )
```

Compile a string that types itself at run time. IMMEDIATE, compile-only.

Side effects: No interpretation semantics, per ANS. The text is read with PARSE up to the closing quote, as for S", so ." " (an empty string) is valid; over 255 characters throws -18.

COMPARE

```
( addr1 len1 addr2 len2 -- n )
```

Compare two strings, signed three-way result.

Side effects: *None.*

SEARCH

```
( addr1 len1 addr2 len2 -- addr3 len3 flag )
```

Find a substring within a string. If found, addr3 len3 is the part of the string that starts at the match (to its end) and flag is true; if not, addr1 len1 are returned unchanged and flag is false.

Side effects: *An empty needle (len2 = 0) matches at the start: addr3 len3 = addr1 len1, flag true, as the standard requires. len3 is the number of characters remaining from the match, not the length of the needle.*

-TRAILING

```
( addr len1 -- addr len2 )
```

Trim trailing spaces from a string's reported length.

Side effects: *None.*

/STRING

```
( addr1 len1 n -- addr2 len2 )
```

Trim n characters from the front of a string.

Side effects: *None.*

REPLACES

```
( addr1 len1 addr2 len2 -- )
```

Register a named substitution for SUBSTITUTE - addr1/len1 is the replacement text, addr2/len2 is the substitution name (matched later between % delimiters). Any number of names can be registered; registering a name again replaces its text.

Side effects: The table (SUBPOOL, 64 bytes at $0200-$023F) keeps its own copy of the name and the text, so the arguments may be overwritten or come from interpreted S" (PAD) afterwards. Each entry takes 5 bytes plus the name and text lengths; three 8-character name and text pairs fit. A redefinition whose text is no longer than the original is done in place; a longer one adds a new entry and the old entry's space is not reclaimed. Names are case sensitive and are not forgotten by MARKER; CLR-REPLACES empties the table. Throws -18 if the name or text is longer than 255 characters, -8 if the pool is full.

CLR-REPLACES

```
( -- )
```

Empty the REPLACES table, forgetting every registered substitution and freeing the whole pool.

Side effects: *Not an ANS word: a ClaudeForth extension. The 64-byte pool (see REPLACES) never reclaims space, so a full pool (throw -8) or the leftovers of redefinitions are cleared by calling this. COLD also starts with an empty table.*

SUBSTITUTE

```
( addr1 len1 addr2 len2 -- addr2 len3 n )
```

Scan addr1/len1 for text between '%' delimiter pairs. Each %name% whose name has been registered with REPLACES has the whole span - delimiters included - replaced by its text; %% becomes a single %; an unregistered name, or a trailing % with no closing delimiter, is copied unchanged. The result is stored at addr2 (capacity len2), length len3.

Side effects: n is the number of substitutions made when zero or positive. A negative n is an error and does not throw: -18 if the result would not fit in len2 characters (nothing is written beyond the buffer), -9 if the destination is the same buffer as the source (the source and destination must not overlap). len3 is unspecified on error.

SNAME

```
( xt -- addr len )
```

Reverse dictionary lookup: xt to name. Non-standard.

Side effects: *Returns zero length if not found.*

UNESCAPE

```
( c-addr1 u1 c-addr2 -- c-addr2 u2 )
```

Replace every '%' character in the source string with two '%' characters; every other character passes through unchanged. Passing a string through UNESCAPE and then SUBSTITUTE returns the original string - this is UNESCAPE's specific purpose, protecting a literal '%' from being read as a SUBSTITUTE delimiter. Has nothing to do with backslash sequences (\n, \t, etc.) despite the name's similarity to that idea in other languages - confirmed against the ANS reference implementation.

Side effects: u2 can be longer than u1 - each '%' in the source adds one extra character to the output. No destination-buffer-overflow check, matching the ANS reference implementation, which also has none (an ambiguous condition per the standard, not a required error case).

3.13 Numeric Output

<#

```
( -- )
```

Begin pictured numeric output conversion.

Side effects: Sets HLD to PAD's address. To get a proper ud1 onto the stack for the # family below, type the number with a trailing '.' (e.g. "123." rather than "123") - the text interpreter converts any number immediately followed by a decimal point to a double-cell number, per ANS's own input convention (forth-standard.org/standard/usage#usage:numbers, 3.4.1.3). "123" alone stays single-cell and is not a valid argument to #.

#

```
( ud1 -- ud2 )
```

Convert one digit.

Side effects: Uses the current BASE. Expects a genuine double-cell ud1 - see <# above for how to get one onto the stack from typed input.

#S

```
( ud1 -- ud2 )
```

Convert remaining digits until the value is zero.

Side effects: *Uses the current BASE.*

#>

```
( ud -- addr len )
```

End pictured conversion, discarding ud.

Side effects: *None.*

HOLD

```
( char -- )
```

Insert one character into the pictured output buffer.

Side effects: *Buffer grows downward from PAD; no bound enforced.*

HOLDS

```
( addr len -- )
```

Insert an entire string into the pictured output buffer.

Side effects: *Depends on HOLD's exact decrement-by-one behavior.*

SIGN

```
( n -- )
```

Insert a minus sign if n is negative.

Side effects: *None.*

.

```
( n -- )
```

Print signed, trailing space.

Side effects: *None.*

U.

```
( u -- )
```

Print unsigned, trailing space.

Side effects: *None.*

.R

```
( n width -- )
```

Print signed, right-justified in a field.

Side effects: *No trailing space; width is a minimum, not a cap.*

U.R

```
( u width -- )
```

Print unsigned, right-justified in a field.

Side effects: *No trailing space; width is a minimum, not a cap.*

?

```
( addr -- )
```

Fetch and print the cell at addr, signed.

Side effects: *None.*

D.

```
( d -- )
```

Print signed double, trailing space.

Side effects: *None.*

D.R

```
( d width -- )
```

Print signed double, right-justified.

Side effects: *No trailing space.*

3.14 Base / Radix Control

BASE

```
( -- addr )
```

Variable holding the current number base.

Side effects: *None.*

DECIMAL

```
( -- )
```
Set BASE to 10.
Side effects: *None.*

HEX
```
( -- )
```
Set BASE to 16.
Side effects: *None.*

BINARY
```
( -- )
```
Set BASE to 2. Not an ANS-standard word.
Side effects: *None.*

3.15 Exception Handling

CATCH
```
( xt -- 0 | n )
```
Execute xt, trapping any THROW.
Side effects: *Restores data stack depth, the input source and HANDLER on either path. The exception frame (12 bytes) is held on the return stack. A CATCH in user code does not restore DPHERE, CODEHERE, VARHERE or LATEST; QUIT does that for the whole input line.*

THROW
```
( n -- )
```
Raise exception n, or do nothing if n is zero.
Side effects: *Nonzero: sets S from HANDLER in one step, discarding all intervening call levels, and makes the matching CATCH return n; falls into ABORT (with n on the stack) if none is active.*

3.16 Comments

(
```
( "ccc<paren>" -- )
```
Discard input up to the next close paren. IMMEDIATE.
Side effects: *Works interpreting or compiling.*

.(
```
( "ccc<paren>" -- )
```
Display the text up to the next close paren at once. IMMEDIATE.
Side effects: *Works interpreting or compiling. The text is shown when .(is parsed, not when a definition containing it later runs.*

\
```
( "ccc<eol>" -- )
```
Discard the remainder of the current input line. IMMEDIATE.
Side effects: *Follows SOURCE, so it operates correctly inside EVALUATE.*

3.17 Environmental & System Queries

ENVIRONMENT?
```
( c-addr u -- false | i*x true )
```
Query implementation-defined characteristics.
Side effects: *Supports /COUNTED-STRING (255), MAX-N (32767), MAX-U (65535), ADDRESS-UNIT-BITS (8), /HOLD (34), /PAD (128), FLOORED (0 - this system uses symmetric, not floored, division), MAX-CHAR (255), MAX-D and MAX-UD (double-cell; push low then high before the true flag), RETURN-STACK-CELLS (384), STACK-CELLS (512), and the word-set queries CORE,*

CORE-EXT, DOUBLE and DOUBLE-EXT. A word-set query returns a flag for the set followed by true for "recognized", so each answers true true. WORDLISTS is deliberately unrecognized (returns plain false, not 0 true) since the Search-Order word set isn't implemented.

SOURCE

```
( -- addr len )
```

The current input source.

Side effects: *Follows terminal input or an active EVALUATE string.*

SAVE-INPUT

```
( -- x1 x2 x3 3 )
```

Save the input source specification.

Side effects: *Saves the source address, the source length and >IN as three cells followed by the count 3.*

RESTORE-INPUT

```
( x1 x2 x3 3 -- flag )
```

Restore the input source specification saved by SAVE-INPUT. Returns false on success.

Side effects: *Returns true, and leaves >IN alone, if the count is not 3 or the source address or length differs from the saved one, so it cannot restore across REFILL or into a different EVALUATE string.*

SOURCE-ID

```
( -- 0 | -1 )
```

Identify the current input source. 0 = terminal, -1 = string.

Side effects: *None.*

REFILL

```
( -- flag )
```

Attempt to refill the input source.

Side effects: *Fails (false) if the current source is a string, per ANS.*

EVALUATE

```
( addr len -- )
```

Interpret a string as Forth source.

Side effects: *Saves and restores the whole input-source state around the call.*

TIB

```
( -- addr )
```

The fixed terminal input buffer address.

Side effects: *Always the terminal buffer (80 bytes, TIBBUFL, at $0106), unaffected by EVALUATE.*

#TIB

```
( -- addr )
```

Variable: count of valid characters in TIB.

Side effects: *Always the terminal buffer's count.*

>IN

```
( -- addr )
```

Variable: scan offset into the current input source.

Side effects: *None.*

SPAN

```
( -- addr )
```

Variable: EXPECT's returned character count.

Side effects: *None.*

BL

`( -- char )`

The blank/space character, 32.

Side effects: *None.*

3.18 Tools

.S

`( -- )`

Print the data stack, top to bottom.

Side effects: *Non-destructive. Each cell is printed with DOT, so signed, in the current BASE, and followed by a space.*

WORDS

`( -- )`

List every word in the dictionary.

Side effects: *Walks the full ROM+RAM chain from LATEST, then sends a CR.*

DUMP

`( addr u -- )`

Hex and ASCII memory dump, 16 bytes per line.

Side effects: *Sends a CR first and after every line. The last line is padded so the ASCII column lines up. Prints no addresses.*

4. Forth Development Notes

This section is written for the audience who will program the completed system: users developing a Forth application on top of ClaudeForth, not those building or modifying the system itself. It assumes the base system already runs and focuses on what's needed to work productively within it day to day.

This section covers five practical aspects of working with the system that fall outside the word-by-word glossary: how serial communication is configured, a brief introduction to Forth syntax and stack operations, worked examples of the Tools word set, how to reclaim dictionary space during interactive development using MARKER, and why the optional Search-Order word set is not part of this implementation.

4.1 Serial I/O and Handshaking

Serial communication parameters — baud-rate divisor, word length, parity, and stop bits — are fixed by the hardware, not selectable from Forth. The 6850 ACIA's control register is written once, at COLDSTRT (INITSERIAL), encoding a ÷16 clock, 8 data bits, no parity and 1 stop bit; no word in this system changes these settings at run time, so the terminal or host at the other end of the link must be configured to match. The baud rate itself is set by the ACIA's clock: the MAME emulation of the MECB6809 uses 57600 baud.

The system can be built with either of two serial drivers, chosen at assembly time by the SERIALPOLL option (Section 6). SERIALPOLL=1, the default, is the polled driver: no interrupts and no ring buffers. KEY, KEY? and EMIT read ACIASR directly, the control register holds CR_POLL ($15, receive interrupt off, RTS held low) and IRQH is an RTI stub. SERIALPOLL=0 is the interrupt-driven driver: the receiver and transmitter are serviced by IRQH through the 64-byte INBUF and OUTBUF ring buffers, with hardware (RTS/CTS) handshaking. The two builds differ in size (Section 2) but present identical behaviour to Forth programs.

Polled driver (SERIALPOLL=1) — software XON/XOFF

The polled driver has no input buffer, so it cannot absorb characters that arrive while the processor is busy. Instead it paces the host with software (XON/XOFF) handshaking applied around each input line. When ACCEPT starts reading a line it transmits XON ($11) with PUTXON, telling the host it may stream the line's characters; when the line is complete (CR received) it transmits XOFF ($13) with PUTXOFF, so the host pauses while INTERPRET runs and KEY is not being polled. A host or terminal emulator must therefore honour XON/XOFF — for example minicom with xonxoff enabled, or the ANS Test Suite runner, which honours XON/XOFF from the target. RTS is held low (asserted) permanently, and the ACIA's receive interrupt is disabled. Incoming XON/XOFF bytes are not recognised by the system: an incoming $11 or $13 is returned by KEY like any other character.

KEY counts the receiver errors it sees — framing (FECOUNT), overrun (OVRNCOUNT) and parity (PECOUNT) — and POLLREADYCNT, which counts the characters that were already waiting when KEY was called (a sign that input is arriving faster than it is consumed). These are single-byte counters in the serial buffer control block (SERBUFCTL, Section 2), cleared at start-up. They can be examined with a monitor or debugger; no Forth word reports them.

Interrupt-driven driver (SERIALPOLL=0) — hardware RTS/CTS

In the interrupt-driven build, RTS — this system's output telling the remote device when to pause sending — is toggled automatically based on the input ring buffer's fill level. IRQH's receive path calls RTSCHECKHI after storing each incoming byte; once the ring reaches INHIWATER (48 of its 64 bytes), RTS

is driven high by writing CR_RTSHI to the ACIA control register, and RTS stays high until KEY, on the mainline side, drains the ring back down to INLOWATER (16 bytes) and calls RTSCHECKLO to drop it low again. The 32-byte gap between the two marks is deliberate hysteresis, avoiding RTS toggling right at a single threshold.

CTS — the remote device's own flow-control signal telling this system when it may transmit — needs no firmware logic at all: the 6850 ACIA inhibits TDRE in hardware whenever CTS is deasserted, and every transmit path in this system (EMIT, IRQH's TXCHK) already gates on TDRE, so CTS is honored automatically with no code changes required.

One real ACIA constraint shapes this design directly: the 6850's control register ties RTS state and transmit-interrupt-enable to the same two bits, so RTS-high and TX-interrupt-enabled can never be selected simultaneously. While RTS is asserted high, EMIT still queues outgoing bytes into the output ring but deliberately does not enable the transmit interrupt to drain them — RTSCHECKLO re-enables it once RTS drops low, restoring CR_RXTX if output is still queued at that point. A second, genuine race is worth naming rather than glossing over: RTSCHECKHI runs from inside the interrupt handler, where hardware has already masked further interrupts, but RTSCHECKLO runs from ordinary mainline code (KEY) and could otherwise be interrupted mid-decision by IRQH's own TXOFF path, which also writes the control register — RTSCHECKLO masks IRQ around its own critical section specifically to prevent that.

The interrupt-driven build performs no software handshaking: it neither transmits XON/XOFF nor recognises those bytes if a remote device sends them. On a genuine 3-wire serial link with no RTS/CTS wiring, that build would have no flow control of any kind; its hardware path only helps when RTS/CTS are physically connected. The polled build's XON/XOFF pacing suits such a link, in the receive direction only.
Connecting to MAME. Hardware handshaking can be handled more reliably by MAME and the host than by software running in the emulated CPU. For RTS/CTS, open MAME's configuration menu (Tab), choose Machine Configuration and set the RS-232 baud rate to 57600 and the handshaking mode to RTS/CTS; for XON/XOFF choose that mode instead. Then bridge MAME's null-modem socket to a pseudo-terminal with socat (crtscts=1 for RTS/CTS, or ixon=1,ixoff=1 for XON/XOFF) and connect minicom to it with a settings file for 57600, 8-N-1 with the matching handshaking. The exact socat, minicom and MAME command lines are in MAME_to_minicom.md, distributed with the source. If character reception is still unreliable, check for competing background load (stray minicom processes, system updates, virus scanners) or raise MAME's priority.

4.2 Forth Syntax and Stack Operations

Forth has no operators, no operator precedence, and no infix expressions. A line of input is simply a sequence of whitespace-delimited words, read and acted on strictly left to right. Each word is either a number, which is pushed onto the data stack, or a name that is looked up in the dictionary and executed immediately. Arithmetic and every other operation work by taking their arguments off the top of the stack and pushing their result back onto it — this is postfix, or reverse Polish, notation: the operator always comes after its operands, never between or before them.

A numerical example:

Computing (3 + 4) × 2 — 14 — is written:

```
3 4 + 2 * .
14   ok
```

Read left to right, tracking the data stack after each word (shown deepest item first, top of stack last):

```
3       ->  ( 3 )             push 3
4       ->  ( 3 4 )           push 4
+       ->  ( 7 )             pop 4 and 3, push their sum
2       ->  ( 7 2 )           push 2
*       ->  ( 14 )            pop 2 and 7, push their product
.       ->  ( )               pop and print: 14
```

The stack-effect notation used throughout the Glossary (Section 3) — for example, +'s (n1 n2 -- sum) — describes exactly this: the items expected on the stack before a word runs, to the left of "--", and what it leaves in their place afterward, to the right. Reading a word's stack effect this way is enough to know how to use it correctly without needing to know anything about how it is implemented internally.

This system does not require building up a vocabulary from nothing: a large set of predefined words — arithmetic, stack manipulation, comparison, control flow, string and memory operations, and more — already exists and is documented word by word in the Glossary. At the interactive prompt, executing WORDS lists every one of these preexisting words currently in the dictionary, from the most recently defined word back through every built-in primitive. See Section 4.3 for a worked example of WORDS and the other Tools words.

4.3 Tools Word Set: Explanation and Examples

The three Tools words in the Glossary — .S, WORDS, and DUMP — are interactive aids rather than words a program typically calls itself. The examples below show plausible session transcripts. One real characteristic of this system worth knowing before reading them: output from a word is written immediately, with no leading newline of its own — the newline seen before "ok" in a normal session comes from QUIT, not from the word that ran. The transcripts below add a line break before each result for readability; on the actual wire, output from .S, WORDS, or DUMP appears immediately after the typed command, on the same line.

.S — print the data stack, top to bottom, non-destructively

Pushes nothing and removes nothing; it only reads the stack from the current top down to SP0 and prints each cell with the same signed formatting as DOT. This makes it safe to run at any point without disturbing whatever a definition is part-way through building on the stack.

```
1 2 3 .S
3 2 1   ok

\ top of stack (3, pushed last) prints first, deepest
\ item (1, pushed first) prints last
```

WORDS — list every word in the dictionary

Walks the LATEST chain from the most recently defined application word all the way back through every ROM word, printing each name separated by a space. The glossary lists 242 words in all, so real output is long — the excerpt below shows the shape of it rather than a complete transcript.

```
WORDS
SQUARE FENCE VALUE TO VARIABLE CONSTANT DOES>
CREATE ... DUP DROP SWAP OVER ROT DEPTH ...
+ - * / MOD ... IF THEN ELSE BEGIN ... ok

\ most-recently-defined application words appear first,
\ since the walk follows LATEST backward through LINK;
\ ROM primitives make up the remainder of the list
```

DUMP — hex and ASCII memory dump, 16 bytes per line

Takes an address and a byte count, and prints that many bytes, 16 per line: each byte as two hex digits, then a matching ASCII column with unprintable bytes shown as a period. The first line starts immediately after the command with no leading newline of its own, like the other Tools words. Worth noting explicitly since it differs from many DUMP implementations: this one does not print a leading address on each line — only the hex bytes and their ASCII rendering — so the caller must track which address a given line corresponds to separately if that matters.

```
HEX
7000 10 DUMP
BD E4 32 8E 00 00 39 00 00 00 00 00 00 00 00 00  ........9.......

\ 16 bytes shown starting at $7000 (the start of APPCODE);
\ actual bytes depend entirely on what has been compiled
\ into code space by the time DUMP is run
```

4.4 Dictionary Management with MARKER

ANS Forth provides no general "undefine" word; instead, the standard's mechanism for reclaiming dictionary space is MARKER. Executing MARKER with a name creates an ordinary dictionary entry that, when later executed itself, forgets its own definition and every definition made after it — restoring the dictionary to exactly the state it was in at the moment MARKER was invoked. This is the standard way to experiment interactively without permanently accumulating discarded or superseded definitions.

In this implementation, a word created by MARKER stores a snapshot of all three region pointers — DPHERE (dictionary), CODEHERE (code), and VARHERE (mutable variable space) — plus LATEST, at the moment of its own creation. Executing the marker word (DOMARKER) simply writes those four saved values back, in one step. Nothing needs to be erased or unwound word by word: every allocation word in the system (comma, ALLOT, HEADER, and their VARHERE counterparts) only ever appends at the current pointer, so rolling the pointer back alone is sufficient to make everything defined afterward unreachable and its space available again for reuse.

Example:

```
MARKER FENCE
: SQUARE  DUP * ;
5 SQUARE .            \ prints 25

FENCE                 \ forgets FENCE itself, and SQUARE

5 SQUARE .            \ throws -13, undefined word: SQUARE
```

MARKER FENCE creates FENCE and captures the current DPHERE/CODEHERE/VARHERE/LATEST as its own PFA contents. SQUARE is then compiled normally, advancing all three pointers past that snapshot. Executing FENCE restores the four saved values, which puts LATEST back to whatever it was immediately before FENCE was defined — so SQUARE is no longer reachable by FIND, and FENCE itself is gone too, along with the dictionary, code, and variable space both of them occupied. A second MARKER call after this point would need a new name, since FENCE no longer exists to be reused.

Caveat: *as noted in the glossary, a MARKER's snapshot only covers the single dictionary chain this system currently maintains. It is not defined against the optional Search-Order word set, which this implementation does not include — see Section 4.5.*

4.5 The Search-Order Word Set Is Not Included

ANS Forth's optional Search-Order word set (FORTH-WORDLIST, WORDLIST, GET-CURRENT, SET-CURRENT, GET-ORDER, SET-ORDER, ALSO, ONLY, PREVIOUS, DEFINITIONS, and SEARCH-WORDLIST) is not implemented in this system. Every one of the 242 words in the Glossary is resolved through a single, unified dictionary chain rooted at LATEST — there is only ever one wordlist, and new definitions always link into it. Search-Order's whole purpose is letting a program maintain several independent wordlists (for example, to keep an application's vocabulary separate from the words used only to build it) and to search a chosen, ordered subset of them; without it, every word ever defined is always visible to FIND, in one flat namespace, for the lifetime of the system.

This was a foundational decision, not an oversight discovered late: the single-chain design was fixed from the very first system-initialization code, where LATEST is set to the top of the ROM dictionary at cold start and every application word links onto that same chain afterward. Adding Search-Order now would not be a small addition — it would touch code that already exists throughout the system, not just introduce new words. FIND itself would need restructuring into an outer loop over an active search order, trying each wordlist in turn; HEADER, which every defining word (colon definitions, CREATE, VARIABLE, CONSTANT, VALUE, and the rest) already funnels through, would need to link new definitions into a selectable "current" wordlist rather than unconditionally into LATEST; and a few words that deliberately read LATEST directly today — RECURSE, the unsmudging step in semicolon, and WORDS — would each need the same substitution or they would silently keep operating on the wrong wordlist once more than one existed. MARKER's snapshot would also need to widen to cover every wordlist's head cell, not just LATEST, with the further complication that ANS itself leaves a marker's interaction with wordlists created after it as implementation-defined, not fully specified.

In practice, this means every program written against this system lives in one shared, flat namespace — there is no mechanism to hide a helper word's name from being visible everywhere else, and no way to have two differently-scoped words share the same name. For the scope of a single-user, single-application embedded system, this is a reasonable simplification rather than a defect; it is flagged here specifically so it is not mistaken for an accidental gap in ANS coverage.

4.6 Exception Handling

CATCH and THROW (Section 3.15) let a definition signal a problem to its caller without forcing every intermediate word on the call chain to check for it. The example below defines a guarded division word that throws rather than dividing by zero, and a caller that traps that exception with CATCH and reports it with an appropriate message in place of a numeric result. When CATCH traps a throw it restores the data-stack pointer to its depth at the time of the CATCH (after the execution token was removed), so the arguments the failed word was given are still on the stack and the handler must drop them.

```
\ SAFE/ throws -10 (the standard ANS code for division
\ by zero) instead of dividing when the divisor is 0
\ GUARDED/ wraps SAFE/'s execution token in CATCH,
\ which traps the THROW and returns its code in place
\ of letting it propagate uncaught; nonzero means SAFE/
\ threw, so GUARDED/ drops n1 and n2 and reports a message instead of the
\ numeric result CATCH's success path leaves

\ Usage:
\ 10 5 GUARDED/
\ 2
\ 10 0 GUARDED/
\ can't divide by zero

: SAFE/ ( n1 n2 -- n1/n2 )
  DUP 0= IF -10 THROW THEN
  / ;

: GUARDED/ ( n1 n2 -- )
  ['] SAFE/ CATCH
  IF 2DROP ." can't divide by zero" CR
  ELSE . CR THEN ;
```

5. Open Items Checklist

Everything below was explicitly flagged during the build as incomplete, unverified, or deliberately deferred. Nothing here is a surprise — each item was named at the point it came up. This is a consolidated list to work from, not a new set of findings. Regenerated to reflect fixes and design decisions made since the original version.

This section is a point-in-time snapshot of the checklist maintained alongside the assembler source, which has since grown considerably beyond what is practical to reproduce here in full. Rather than continuing to expand this section, further additions are tracked in that external checklist directly (OPEN ITEMS CHECKLIST part1.md, part2.md and part3.md, distributed alongside the source) — see those files for the current, complete state. This section itself is revised only when an item listed below is genuinely addressed, not to keep pace with ongoing work. At this revision every item marked as open below was re-checked against the implementation and none has been completed since the previous revision.

5.1 Known bugs — resolved since the original checklist

✅ **DUMP's partial-final-line bug** — fixed via DUVALID tracking (`25_tools_word_set.asm`). The ASCII column no longer reads past the valid bytes on a short final line.

✅ **SUBSTITUTE — completed.** It performs a real bounds-checked copy (prefix / replacement / suffix) via a shared SUBCOPY helper, reusing SEARCHW for the match. It now draws on a multi-entry REPLACES table; see the next items.

✅ **SEARCH, REPLACES and SUBSTITUTE brought into line with the standard — resolved (October 2026).** The optional String word tests (ANS Annex F) showed that the tests were right and the implementation wrong. SEARCH returned the needle's length as len3 instead of the characters remaining from the match, and treated an empty needle as not found. REPLACES held a single registration, so a second call lost the first; it is now a table in SUBPOOL (64 bytes, $0200-$023F, three 8-character pairs) that keeps its own copy of each string. SUBSTITUTE threw -1 on overflow; it now returns a negative n (-18 output overflow, -9 source and destination the same buffer) as the standard requires. The assembler unit test for SEARCH had the same wrong expectation and was corrected. New tests cover empty needles, several names, redefinition, overflow and same-buffer.

✅ **BLANK and CLR-REPLACES added — done.** BLANK (addr u --) is 32 FILL, with tests for BLANK, CMOVE and CMOVE> (overlap in both directions and counts over 255) in both the assembler unit tests and the ANS file. CLR-REPLACES is a ClaudeForth extension that empties the REPLACES table, the recovery when the 64-byte pool fills.

✅ **S", ." and ABORT" with an empty string hung compilation — resolved (October 2026).** They parsed their text with WORD, which skips leading delimiters, so S" " skipped its own closing quote and ran on to the next quote in the source, leaving the definition open. They now use PARSE through a shared helper (PARSEQUOTE); an empty string is valid and a string over 255 characters throws -18. Found by the ANS test for the String words.

✅ **Static temporaries in page zero reduced — done (stages 5a to 5e, October 2026).** The page-zero cells that held a routine's working values between instructions were moved onto the stacks: parameters stay on the data stack and are reached U-relative, and locals live in short frames on either stack. This covers CMOVE, FILL, COMPARE, SEARCH, TYPE, SPACES, ROLL, 2>R, 2R>, 2R@, .S, WORDS, .R, U.R, D.R, DUMP, UNESCAPE, SUBSTITUTE and ACCEPT, among others. The GLOBALS block now uses 127 of its 256 bytes (it was 255), MVSCRATCH is gone, and BASECODE shrank by about 210

bytes in the polled build. A unit test harness defect found on the way (the tests ran before COLD with CODEHERE at zero, so PAD fell in page zero) was fixed in TSTRUNNER.

☑ **Page-zero scratch removed from WORD, PARSE, FIND, number conversion and more — done (stages 5k to 5m, October 2026).** Stage 5k moved the scratch of WORD, PARSE, PARSE-NAME, -TRAILING, HEADER, SNAME, ENVIRONMENT?, ELSE, ENDOF, UNTIL, AGAIN, LOOP, +LOOP, REPEAT and EXIT into registers and short S frames, and the S", ." and ABORT" family now passes its string as c-addr u on the data stack (PARSEQUOTE) and compiles it through one shared helper (CSTRCOMP). Stage 5l did the same for FIND (the c-addr stays on the data stack and is replaced by the result) and for SETDOES. Stage 5m made >NUMBER work in place on its (ud c-addr u) frame on the data stack, which NUMBER? now builds itself, made # and #S divide the double in place, and moved the sign cell of . .R D. and D.R onto S. The GLOBALS block now uses 69 of its 256 bytes (127 before stage 5k) and BASECODE is 310 bytes smaller than at stage 5j in the polled build. CURXT, which used to share a cell with a scratch variable, now has a cell of its own.

☑ **Page-zero reduction stopped at stage 5m — decision (October 2026).** What remains in page zero is the shared multiply and divide convention (PRODHI/PRODLO, DIVNUM, DIVDEN, DIVREM, the MAHI to MBLO operand bytes, the sign flags) and the four MSCR cells, about 45 bytes in all. UM*, UM/MOD, SM/REM, FM/MOD, */, */MOD, M* and D. deliberately share them. Moving them onto the stack would rewrite about 800 lines of arithmetic, make it larger and slower (indexed stack access costs more than direct-page access in the multiply and divide loops) and free space that nothing needs, so it was not done.

☑ **Double-Number word set completed — done (stage 6a, October 2026).** D0=, D0<, D2*, D2/, M*/, 2LITERAL and 2VALUE were the missing words, and TO now recognizes a 2VALUE. M*/ forms the triple-cell product of |d1| and |n1| with two UMUL32 steps, then divides it by |n2| in three 32/16 UDIV32 steps, and negates the result when an odd number of the three signs are negative. The method was validated against exact integer arithmetic in a Python model (200000 random cases including the extremes) before it was written in assembler. Division is symmetric like the rest of the system, and a zero divisor throws -10. A new ANS test file (28_double_word) was built from the Annex F Double-Number tests; its constants are now computed from M*/, D2/ and the MAX-D environment query rather than hard-coded, and every test passes in MAME.

☑ **ENVIRONMENT? answers the word-set queries — done (stages 6a and 6b, October 2026).** CORE, CORE-EXT, DOUBLE and DOUBLE-EXT now answer. A word-set query returns a flag for the set and then the recognized flag, so each gives true true; an unknown name still returns a single false. The ANS environmental-query test file now checks the whole standard query table (/COUNTED-STRING, /HOLD, /PAD, ADDRESS-UNIT-BITS, MAX-CHAR, MAX-N, MAX-U, MAX-D, MAX-UD, FLOORED, the four word sets and both stack sizes), with /HOLD and /PAD checked against the standard minimums.

☑ **Core Extension word set completed — done (stages 6b and 6c, October 2026).** An audit against the standard found six words missing: .(, C", SAVE-INPUT, RESTORE-INPUT, S\" and the obsolescent CONVERT. All six are now in the dictionary, so ENVIRONMENT? can answer CORE-EXT true. SAVE-INPUT saves the source address, source length and >IN; RESTORE-INPUT refuses to restore across REFILL or into a different source. S\" translates its escapes into PAD and then shares the S" code path. A new test file (29_coreext_strings) covers them. Stage 6c grew the base code enough that the interrupt-driven build no longer fitted below INITCODE, so BASECODE and BASEDICT were lowered together (BASECODE now $DC44, BASEDICT $D395); that build now has 78 bytes of headroom, so the next addition to the base code will need the same treatment or a saving elsewhere.

✅ **Unit test group 11 split — done (stage 6b, October 2026).** With the new words the StrParse unit tests no longer fitted below BASEDICT in the interrupt-driven build (the assembler reported an invalid FILL length). The tests for REPLACES, SUBSTITUTE, CLR-REPLACES, SNAME and UNESCAPE moved to a new group, TSTSELECTOR=18 (StrParse2). Group 11 keeps the rest, and the default selector-0 run calls both.

✅ **VALUE's PFA** — moved from CODEHERE (immutable space) to VARHERE (mutable space), matching VARIABLE's pattern, so every TO-driven update now writes into the correct region instead of into code space. Not on the original checklist (found and fixed after this list was first generated); logged here for the record.

✅ **Every scratch/global cell now has a real RMB-assigned address in page zero** (`00_memory_map_and_globals.asm`), applied rather than left as a placeholder. Two real findings came out of doing this: the budget is **253 of 256 bytes used — only 3 bytes of headroom** in the GLOBALS page, and SNEND (documented in the original placeholder list) turned out to be genuinely dead — never read or written anywhere — and was dropped rather than given an address. Any future scratch cell added to this system will need to fit in that remaining 3 bytes or the page-zero fast-addressing property (DP = $00) stops covering it. (Historical: page zero is no longer nearly full. See the "Static temporaries in page zero reduced" item below; GLOBALS now uses 69 of 256 bytes.)

✅ **The ROM base dictionary is now real** (`27_forth_dictionary.asm`) — every primitive with actual code got a real header, chained via LINK, CFA pointing straight at its code label. ABORTHDR's LINK field, a placeholder 0 since it was first built, is now resolved to the chain's newest entry. Building this surfaced two real findings, not just mechanical work: the original 1024-byte BASEDICT could not hold the header table (ultimately 1954 bytes needed, once DOES> was added — see below), so it was resized to 2048 bytes, taking the space from BASECODE (now 6080 bytes, was 7104) — a resize based purely on the header-table budget, not on any actual measurement of BASECODE's assembled size, which has never been checked with a real 6809 assembler. Generating this table also surfaced that **DOES> had no corresponding code anywhere in the file** — resolved in a follow-up pass, see below.

✅ **DOES> — resolved.** SETDOES (the patching runtime) was added beside DODOES/DOESRT0, and DOES> itself (code label DOESGT, since a literal ">" is not a valid 6809 assembler label) was added right after CREATE. DOESBEH was added to the GLOBALS layout to support it, using 2 of the page's last 3 free bytes — only 1 byte of headroom now remains in page zero. DOES> also has a real dictionary header now (H_DOESGT, the chain's newest entry); ABORTHDR's LINK was updated again to point at it.

✅ **Four internal loop-label names were reused across unrelated routines: FLOOP, UDLOOP, DRPOS, CMLOOP.** Resolved — each pair renamed to a distinct name scoped to its own routine: FIND's FLOOP → FFLOOP; FILLW's FLOOP → FILLOOP; UDIV16's UDLOOP → UD16LOOP; UDDIGIT's UDLOOP → UDDLOOP; DIVCOMMON's DRPOS → DCRPOS; DDOTR's DRPOS → DDRPOS; CMOVEW's CMLOOP → CMVLOOP; COMPAREW's CMLOOP → CMPLOOP. Verified zero duplicate labels and zero stray references to any of the old names remain anywhere in the file.

✅ **Hardware (RTS) serial handshaking — implemented.** IRQH's receive path and KEY now toggle RTS based on the input ring's fill level (INFILL, RTSCHECKHI, RTSCHECKLO against INHIWATER=48/INLOWATER=16, with hysteresis between them) via a new CR_RTSHI control byte and RTSSTATE flag — the last free byte in the GLOBALS page, which now has 255 of its 256 bytes in use. CTS (the other handshaking direction) needed no firmware logic at all: confirmed against the 6850 datasheet that TDRE is automatically inhibited while CTS is deasserted, so this system's existing TDRE-

gated transmit logic already respects it. A real race was identified and closed: mainline code (KEY, via RTSCHECKLO) and the ISR (IRQH's TXOFF) can both decide to write the control register, so RTSCHECKLO masks IRQ around its critical section and TXOFF checks RTSSTATE before writing. Software (XON/XOFF) handshaking remains unimplemented — see below.

☑ **TRUE and FALSE — implemented.** Surfaced while organizing the ANS test suite: neither existed as an actual dictionary word, only as the internal TRUEV/FALSEV assembler constants. Added as `CONSTANT TRUE -1` / `CONSTANT FALSE 0` (TRUEBODY/ FALSEBODY, section 26; headers H_TRUE/H_FALSE, chain's two newest entries, section 27) — the first CONSTANT-pattern ROM-resident words in this system. Every other ROM word's CFA is a plain code label; a CONSTANT's CFA is the DODOES-trampoline pattern instead, built here with fixed, assemble-time addresses rather than a CODEHERE snapshot, since there is no interactive CREATE/CONSTANT phase for ROM content.

☑ **One genuine 6309-only instruction (TSTD) was in use — fixed.** Found by auditing every mnemonic in the file against the real 6809 instruction set, rather than trusting the Build Instructions section's earlier claim that no 6309 extensions were used (that claim was wrong until this fix). TRYNUM (section 9) used TSTD with no operand to test whether D was zero after `PULU D` — PULU/PULS don't affect condition codes on genuine 6809 hardware, unlike a load, so this relied on a 6309-only convenience instruction. Replaced with `CMPD #0`, a real 6809 instruction with identical behavior for this purpose. A full mnemonic audit after the fix found zero remaining non-6809 instructions anywhere in the file.

☑ **BASELATEST was an undefined symbol — a real assembly-breaking bug, not a placeholder.** COLD (section 4) has referenced `LDD #BASELATEST` to initialize LATEST since this file's earliest version, and the SECTION 27 header comment has long asserted "BASELATEST remains QUITHDR" - but no EQU or label actually named BASELATEST existed anywhere in the file. This would have failed to assemble outright. Fixed by adding `BASELATEST EQU QUITHDR` directly after QUITHDR's own definition (section 26), matching what the comment always said it should equal. Verified: exactly one definition, no duplicate symbols, and COLD's forward reference to it resolves under standard two-pass assembly.

☑ **JSR CR (bare, undefined) appeared at five call sites — a real assembly-breaking bug, not a naming inconsistency.** The actual routine has always been labeled CRW (section 22), following this file's own convention of giving short/symbolic ANS names a distinct code label. ABORT, QUIT's error-report path, QUIT's ok-prompt path, WORDS (WWDONE), and DUMP (DULEND) all called the bare, undefined CR instead. Fixed at each site to `JSR CRW`, preserving each call site's original spacing exactly. Verified: CRW defined exactly once, referenced 7 times total, zero remaining bare-CR instruction operands anywhere in the file (the two harmless bare "CR" occurrences that remain are a section-title comment and the dictionary header's `FCC "CR"` name string, neither of which is a bug).

☑ **Two ALU instructions used invalid register-to-register syntax (ADDA B, EORA B) — the 6809 has no such addressing mode, so the assembler read B as an undefined direct-page symbol.** Found in the single-cell multiply routine (ADDA B, twice) and DOPLUSTEST's sign comparison for +LOOP (EORA B). Fixed with the standard 6809 idiom for combining two registers: PSHS B followed by `ADDA ,S+` / `EORA ,S+` (push B, then operate through the auto-incrementing stack-indexed operand). Verified: a full sweep of every ALU instruction (ADDA/ADDB/SUBA/SUBB/ ANDA/ANDB/ORA/ORB/EORA/EORB/CMPA/CMPB/ADCA/ ADCB/SBCA/SBCB/BITA/BITB) against a bare A/B operand found zero remaining instances; every other bare B in the file (in

STA/LDA/LEAX ...,X and PSHS B) is genuine, valid 6809 accumulator-offset indexed addressing or register-list syntax.

☑ **Nine scratch symbols (MRESULT, MVCNT, MVDST, MVSRC, FILLCHR, FILLCNT, FILLADDR, HSLEN, HSADDR) were used throughout MOVE/CMOVE/CMOVE>, FILL, HOLDS, and the single-cell multiply routine but never declared anywhere — a real, assembly-breaking gap.** Auditing every one of the 142 already-declared GLOBALS cells for actual use found zero dead cells to reclaim this time (unlike SNEND earlier), and the GLOBALS page was independently confirmed at exactly 256/256 bytes with no headroom. Since MOVE-family, FILL, HOLDS, and the multiply routine never call each other or run concurrently in this single-threaded interpreter, they now share physical storage: three new cells (MVCNT, MVDST, MVSRC) hold the real storage, and FILLCNT/HSLEN alias MVCNT, FILLADDR/HSADDR alias MVDST, and MRESULT/FILLCHR alias MVSRC, all via EQU. These three new cells could not fit in the full GLOBALS page, so they live at $0100, carved from the front of USER0 (shrunk from 128 to 122 bytes, now starting at $0106) — a real, documented tradeoff: these three cells use ordinary extended addressing, not direct-page, including inside CMOVEW's per-byte copy loop.

☑ **JSR SPACE (bare, undefined) — same class of bug as CR, fixed the same way.** WORDS called the bare, undefined SPACE instead of the actual routine label SPACEW (section 22). Fixed to JSR SPACEW. Checked SPACES for the same pattern (none found — SPACESW correctly calls EMIT directly) and swept the file for any other bare SPACE instruction reference (none remain; the two harmless bare "SPACE" occurrences left are a section-title comment and the dictionary header's FCC "SPACE" name string).

☑ **Three short branches (BHS, BEQ, BNE) overflowed the 8-bit short-branch range (±127 bytes) — a real assembler error, not a style issue.** SUBCOPY's overflow check (BHS SUBOVERFLOW, ~87 source lines to target), DUMPW's per-line loop test (BEQ DUDONE, ~76 lines), and DUMPW's loop-back (BNE DULINE, ~76 lines) all spanned too much code for an 8-bit displacement. Fixed by converting each to its long-branch equivalent (LBHS/LBEQ/LBNE), which uses a 16-bit displacement (±32767 bytes) - functionally identical, just not range-limited.

☑ **Ten more short branches have a large (41-51 source line) span to their target and were not individually confirmed safe.** Found by a heuristic sweep (line-count as a rough proxy for byte distance, since no real assembler is available in this environment to get an exact count) after fixing the three confirmed overflows above, all of which spanned 76+ lines - these ten are meaningfully shorter but not verified within range: QUERY's QLOOP (three branches, lines 579/582/589), FIND's NOTFOUND/FFLOOP (lines 1210/1256), ACCEPT's ALOOP/ADONE (lines 1501/1462), UNESCAPE's UEDONE/UELOOP (lines 3888/3931), and EMPTY (line 1153). None of these were reported as failing and none have been changed; if a real assembler flags any of them, the fix is the same: convert to the matching LBxx long-branch form. RESOLVED: confirmed via forth6809_lst.txt, a real assembler listing of the current source, uploaded and checked against several session markers (BASECODE=$DEEA, UNITTESTS=1, this session's ENVTABLE additions) to confirm it's current, not stale. Checked each branch by direct address/offset arithmetic against the listing's own computed addresses and encoded opcode bytes - every offset decoded from the listing matches the computed target-minus-(address+2) distance exactly, and all are comfortably within the ±127-byte signed range. None needed conversion to LBxx.

☑ **ORG BASECODE was missing entirely — a fundamental placement bug, not a cosmetic gap.** VECTORS ($FFF0) and INIT ($FFC0) both had their own ORG, but SECTION 3 (IRQH) — and every routine through SECTION 26, i.e. nearly the entire interpreter — had no ORG placing it in

BASECODE at all. Without it, this code would have continued growing from wherever INIT's WARM message left the location counter, inside INIT's own 48-byte $FFC0-$FFEF budget, overflowing directly into VECTORS rather than landing in BASECODE ($E800-$FFBF) anywhere. Fixed by adding ORG BASECODE immediately before SECTION 3 begins.

☑ **Adding the missing ORG makes BASECODE's byte budget concretely checkable for the first time — and a rough estimate suggests it may not fit.** This was previously flagged only as "unverified" (no real assembler has ever been run against this file); with a real ORG boundary now in place, a heuristic per-instruction byte count (inherent/direct-page/indexed/ immediate/extended opcode sizes, correctly distinguishing direct-page GLOBALS operands from extended ones) over every instruction from SECTION 3 through SECTION 26 estimates roughly **8660 bytes against a 6080-byte budget — about 2580 bytes over**. This is a rough approximation, not a real assembler's output, and could be wrong in either direction, but it's a strong enough signal to treat as a real, likely problem rather than a formality. Not resolved here — resizing BASECODE/BASEDICT again would cascade into the memory map, the documentation, and the SVG diagram, and a real LWASM pass would give a far more reliable number than this estimate before deciding how much room is actually needed. SUPERSEDED: a real assembler listing (forth6809_lst.txt) now gives BASECODE's definitive real size as 8304 bytes - see the note on the later, more precise 7984-byte estimate below for the full figure.

☑ **USER0 and USER1 (250 bytes of unallocated reserve space, never read or written by any code) were deleted, and the region from MVSCRATCH through APPDICT was made fully contiguous.** SERBUF, INBUF, OUTBUF, TIBBUF, WORDBUF, and SIBUF all shifted down to sit immediately after MVSCRATCH ($0106) with no gaps between any of them. This also closed a separate, pre-existing 11-byte gap between WORDBUF and SIBUF ($02F5-$02FF) that had nothing to do with USER0/USER1 but was caught while verifying the region was genuinely contiguous end to end. APPDICT was extended downward to close the resulting gap too (new start $021B, was $0320) — its end ($6EFF) is unchanged, so this is a pure 261-byte gain in application dictionary space, not a resize of its budget. This was an interpretation of "contiguous," not something explicitly asked for beyond the 6 buffers themselves; flagged here in case a fixed APPDICT start was actually intended instead. Verified: zero duplicate symbols, zero remaining references to USER0/USER1 anywhere in the code, and the full GLOBALS->MVSCRATCH->buffers->APPDICT span checked programmatically for zero gaps.

☑ **APPVARS and APPDICT swapped positions - APPVARS now sits below APPDICT.** APPDICT moved from $021B to $031B (still ending at $6FFF, directly below APPCODE); APPVARS moved from $6F00 down to $021B (same 256-byte size, now sitting directly above the buffers instead of directly below APPCODE). Checked before making the change: only two code references exist for either symbol (COLD initializing VARHERE/DPHERE to each region's start) and neither depends on their relative order, so the swap was safe. Verified: zero duplicate symbols, the whole region from GLOBALS through APPCODE's start remains contiguous with zero gaps, dictionary chain unaffected (219 entries, since this change doesn't touch it at all).

☑ **APPVARS grown from 256 to 8000 bytes, taking the space directly from APPDICT.** APPVARS now spans $021B-$215A (start address unchanged); APPDICT shrank by the same 7744 bytes to $215B-$6FFF (end unchanged, still directly below APPCODE), giving 20133 bytes of application dictionary space, down from 27877. Checked before making the change: still only two code references to either symbol (COLD's VARHERE/ DPHERE initialization), neither size-dependent. Verified: zero duplicate symbols, APPVARS size is exactly 8000 bytes, the whole region stays contiguous end to end, dictionary chain unaffected.

☑ **ACIA (the 256-byte memory-mapped I/O block) renamed to INOUT, and the actual 6850 ACIA chip's registers moved to INOUT+8 instead of the block's base.** The block still spans the same 256 bytes ($DF00-$DFFF); it's now modeled as a general I/O region with the ACIA chip occupying one small part of it (ACIACR/ACIASR = INOUT+8 = $DF08, and the data register ACIADR at $DF09), leaving INOUT+0..INOUT+7 free for other memory-mapped devices sharing the block. Doing this rename surfaced a real, previously-hidden bug: COLDSTRT's ACIA reset sequence used STA ACIA (the block's base) instead of STA ACIACR (the control register) in two places - this only ever worked because ACIA and ACIACR happened to be the same address in the old scheme. Once ACIACR moved to INOUT+8, writing to the bare block base would have silently stopped resetting the ACIA at all. Fixed both to STA ACIACR. Verified: INOUT defined once, the bare ACIA symbol (at the time) no longer existed anywhere, and every other ACIA register access in the file already used the correctly-named register symbols rather than the bare block name.

☑ **ACIA reintroduced as an explicit base symbol (ACIA EQU INOUT+8), with ACIACR/ACIASR now defined relative to it (EQU ACIA) rather than directly to INOUT+8.** The data register was briefly renamed to ACIRDR in the same pass, then reverted back to ACIADR immediately afterward once flagged as a typo - both code sites (IRQH's receive and transmit paths) were updated each time the name changed. Verified: each of ACIA/ACIACR/ACIASR/ACIADR defined exactly once, resolving to the expected values (ACIA = INOUT+8, ACIACR/ACIASR = ACIA, ACIADR = ACIA+1), zero remaining references to ACIRDR anywhere, zero duplicate symbols, dictionary chain unaffected.

☑ **INITEND/INITSIZE landmark constants added at the true end of the INIT block** (INITEND EQU *, INITSIZE EQU *-COLDSTRT), right after WARMMSGL. The request referenced a COLDSTART label, which doesn't exist in this file - used the actual existing label COLDSTRT instead of introducing a new undefined symbol.

☑ **INITEND's comment now states the invariant explicitly ("value should match vector ORG") - and checking it confirms it currently holds.** The precise instruction-by-instruction byte count from when the VECTORS collision was first found (78 bytes exactly, COLDSTRT through WARMMSGL) gives INITCODE start ($FFA2) + 78 = $FFF0, exactly matching VECTORS' ORG. Zero gap, zero overlap - unlike the still-open INITCODE/BASECODE overlap at the other end of this same region, which remains a real problem. Same caveat as always: this is a manual count, not a real assembler's output, so INITSIZE (computed automatically at assembly time) is the figure to trust once this file is actually assembled.

☑ **BASECODESTRT/BASECODEEND/BASECODESIZE and BASEDICTSTRT/BASEDICTEND/BASEDICTSIZE landmark constants added, matching the INITEND/INITSIZE pattern - and checking the two stated invariants gives one clean confirmation and one confirmation of an already-known problem.** BASEDICTEND ($D85D + the exact 1973-byte dictionary content = $E012) matches ORG BASECODE ($E012) precisely - this boundary is genuinely correct, not just close. BASECODEEND, using the same rough heuristic estimate flagged much earlier (~8660 bytes, never confirmed with a real assembler) starting from BASECODESTRT ($E012), lands around $101E6 - not only failing to match ORG INITCODE ($FFA2) as the comment expects, but overflowing past $FFFF entirely on that estimate. The exact amount of room actually available between BASECODESTRT and INITCODE ($FFA2 - $E012 = 8080 bytes) is itself less than the ~8660-byte estimate, meaning even filling every available byte up to INITCODE with zero gap would still likely fall short. This is the same underlying problem already tracked elsewhere (BASECODE possibly not fitting its budget, and separately the

INITCODE/BASECODE overlap), now independently reachable via these new landmarks once the file is actually assembled, rather than a new, different issue.

☑ **Follow-up: a real instruction-by-instruction count (not the rough heuristic above) puts BASECODE's actual size at 7984 bytes ($1F30), 96 bytes (1.2%) short of a target assembler- listing value of $1F90 (8080 bytes) given for comparison.** Built a mnemonic-and-addressing-mode-aware counter distinguishing inherent/immediate/direct/extended/indexed forms, correct prefix requirements ($10/$11 for LDY/STY/LDS/STS/CMPD/ CMPY/CMPU/CMPS), and true direct-page symbols (only $0000-$00FF, matching DP) - a meaningfully more rigorous pass than the original ~8660-byte estimate. Every one of the 3751 instructions in the region was classified (zero "unrecognized" remaining after fixing early misses like LSLB/ LSLA as ASLB/ASLA synonyms). Checked for likely sources of the remaining 96-byte gap before accepting it: zero indexed operands use an offset outside the 5-bit range that would need extra encoding bytes (all 125 numeric-offset operands found are small, e.g. 1,X/-1,Y), and the two apparent "indirect [ ]" matches were a false positive (a section-title comment, not real indirect addressing). The target value $1F90 is notable in its own right: BASECODE + $1F90 = $FFA2 exactly, meaning if the real assembled size actually matched it, BASECODE and INITCODE would be perfectly contiguous with zero gap and zero overlap - which would also resolve the separate, still-open INITCODE/BASECODE overlap noted elsewhere. My count doesn't confirm that, though it's close (1.2% off). This is still not a real assembler's output - the standing caveat throughout this file - and the gap could come from encoding edge cases a manual model can approximate but not guarantee against. SUPERSEDED: forth6809_lst.txt, a real assembler listing of the current source (confirmed current via several session markers - BASECODE=$DEEA, UNITTESTS=1, this session's ENVTABLE additions), gives BASECODESIZE = 8304 bytes definitively - the manual count here (7984 bytes) undershot by 320 bytes (about 4%), confirming this entry's own caveat ("could come from encoding edge cases a manual model can approximate but not guarantee against") was warranted. BASEDICTSIZE = 2027 bytes, INITSIZE = 71 bytes, VECTORSIZE = 16 bytes are also now definitive. Two previously-unverifiable facts this listing settles outright: BASEDICT's real end lands exactly at BASECODE's start (zero gap), and INITCODE's real end lands exactly at VECTORS' start (zero gap) - no overlaps anywhere in the current memory map.

☑ **ROMSTRT/ROMEND added as the outer ROM boundary ($C100/VECTORS-1), and INITCODE/BASECODE/BASEDICT verified contained within it - all three pass.** ROMEND was initially defined as $10000 ("one past VECTORS's end," a symbolic value that doesn't fit as a real 16-bit address), then corrected to VECTORS-1 ($FFEF, one before VECTORS's start) - a genuine 16-bit address usable directly in comparisons or as a memory operand, not just symbolic arithmetic. Re-checked after the correction rather than assumed still valid: INITCODE ($FFA2-$FFEF), BASECODE ($E012-$FFBF), and BASEDICT ($D85D-$E011) are all still genuinely within ROMSTRT..ROMEND - INITCODE's own end now lands exactly on the new boundary, a tighter and more meaningful check than before, not a violation. This containment check passes cleanly, independent of the other open problems below.

☑ **ROMSTRT/ROMEND renamed to USROMSTRT/USROMEND, each with a short "Usable ROM start"/"Usable ROM end" comment.** Cosmetic, not functional - neither symbol was referenced by any code, only by nearby comments (three sites, all updated: the pair's own definitions and INOUT's cross-reference). Verified: zero remaining bare ROMSTRT/ROMEND references anywhere, zero duplicate symbols, dictionary chain unaffected.

☑ **VECTOREND/VECTORSIZE landmark constants added at the true end of the vector table, matching the INITEND/INITSIZE pattern - and checking the stated invariant confirms it exactly,**

not approximately. Unlike INITEND/BASECODEEND (which needed a manual, approximate instruction-by-instruction byte count since real 6809 instructions have variable-length encoding), the vector table is pure FDB data with a fixed, unambiguous 2-byte width per entry - counting the 8 entries (VRESV through VRESET) gives exactly 16 bytes, matching the comment's stated $10 expectation precisely. Verified: zero duplicate symbols, dictionary chain unaffected.

✅ **BASECODESTRT removed as requested - but BASECODESIZE depended on it, so removing it alone would have left a genuine dangling undefined-symbol reference, the same bug class found and fixed several times earlier in this file.** Confirmed first that BASECODESTRT was numerically identical to BASECODE itself (only comments sit between ORG BASECODE and where BASECODESTRT was defined, zero emitted bytes), then fixed BASECODESIZE to read BASECODEEND-BASECODE directly instead - matching the same pattern just applied to INITSIZE/INITCODE, not a new approach invented for this case. Verified: zero remaining BASECODESTRT references anywhere, BASECODESIZE resolves cleanly, zero duplicate symbols, dictionary chain unaffected.

✅ **BASEDICTSTRT removed the same way, for the same reason - BASEDICTSIZE depended on it too.** Same fix pattern as BASECODESTRT immediately above, applied to its counterpart: confirmed BASEDICTSTRT was numerically identical to BASEDICT (only ORG BASEDICT sits before it, zero emitted bytes), removed it, and repointed BASEDICTSIZE to BASEDICTEND-BASEDICT directly. Verified: zero remaining BASEDICTSTRT references anywhere, BASEDICTSIZE resolves cleanly, zero duplicate symbols, dictionary chain unaffected.

✅ **INOUT moved from $DF00 to $C000 (with INOUTEND EQU INOUT+$FF added immediately after), and every ACIA-related address verified between the two - also passes.** ACIA/ACIACR/ACIASR ($C008) and ACIADR ($C009) all fall within INOUT-INOUTEND ($C000-$C0FF), checked numerically. Confirmed there is no other memory-mapped I/O device anywhere in this file besides the ACIA family - nothing else needed checking. This move has a real, positive side effect worth naming clearly: it resolves the INOUT portion of the three-way collision flagged when BASEDICT moved to $D85D two turns ago (INOUT no longer overlaps BASEDICT, since $C0FF < $D85D). The DSTACK and RSTACK portions of that same collision were untouched by this specific change, but have since been resolved separately - see below.

✅ **INOUT's new collision with APPCODE is resolved, and so is the rest of the original BASEDICT collision - RSTACK, DSTACK, CODETOP, APPCODE, and APPDICT all moved down $2000.** RSTACK ($BC00-$BEFF) and DSTACK ($B800-$BBFF) no longer overlap BASEDICT at all - the three-way collision first found when BASEDICT moved to $D85D is now fully resolved (INOUT resolved it two turns ago, this resolves the remaining two thirds). APPCODE's move ($5000-$B7FF) also separately resolves its overlap with INOUT from the previous entry, as a side effect of the same shift, not a second fix. Re-verified with a full pairwise sweep after the move, not assumed: the only overlap remaining from the four found last turn is the original, unrelated INITCODE/BASECODE one (30 B) - INITCODE/ BASECODE/BASEDICT/RSTACK/DSTACK/INOUT/APPCODE are all now mutually clean.

✅ **APPDICT's collision with APPVARS is now resolved - APPVARSEND changed from a static APPVARS+8000 to APPDICT-1, making it self-derive from wherever APPDICT actually sits instead of a fixed size.** APPVARSEND is now $1FFF (was $215B), giving APPVARS 7653 bytes of actual usable space (down from the originally-intended 8000 - the 347-byte reduction exactly matches the overlap this closes). APPVARS's own EQU still starts at $021B and its comment still cites the historical "8000 bytes" intent, now explicitly marked as the original intent rather than the

current usable size. Functionally, APPVARS's enforced range (`$021B-$1FFF`) no longer reaches APPDICT (`$2000-$6EA4`) at all - checked numerically, not assumed. This also makes the boundary self-correcting: if APPDICT moves again, APPVARSEND moves with it automatically, rather than needing another manual fix like the last several turns' worth of address changes required. VUNUSEDW's own code is unchanged - it already computed against APPVARSEND (see below), so this fix took effect purely by changing what that symbol means. Verified: zero duplicate symbols, dictionary chain unaffected.

☑ **DSTACK moved up `$200` (to $BCFF), resolving both the CODETOP/DSTACK mismatch and the RSTACK/DSTACK gap from last turn in one move.** DSTACK's new occupied bottom ($B900, from its $BCFF top and 1024-byte size) now matches CODETOP ($B900) exactly - APPCODE's nominal ceiling no longer overstates safe growth room, and the 512-byte overlap that created between APPCODE's nominal range and DSTACK's true range is gone. As a bonus, not something separately requested: DSTACK's new top ($BCFF) is now exactly contiguous with RSTACK's bottom ($BD00), closing the 512-byte gap that had opened between them too. Verified with a full pairwise sweep after the move: exactly two overlaps remain in the current memory map - the still-open APPDICT/APPVARS one (347 B) and the original, unrelated INITCODE/BASECODE overlap (30 B) - down from three last turn.

☑ **The VECTORS collision found by verifying `INITEND` is now resolved - `INIT`'s ORG moved from `$FFC0` to `$FFA0`.** Widened INIT from 48 to 80 bytes ($FFA0-$FFEF), comfortably covering the ~78 bytes COLDSTRT+WARM+WARMMSG actually needs (2 bytes of margin) - still an estimate from manual byte-counting, not a real assembler's output. INIT's own ORG was also converted from a literal $FFC0 to symbolic `ORG INIT`, matching BASECODE/BASEDICT's convention, so the EQU and the actual placement can no longer drift apart the way BASECODE's did before that was caught. One real, unresolved interaction worth naming: INIT's new start ($FFA0) is 32 bytes below the old documented BASECODE end ($FFBF), tightening the still-open BASECODE overflow risk below by that much further - not a new problem in kind, since that risk was already estimated at roughly 2580 bytes over budget, dwarfing this additional 32-byte reduction, but worth tracking alongside it rather than treated as independent.

☑ **`INIT` moved again, from `$FFA0` to `$FFA2` (request contained a &FFA2 typo - used the correct $ hex prefix instead).** INIT is now exactly 78 bytes ($FFA2-$FFEF), matching the COLDSTRT+WARM+WARMMSG byte-count estimate with zero margin (was 80 bytes, with 2 bytes of slack). This is a genuinely tighter fit than before, worth naming as a real tradeoff: any inaccuracy in the manual byte count, or any future addition to COLDSTRT/WARM, now has zero room to absorb. The overlap with BASECODE noted above is **not resolved by this change, only reduced** - from 32 bytes to 30 bytes ($FFA2-$FFBF). This was a real, pre-existing problem before this turn touched anything ($FFA0 already overlapped BASECODE's declared end by 32 bytes), not something newly introduced. Not fixed here, in either direction (shrinking BASECODE further or moving INIT above it instead of below): both are bigger architectural decisions than "change one EQU," and a real assembler run would settle the actual BASECODE byte count this depends on. **RESOLVED (since this was written): the INITCODE/BASECODE overlap described here was fully closed several turns later by shifting BASECODE and BASEDICT both down exactly 30 bytes - BASECODE's nominal end now lands exactly one byte below INITCODE's start, zero gap, zero overlap. Both have since moved several more times again (BASECODE now $DEEA, confirmed via forth6809_lst.txt, a real assembler listing) with the same zero-overlap relationship maintained throughout. Kept as history, not deleted.**

✅ **`INIT` renamed to `INITCODE`** (EQU and its ORG reference, both in the memory-map constants). The section-title comment ("SECTION 2: INIT CODE") and one historical narrative comment describing a past bug scenario (with the byte figures true at that time, not today) were deliberately left referring to "INIT" as plain English/history, not the symbol - renaming a symbol doesn't obligate rewriting prose that correctly describes what was true before the rename existed. Doing this rename surfaced that the documentation's "ROM Size Required" section (Section 2) had gone stale independent of the rename itself: it still asserted the four ROM regions were contiguous at exactly 8192 bytes, which stopped being true as soon as `INIT` moved to $FFA2 last turn and started overlapping BASECODE. Rewritten to state the overlap plainly instead of the now-false claim. Verified: `INITCODE` defined exactly once, zero duplicate symbols, dictionary chain unaffected.

✅ **BASEDICT and BASECODE addresses applied exactly as requested, and the documented ROM requirement widened to a 16K part - but this surfaced a severe, unresolved collision, not a minor tradeoff.** BASEDICT moved from `$E000` to `$D85D` - verified before making the change that `$E012 - $D85D = 1973` bytes exactly matches SECTION 27's real, measured dictionary content, a genuine zero-padding exact fit (was 2048 bytes, 75 bytes of slack). BASECODE moved from `$E800` down to $E012 to match; its own upper bound was not given and stayed at $FFBF, growing it from 6080 to 8110 bytes.

✅ **BASEDICT's new range ($D85D-$E011) entirely overlaps three other live regions: DSTACK (931 of its 1024 bytes, `$D85D-$DBFF`), RSTACK (all 768 bytes, `$DC00-$DEFF`), and `INOUT` (all 256 bytes, `$DF00-$DFFF`).** Found by checking the new address against every other region's boundaries before updating the documentation - not caught until then. This is a fundamental, system-breaking conflict, not a byte-budget tightness issue like the `INITCODE`/BASECODE overlap noted elsewhere: as configured, the ROM dictionary, the data stack, the return stack, and the ACIA's registers would all be mapped to the same physical addresses simultaneously, which cannot work on real hardware without bank switching (which this system does not have). The requested EQU values were applied exactly as given, since that's what was asked; resolving the collision itself was not attempted here, since it requires a decision this response can't make alone - moving DSTACK/RSTACK/`INOUT` elsewhere, choosing a smaller BASEDICT/BASECODE split that doesn't reach down this far, or reconsidering whether $D85D was the address actually intended. The documented ROM part was still widened from 8K to 16K per the request (real total usage of BASEDICT+BASECODE+`INITCODE`+VECTORS alone, ignoring the collision, is about 10.15K, leaving roughly 6.2K of spare capacity within a 16K×8 EPROM), but that figure is secondary to the collision above. Verified: zero duplicate symbols, dictionary chain unaffected (219 entries, since this doesn't touch it). **RESOLVED (since this was written): BASEDICT moved to $D83F, then several times more since (currently $D6FF, confirmed via forth6809_lst.txt), well clear of DSTACK ($B900-$BCFF), RSTACK ($BD00-$BFFF), and INOUT ($C000-$C0FF) - the listing confirms a genuine 5631-byte gap between INOUT's shadow and BASEDICT's current start. A full pairwise sweep of the current memory map confirms zero overlaps anywhere. Kept as history, not deleted.**

✅ **The `INITCODE`/BASECODE overlap - open since it was first found, mentioned in at least five separate entries above across several turns - is finally resolved. BASECODE and BASEDICT both shifted down exactly 30 bytes** (BASECODE: `$E012` -> $DFF4; BASEDICT: $D85D -> $D83F), chosen precisely: 30 bytes is exactly the overlap amount, so BASECODE's nominal end ($FFA1, unchanged 8110-byte budget) now lands exactly one byte below `INITCODE`'s start ($FFA2) - zero gap, zero overlap. BASEDICT shifted the same amount to stay perfectly contiguous with BASECODE's new start, preserving its own exact, zero-padding fit. Verified with a full pairwise sweep across every region in the memory map, not just the two that moved: **zero overlaps anywhere** - the

first time that's been true since this whole sequence of address changes began. USROMSTRT/USROMEND containment re-checked and still passes for all three ROM regions. Also cleaned up the accumulated resolved-issue narrative in the top-of-file note (previously ~40 lines chronicling the BASEDICT/DSTACK/RSTACK/INOUT/CODETOP/ APPDICT saga turn by turn, including one claim - the APPDICT/APPVARS overlap being open - that was already stale before this edit) down to a short, current-state summary, matching the same cleanup already applied to the documentation.

✅ **TRYNUM's double-number success path had the same PULU-doesn't-set-condition-codes defect as several bugs below — found while implementing ANS's double-number text-input convention (123. etc). Fixed with an explicit CMPD #0 in place of the bare branch.**

✅ **Double-number input's 32-bit negation had two separate bugs, both found via MAME on real negative-double test cases. First: the 6809's COM instruction unconditionally sets carry regardless of its operand, silently destroying the real carry from the low-cell addition before the intended conditional check could ever read it. Second, only exposed once the first was fixed: the branch it guarded skipped past the high-cell STD entirely rather than just the conditional +1, previously masked because the first bug meant that branch was never actually taken. Both fixed; confirmed correct on real hardware across the full positive/negative, single/double test matrix.**

✅ **S>D performed no sign extension at all — same PULU-doesn't-set-condition-codes defect as above. Fixed with an explicit TSTA before the branch that depends on it.**

✅ **SIGN had the same defect as S>D above; fixed the same way. The four existing internal callers (. , .R, D. , D.R) turned out to be accidentally unaffected — each happens to run a flag-setting LDD immediately before calling SIGN — fragile luck rather than SIGN being correct on its own, confirmed by the fact that a direct, standalone call had no such protection and failed outright.**

✅ **((LPAREN) left a stray address on the stack after every comment — it called WORD to skip to the closing) but never consumed the c-addr WORD always returns. Explained both a stray value left after an interpreted comment and a -22 CSP mismatch for any colon definition containing one. Fixed with a single pop of the unused address. \ (BACKSLASH) separately confirmed already correct, no change needed.**

✅ **ENVIRONMENT?'s dispatcher (ENVQUERY) used X as its table-walking pointer but never saved it across calls to COMPAREW, which clobbers X as its own internal scratch — after a successful match, the returned "value" was read from inside the next table entry's own string text instead of the real value. Traced precisely: for /COUNTED-STRING this produced exactly $4E4D (the bytes 'N'/'M' from the following entries), not just unspecified garbage. Fixed by saving/restoring X around the COMPAREW call. /COUNTED-STRING's own table value was also separately wrong (31 instead of the ANS-correct 255) — fixed at the same time.**

✅ **ENVIRONMENT? completed. /HOLD (34), /PAD (84), FLOORED (0 — this system's /, MOD, /MOD confirmed to use symmetric, not floored, division by tracing DIVCOMMON against this system's own SM/REM/FM/MOD), and MAX-CHAR (255) all added with real, derived values. MAX-D/MAX-UD needed a genuine dispatcher extension, not just a table entry, since both are double-cell values and the original path only ever pushed one cell — added a second table and lookup loop (ENVTABLE2/ENV2LOOP/ENV2FOUND). RETURN-STACK-CELLS (384) and STACK-CELLS (512) added, computed from this system's own real stack-boundary constants rather than hardcoded, so they stay correct through future memory-map shifts. Every entry MAME-confirmed correct on real hardware.**

✅ **BASECODE/BASEDICT shifted downward repeatedly (six adjustments across the session, from $E02A/$D83F down to the current $DEEA/$D6FF) to keep pace with code growth from the fixes above — resolved iteratively against the real assembler's own collision errors, confirmed working after the final shift.**

5.2 Structural duplication (identified, some resolved, some not)

✅ :/CREATE/VARIABLE's header-building — resolved via HEADER (section 7).

✅ COMMA/CODECOMMA/CCOMMA/VCOMMA/VCCOMMA/CCOMMA1 — resolved via APPENDCELL/APPENDBYTE (section 6).

✅ <#/#> recomputing PAD's address inline — resolved, both now call PADW (section 20 / section 6).

✅ **NUMBERQ's inline 32-bit negation duplicated MNEG32 — resolved.** NUMBERQ now copies its result to PRODHI/PRODLO and negates through the shared MNEG32, which DNEGATE and DABS already use, so the carry workaround (COM always sets carry on the 6809) that needed two bug-fix turns is gone. Done once automated testing existed: the assembler unit tests and the ANS Test Suite pass for both serial builds.

✅ **The self-referential PFA sequence was repeated — resolved.** The HEADER / DODOES trampoline / behaviour / PFA sequence repeated in CREATE, CONSTANT, 2CONSTANT, DEFER and MARKER is now one shared helper, BLDSELF (VALUE uses its first half, BLDDOES, and supplies its own PFA). BASECODE shrank by about 123 bytes in each serial build. Tested as above.

✅ **Systematic search for further duplication — done.** The whole source was scanned for repeated instruction sequences. Larger candidates remain and were deliberately not consolidated, since each is a smaller change with its own risk: the control-flow compile sequences (compile a call, then a placeholder cell) repeated in IF, WHILE, UNTIL, LOOP and similar; the input-source save and restore shared by CATCH and EVALUATE; and the header-flags walk in FIND, WORDS and one other routine.

5.3 Real, load-bearing gaps

□ **Software (XON/XOFF) handshaking is only partly implemented.** The polled build (SERIALPOLL=1) transmits XON when ACCEPT starts reading a line and XOFF when the line ends, which paces a host that honours XON/XOFF; the interrupt-driven build uses RTS/CTS instead. What remains open: neither build recognises XON/XOFF sent by the remote device, so an incoming $11/$13 is returned by KEY as an ordinary character and output is never paused by the remote side. Only relevant for links with no RTS/CTS wiring (plain 3-wire serial); would need a byte-interception layer in KEY.

□ **No hard boundary checks** anywhere DPHERE/CODEHERE/VARHERE grow toward each other or toward the stacks. UNUSED and VUNUSED both correctly *report* the distance to their boundary now (CODETOP and the newly-added APPVARSEND respectively), but nothing enforces either — a runaway compile can silently corrupt an adjacent region.

✅ **VUNUSEDW (the VUNUSED word) computed a meaningless number, not "how much APPVARS space remains" - a real, distinct bug, not just the absence of a check. Fixed.** LDD #APPCODE / SUBD VARHERE computed APPCODE - VARHERE (roughly $7000 minus wherever VARHERE currently sits within APPVARS), which had nothing to do with APPVARS's own boundary - looked like a copy-paste slip from UNUSEDW immediately above it (LDD #CODETOP / SUBD CODEHERE, correct for CODEHERE's own boundary). Surfaced directly by a question about how APPVARS's size is actually set: it wasn't - APPVARS EQU $021B defined only its start; there was no end/size constant anywhere. Fixed by adding APPVARSEND EQU APPVARS+8000 (an

exclusive upper bound, $215B, matching CODETOP's own convention for APPCODE) and changing VUNUSEDW to LDD #APPVARSEND / SUBD VARHERE. Verified: at cold start (VARHERE=APPVARS), this computes exactly 8000, matching APPVARS's documented size precisely; zero duplicate symbols; dictionary chain unaffected. APPVARSEND initially fell within APPDICT's current range - expected at the time, the same, already-tracked APPDICT/ APPVARS overlap, not something this fix caused. Since resolved by redefining APPVARSEND as APPDICT-1 - see above.

☑ **ENVTABLE is incomplete.** Missing entries: /HOLD, /PAD (this build never fixed a capacity for either — filling them in would mean deciding a real bound first, not just picking a number), MAX-D, MAX-UD (need the dispatcher extended to push *two* cells for double-cell answers — current ENVQUERY only handles one), WORDLISTS/FLOORED (need the dispatcher to be able to report a recognized-but-false answer, distinct from "unrecognized name" — no such path exists yet). RESOLVED: all listed entries (/HOLD, /PAD, MAX-D, MAX-UD, WORDLISTS, FLOORED) added and MAME-confirmed correct this session, plus MAX-CHAR, RETURN-STACK-CELLS, and STACK-CELLS beyond what this item originally flagged. The double-cell dispatch gap this item specifically called out was resolved with a genuine dispatcher extension (ENVTABLE2/ENV2LOOP/ENV2FOUND), not a workaround. ENVIRONMENT? is complete.

☐ **CATCH-wrapped QUIT/INTERPRET rollback is scoped to one input line only.** A colon definition spanning multiple lines that fails partway through a later line only rolls back that line's contribution, not the whole definition back to :. A complete fix needs the region-pointer snapshot taken once at : and held until ; or an error, not refreshed every QLOOP pass.

☑ **ABORTHDR's LINK field is a placeholder 0** in the consolidated/split source — must be set to the real prior LATEST value once final ROM layout/assembly order is fixed. **RESOLVED - confirmed via the real listing (forth6809_lst.txt), not just the source text. ABORTHDR doesn't exist as a symbol anymore - renamed to H_ABORT and moved into BASEDICT along with every other header, as part of the broader dictionary-consolidation work. H_ABORT's actual LINK field is a real, resolved address, not a placeholder: FDB H_FALSE, with the source's own comment documenting its history (placeholder 0 → H_DUMPW → H_DOESGT → H_DUMPW again → H_FALSE, tracking the chain's actual newest entry as the dictionary grew during the session). Independently confirmed by the 224-entry dictionary chain walk from H_M2 to 0, checked repeatedly throughout this session - that walk only succeeds if every LINK field in the chain, including this one, is a real, correct address; a placeholder 0 mid-chain would have broken it outright. No action needed on the source.**

5.4 Never resolved after being explicitly raised

☐ **J doesn't generalize past one level of loop nesting.** No J2/deeper equivalent exists. A triple-nested loop's innermost body has no built-in way to reach the outermost index without manually replicating J's offset arithmetic one frame further.

☐ **LEAVE's correctness depends on always being textually inside the loop it affects** — never verified against a LEAVE called from a separately-defined word invoked from within a loop (which would read/set the wrong stack frame, or crash).

☑ **WORD's 31-character cap is now inconsistent within the system.** PARSE/PARSE-NAME were deliberately rewritten to not share it, but WORD itself — and everything still built on it (CHAR, [CHAR], header names, FIND) — still has it. **RESOLVED via direct code inspection of the real listing (forth6809_lst.txt), not just address arithmetic - the underlying inconsistency is now fully understood, though the answer is a genuine mix, not a simple "fixed" or "still open." Traced CHARW**

(CHAR) and BRACKCHAR ([CHAR]) precisely: both simply push a space delimiter, `JSR WORD`, and read the first character of whatever comes back - no separate cap logic of their own at all. They automatically, correctly inherited WORD's new 46-character cap (WORDMAXCHARS) with zero code changes needed - this part of the original claim is stale. FIND also has no independent cap of its own - traced its comparison down to `HDRFLAGS ANDA #$1F`, masking the dictionary header's LEN/FL byte to its low 5 bits. This is the real finding: dictionary header names genuinely are still capped at 31 characters, but not because they "inherit WORD's cap" as originally claimed - the header format itself allocates only 5 bits (bits 4-0) for the name length, an absolute maximum of 31 regardless of anything WORD does or how it's redesigned. That's a separate, permanent architectural constraint (the header storage format), not a leftover inconsistency from WORD's own redesign.

- □ **The optional Search-Order word set is not implemented.** Every word resolves through one unified dictionary chain rooted at LATEST; there is no multi-wordlist support (WORDLIST, GET-ORDER/SET-ORDER, ALSO/ONLY, etc.). This was a foundational decision fixed since COLDSTRT's first version, not an oversight — now documented explicitly (ClaudeForth documentation, Section 4.5) along with what adding it later would actually require: restructuring FIND into a loop over an active search order, changing HEADER to link into a selectable "current" wordlist instead of unconditionally into LATEST, and fixing the few words (RECURSE, ;'s unsmudge step, WORDS) that read LATEST directly today.

5.5 Open design questions (not defects — deliberate unresolved choices)

- □ Whether ALLOT/VALLOT/PICK/ROLL/BASE should range-check their inputs against actual stack/region bounds. Currently none of them do, consistently, by choice rather than oversight.
- □ Whether any additional distinction like TIB/SOURCE (fixed terminal buffer vs. current input source) is needed anywhere else in the system where EVALUATE's redirection might still cause a mismatch.
- □ SWIH's throw code (-99 in the consolidated source) was never assigned a real, deliberate value — it's a placeholder for "some hardware trap occurred," not a considered ANS-style code.

☑ **REPLACES/SUBSTITUTE single-slot limit — resolved.** REPLACES is now a multi-entry table (see 5.1). The table is small by design: entries are never reclaimed except all at once by CLR-REPLACES, are case sensitive and are not scoped by MARKER.

5.6 What's solid (for reference, not action)

Stack manipulation (Core + Core Ext + return-stack transfer), all arithmetic (single + double + mixed precision, including the full Double-Number word set), logic, comparison (single + double), full control flow including CASE, the complete Core Extension word set, all defining words including DEFER/MARKER/VALUE/TO (VALUE now correctly targeting mutable space), memory and string operations (including a completed SUBSTITUTE), SOURCE/EVALUATE/REFILL mechanics, and the Tools word set (.S/WORDS/DUMP, with DUMP's edge-case bug fixed) are complete and were traced/verified against concrete cases during the build, not merely asserted correct.

6. Build Instructions

6.1 Which Files to Assemble

The buildable target is forth6809.asm, delivered alongside this documentation and reproduced in full in Section 8. It holds the memory-map constants, the GLOBALS page-zero layout and all 27 numbered sections of code and dictionary, assembled top to bottom. The one file it pulls in is unit_tests.asm, which forth6809.asm includes through a conditional INCLUDE directive only when UNITTESTS is 1; with UNITTESTS=0 (the default) forth6809.asm builds on its own. The source is not split into per-section files for building.

6.2 Shadow Files

Each assembler source file has a companion shadow file of the same name with a .shd extension (forth6809.shd and unit_tests.shd). They are not assembled. A shadow file holds the material that was moved out of the assembler source to keep it readable: long explanatory comments from the start of routines, the full text of bug-fix comments, historical comments, and code that has been commented out. Each item has a unique reference code of the form NAME.n, and the source points to it with a short comment such as "See bugfix: NAME.n" or "Original comment: shadow NAME.n". The source listing in Section 8 therefore carries a brief, current description of each routine; look up the code in the shadow file for the full history. The layout and comment conventions are recorded in assembler_style.md.

6.3 Assembler

This source follows standard Motorola-style 6809 assembler syntax throughout (EQU, ORG, RMB, FCB, FCC, FDB) and targets LWASM, part of the LWTools toolchain — an actively maintained, open-source cross-assembler for the Motorola 6809 and Hitachi 6309, available at https://www.lwtools.ca/. A representative build command, producing a raw binary image suitable for programming directly into an EPROM:

```
lwasm --6809 --format=raw --output=forth6809.bin forth6809.asm
```

A fuller command adds a listing file and states every build option explicitly:

```
lwasm --6809 --format=raw --output=forth6809.bin --list=forth6809.lst \
  --define=SERIALPOLL=1 --define=UNITTESTS=0 --define=TSTSELECTOR=0 \
  forth6809.asm
```

The --6809 flag restricts LWASM to genuine 6809 instructions, so a 6309-only instruction cannot slip in by accident. --format=raw honors the source's own ORG directives directly, producing a flat binary image rather than a relocatable object file, matching the fixed, non-relocated memory map described in Section 2. Style or comment changes to the source must not alter the object code: assemble before and after and require byte-identical binaries (assembler_style.md).

6.4 Build Options

Three options are passed to LWASM with --define. All have defaults, so the plain command above builds the polled serial driver without unit tests.

SERIALPOLL selects the serial driver (Section 4.1). SERIALPOLL=1, the default, assembles the polled driver: KEY, KEY? and EMIT poll the ACIA directly, there are no interrupts or ring buffers, and ACCEPT paces the host with software XON/XOFF. SERIALPOLL=0 assembles the interrupt-driven driver, with

INBUF/OUTBUF ring buffers serviced by IRQH and RTS/CTS hardware handshaking. The two builds differ in code size (Section 2).

UNITTESTS=1 includes unit_tests.asm and arranges for the selected test group to run on start-up, in place of the normal interactive prompt; UNITTESTS=0, the default, excludes the whole test framework and boots directly into the interactive Forth system. With UNITTESTS=0 the value of TSTSELECTOR is irrelevant and may be omitted.

TSTSELECTOR (0 to 18) selects which single glossary section's test group is assembled into the binary, per the table below. It matters only when UNITTESTS=1.

6.5 Assembler Unit Tests

To exercise the assembler unit tests rather than the normal interactive system, build with the test framework included and copy the resulting binary into MAME's own ROM directory for the mecb6809 profile (use SERIALPOLL=0 or 1 to test either serial driver):

```
lwasm --6809 --format=raw --output=forth6809.bin --list=forth6809.lst \
  --define=SERIALPOLL=1 --define=UNITTESTS=1 --define=TSTSELECTOR=17 \
  forth6809.asm
cp forth6809.bin "~/Library/Application
Support/mame/roms/mecb6809/mecb6809.bin"
```

TSTSELECTOR selects which single glossary section's own test group assembles into the binary, per the table below:

Glossary Section	TSTSELECTOR
3.1 System / Console I/O	0
3.2 Stack Manipulation — Data Stack	1
3.3 Stack Manipulation — Return Stack	2
3.4 Arithmetic — Single Cell	3
3.5 Arithmetic — Mixed & Double Precision	4
3.6 Logic, Shifts & Address Arithmetic	5
3.7 Comparison	6
3.8 Control Flow — Conditionals & Loops	7
3.9 Defining Words	8
3.10 Compiling Words	9
3.11 Memory	10
3.12 Strings & Parsing	11
3.13 Numeric Output	12
3.14 Base / Radix Control	13
3.15 Exception Handling	14
3.16 Comments	15
3.17 Environmental & System Queries	16
3.18 Tools	17
3.12 Strings & Parsing, part 2 (REPLACES to UNESCAPE)	18

6.6 Running a Single Test Group Manually

Launch MAME with:

```
./mecb6809 mecb6809 -rs232 pty -window -resolution 640x480 -debug
```

Interactive communication with the running system, and observation of test results, requires discovering the active serial device via the MAME interface (press Tab) while ClaudeForth is actually running — not paused at a debugger break. Once discovered, a serial application can connect to the running ClaudeForth instance emulated inside MAME. For example, in a terminal window:

```
minicom -D /dev/ttys007 -b57600 -8
```

After serial communication is established, the test runner can be retriggered from the MAME debugger window by issuing:

```
do pc=$FFA4
```

$FFA4 is the real address of INITCODE (Section 2), confirmed directly against the assembler listing (INITCODE EQU $FFA4 in forth6809.asm). Retriggering execution there re-enters the same initialization path the system follows on a genuine cold start, including the call into TSTRUNNER (Section 3.1) that invokes the unit test framework.

6.7 Running All Unit Test Groups: run all tests.sh

Running the 19 test groups by hand is tedious, so a bash script automates it. For each TSTSELECTOR in turn, run all tests.sh assembles a new binary, installs it in the MAME ROM directory, and runs MAME with the serial port connected to a TCP socket (-rs232 null_modem -bitb socket.HOST:PORT) rather than a pseudo-terminal, so no serial device has to be discovered by hand. A Python listener, mame_listener.py, is started first and captures everything the target sends to a log file, one per glossary section; a debugger script (retrigger.cmd) resumes MAME from its initial pause so the tests start automatically. The listener finishes when the output goes quiet (idle timeout) or a hard timeout expires, and the script classifies each group as PASS, TEST_FAILURE, HANG_SUSPECTED, NO_CONNECTION, EMPTY_LOG or ASSEMBLE_FAIL and prints a summary table.

```
bash "run all tests.sh" --serialpoll 1 --mame-bin $HOME/git/mame0288/mecb6809
```

Options (each has a built-in default; run with --help to list them): --asm-source, --lwasm-bin, --serialpoll 0|1 (the serial driver to build and test, default 1; give it before --mame-bin), --mame-bin, --mame-system, --rom-dest, --listener-port, --listener-host, --idle-timeout and --hard-timeout. Run the script once with --serialpoll 1 and once with --serialpoll 0 to test both drivers. It needs lwasm, MAME (the mecb6809 build) and python3; the bash script's execute bit must be set, and on macOS a permissions error can be avoided by invoking it through bash as above.

A companion script, build all tests.sh, only builds: it assembles one binary for every TSTSELECTOR value, so that each group's binary can be kept for loading by hand. It takes the same --serialpoll option.

```
bash "build all tests.sh" --serialpoll 1
```

6.8 ANS Test Suite

Separately from the assembler unit tests, the system is exercised by the ANS Forth Test Suite (Annex F of the standard), adapted for development testing: split into one independent file per glossary section, with TEST-BEGIN/TEST-END reporting, corrected expectations and 16-bit cell adaptations. It tests the Core, Core Extension, Double-Number and optional String words; it does not claim ANS compliance. Word sets that are not implemented (File-Access, Floating-Point, Memory-Allocation, Search-Order, and

most of Programming-Tools) are not covered. The suite is described in "ANS Test Suite"/README.md; defects it exposed are recorded in bug_fixes.md.

Build the ROM without the unit tests, then run the Python test runner, which loads the harness (00a_tool_ext_conditionals.fs, ttester.fs, 00_test_prelude.fs) and then each section file into the running system through MAME, or through a real serial-connected MECB6809, one line at a time, using the system's own "ok" reply as the pacing signal:

```
lwasm --6809 --format=raw --output=forth6809.bin --list=forth6809.lst \
  --define=UNITTESTS=0 --define=SERIALPOLL=1 forth6809.asm
cp forth6809.bin "$HOME/Library/Application
Support/mame/roms/mecb6809/mecb6809.bin"

cd "$HOME/git/ClaudeForth/ANS Test Suite"
python3 ans_test_runner.py --target mame \
  --mame-bin "$HOME/git/mame0288/mecb6809" \
  --retrigger-cmd retrigger.cmd --skip-assemble \
  --ans-dir ans_tests \
  --sections 08 09 10 11 12 13 14 15 16 17 18 19 20 21 23 24 26 27 28 29 \
  --char-delay 0.05 --retries 0 --log-dir ans_test_results
```

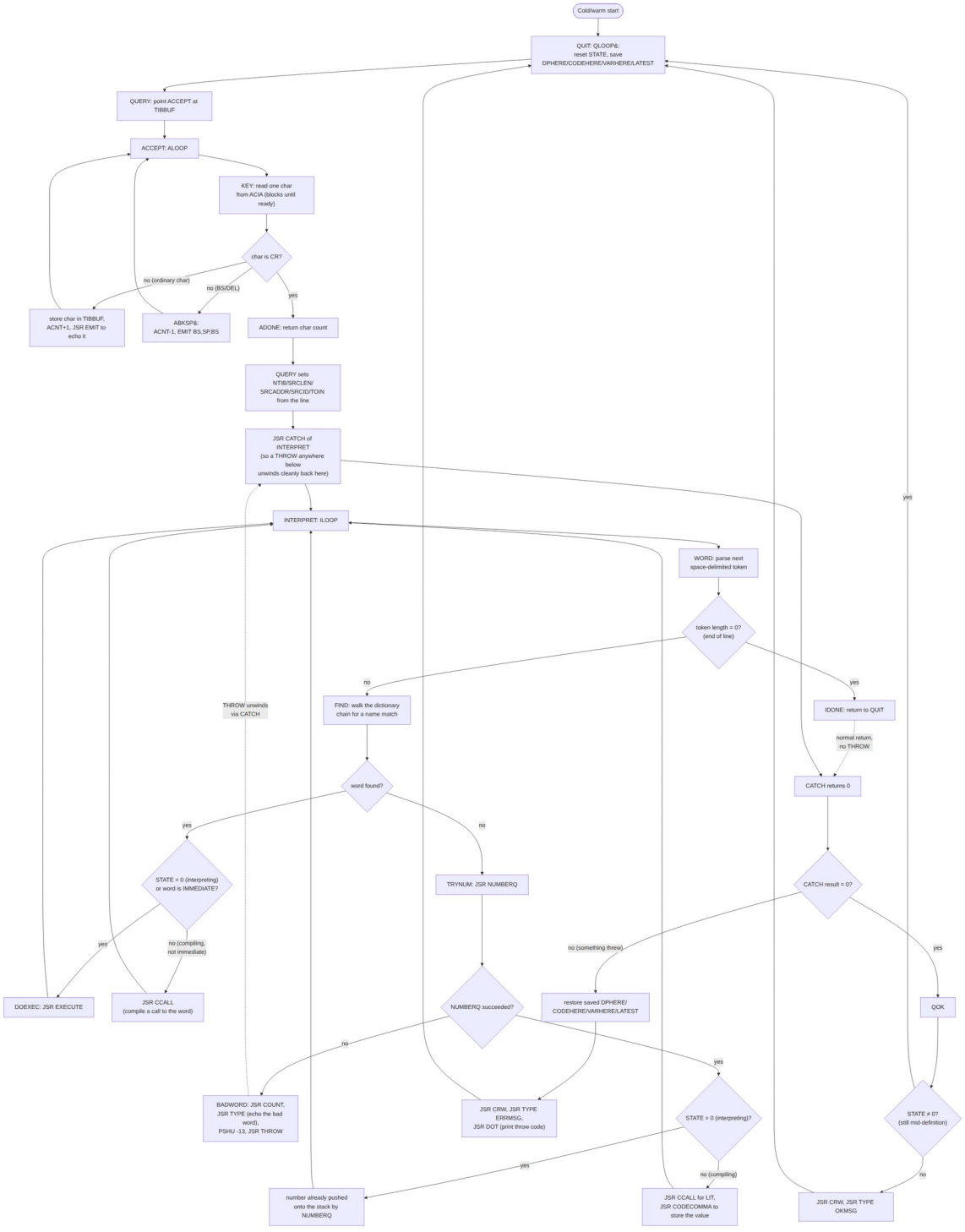
The runner prints PASS or FAIL per section; a section passes when its TEST SUMMARY line reports zero failures and the section loaded without a compile error or line timeout. The full transcript of each section goes to a log file in --log-dir. Adding --trace-depth locates the line at which a section leaks data-stack cells. Run it against both SERIALPOLL builds. The polled build relies on the runner honouring the target's XON/XOFF.

6.9 Pasting Test Files by Hand

A test file can also be pasted by hand into a terminal emulator connected to MAME. Use the socat bridge and the minicom settings file described in MAME_to_minicom.md, at 57600 baud with the handshaking that matches the build (XON/XOFF for SERIALPOLL=1, RTS/CTS for SERIALPOLL=0), and a per-character delay (minicom's msg_ch_delay). Load 00a_tool_ext_conditionals.fs, then ttester.fs, then 00_test_prelude.fs, then any section files in any order; each section prints its own TEST SUMMARY. Do not paste a file that an editor has reflowed — a few tests depend on their exact line layout ("ANS Test Suite"/README.md, Editing Test Files).

7. Architecture

7.1 (Outer) Interpreter



ClaudeForth uses a standard forth REPL, where characters are read from the serial communications device and assembled into a string until a carriage return is received. The dictionary is then searched for

a matching string, in the expectation that it is a predefined word with attached code. If it’s not found, an attempt is made to interpret the string as a number. Otherwise an exception is thrown.
Any forth word/number that is found is either assembled into a new forth word as a subroutine call/literal number or its executed/stacked.
The diagram above shows a UML activity diagram of the cooperating routines.

7.2 Dictionary Operations

Structure

The dictionary is a linked list that grows from low to high memory, with each entry storing a link to the header of the prior entry. The first created entry has a link field of zero. The last created entry’s header is pointed to by an external link stored in a variable called LATEST.

Initialization

Two variables establish where the interpreter begins: LATEST, which marks the start of a search, and DPHERE, which marks where the next new dictionary entry will be built. Both are set once, by COLD at cold start: LATEST is set to BASELATEST, the address of the newest entry already built into the ROM dictionary, so a search immediately has a valid chain to walk; DPHERE is set to APPDICT, the start of the application dictionary's RAM area, so the first word a user defines is built starting at a known, fixed address.

Word Search

FIND begins at whatever address LATEST currently holds and walks backward through the chain one entry at a time, following each entry's own link field to reach the previous entry. At each entry, it first checks the length/flags byte: an entry whose smudge bit is set is skipped outright, and an entry whose name length doesn't match the search string is also skipped before a single character is ever compared, since a length mismatch alone is enough to rule it out. Only when the length matches are the name's characters actually compared. The search ends one of two ways: a full name match, in which case the entry's code-field address is returned; or the link field read is zero, meaning the chain's original first entry has been reached with no match, which signals the word was not found.

Word Append

HEADER builds a new entry by first capturing the current value of DPHERE as the address the new entry will occupy. It writes the length/flags byte and the name's characters starting there, advancing a working pointer as it goes. It then writes the current value of LATEST — the address of whatever is still the newest entry at this point — into the new entry's own link field, so the new entry points back to the one before it. A code-field address follows the link field. Only once the whole entry has been written are the two pointers updated: DPHERE is advanced past the new entry, so the next word will be built after it, and LATEST is set to the new entry's own address, so it becomes the newest entry — the one a search will now start from, and the one the next appended entry's link field will point back to.

8. Assembler Source

The complete 6809 assembly source for this system, forth6809.asm, as assembled for the final ROM. This matches the forth6809.asm file delivered alongside this documentation; the section breakdown below follows the numbered SECTION banners used in that file (Section 8.n is SECTION n-1; Section 8.1 is the memory map, constants and GLOBALS layout that precede SECTION 27). The file is arranged with the dictionary headers (SECTION 27, Section 8.28) ahead of the code (SECTIONS 3 to 26), then the init code (SECTION 2) and the hardware vectors (SECTION 1) at the end. The unit test framework itself (gated by UNITTESTS, see Section 6) lives in a separate file, unit_tests.asm, included from forth6809.asm via a conditional INCLUDE directive — it is not reproduced here. The long explanatory comments, bug-fix descriptions and commented-out code that were moved out of this listing are kept in the companion shadow file forth6809.shd and referred to from the listing by reference codes (Section 6).

8.1 Memory Map, Constants & GLOBALS Layout

```
; ============================================================
; 6809 ANS FORTH - subroutine threaded
;
; Layout of this file: memory map and constants, GLOBALS (page zero),
; serial buffers, SECTION 27 (dictionary headers), SECTIONS 3-26 (code),
; SECTION 2 (init code), SECTION 1 (vectors).
;
; Build options (lwasm -D): SERIALPOLL (1 = polled ACIA, 0 = interrupt
; driven), UNITTESTS (1 = include unit_tests.asm), TSTSELECTOR.
;
; Long explanatory and bug-fix comments are kept in the shadow file
; forth6809.shd, and are referred to here by their NAME.n codes.
; The original header comment is shadow HEADER.0.
; ============================================================

; ------------------------------------------------------------
; MEMORY MAP
; ------------------------------------------------------------
ROMSTRT     EQU   $C000             ; physical start of the 16K EPROM
USROMSTRT   EQU   $C100             ; usable ROM start (above INOUT)
USROMEND    EQU   VECTORS-1         ; usable ROM end (VECTORS-1)
VECTORS     EQU   $FFF0
INITCODE    EQU   $FFA4             ; start of the init code (COLDSTRT)
            ; Commented-out code moved to shadow: MEMMAP.5
BASECODE    EQU   $DC44             ; base code. See shadow MEMMAP.6
            ; Commented-out code moved to shadow: MEMMAP.7
BASEDICT    EQU   $D395             ; base dictionary. See shadow MEMMAP.8
INOUT       EQU   $C000             ; I/O block (ACIA), 256 bytes
INOUTEND    EQU   INOUT+$FF
RSTACK      EQU   $BFFF             ; return stack top
DSTACK      EQU   $BCFF             ; data stack top
CODETOP     EQU   $B900             ; code space ceiling

; ANS transient-region sizes (ANS 3.3.3.6):
;   PADMINSIZE  - size of PAD's scratch region, in characters.
;   WORDMINSIZE - minimum size for WORD's region (33).
;   HOLDMINSIZE - pictured numeric output buffer, (2 * 16) + 2 = 34;
;                 it grows downward from PAD into the CODEHERE-to-PAD
;                 gap.
; PADOFFSET (the CODEHERE-to-PAD gap) equals PADMINSIZE. Original
; comment: shadow PADSIZE.0.
PADMINSIZE  EQU   128
WORDMINSIZE EQU   33
HOLDMINSIZE EQU   34
PADOFFSET   EQU   PADMINSIZE        ; CODEHERE-to-PAD gap
WORDMAXCHARS EQU   PADOFFSET-HOLDMINSIZE-1-3
            ; Maximum characters WORD can scan in the CODEHERE-to-PAD gap:
            ; PADOFFSET - HOLDMINSIZE - 1 (count byte) - 3. The 3 bytes were
```

```
            ; once reserved by S", ." and ABORT" (they now use PARSE, stage
            ; 5i). See shadow WORDMAXCHARS.2. Original comment: shadow
            ; WORDMAXCHARS.1.

APPCODE     EQU   $7000             ; application code area
APPDICT     EQU   $2000             ; application dictionary
SUBPOOL     EQU   $0200             ; REPLACES substitution table. See shadow
SUBPOOLSZ   EQU   64                ; REPLACES.1; $0200-$023F
APPVARS     EQU   SUBPOOL+SUBPOOLSZ ; application variables ($0240)
APPVARSEND  EQU   APPDICT-1         ; end of application variables
; $01DD-$01FF were formerly WORDBUF and SIBUF; $0200 now starts SUBPOOL.
; Original comment: shadow MEMMAP.17.

; Commented-out code moved to shadow: MEMMAP.18
TIBBUF      EQU   $0106             ; terminal input buffer
TIBBUFL     EQU   80                ; length of TIBBUF

GLOBALS     EQU   $0000

SP0         EQU   DSTACK+1
RP0         EQU   RSTACK+1

; ------------------------------------------------------------
; SERIALPOLL - conditional-assembly switch for serial I/O.
; 1 (default): KEY/KEYQ/EMIT poll ACIASR directly, no interrupts,
; no ring buffers, no RTS/CTS handshaking - IRQH is a harmless RTI
; stub, matching the other unused vectors (NMIH/FIRQH/SWI2H/
; SWI3H). 0: the original interrupt-driven implementation, with
; INBUF/OUTBUF ring buffers serviced by IRQH and RTS-based flow
; control via INFILL/RTSCHECKHI/RTSCHECKLO. Uses LWASM's IFEQ/
; ELSE/ENDC (a numeric-expression test, not IFDEF/IFNDEF, since
; this is a value to compare, not a symbol's mere presence).
;
; SERIALPOLL can be chosen at build time with lwasm -DSERIALPOLL=0 or
; -DSERIALPOLL=1 (the default). Original comment: shadow SERIALPOLL.1.
; ------------------------------------------------------------
            IFNDEF SERIALPOLL
SERIALPOLL  SET   1                 ; default if -D not passed
            ENDC

; ------------------------------------------------------------
; ACIA (6850) constants - the chip sits at INOUT+8, not at the
; base of the I/O block, leaving INOUT+0..INOUT+7 free for other
; memory-mapped devices sharing this 256-byte region
; ------------------------------------------------------------
ACIA        EQU   INOUT+8
ACIACR      EQU   ACIA
ACIASR      EQU   ACIA
ACIADR      EQU   ACIA+1

; ACIA Status Register Bits

SR_IRQ      EQU   %10000000         ; ($80) (Interrupt Request)
SR_RDRF     EQU   %00000001         ; ($01) (Receive Data Register Full)
SR_TDRE     EQU   %00000010         ; ($02) (Transmit Data Register Empty)
SR_CTS      EQU   %00001000         ; ($08) CTS blocks transmit

; Errors only meaningful together with RDRF
SR_FE       EQU   %00010000         ; ($10) (Framing Error)
SR_OVRN     EQU   %00100000         ; ($20) (Overrun Error)
SR_PE       EQU   %01000000         ; ($40) (Parity Error)

; ACIA Control Register Bits

CR_RESET    EQU   %00000011         ; ($03) (Master Reset mode)
CR_BASE     EQU   %10010101         ; ($95) (Rx Int Enabled, 8-N-1, /16 Clock)
CR_RXON     EQU   %10010101         ; ($95) (Rx Int Enabled, 8-N-1, /16 Clock)
CR_RXTX     EQU   %10110101         ; ($B5) Rx Int, Tx Int/RTS, 8-N-1
CR_POLL     EQU   %00010101         ; ($15) Polling, no ints, 8-N-1
CR_RTSHI    EQU   %11010101         ; ($D5) RTS high, Tx int off, 8-N-1
```

```
; CR_POLL

; bit7=0 (RX interrupt disabled), bits6-5=00 (RTS
; low, TX interrupt disabled) - CR_RXON ($95) with
; only the RX-interrupt-enable bit cleared. Used
; only when SERIALPOLL=1; RTS stays permanently
; low (asserted), since polling mode has no ring
; buffer to overflow and so needs no flow control

; CR_RTSHI

; bits6-5=10: RTS high, TX int disabled, RX int enabled -
; derived from CR_RXON ($95) with bits6-5 changed from
; 00 to 10; the ACIA has no combination offering RTS
; high AND TX interrupt enabled simultaneously (bits6-5
; only has 00/01/10/11, and only 01 enables TX interrupt,
; which always ties RTS low) - EMIT/IRQH's OUTCHAR must
; respect this and defer transmission while RTS is high

; Ring buffer control
SERBUFCTL   EQU   $0176
SERBUF      EQU   $0180             ; ring buffer start, 64-byte aligned
INBUFSZ     EQU   64
OUTBUFSZ    EQU   64
INHIWATER   EQU   48                ; input ring level: RTS high
INLOWATER   EQU   16                ; input ring level: RTS low

; Input control
; /RTS flag states
INACCEPT    EQU   $00
INREJECT    EQU   $FF

; Output control
; Transmit buffer interrupt active flag states.
OUTIDLE     EQU   $00
OUTBUSY     EQU   $FF

; Software (in-band) flow control - polling build only. The
; interrupt-driven build has real RTS/CTS hardware flow control
; already (CHKHI/CHKLO/UPDATE_RTS); polling mode has no ring
; buffer and no hardware handshaking at all, so it needs its own
; substitute - see ACCEPT and PUTXON/PUTXOFF.
XONCH       EQU   $11
XOFFCH      EQU   $13

; ------------------------------------------------------------
; Flag / opcode constants
; ------------------------------------------------------------
TRUEV       EQU   $FFFF
FALSEV      EQU   $0000
OPJSR       EQU   $BD
OPRTS       EQU   $39               ; used by section 3.8's control-flow test
                                    ; harness to terminate each compiled test
                                    ; snippet, compiled via CCOMMA
RTSOPC      EQU   $39

; ------------------------------------------------------------
; Control-flow compile-time tags
; ------------------------------------------------------------
TAGFWD      EQU   1
TAGBACK     EQU   2
TAGDO       EQU   3
TAGCASE     EQU   4
TAGOF       EQU   5
TAGENDOF    EQU   6
TAGQDO      EQU   7                 ; ?DO marker. See bugfix: TAGQDO.1

; ============================================================
; GLOBALS - scratch and state cells in page zero (DP = $00, set in
; COLDSTRT), laid out in RMB order. 69 of 256 bytes are used
; (GLOBALS_USED). Original comment: shadow GLOBALS.0.
```

```
; ============================================================
            ORG   $0000             ; GLOBALS page (DP = $00)
STATE       RMB   2                 ; offset $00
BASE        RMB   2                 ; offset $02
LATEST      RMB   2                 ; offset $04
DPHERE      RMB   2                 ; offset $06
CODEHERE    RMB   2                 ; offset $08
VARHERE     RMB   2                 ; offset $0A
HANDLER     RMB   2                 ; offset $0C
THROWN      RMB   2                 ; offset $0E
TOIN        RMB   2                 ; offset $10
NTIB        RMB   2                 ; offset $12
MSCR        RMB   2                 ; offset $14 (shared: -, *, WITHIN...)
MSCR2       RMB   2                 ; offset $16
MSCR3       RMB   2                 ; offset $18
MSCR4       RMB   2                 ; offset $1A
HLD         RMB   2                 ; offset $1C
EMITCH      RMB   1                 ; offset $1E
; CURXT holds the xt of the word being compiled, for RECURSE.
; Original comment: shadow CURXT.1.
CURXT       RMB   2                 ; offset $1F
CSP         RMB   2                 ; offset $21
PRODHI      RMB   2                 ; offset $23
PRODLO      RMB   2                 ; offset $25
PSIGN       RMB   1                 ; offset $27
DIVNUM      RMB   2                 ; offset $28
DIVDEN      RMB   2                 ; offset $2A
DIVREM      RMB   2                 ; offset $2C
DIVCNT      RMB   1                 ; offset $2E
DNSIGN      RMB   1                 ; offset $2F
DVSIGN      RMB   1                 ; offset $30
DVOWNSIGN   RMB   1                 ; offset $31
MAHI        RMB   1                 ; offset $32
MALO        RMB   1                 ; offset $33
MBHI        RMB   1                 ; offset $34
MBLO        RMB   1                 ; offset $35
MSIGN       RMB   1                 ; offset $36
PRSIGN      RMB   1                 ; offset $37
SHCNT       RMB   1                 ; offset $38
SRCADDR     RMB   2                 ; offset $39
SRCLEN      RMB   2                 ; offset $3B
SRCID       RMB   2                 ; offset $3D
SPAN        RMB   2                 ; offset $3F

SUBHEAD     RMB   2                 ; offset $41: newest REPLACES entry, 0 = none
SUBFREE     RMB   2                 ; offset $43: next free byte of SUBPOOL, 0 = empty

GLOBALS_USED EQU   69               ; bytes used, of 256 available

; $0100-$0105 is unclaimed (was MVSCRATCH). See shadow MVSCRATCH.1

; ------------------------------------------------------------
; Ring buffers for serial communications.
; One for input, one for output.
; The 64 byte alignment is supposed to allow some speed tricks.
; The tricks are not used.
; ------------------------------------------------------------
            ORG   SERBUFCTL         ; $176.

INHEAD      RMB   1
INTAIL      RMB   1
; Open question moved to shadow: RTSSTATE.1
RTSSTATE    RMB   1                 ; $00 = RTS low, $FF = RTS high

OUTHEAD     RMB   1
OUTTAIL     RMB   1
OUTACTIVITY RMB   1                 ; $00 = Idle, $FF = Busy (tx_active)

FECOUNT     RMB   1                 ; framing errors
OVRNCOUNT   RMB   1                 ; overrun count, same as above
PECOUNT     RMB   1                 ; parity-error count, same as above
```

```
POLLREADYCNT
            RMB   1                 ; polling build; see shadow POLLREADYCNT.1

            ORG   SERBUF            ; $180, a 64 byte boundary.

            ALIGN 64                ; Force 64-byte boundary alignment
INBUF       RMB   INBUFSZ           ; Receive circular queue buffer

            ALIGN 64                ; Force 64-byte boundary alignment
OUTBUF      RMB   OUTBUFSZ          ; Transmit circular queue buffer

; ------------------------------------------------------------
; Provide padding, to ensure the correct ROM & .bin file size (and
; opcode offsets) for the MAME emulation and flash memory burn.
; ------------------------------------------------------------
            ORG   USROMSTRT

            IFNDEF UNITTESTS
UNITTESTS   SET   0                 ; default if -D not passed
            ; Default 0: unit tests excluded; -DUNITTESTS=1 includes them.
            ; Original comment: shadow UNITTESTS.1.
            ENDC

            IFNDEF TSTSELECTOR
TSTSELECTOR SET   2                 ; default if -D not passed
            ENDC

            IFNE  UNITTESTS         ; >>>>>>>>>>

; ------------------------------------------------------------
; The unit test framework itself (explanatory comment plus all
; test code) lives in unit_tests.asm, a separate file included
; here only when UNITTESTS is nonzero - see unit_tests.asm for
; the full framework description, or OPEN_ITEMS_CHECKLIST_
; part1.md/part2.md for its own development history.
; ------------------------------------------------------------
            INCLUDE unit_tests.asm

            ENDC                    ; <<<<<<<<<<

ROM:
            FILL  $FF,BASEDICT-ROM
```

8.2 Hardware Vector Table

```
; ============================================================
; SECTION 1: HARDWARE VECTOR TABLE
; ============================================================
            ORG   VECTORS           ; hardware vector table
VRESV       FDB   $0000
VSWI3       FDB   SWI3H             ; SWI3
VSWI2       FDB   SWI2H             ; SWI2
VFIRQ       FDB   FIRQH             ; FIRQ
VIRQ        FDB   IRQH              ; IRQ
VSWI        FDB   SWIH              ; SWI
VNMI        FDB   WARM              ; NMI -> warm restart
VRESET      FDB   COLDSTRT          ; reset

VECTOREND   EQU   *                 ; vectors size, should be $10
VECTORSIZE  EQU   VECTOREND-VECTORS

; ============================================================
; END OF CONSOLIDATED SOURCE
; ============================================================
```

8.3 Init Code (COLDSTRT / WARM)

```
; ============================================================
; SECTION 2: INIT CODE (COLDSTRT / WARM)
; ============================================================
```

```
            ORG   INITCODE          ; init code block

; ------------------------------------------------------------
; Cold start, entered from the reset vector: mask interrupts, set up
; the stacks and the direct page, clear GLOBALS, initialise the ACIA,
; unmask interrupts, run the unit tests if they are built in, then
; enter COLD.
; COLDSTRT
;    Inputs:
;        hardware reset
;    Outputs:
;        does not return
;    Registers: all registers initialised.
; ------------------------------------------------------------
COLDSTRT:
            ORCC  #$50              ; Disable IRQ & FIRQ
            LDS   #RSTACK+1
            LDU   #DSTACK+1
            CLRA
            TFR   A,DP

            LDX   #GLOBALS
            LDB   #0
CLRGLOB:    CLR   ,X+
            DECB
            BNE   CLRGLOB

            JSR   INITSERIAL

            ANDCC #$AF              ; Enable IRQ & FIRQ

            ; IRQ and FIRQ are unmasked above. See bugfix: COLDSTRT.1

            IFNE  UNITTESTS         ; >>>>>>>>>>
            JSR   TSTRUNNER
            ELSE                    ; <<<<<>>>>>
            NOP                     ; same size as JSR TSTRUNNER
            NOP                     ; See bugfix: COLDSTRT.2
            NOP
            ENDC                    ; <<<<<<<<<<

            JMP   COLD

; ------------------------------------------------------------
; Warm start, entered from the NMI vector: mask interrupts, reset the
; stacks and direct page, re-initialise the ACIA, print the warm-start
; message, unmask interrupts and ABORT. GLOBALS is not cleared.
; WARM
;    Inputs:
;        NMI
;    Outputs:
;        does not return
;    Registers: all registers initialised.
; ------------------------------------------------------------
WARM:       ORCC  #$50              ; Disable IRQ & FIRQ
            CLRA
            TFR   A,DP
            LDU   #SP0
            LDS   #RP0

            JSR   INITSERIAL        ; See bugfix: WARM.1

            LDX   #WARMMSG
            PSHU  X
            LDD   #WARMMSGL
            PSHU  D
            JSR   TYPEW
            ANDCC #$AF              ; Enable IRQ & FIRQ
            JMP   ABORT             ; restart the interpreter

WARMMSG:    FCC   "  warm"
```

```
WARMMSGL    EQU   *-WARMMSG

            ; Verify no collision with vectors: INITEND should equal VECTORS.
INITEND     EQU   *
INITSIZE    EQU   INITEND-INITCODE

; Prevent the assembler from extinguishing the gap between the
; INITCODE block and the VECTORS block when it generates the
; .bin file.
            FILL  $FF,VECTORS-INITEND
```

8.4 ACIA Interrupt Handler

```
; ============================================================
; SECTION 3: ACIA INTERRUPT HANDLER
; ============================================================
            ORG   BASECODE          ; See bugfix: BASECODE.1

; ------------------------------------------------------------
; Reset the serial ring buffers, flow-control state and
; receiver-error counters to zero.
; SERBUFCLR
;    Inputs:
;        none
;    Outputs:
;        INHEAD, INTAIL, OUTHEAD, OUTTAIL, RTSSTATE, OUTACTIVITY,
;        FECOUNT, OVRNCOUNT, PECOUNT and POLLREADYCNT all cleared
;    Registers: only CC changed.
; Original comment: shadow SERBUFCLR.0.
; See bugfix: SERBUFCLR.1.
; ------------------------------------------------------------

SERBUFCLR:  CLR   INHEAD
            CLR   INTAIL
            CLR   RTSSTATE          ; $00 = RTS Low (Clear)
            CLR   OUTHEAD
            CLR   OUTTAIL
            CLR   OUTACTIVITY       ; $00 = Idle


            CLR   FECOUNT
            CLR   OVRNCOUNT
            CLR   PECOUNT
            CLR   POLLREADYCNT
            RTS

; ------------------------------------------------------------
; Initialise the ACIA (6850): master reset, then select
; interrupt-driven or polling operation per SERIALPOLL.
; INITSERIAL
;    Inputs:
;        none
;    Outputs:
;        ring buffers cleared (SERBUFCLR); ACIACR set to the run mode
;    Registers: RegA and CC changed.
; Original comment: shadow INITSERIAL.0.
; See bugfix: INITSERIAL.1.
; ------------------------------------------------------------
INITSERIAL: JSR   SERBUFCLR
            LDA   #CR_RESET         ; Master Software Reset. See bugfix: INITSERIAL.2
            STA   ACIACR
            NOP                     ; Settling Delay
            NOP

            IFEQ  SERIALPOLL        ; >>>>>>>>>>
            LDA   #CR_RXON          ; interrupt-driven mode: RX interrupt on
            ELSE                    ; <<<<<>>>>>
            LDA   #CR_POLL          ; polling mode: no interrupts, RTS held low
            ENDC                    ; <<<<<<<<<<
            STA   ACIACR            ; See bugfix: INITSERIAL.1
```

```
            RTS

            IFEQ  SERIALPOLL        ; >>>>>>>>>>
; ------------------------------------------------------------
; Return the input ring buffer fill level (0 to INBUFSZ-1).
; INBUFSZ is a power of two and both indices stay in range, so a
; masked subtraction gives the true distance across the wrap.
; INFILL
;    Inputs:
;        none
;    Outputs:
;        RegA = (INHEAD - INTAIL) mod INBUFSZ
;    Registers: RegA and CC changed.
; ------------------------------------------------------------
INFILL:     LDA   INHEAD
            SUBA  INTAIL
            ANDA  #INBUFSZ-1        ; Output: RegA = fill level
            RTS
; ------------------------------------------------------------
; Assert RTS high (ask the sender to pause) if the input ring has
; reached INHIWATER and RTS is not already high. Called from the
; receive path with interrupts already masked.
; RTSCHECKHI
;    Inputs:
;        none
;    Outputs:
;        RTSSTATE = 1 and ACIACR = CR_RTSHI if the threshold was reached
;    Registers: RegA and CC changed.
; ------------------------------------------------------------
RTSCHECKHI: JSR   INFILL
            CMPA  #INHIWATER
            BLO   RTSCHIDONE
            TST   RTSSTATE
            BNE   RTSCHIDONE        ; already high - nothing to do
            LDA   #CR_RTSHI
            STA   ACIACR
            LDA   #1
            STA   RTSSTATE
RTSCHIDONE: RTS
; ------------------------------------------------------------
; Write the ACIA control byte for the current RTSSTATE and
; OUTACTIVITY: RTS high disables the Tx interrupt; RTS low
; enables it only while output is pending.
; UPDATE_RTS
;    Inputs:
;        RTSSTATE, OUTACTIVITY
;    Outputs:
;        ACIACR written (OUTACTIVITY cleared when RTS is high)
;    Registers: RegA and CC changed.
; ------------------------------------------------------------
UPDATE_RTS:
            TST   RTSSTATE
            BNE   SET_RTS_HI_TX_OFF

            TST   OUTACTIVITY       ; Is active?
            BEQ   SET_RTS_LO_TX_OFF ; No! Disable Transmit interrupt.

                                    ; $B5/%1011_0101
            LDA   #CR_RXTX          ; Yes! Accept input & transmit output
            BRA   WRITE_CR          ; RTS = Low, Tx Interrupt = Enabled

SET_RTS_LO_TX_OFF:                  ; $95/%10010101
            LDA   #CR_RXON          ; RTS = Low, Tx Interrupt = Disabled
            BRA   WRITE_CR          ; Open question moved to shadow: UPDATE_RTS.1

SET_RTS_HI_TX_OFF:                  ; $D5/%1101_0101
            LDA   #CR_RTSHI         ; RTS = High, Tx Interrupt = Disabled
            CLR   OUTACTIVITY       ; Interrupts are hardware-disabled; clear state
WRITE_CR:
            STA   ACIACR
            RTS
```

```
; ------------------------------------------------------------
; If the input ring is near full and RTS is not already high,
; set RTSSTATE to INREJECT and update the ACIA to drop RTS.
; Called after a character is added to the ring.
; CHKHI
;    Inputs:
;        RegB = input ring fill level
;    Outputs:
;        RTSSTATE and ACIACR updated if the high-water mark was reached
;    Registers: RegA and CC changed.
; Original comment: shadow CHKHI.0.
; ------------------------------------------------------------
CHKHI:
            CMPB  #INHIWATER        ; RegB = fill level. Near full?
            BLO   CHKHIDONE         ; No! Do nothing.
            TST   RTSSTATE
            BNE   CHKHIDONE         ; already high - nothing to do
            LDA   #INREJECT         ; Yes! De-assert RTS.
            STA   RTSSTATE
            JSR   UPDATE_RTS
CHKHIDONE:  RTS

; ------------------------------------------------------------
; If the input ring is near empty and RTS is high, set
; RTSSTATE to INACCEPT and update the ACIA to raise RTS.
; Called after a character is removed from the ring.
; CHKLO
;    Inputs:
;        RegB = input ring fill level
;    Outputs:
;        RTSSTATE and ACIACR updated if the low-water mark was reached
;    Registers: RegA and CC changed.
; ------------------------------------------------------------
CHKLO:
            CMPB  #INLOWATER        ; RegB = fill level. Near empty?
            BHS   CHKLODONE         ; No! Do nothing.
            TST   RTSSTATE
            BEQ   CHKLODONE         ; already low - nothing to do
            LDA   #INACCEPT         ; Yes! Assert RTS.
            STA   RTSSTATE
            JSR   UPDATE_RTS
CHKLODONE:  RTS
; ------------------------------------------------------------
; Send at most one character from the output ring to the ACIA
; by polling, then return. Nothing is sent if the ring is empty,
; the remote receiver is blocking (CTS) or the transmitter is not
; ready; the caller simply tries again on its next call.
; FLUSHOUTBUFFER
;    Inputs:
;        none
;    Outputs:
;        one character sent and OUTTAIL advanced, or no change
;    Registers: RegA, RegB, RegX and CC changed.
; See bugfix: FLUSHOUTBUFFER.1.
; ------------------------------------------------------------
FLUSHOUTBUFFER:
            LDB   OUTTAIL
            CMPB  OUTHEAD           ; Is transmit buffer empty?
            BEQ   FLUSHED           ; Yes! Exit.

            LDA   ACIASR            ; Is transmission blocked by remote receiver?
            BITA  #SR_CTS
            BNE   FLUSHED           ; Yes! Exit.

            BITA  #SR_TDRE          ; Is transmit register empty?
            BEQ   FLUSHED           ; No! Exit - try again next call rather
                                    ; than spin-wait here.

            LDX   #OUTBUF           ; The out buffer is occupied.
            LDA   B,X               ; Get the character from the out buffer.
            STA   ACIADR            ; Transmit character.
```

```
            INCB                    ; Update tail pointer to next character slot.
            ANDB  #OUTBUFSZ-1
            STB   OUTTAIL
                                    ; One character sent - return rather than
                                    ; loop back for more (see header comment).
FLUSHED:
            RTS
; ------------------------------------------------------------
; Queue a character for transmission. The character is discarded
; if the output ring is full. Runs as a critical section.
; PUTCHAR
;    Inputs:
;        RegA = character to insert into transmit buffer
;    Outputs:
;        none
;    Registers: RegA and RegB changed; RegX and CC preserved.
; ------------------------------------------------------------
PUTCHAR:
            PSHS  CC,X
            ORCC  #$50              ; Enter Critical Section. Disable IRQ & FIRQ

            LDB   OUTHEAD           ; Is buffer full?
            INCB
            ANDB  #OUTBUFSZ-1
            CMPB  OUTTAIL
            BEQ   PUT_EXIT          ; Yes! Exit discarding character.

            LDX   #OUTBUF           ; Put character into character slot.
            LDB   OUTHEAD
            STA   B,X               ; Input: RegA = character

            INCB                    ; Update tail pointer to next character slot.
            ANDB  #OUTBUFSZ-1
            STB   OUTHEAD

            TST   RTSSTATE          ; Is RTS currently low (chars still being accepted)?
            BEQ   CHK_INT_PATH      ; Yes! Interrupt-driven TX is still available.

            ; RTS is high so the Tx interrupt is off: drain by polling,
            ; with IRQ and FIRQ unmasked meanwhile. See bugfix: PUTCHAR.1
            ANDCC #$AF              ; unmask IRQ+FIRQ for the flush only
            JSR   FLUSHOUTBUFFER    ; Fallback to transmit by polling.
            ORCC  #$50              ; re-mask - PUT_EXIT below still
                                    ; expects the critical section active

            BRA   PUT_EXIT

; Re-start the interrupt driven character pump,
; priming the pump by transmitting a character.
CHK_INT_PATH:
            TST   OUTACTIVITY       ; Is the interrupt handler transmitting?
            BNE   PUT_EXIT          ; Yes! Exit, no action required.

                                    ; No! Restart.
            LDX   #OUTBUF           ; Pull character from the out buffer.
            LDB   OUTTAIL
            LDA   B,X
            STA   ACIADR            ; Load the char for transmission.

            INCB                    ; Update the tail pointer
            ANDB  #OUTBUFSZ-1       ; to point to the next character.
            STB   OUTTAIL

            LDA   #OUTBUSY          ; Flag restart of transmit interrupt handler.
            STA   OUTACTIVITY
            JSR   UPDATE_RTS        ; Turn on the Tx interrupt.

PUT_EXIT:
            PULS  CC,X,PC           ; Leaving the critical section,
                                    ; by restoring the CC.
```

```
; ------------------------------------------------------------
; Fetch the next character from the input ring, if any, and let
; the flow-control check re-assert RTS when the ring has drained.
; GETCHAR
;    Inputs:
;        none
;    Outputs:
;        RegA = next character from the receive buffer
;        Carry clear = character returned, set = ring empty
;    Registers: RegA, RegB, RegX and CC changed.
; ------------------------------------------------------------
GETCHAR:
            LDB   INTAIL            ; Is the out buffer populated?
            CMPB  INHEAD
            BEQ   GET_NO_CHAR       ; No! Exit.

            LDX   #INBUF            ; Pull character from the out buffer.
            LDA   B,X               ; Output: RegA = character

            INCB                    ; Update the tail pointer
            ANDB  #INBUFSZ-1        ; to point to the next character.
            STB   INTAIL

            ; Mask IRQ so the flow-control check cannot race the
            ; receive interrupt. See bugfix: GETCHAR.1
            ORCC  #$10
            LDB   INHEAD            ; Calculate current fill level fresh -
            SUBB  INTAIL            ; both pointers may have moved since
            ANDB  #INBUFSZ-1        ; the ISR last ran, so (unlike INCHAR)
                                    ; there's no already-loaded value here
                                    ; worth reusing.
            JSR   CHKLO
            ANDCC #$EF

            ANDCC #$FE              ; Carry flag %0 = valid character
            RTS

GET_NO_CHAR:
            ORCC  #$01              ; Carry flag %1 = no valid character.
            RTS
; ------------------------------------------------------------
; Raise RTS (accept input again) once the input ring has drained
; to INLOWATER or below, re-enabling the Tx interrupt if output is
; pending. Called from mainline code, so IRQ is masked while it runs.
; RTSCHECKLO
;    Inputs:
;        none
;    Outputs:
;        RTSSTATE = 0 and ACIACR rewritten if RTS was high and the ring is low
;    Registers: RegA, RegB and CC changed.
; Original comment: shadow RTSCHECKLO.0.
; ------------------------------------------------------------
RTSCHECKLO:
            ORCC  #$10              ; mask IRQ for the critical section

            JSR   INFILL
            CMPA  #INLOWATER
            BHI   RTSCLODONE
            TST   RTSSTATE
            BEQ   RTSCLODONE        ; already low - nothing to do


            CLR   RTSSTATE          ; Set flag asserting acceptance of chars

            LDB   OUTTAIL           ; Check if chars are available.
            CMPB  OUTHEAD
            BEQ   RTSCLONOTX        ; No! Just configure the ACIA to accept chars.

            LDA   #CR_RXTX          ; Yes! Configure the ACIA to accept chars.
            STA   ACIACR            ; & to generate transmit slot available interrupts.
            BRA   RTSCLOUNMASK
```

```
RTSCLONOTX: LDA   #CR_RXON          ; Configure the ACIA to accept chars.
            STA   ACIACR

RTSCLOUNMASK:
            ANDCC #$EF

RTSCLODONE: RTS

; ------------------------------------------------------------
; ACIA (6850) interrupt handler (SERIALPOLL=0). Services received
; characters (into the input ring, counting framing, overrun and
; parity errors) and transmit-ready (from the output ring), with a
; single exit point. IRQ is hardware-masked while it runs.
; IRQH
;    Inputs:
;        none (hardware interrupt entry)
;    Outputs:
;        none (returns with RTI)
;    Registers: all registers are saved and restored by the interrupt.
; Original comment: shadow IRQH.0.
; ------------------------------------------------------------
IRQH:       LDA   ACIASR            ; Get the status.

            BITA  #SR_IRQ           ; Is the ACIA the interrupt source?
            BEQ   IRQDONE0          ; No! Just exit.
            BITA  #SR_RDRF          ; Is an incoming character available?
            BNE   INCHAR
            BITA  #SR_TDRE          ; Is the slot for an outgoing character available?
            BNE   OUTCHAR
                                    ; Ignore all other interrupts such as DCD change.
IRQDONE0:   RTI                     ; Single point of exit - re-enables interrupts
                                    ; & restores state.

INCHAR:
            LDB   ACIADR            ; Get the char & clear RDRF & error flags.
            BITA  #SR_FE+SR_OVRN+SR_PE
                                    ; Any receiver error flagged?
            BEQ   INOK              ; No! - character is good, keep it.

            BITA  #SR_FE            ; Yes! char is corrupted, discard it.
            BEQ   INXFE             ; Tally each flagged error independently -
            INC   FECOUNT           ; more than one bit can be set at once.
INXFE:      BITA  #SR_OVRN
            BEQ   INXOVRN
            INC   OVRNCOUNT
INXOVRN:    BITA  #SR_PE
            BEQ   INXPE
            INC   PECOUNT
INXPE:      RTI

INOK:       TFR   B,A               ; Transfer good character from the receiver.
            LDX   #INBUF            ; Store it in the empty in buffer slot.
            LDB   INHEAD
            STA   B,X
            INCB                    ; Figure out the new head,
            ANDB  #INBUFSZ-1        ; pointing to the next the empty in buffer slot.
            CMPB  INTAIL            ; Would the new head slot meet the tail?
            BEQ   IRQDONE1          ; Don't allow it - buffer full, drop the character.
            STB   INHEAD            ; New head pointer is OK so store it.
            ; Commented-out code moved to shadow: INCHAR.2
            ; Reuse RegB (the new head) for the fill level.
            ; See shadow: INCHAR.1
            SUBB  INTAIL            ; RegB: new head - tail = current fill level.
            ANDB  #INBUFSZ-1
            JSR   CHKHI             ; Protect the in buffer from overflow.
IRQDONE1:   RTI

            ; Commented-out code moved to shadow: OUTCHAR.1

OUTCHAR:    LDB   OUTTAIL           ; Is the out buffer empty?
```

```
            CMPB  OUTHEAD
            BEQ   TXOFF             ; Yes! Stop transmitting.

            LDX   #OUTBUF           ; No! The out buffer is occupied.
            LDA   B,X               ; Get the character from the out buffer.
            STA   ACIADR            ; Transmit the char.

            INCB                    ; Update the tail pointer pointing to the next char.
            ANDB  #OUTBUFSZ-1
            STB   OUTTAIL
IRQDONE2    RTI

TXOFF:                              ; The output ring is empty.
            ; Commented-out code moved to shadow: TXOFF.1
            CLR   OUTACTIVITY
            JSR   UPDATE_RTS
IRQDONE3    RTI

            ELSE                    ; <<<<<>>>>>

IRQH:       RTI                     ; polling mode (SERIALPOLL=1) - ACIA
                                    ; interrupts are never enabled (see
                                    ; COLDSTRT's CR_POLL init), so this should
                                    ; never fire; kept as a safe stub matching
                                    ; the other unused vectors below
            ENDC                    ; <<<<<<<<<<

; ------------------------------------------------------------
; Unused interrupt handlers: return immediately. SWIH instead
; pushes the throw code -99 and jumps to THROW.
; ------------------------------------------------------------
SWI3H:      RTI
SWI2H:      RTI
FIRQH:      RTI
NMIH:       RTI                     ; unused now that NMI -> WARM
SWIH:       LDD   #-99              ; placeholder hardware-trap code; push and
            PSHU  D                 ; JMP THROW, per the CATCH/THROW turn
            JMP   THROWW
```

8.5 COLD / ABORT / QUIT

```
; ============================================================
; SECTION 4: COLD / ABORT / QUIT  (with CATCH-wrapped INTERPRET)
; ============================================================
; ------------------------------------------------------------
; Cold start: set the dictionary, code and variable pointers to
; their application areas, BASE to 10 and the input source to the
; terminal buffer, print the sign-on banner, then enter ABORT.
; COLD
;    Inputs:
;        none (entered at reset)
;    Outputs:
;        none (does not return)
;    Registers: all changed.
; ------------------------------------------------------------
COLD:       LDD   #APPVARS
            STD   VARHERE
            LDD   #APPCODE
            STD   CODEHERE
            LDD   #APPDICT
            STD   DPHERE
            LDD   #BASELATEST
            STD   LATEST

            LDD   #10
            STD   BASE

            LDD   #TIBBUF
            STD   SRCADDR
            LDD   #0
```

```
            STD   SRCLEN
            STD   SRCID

            LDX   #SIGNON
            PSHU  X                 ; c-addr
            LDD   #SIGNONL
            PSHU  D                 ; u
            JSR   TYPEW
            JSR   CRW
            BRA   ABORT             ; Go to ABORT, skipping the ABORTW word.

; ------------------------------------------------------------
; ABORT  ( i*x -- ) ( R: j*x -- )
; Throw -1. With no CATCH frame, THROW falls into the ABORT reset below.
; ------------------------------------------------------------
ABORTW:     LDD   #-1
            PSHU  D                 ; throw code (-1)
            JMP   THROWW

; ------------------------------------------------------------
; Reset the data stack to its base, then fall through into QUITW.
; ABORT
;    Inputs:
;        none
;    Outputs:
;        none (does not return)
;    Registers: RegU changed.
; ------------------------------------------------------------
ABORT:      LDU   #SP0
            ; falls through into QUIT

; ------------------------------------------------------------
; QUIT  ( -- ) ( R: i*x -- )
; Empty the return stack, set interpretation state, then loop reading
; a line and interpreting it under CATCH. Errors are reported as
; "ERROR n"; -1 (ABORT) is silent. Never returns.
; ------------------------------------------------------------
QUITW:      LDS   #RP0
            LDD   #0
            STD   STATE             ; See bugfix: QUITW.1

QLOOP:      JSR   QUERYW

            LDD   DPHERE            ; Checkpoint on the return stack. CATCH
            PSHS  D                 ; and THROW restore S, so the frame is
            LDD   CODEHERE          ; intact whichever way INTERPRET ends.
            PSHS  D                 ; See shadow QLOOP.2
            LDD   VARHERE
            PSHS  D
            LDD   LATEST
            PSHS  D

            LDD   #INTERPRET
            PSHU  D                 ; xt
            JSR   CATCHW
            PULU  D                 ; throw code (0 = no error)
            PSHS  D                 ; frame: 0,S code  2,S LATEST  4,S VARHERE
            CMPD  #0                ;        6,S CODEHERE  8,S DPHERE
            BEQ   QOK

            LDD   8,S
            STD   DPHERE
            LDD   6,S
            STD   CODEHERE
            LDD   4,S
            STD   VARHERE
            LDD   2,S
            STD   LATEST
            LDU   #SP0

            ; Return to interpretation state after a caught error.
```

```
            ; See bugfix: QLOOP.1
            LDD   #0
            STD   STATE

            LDD   ,S                ; -1 is ABORT: reset silently, no "ERROR -1"
            CMPD  #-1
            BNE   QSHOWERR
            LEAS  10,S              ; drop the frame
            LBRA  QLOOP

QSHOWERR:   JSR   CRW
            LDX   #ERRMSG
            PSHU  X                 ; c-addr
            LDD   #ERRMSGL
            PSHU  D                 ; u
            JSR   TYPEW
            LDD   ,S
            PSHU  D                 ; n
            JSR   DOTW
            LEAS  10,S              ; drop the frame
            BRA   QLOOP

QOK:        LEAS  10,S              ; drop the frame
            JSR   CRW               ; See bugfix: QOK.1
            LDD   STATE
            BNE   QLOOP
            LDX   #OKMSG
            PSHU  X                 ; c-addr
            LDD   #OKMSGL
            PSHU  D                 ; u
            JSR   TYPEW
            BRA   QLOOP

SIGNON:     FCC   "6809 FORTH v1.0"
SIGNONL     EQU   *-SIGNON
OKMSG:      FCC   "  ok"
OKMSGL      EQU   *-OKMSG
ERRMSG:     FCC   "  ERROR "
ERRMSGL     EQU   *-ERRMSG
```

8.6 Inner-Interpreter Support

```
; ============================================================
; SECTION 5: INNER-INTERPRETER SUPPORT (LIT, ZBRANCH, BRANCH,
; DODOES, DODEFER, EXECUTE)
; ============================================================
; ------------------------------------------------------------
; Run-time for a literal: fetch the inline cell that follows the
; calling JSR, push it, and advance the return address past it.
; LIT   ( -- x )
;    Inputs:
;        return stack top = address of the inline 16-bit literal
;    Outputs:
;        x pushed on the data stack; return address advanced by 2
;    Registers: RegD, RegX and CC changed.
; ------------------------------------------------------------
LIT:        PULS  X
            LDD   ,X++
            PSHU  D                 ; x
            PSHS  X
            RTS

; ------------------------------------------------------------
; Run-time for a conditional branch: take the inline offset if the
; flag is false, otherwise skip it. The offset is relative to its own
; address.
; ZBRANCH   ( flag -- )
;    Inputs:
;        flag on the data stack
;        return stack top = address of the inline 16-bit offset
```

```
;    Outputs:
;        flag removed; return address set to the branch target or past
;        the offset
;    Registers: RegD, RegX and CC changed.
; ------------------------------------------------------------
ZBRANCH:    PULU  D                ; flag
            PULS  X
            CMPD  #0
            BNE   ZSKIP
            LDD   ,X
            LEAX  D,X
            PSHS  X
            RTS
ZSKIP:      LEAX  2,X
            PSHS  X
            RTS

; ------------------------------------------------------------
; Run-time for an unconditional branch: add the inline offset
; (relative to its own address) to the return address.
; BRANCH
;    Inputs:
;        return stack top = address of the inline 16-bit offset
;    Outputs:
;        return address set to the branch target
;    Registers: RegD, RegX and CC changed.
; ------------------------------------------------------------
BRANCH:     PULS  X
            LDD   ,X
            LEAX  D,X
            PSHS  X
            RTS

; ------------------------------------------------------------
; Run-time entry of a CREATEd or DOES>-defined word. Its code is a
; trampoline: JSR DODOES, a BEHAVIOR address, then the data-field
; address. Pushes the data-field address and jumps to BEHAVIOR.
; DODOES   ( -- pfa )
;    Inputs:
;        return stack top = address of the BEHAVIOR cell in the
;        trampoline
;    Outputs:
;        pfa pushed on the data stack; control passes to BEHAVIOR
;    Registers: RegD, RegX, RegY and CC changed.
; ------------------------------------------------------------
DODOES:     PULS  X
            LDY   ,X++
            LDD   ,X
            PSHU  D                ; pfa
            JMP   ,Y

; ------------------------------------------------------------
; Default BEHAVIOR for a CREATEd word: do nothing, leaving the data-
; field address on the stack.
; DOESRT0
;    Inputs:
;        none
;    Outputs:
;        none
;    Registers: none.
; ------------------------------------------------------------
DOESRT0:    RTS

; ------------------------------------------------------------
; Run-time for DOES>: patch the BEHAVIOR cell of the most recent
; definition with the code that follows the calling JSR, then return
; two levels up, skipping the rest of the defining word.
; SETDOES
;    Inputs:
;        return stack top = address after "JSR SETDOES" (the new
;        BEHAVIOR)
```

```
;        next on the return stack = the defining word's own return
;        address
;    Outputs:
;        BEHAVIOR cell of LATEST patched; control returns to the
;        defining word's caller
;    Registers: RegD, RegX, RegY and CC changed.
; ---------------------------------------------------------------
SETDOES:    PULS  X                 ; X = addr right after "JSR SETDOES" - new BEHAVIOR
            TFR   X,Y               ; Y = new BEHAVIOR, kept across the walk

            LDX   LATEST
            LDB   ,X
            ANDB  #$1F
            CLRA
            LEAX  D,X               ; header + name length
            LDD   3,X               ; D = CFA (trampoline address), past name and LINK
            ADDD  #3                ; +3 -> BEHAVIOR field (past JSR DODOES)
            TFR   D,X

            STY   ,X                ; patch it

            PULS  X                 ; X = the OUTER defining word's own return addr
            JMP   ,X                ; jump there directly - "double RTS"

; ---------------------------------------------------------------
; BEHAVIOR of a DEFERred word: execute the xt stored in its data
; field. Tail-calls the xt, so it returns to the original caller.
; DODEFER   ( pfa -- )
;    Inputs:
;        pfa on the data stack
;    Outputs:
;        pfa removed; the stored xt is executed
;    Registers: RegD, RegX and CC changed.
; ---------------------------------------------------------------
DODEFER:    PULU  X                 ; pfa
            LDD   ,X
            TFR   D,X
            JMP   ,X

; ---------------------------------------------------------------
; Initial xt of a DEFER: throws -21 until IS stores a real xt.
; DOABORTUNDEF
;    Inputs:
;        none
;    Outputs:
;        none (does not return)
;    Registers: RegD changed.
; ---------------------------------------------------------------
DOABORTUNDEF:
            LDD   #-21
            PSHU  D                 ; throw code (-21)
            JMP   THROWW

; ---------------------------------------------------------------
; BEHAVIOR of a MARKER word: restore the dictionary, code and variable
; pointers and LATEST from the four cells saved when it was defined.
; DOMARKER   ( pfa -- )
;    Inputs:
;        pfa on the data stack (the four saved cells)
;    Outputs:
;        pfa removed; DPHERE, CODEHERE, VARHERE and LATEST restored
;    Registers: RegD, RegX and CC changed.
; ---------------------------------------------------------------
DOMARKER:   PULU  X                 ; pfa
            LDD   ,X
            STD   DPHERE
            LDD   2,X
            STD   CODEHERE
            LDD   4,X
            STD   VARHERE
            LDD   6,X
```

```
            STD   LATEST
            RTS

; ------------------------------------------------------------
; EXECUTE  ( i*x xt -- j*x )
; Execute the word whose execution token is xt.
; ------------------------------------------------------------
EXECUTEW:   PULU  X                 ; xt
            JSR   ,X
            RTS
```

8.7 Comma Family

```
; ============================================================
; SECTION 6: COMMA FAMILY (factored via APPENDCELL/APPENDBYTE)
; ============================================================
; ------------------------------------------------------------
; Append a 16-bit cell to the area whose "here" pointer is at RegX
; (CODEHERE or VARHERE), and advance that pointer by 2.
; APPENDCELL   ( x -- )
;    Inputs:
;        RegX = address of the here pointer
;        x on the data stack
;    Outputs:
;        x stored at the old here value; pointer advanced by 2
;    Registers: RegD, RegY and CC changed.
; ------------------------------------------------------------
APPENDCELL: PULU  D                 ; x
            LDY   ,X
            STD   ,Y++
            STY   ,X
            RTS

; ------------------------------------------------------------
; Append the low byte of the top cell to the area whose "here" pointer
; is at RegX (CODEHERE or VARHERE), and advance that pointer by 1.
; APPENDBYTE   ( char -- )
;    Inputs:
;        RegX = address of the here pointer
;        char on the data stack
;    Outputs:
;        char stored at the old here value; pointer advanced by 1
;    Registers: RegD, RegY and CC changed.
; ------------------------------------------------------------
APPENDBYTE: PULU  D                 ; char
            LDY   ,X
            STB   ,Y+
            STY   ,X
            RTS

; ------------------------------------------------------------
; ,  ( x -- )
; Append x to the code/data area (CODEHERE).
; ------------------------------------------------------------
COMMAW:     LDX   #CODEHERE
            JMP   APPENDCELL

; ------------------------------------------------------------
; CODECOMMA  ( x -- )
; Internal name for ",": append x at CODEHERE.
; ------------------------------------------------------------
CODECOMMA:  LDX   #CODEHERE
            JMP   APPENDCELL

; ------------------------------------------------------------
; C,  ( char -- )
; Append char to the code/data area (CODEHERE).
; ------------------------------------------------------------
CCOMMAW:    LDX   #CODEHERE
            JMP   APPENDBYTE
```

```
; ------------------------------------------------------------
; CCOMMA1  ( char -- )
; Internal name for "C,": append char at CODEHERE.
; ------------------------------------------------------------
CCOMMA1:    LDX   #CODEHERE
            JMP   APPENDBYTE

; ------------------------------------------------------------
; V,  ( x -- )
; Append x to the variable area (VARHERE).
; ------------------------------------------------------------
VCOMMAW:    LDX   #VARHERE
            JMP   APPENDCELL

; ------------------------------------------------------------
; VC,  ( char -- )
; Append char to the variable area (VARHERE).
; ------------------------------------------------------------
VCCOMMAW:   LDX   #VARHERE
            JMP   APPENDBYTE

; ------------------------------------------------------------
; ALLOT  ( n -- )
; Reserve n bytes in the code/data area by advancing CODEHERE.
; ------------------------------------------------------------
ALLOTW:     PULU  D                 ; n
            LDX   CODEHERE
            LEAX  D,X
            STX   CODEHERE
            RTS

; ------------------------------------------------------------
; VALLOT  ( n -- )
; Reserve n bytes in the variable area by advancing VARHERE.
; ------------------------------------------------------------
VALLOTW:    PULU  D                 ; n
            LDX   VARHERE
            LEAX  D,X
            STX   VARHERE
            RTS

; ------------------------------------------------------------
; HERE  ( -- addr )
; Return the next free address in the code/data area (CODEHERE).
; ------------------------------------------------------------
HEREW:      LDD   CODEHERE
            PSHU  D                 ; addr
            RTS

; ------------------------------------------------------------
; VHERE  ( -- addr )
; Return the next free address in the variable area (VARHERE).
; ------------------------------------------------------------
VHEREW:     LDD   VARHERE
            PSHU  D                 ; addr
            RTS

; ------------------------------------------------------------
; PAD  ( -- c-addr )
; Return the scratch buffer address, PADOFFSET bytes above CODEHERE.
; ------------------------------------------------------------
PADW:       LDD   CODEHERE
            ADDD  #PADOFFSET
            PSHU  D                 ; c-addr
            RTS

; ------------------------------------------------------------
; UNUSED  ( -- u )
; Return the number of free bytes remaining in the code/data area.
; ------------------------------------------------------------
```

```
UNUSEDW:    LDD   #CODETOP
            SUBD  CODEHERE
            PSHU  D                 ; u
            RTS

; ------------------------------------------------------------
; VUNUSED  ( -- u )
; Return the number of free bytes remaining in the variable area.
; ------------------------------------------------------------
VUNUSEDW:   LDD   #APPVARSEND       ; See bugfix: VUNUSEDW.1
            SUBD  VARHERE
            PSHU  D                 ; u
            RTS
```

8.8 HEADER

```
; ============================================================
; SECTION 7: HEADER (factored from :/CREATE/VARIABLE)
; ============================================================
; ------------------------------------------------------------
; Build a dictionary header for the next name in the input: parse the
; name, then lay down the length byte (with the smudge bit set if
; requested), the name, the link to the previous header and the code
; field address (CODEHERE). LATEST and DPHERE are updated.
; HEADER   ( smudge-flag "name" -- )
;    Inputs:
;        smudge-flag on the data stack (non-zero hides the new word)
;        name parsed from the input stream
;    Outputs:
;        header built at DPHERE; LATEST = address of the new header
;    Registers: RegA, RegB, RegX, RegY and CC changed.
; ------------------------------------------------------------
HEADER:     LDD   #32
            PSHU  D
            JSR   WORDW
            PULU  X                 ; c-addr of the name
            PULU  D                 ; smudge flag in B
            LDA   ,X+               ; name length, X = name text
            PSHS  A                 ; 2,S = length (5k: S-frame, no scratch)
            TSTB
            BEQ   HDNOSM
            ORA   #$40
HDNOSM:     LDY   DPHERE
            PSHS  Y                 ; 0,S = address of the new header
            STA   ,Y+
            LDB   2,S
            BEQ   HDNONM
HDCPY:      LDA   ,X+
            STA   ,Y+
            DECB
            BNE   HDCPY
HDNONM:     LDD   LATEST
            STD   ,Y++
            LDD   CODEHERE
            STD   ,Y++
            STY   DPHERE
            PULS  D
            STD   LATEST
            LEAS  1,S
            RTS
```

8.9 Defining Words

```
; ============================================================
; SECTION 8: DEFINING WORDS
; ============================================================
; ------------------------------------------------------------
; :  ( "name" -- )
; Start a colon definition: parse the name, build a smudged header,
; record the new xt in CURXT, note the control-flow stack position in
```

```
; CSP and enter compile state.
; ------------------------------------------------------------
COLONW:     LDD   #TRUEV
            PSHU  D
            JSR   HEADER
            LDD   CODEHERE          ; Record this word's xt for RECURSE.
            STD   CURXT             ; See bugfix: COLONW.1
            TFR   U,D
            STD   CSP
            LDD   #-1
            STD   STATE
            RTS

; ------------------------------------------------------------
; ;  ( -- )  IMMEDIATE
; End a colon definition: compile RTS, check that the control-flow
; stack is balanced (error -22 if not), clear the smudge bit of the
; new word and return to interpret state.
; ------------------------------------------------------------
SEMIW:      LDD   #RTSOPC
            PSHU  D
            JSR   CCOMMA1
            TFR   U,D
            CMPD  CSP
            BEQ   SEMIOK
            JSR   CFERR
SEMIOK:     LDX   LATEST
            LDA   ,X
            ANDA  #$BF
            STA   ,X
            LDD   #0
            STD   STATE
            RTS

; ------------------------------------------------------------
; :NONAME  ( -- xt )
; Start an anonymous definition: no header is built and LATEST is left
; alone, so the word can never be found. Leaves the xt (the current
; CODEHERE) on the stack, records it in CURXT, notes the control-flow
; stack position in CSP and enters compile state.
; Original comment: shadow NONAMEW.0.
; ------------------------------------------------------------
NONAMEW:    LDD   CODEHERE
            PSHU  D                 ; xt
            STD   CURXT             ; See bugfix: NONAMEW.1
            TFR   U,D
            STD   CSP
            LDD   #-1
            STD   STATE
            RTS

; ------------------------------------------------------------
; Common start of every DODOES-style defining word: build a header
; and compile the DODOES trampoline followed by the behaviour address.
; BLDDOES
;    Inputs:
;        RegD = address of the behaviour routine
;    Outputs:
;        header built, trampoline and behaviour cell compiled
;    Registers: all changed.
; BLDSELF
;    Inputs:
;        RegD = address of the behaviour routine
;    Outputs:
;        as BLDDOES, then the PFA cell compiled pointing at the
;        cell that follows it (the word's own data field)
;    Registers: all changed.
; Used by CREATE, CONSTANT, 2CONSTANT, DEFER and MARKER (BLDSELF)
; and by VALUE (BLDDOES, which supplies its own PFA).
; ------------------------------------------------------------
BLDDOES:    PSHS  D                 ; behaviour address
```

```
            LDD   #0
            PSHU  D
            JSR   HEADER
            LDD   #DODOES
            PSHU  D
            JSR   CCALL
            PULS  D
            PSHU  D                 ; behaviour address
            JSR   CODECOMMA
            RTS

BLDSELF:    JSR   BLDDOES
            LDD   CODEHERE          ; PFA = the cell after this one
            ADDD  #2                ; See bugfix: CONSTANTW.1
            PSHU  D
            JSR   CODECOMMA
            RTS

; ------------------------------------------------------------
; CREATE  ( "name" -- )
; Build a header whose code is a DODOES trampoline. The default
; behaviour (DOESRT0) leaves the data-field address, the next free
; address in the code area, on the stack.
; ------------------------------------------------------------
CREATEW:    LDD   #DOESRT0
            JSR   BLDSELF           ; See bugfix: BLDSELF.1
            RTS

; ------------------------------------------------------------
; DOES>  ( -- )  IMMEDIATE, compile-only
; Compile a call to SETDOES, which at run time patches the behaviour
; of the word being defined.
; Original comment: shadow DOESGTW.0.
; ------------------------------------------------------------
DOESGTW:    LDD   #SETDOES
            PSHU  D
            JSR   CCALL
            RTS

; ------------------------------------------------------------
; VARIABLE  ( "name" -- )
; Build a word that returns the address of a new cell in the variable
; area, initialised to zero.
; ------------------------------------------------------------
VARIABLEW:  LDD   #0
            PSHU  D
            JSR   HEADER
            LDD   #DODOES
            PSHU  D
            JSR   CCALL
            LDD   #DOESRT0
            PSHU  D
            JSR   CODECOMMA
            LDD   VARHERE
            PSHU  D
            JSR   CODECOMMA
            LDD   #0
            LDX   VARHERE
            STD   ,X++
            STX   VARHERE
            RTS

; ------------------------------------------------------------
; @  ( a-addr -- x )
; Fetch the cell at a-addr. Also the behaviour of CONSTANT words.
; ------------------------------------------------------------
FETCHW:     PULU  X                 ; a-addr
            LDD   ,X
            PSHU  D                 ; x
            RTS
```

```
; ------------------------------------------------------------
; CONSTANT  ( x "name" -- )
; Build a word that returns x. The value is stored in the code area
; after the trampoline and fetched with @.
; ------------------------------------------------------------
CONSTANTW:  LDD   #FETCHW
            JSR   BLDSELF           ; See bugfix: BLDSELF.1
            JSR   COMMAW
            RTS

; ------------------------------------------------------------
; Behaviour of a VALUE word: fetch the value cell.
; DOVALUE   ( pfa -- x )
;    Inputs:
;        pfa on the data stack (address of the value cell)
;    Outputs:
;        pfa replaced by the value
;    Registers: RegD, RegX and CC changed.
; ------------------------------------------------------------
DOVALUE:    PULU  X                 ; pfa
            LDD   ,X
            PSHU  D                 ; x
            RTS

; ------------------------------------------------------------
; VALUE  ( x "name" -- )
; Build a word that returns x, which can be changed with TO. The value
; cell is kept in the variable area so that it is writable.
; ------------------------------------------------------------
VALUEW:     LDD   #DOVALUE
            JSR   BLDDOES
            LDD   VARHERE           ; PFA = VARHERE (writable space)
            PSHU  D                 ; See bugfix: VALUEW.1
            JSR   CODECOMMA
            JSR   VCOMMAW           ; store x in the variable area
            RTS

; ------------------------------------------------------------
; Behaviour of a 2VALUE word: fetch the two-cell value, x2 at the
; lower address as 2@ expects.
; DO2VALUE   ( pfa -- x1 x2 )
; ------------------------------------------------------------
DO2VALUE:   JMP   TWOFETCHW

; ------------------------------------------------------------
; 2VALUE  ( x1 x2 "name" -- )
; Build a word that returns x1 x2, which can be changed with TO. The
; two cells are kept in the variable area so that they are writable.
; ------------------------------------------------------------
TWOVALUEW:  LDD   #DO2VALUE
            JSR   BLDDOES
            LDD   VARHERE           ; PFA = VARHERE (writable space)
            PSHU  D
            JSR   CODECOMMA
            JSR   VCOMMAW           ; x2 at the lower address
            JSR   VCOMMAW           ; then x1
            RTS

; ------------------------------------------------------------
; TO  ( x "name" -- )  IMMEDIATE
; Store x in the named VALUE. When compiling, instead compile code to
; store a run-time x in it. Error -13 if the name is not found.
; ------------------------------------------------------------
TOW:        LDD   #32
            PSHU  D
            JSR   WORDW
            JSR   FINDW
            PULU  D
            CMPD  #0
            BNE   TOFOUND
            PULU  D
```

```
            LDD   #-13
            PSHU  D
            JSR   THROWW
TOFOUND:    LDX   ,U                ; xt: is it a 2VALUE? (behaviour cell at xt+3)
            LDD   3,X
            CMPD  #DO2VALUE
            BEQ   TOTWO
            JSR   TOBODYW
            LDD   STATE
            BEQ   TOIMMED
            JSR   LITERALW
            LDD   #STOREW
            PSHU  D
            JSR   CCALL
            RTS
TOIMMED:    PULU  X                 ; a-addr of the value cell
            PULU  D                 ; x
            STD   ,X
            RTS
TOTWO:      JSR   TOBODYW           ; 2VALUE: store x1 x2 with 2!
            LDD   STATE
            BEQ   TOTWOIMM
            JSR   LITERALW
            LDD   #TWOSTOREW
            PSHU  D
            JSR   CCALL
            RTS
TOTWOIMM:   JMP   TWOSTOREW

; ------------------------------------------------------------
; 2VARIABLE  ( "name" -- )
; Build a word that returns the address of a new two-cell variable in
; the variable area, initialised to zero.
; ------------------------------------------------------------
TWOVARIABLEW:
            LDD   #0
            PSHU  D
            JSR   HEADER
            LDD   #DODOES
            PSHU  D
            JSR   CCALL
            LDD   #DOESRT0
            PSHU  D
            JSR   CODECOMMA
            LDD   VARHERE
            PSHU  D
            JSR   CODECOMMA
            LDD   #0
            LDX   VARHERE
            STD   ,X++
            STD   ,X++
            STX   VARHERE
            RTS

; ------------------------------------------------------------
; 2CONSTANT  ( x1 x2 "name" -- )
; Build a word that returns x1 x2. The pair is stored in the code area
; with x2 at the lower address, as 2@ and 2! expect.
; ------------------------------------------------------------
TWOCONSTANTW:
            LDD   #TWOFETCHW
            JSR   BLDSELF           ; See bugfix: BLDSELF.1
            PULU  D                 ; x2
            STD   MSCR
            LDD   MSCR
            PSHU  D
            JSR   COMMAW            ; x2 -> lower address
            JSR   COMMAW            ; x1 -> higher. See bugfix: TWOCONSTANTW.1
            RTS

; ------------------------------------------------------------
```

```
; BUFFER:  ( u "name" -- )
; Build a word that returns the address of a u-byte buffer reserved in
; the variable area.
; ------------------------------------------------------------
BUFFERCOLONW:
            PULU  D                 ; u
            STD   MSCR2
            LDD   #0
            PSHU  D
            JSR   HEADER
            LDD   #DODOES
            PSHU  D
            JSR   CCALL
            LDD   #DOESRT0
            PSHU  D
            JSR   CODECOMMA
            LDD   VARHERE
            PSHU  D
            JSR   CODECOMMA
            LDD   MSCR2
            PSHU  D
            JSR   VALLOTW
            RTS

; ------------------------------------------------------------
; DEFER  ( "name" -- )
; Build a deferred word. Until IS stores an xt in it, executing it
; throws -21.
; ------------------------------------------------------------
DEFERW:     LDD   #DODEFER
            JSR   BLDSELF           ; See bugfix: BLDSELF.1
            LDD   #DOABORTUNDEF
            PSHU  D
            JSR   COMMAW
            RTS

; ------------------------------------------------------------
; DEFER@  ( xt1 -- xt2 )
; Return the xt currently stored in the deferred word xt1.
; ------------------------------------------------------------
DEFERFETCHW:
            JSR   TOBODYW
            PULU  X                 ; body address
            LDD   ,X
            PSHU  D                 ; xt2
            RTS

; ------------------------------------------------------------
; DEFER!  ( xt2 xt1 -- )
; Store xt2 as the action of the deferred word xt1.
; ------------------------------------------------------------
DEFERSTOREW:
            JSR   TOBODYW
            PULU  X                 ; body address of xt1
            PULU  D                 ; xt2
            STD   ,X
            RTS

; ------------------------------------------------------------
; IS  ( xt "name" -- )  IMMEDIATE
; Store xt as the action of the named deferred word. When compiling,
; compile code to do so at run time. Error -13 if the name is not
; found.
; ------------------------------------------------------------
ISW:        LDD   #32
            PSHU  D
            JSR   WORDW
            JSR   FINDW
            PULU  D
            CMPD  #0
            BNE   ISFOUND
```

```
            PULU  D
            LDD   #-13
            PSHU  D
            JSR   THROWW
ISFOUND:    PULU  X
            LDD   STATE
            BEQ   ISIMMED
            PSHU  X
            JSR   LITERALW
            LDD   #DEFERSTOREW
            PSHU  D
            JSR   CCALL
            RTS
ISIMMED:    PSHU  X
            JSR   DEFERSTOREW
            RTS

; ------------------------------------------------------------
; ACTION-OF  ( "name" -- xt )  IMMEDIATE
; Return the xt currently stored in the named deferred word. When
; compiling, compile code to do so at run time. Error -13 if the name
; is not found.
; ------------------------------------------------------------
ACTIONOFW:  LDD   #32
            PSHU  D
            JSR   WORDW
            JSR   FINDW
            PULU  D
            CMPD  #0
            BNE   AOFOUND
            PULU  D
            LDD   #-13
            PSHU  D
            JSR   THROWW
AOFOUND:    PULU  X
            LDD   STATE
            BEQ   AOIMMED
            PSHU  X
            JSR   LITERALW
            LDD   #DEFERFETCHW
            PSHU  D
            JSR   CCALL
            RTS
AOIMMED:    PSHU  X
            JSR   DEFERFETCHW
            RTS

; ------------------------------------------------------------
; MARKER  ( "name" -- )
; Save DPHERE, CODEHERE, VARHERE and LATEST in a new word, which when
; executed restores them (forgetting everything defined since).
; ------------------------------------------------------------
MARKERW:    LDD   DPHERE
            PSHS  D                 ; 6,S
            LDD   CODEHERE
            PSHS  D                 ; 4,S
            LDD   VARHERE
            PSHS  D                 ; 2,S
            LDD   LATEST
            PSHS  D                 ; 0,S
            LDD   #DOMARKER
            JSR   BLDSELF           ; See bugfix: BLDSELF.1
            LDD   6,S
            PSHU  D
            JSR   COMMAW
            LDD   4,S
            PSHU  D
            JSR   COMMAW
            LDD   2,S
            PSHU  D
            JSR   COMMAW
```

```
            LDD   ,S
            PSHU  D
            JSR   COMMAW
            LEAS  8,S
            RTS
```

8.10 Outer Interpreter

```
; ============================================================
; SECTION 9: OUTER INTERPRETER (INTERPRET / WORD / FIND / NUMBER?)
; ============================================================
; ------------------------------------------------------------
; INTERPRET  ( -- )
; Interpret the rest of the input source: parse each word and look it
; up. A found word is executed, or compiled when STATE is compile and
; it is not immediate. Anything else is converted as a single or
; double number (compiled as literals when compiling). An unknown word
; is typed and THROW -13 is raised.
; ------------------------------------------------------------
INTERPRET:
ILOOP:      LDD   #32               ; Delimiter: space. See bugfix: INTERPRET.1
            PSHU  D                 ; char
            JSR   WORDW
            LDX   ,U
            LDA   ,X
            BEQ   IDONE

            JSR   FINDW
            PULU  D
            TSTB
            LBEQ  TRYNUM

            LDA   STATE+1
            BEQ   DOEXEC
            TSTB
            BPL   DOEXEC
            JSR   CCALL
            BRA   ILOOP

DOEXEC:     JSR   EXECUTEW
            BRA   ILOOP

TRYNUM:     JSR   NUMBERQ
            PULU  D
            CMPD  #0                ; flag (PULU does not set CC)
            BEQ   BADWORD

            CMPD  #1                ; 1 = double-number result
            BEQ   TNDOUBLE          ; See shadow: INTERPRET.3

            LDD   STATE
            BEQ   ILOOP
            LDD   #LIT
            PSHU  D
            JSR   CCALL
            JSR   CODECOMMA
            BRA   ILOOP

TNDOUBLE:                           ; U currently holds [UDLO(bottom), UDHI(top)], matching
                                    ; ANS's (ud1) stack order directly.
            LDD   STATE
            BEQ   ILOOP             ; interpreting: already correctly placed,
                                    ; nothing more to do
            ; compiling: need two literals compiled in the order they
            ; must execute at runtime - low first (lands deep), high
            ; second (lands on top). UDHI is currently on top of U, so
            ; pop it aside first, freeing UDLO to be compiled first.
            PULU  D
            STD   MSCR4             ; stash UDHI - confirmed safe scratch,
                                    ; untouched by CCALL/CODECOMMA
```

```
            LDD   #LIT
            PSHU  D
            JSR   CCALL
            JSR   CODECOMMA         ; compiles UDLO (still on U underneath)
            LDD   #LIT
            PSHU  D
            JSR   CCALL
            LDD   MSCR4
            PSHU  D
            JSR   CODECOMMA         ; compiles UDHI
            BRA   ILOOP

BADWORD:    JSR   COUNTW
            JSR   TYPEW
            LDD   #-13
            PSHU  D
            JSR   THROWW

IDONE:      PULU  X                 ; c-addr from WORD (empty). See bugfix: INTERPRET.4
            RTS

; ------------------------------------------------------------
; WORD  ( char "<chars>ccc<char>" -- c-addr )
; Skip leading delimiters, then parse text up to the next char from
; the input source. The text is stored as a counted string at
; CODEHERE, which is not advanced. An exhausted input gives a zero
; count. Its scratch lives in an S frame; see shadow WORDW.7.
; ------------------------------------------------------------
WORDW:      PULU  D                 ; char
            LEAS  -3,S              ; frame (5k): 0,S = delimiter,
            STB   ,S                ; 1,S = start of the text
            LDD   TOIN
            LDX   SRCADDR
            LEAX  D,X
            LDD   SRCLEN
            SUBD  TOIN
            LBLO  EMPTY             ; See bugfix: WORDW.1
            TFR   D,Y

SKIPLP:     CMPY  #0
            BEQ   EMPTY
            LDA   ,X
            CMPA  ,S
            BNE   STARTW
            LEAX  1,X
            LEAY  -1,Y
            BRA   SKIPLP

STARTW:     STX   1,S
            LDB   #0

SCANLP:     CMPY  #0
            BEQ   ENDW
            LDA   ,X
            CMPA  ,S
            BEQ   CONSUME
            CMPB  #WORDMAXCHARS     ; Length limit. See shadow: WORDW.2
            BEQ   ENDW
            LEAX  1,X
            LEAY  -1,Y
            INCB
            BRA   SCANLP

CONSUME:    LEAX  1,X
            LEAY  -1,Y
ENDW:       PSHS  B                 ; Save the count. See bugfix: WORDW.3
            TFR   X,D
            SUBD  SRCADDR
            STD   TOIN
            PULS  B
```

```
            LDX   CODEHERE          ; Result buffer; CODEHERE is not advanced.
                                    ; See shadow: WORDW.4
            STB   ,X+
            LDY   1,S
COPYLP:     TSTB
            BEQ   COPYDONE
            LDA   ,Y+
            STA   ,X+
            DECB
            BRA   COPYLP
COPYDONE:   LEAS  3,S               ; drop the frame
            LDX   CODEHERE          ; Result address (see WORDW.4)
            PSHU  X                 ; c-addr
            RTS

EMPTY:      LEAS  3,S               ; drop the frame
            LDX   CODEHERE          ; Empty string at the result buffer
            CLR   ,X
            PSHU  X                 ; c-addr
            RTS

; ------------------------------------------------------------
; FIND  ( c-addr -- c-addr 0 | xt 1 | xt -1 )
; Look up the counted string in the dictionary, newest first, ignoring
; smudged entries. Returns the xt and 1 if the word is immediate, or
; -1 if not. If not found, returns c-addr and 0.
; Holds no scratch cells; see shadow FINDW.1.
; ------------------------------------------------------------
FINDW:      LDX   LATEST            ; c-addr stays at ,U until the result
FFLOOP:     CMPX  #0                ; (5l: no scratch cells)
            BEQ   NOTFOUND
            LDA   ,X
            BITA  #$40
            BNE   FNEXT             ; smudged
            ANDA  #$1F
            LDY   ,U                ; counted string
            CMPA  ,Y+
            BNE   FNEXT
            PSHS  X                 ; header, kept across the compare
            LEAX  1,X
            TFR   A,B
            BEQ   FMATCH
CMPLP:      LDA   ,X+
            CMPA  ,Y+
            BNE   FMISS
            DECB
            BNE   CMPLP

FMATCH:     PULS  X                 ; header
            LDB   ,X
            PSHS  B                 ; length|flags
            ANDB  #$1F
            CLRA
            LEAX  1,X
            LEAX  D,X
            LDD   2,X               ; CFA field
            STD   ,U                ; xt replaces c-addr
            PULS  A
            BITA  #$80
            BEQ   FISNORM
            LDD   #1
            BRA   FPUSH
FISNORM:    LDD   #-1
FPUSH:      PSHU  D                 ; 1 = immediate, -1 = normal
            RTS

FMISS:      PULS  X
FNEXT:      LDB   ,X
            ANDB  #$1F
            CLRA
            LEAX  1,X
```

```
            LEAX  D,X
            LDX   ,X                ; LINK field
            BRA   FFLOOP

NOTFOUND:   LDD   #0
            PSHU  D                 ; 0 = not found, c-addr still beneath
            RTS

; ------------------------------------------------------------
; Multiply the unsigned double ud on the data stack by BASE and add a
; digit. (5m: the accumulator lives on the data stack, in the frame of
; >NUMBER, so no page-zero cells are needed.)
; UDMULADD
;    Inputs:
;        ud at 4,U (high cell) and 6,U (low cell), as in the >NUMBER
;        frame ( ud c-addr u )
;        RegB = digit value to add
;        BASE
;    Outputs:
;        ud = ud * BASE + digit
;    Registers: RegA, RegB and CC changed.
; ------------------------------------------------------------
UDMULADD:   LEAS  -2,S              ; 0,S = carry, 1,S = BASE multiplier
            STB   ,S
            LDA   BASE+1
            STA   1,S
            LDA   7,U
            LDB   1,S
            MUL
            ADDB  ,S
            BCC   UM0
            INCA
UM0:        STB   7,U
            STA   ,S
            LDA   6,U
            LDB   1,S
            MUL
            ADDB  ,S
            BCC   UM1
            INCA
UM1:        STB   6,U
            STA   ,S
            LDA   5,U
            LDB   1,S
            MUL
            ADDB  ,S
            BCC   UM2
            INCA
UM2:        STB   5,U
            STA   ,S
            LDA   4,U
            LDB   1,S
            MUL
            ADDB  ,S
            BCC   UM3
            INCA
UM3:        STB   4,U
            LEAS  2,S
            RTS

; ------------------------------------------------------------
; >NUMBER  ( ud1 c-addr1 u1 -- ud2 c-addr2 u2 )
; Convert the string into ud1, in BASE, stopping at the first
; unconvertible character. Returns the accumulated ud2 and the
; remaining string. The conversion works in place on the three
; parameters (NUMLOOP), which are also the frame NUMBERQ builds.
; NUMLOOP
;    Inputs:
;        ( ud c-addr u ) on the data stack
;    Outputs:
;        ud accumulated; c-addr u = first unconverted character and
```

```
;        the count remaining
;    Registers: RegA, RegB, RegX and CC changed.
; ------------------------------------------------------------
TONUMBERW:
NUMLOOP:    LDD   ,U
            BEQ   NLDONE
            LDX   2,U
            LDA   ,X
            CMPA  #'0'
            BLO   NLDONE
            CMPA  #'9'
            BHI   NLALPHA
            SUBA  #'0'
            BRA   NLGOT
NLALPHA:    ANDA  #$DF
            CMPA  #'A'
            BLO   NLDONE
            CMPA  #'Z'
            BHI   NLDONE
            SUBA  #'A'-10
NLGOT:      CMPA  BASE+1
            BHS   NLDONE
            TFR   A,B
            JSR   UDMULADD
            LDX   2,U
            LEAX  1,X
            STX   2,U
            LDD   ,U
            SUBD  #1
            STD   ,U
            BRA   NUMLOOP
NLDONE:     RTS

; ------------------------------------------------------------
; Convert a counted string to a number in BASE. A leading "-" negates
; it; a trailing "." makes it a double-cell number.
; NUMBERQ   ( c-addr -- n -1 | ud 1 | c-addr 0 )
;    Inputs:
;        c-addr of the counted string on the data stack
;    Outputs:
;        single number: n and -1
;        double number: ud (low cell below high cell) and 1
;        not a number: c-addr and 0
;    Registers: RegA, RegB, RegX, RegY and CC changed.
; The original c-addr and the negate flag live in an S frame (0,S and
; 2,S); the count is kept in RegB. See shadow NUMBERQ.1 and
; NUMBERQ.6.
; ------------------------------------------------------------
NUMBERQ:    PULU  X                 ; c-addr
            LEAS  -3,S
            STX   ,S
            LDB   ,X
            BEQ   NQBAD
            LEAX  1,X

            ; A trailing "." means a double number: leave it out of the
            ; digit count. See shadow: NUMBERQ.1
            DECB
            LDA   B,X
            INCB
            CMPA  #'.'
            BNE   NQNOSIGN
            DECB
            BEQ   NQBAD             ; lone "." with nothing before it

NQNOSIGN:   CLR   2,S
            LDA   ,X
            CMPA  #'-'
            BNE   NQNOSIGN2
            COM   2,S
            LEAX  1,X
```

```
            DECB
            BEQ   NQBAD

NQNOSIGN2:  LDY   #0
            PSHU  Y                 ; ud low
            PSHU  Y                 ; ud high
            PSHU  X                 ; c-addr
            CLRA
            PSHU  D                 ; u

            JSR   NUMLOOP

            LDD   ,U
            BNE   NQBADF            ; unconverted characters remain
            LEAU  4,U               ; drop u and c-addr: ud remains

            ; Negate the full 32-bit value. See bugfix: NUMBERQ.2
            TST   2,S
            BEQ   NQPOS32
            LDD   ,U
            COMA
            COMB
            STD   ,U
            LDD   2,U
            COMA
            COMB
            ADDD  #1
            STD   2,U
            BCC   NQPOS32
            LDD   ,U
            ADDD  #1
            STD   ,U

NQPOS32:                            ; re-derive from the original c-addr whether the
            ; last character was '.' - determines which return convention
            LDX   ,S
            LDB   ,X
            DECB
            LEAX  1,X
            LDA   B,X
            LEAS  3,S
            CMPA  #'.'
            BNE   NQSINGLE

            ; DOUBLE success: ud is already low below high; add a
            ; distinct success code (1, not -1) so TRYNUM can tell
            ; single and double apart
            LDD   #1
            PSHU  D
            RTS

NQSINGLE:   LEAU  2,U               ; SINGLE success: drop the high cell
            LDD   #-1
            PSHU  D
            RTS

NQBADF:     LEAU  8,U               ; drop the frame
NQBAD:      LDX   ,S
            LEAS  3,S
            PSHU  X
            LDD   #0
            PSHU  D
            RTS
```

8.11 QUERY / ACCEPT / EXPECT / KEY / KEY? / EMIT

```
; ============================================================
; SECTION 10: QUERY / ACCEPT / EXPECT / KEY / KEY? / EMIT
; ============================================================
            IFEQ  SERIALPOLL        ; >>>>>>>>>>
```

```
; Unused alternative to KEY removed; see shadow X_KEY.1.

; ------------------------------------------------------------
; KEY  ( -- char )
; Wait for the next received character and return it. While waiting,
; drain the output ring by polling if RTS is high.
; ------------------------------------------------------------
KEYW:

            TST   RTSSTATE          ; Is throttling?
            BEQ   TRY_READ          ; No! Retrieve character from input buffer.

            ; IRQ stays unmasked during the flush. See bugfix: KEYW.1
            JSR   FLUSHOUTBUFFER    ; Drain the output buffer,
                                    ; by transmitting all chars.

TRY_READ:
            JSR   GETCHAR           ; Is char available in input buffer.
            BCS   KEYW              ; No? Try again to receive a char,
                                    ; while still transmitting!

            TFR   A,B               ; Move char result to Reg B (LSB of D)
            CLRA                    ; Clear MSB.
            PSHU  D                 ; char

            RTS

; ------------------------------------------------------------
; KEY?  ( -- flag )
; Return true if a received character is waiting in the input ring.
; ------------------------------------------------------------
KEYQW:
            ; Commented-out code moved to shadow: KEYQW.1

            LDA   INHEAD            ; Characters received?
            CMPA  INTAIL

            ; Commented-out code moved to shadow: KEYQW.2

            BNE   KQTRUE            ; Yes!

            LDD   #FALSEV           ; No! Return false result.
            PSHU  D                 ; flag = false

            RTS

KQTRUE:
            LDD   #TRUEV            ; Yes! Return true result.
            PSHU  D                 ; flag = true
            RTS

; Unused alternative to EMIT removed; see shadow X_EMIT.1.

; ------------------------------------------------------------
; EMIT  ( char -- )
; Transmit char through PUTCHAR (the output ring). The character is
; discarded if the ring is full.
; ------------------------------------------------------------
EMITW:
            PULU  D                 ; char
            TFR   B,A
            JSR   PUTCHAR           ; Transmit char.
            RTS

            ELSE                    ; <<<<<>>>>>
; ------------------------------------------------------------
; KEY  ( -- char )  [polling build, SERIALPOLL=1]
; Wait for RDRF and return the received character. There are no ring
; buffers, interrupts or hardware handshaking in this build; ACCEPT
; uses software XON/XOFF instead. Framing, overrun and parity errors
; are counted in FECOUNT, OVRNCOUNT and PECOUNT, and POLLREADYCNT
```

```
; counts characters already waiting when KEY was called.
; Original comment: shadow KEYW.2. KEY?, EMIT, PUTXON and PUTXOFF
; follow.
; ------------------------------------------------------------
KEYW:
            LDA   ACIASR
            BITA  #SR_RDRF
            BEQ   KWAIT
            INC   POLLREADYCNT      ; character was already queued when we
            BRA   KGOTSTAT          ; came back to poll - didn't need to wait
KWAIT:
KSPIN:      LDA   ACIASR
            BITA  #SR_RDRF
            BEQ   KSPIN
KGOTSTAT:                           ; A holds the status byte exactly as it was
                                    ; when RDRF first went true - check the
                                    ; error bits from THIS copy, not a fresh
                                    ; read, since reading ACIADR below clears
                                    ; RDRF and the latched error bits together
            BITA  #SR_FE+SR_OVRN+SR_PE
            BEQ   KGETCH
            BITA  #SR_FE
            BEQ   KXFE
            INC   FECOUNT
KXFE:       BITA  #SR_OVRN
            BEQ   KXOVRN
            INC   OVRNCOUNT
KXOVRN:     BITA  #SR_PE
            BEQ   KGETCH
            INC   PECOUNT
KGETCH:     LDA   ACIADR
            TFR   A,B
            CLRA
            PSHU  D                 ; char
            RTS

; ------------------------------------------------------------
; Send the software flow-control character XOFF (PUTXOFF) or XON
; (PUTXON) straight to the ACIA, waiting for TDRE. The data stack is
; not touched, so ACCEPT can call these at any point.
; PUTXON / PUTXOFF
;    Inputs:
;        none
;    Outputs:
;        XON or XOFF transmitted
;    Registers: RegA, RegB and CC changed.
; Original comment: shadow PUTXON.0.
; ------------------------------------------------------------
PUTXOFF:    LDA   #XOFFCH
            BRA   PUTXCH
PUTXON:     LDA   #XONCH
PUTXCH:     PSHS  A
PXWT:       LDB   ACIASR
            BITB  #SR_TDRE
            BEQ   PXWT
            PULS  A
            STA   ACIADR
            RTS

; ------------------------------------------------------------
; KEY?  ( -- flag )  [polling build]
; Return true if the ACIA has a received character (RDRF set).
; ------------------------------------------------------------
KEYQW:      LDA   ACIASR
            BITA  #SR_RDRF
            BEQ   KQFALSE
            LDD   #TRUEV
            PSHU  D                 ; flag = true
            RTS
KQFALSE:    LDD   #FALSEV
            PSHU  D                 ; flag = false
```

```
            RTS

; ------------------------------------------------------------
; EMIT  ( char -- )  [polling build]
; Wait for TDRE, then transmit char directly.
; ------------------------------------------------------------
EMITW:      PULU  D                 ; char
            STB   EMITCH
EMITWT:     LDA   ACIASR
            BITA  #SR_TDRE
            BEQ   EMITWT
            LDA   EMITCH
            STA   ACIADR
            RTS

            ENDC                    ; <<<<<<<<<<

; ------------------------------------------------------------
; ACCEPT  ( c-addr +n1 -- +n2 )
; Read a line of up to +n1 characters into the buffer at c-addr,
; echoing as it goes. Backspace and DEL erase the last character; LF
; is ignored; CR ends the line. Returns the count +n2.
; ------------------------------------------------------------
ACCEPTW:    LEAU  -2,U              ; local count at 0,U; then +n1 at 2,U and
            LDD   #0                ; c-addr at 4,U. See shadow ACCEPTW.5
            STD   ,U

            IFEQ  SERIALPOLL        ; >>>>>>>>>>  interrupt-driven build: real
                                    ; RTS/CTS hardware flow control already
                                    ; covers this, nothing extra needed
            ELSE                    ; <<<<<>>>>>  polling build: tell the host it's safe to
                                    ; stream this line's characters
            JSR   PUTXON
            ENDC                    ; <<<<<<<<<<

ALOOP:      JSR   KEYW
            PULU  D
            TFR   D,Y               ; Y = the character

            CMPB  #13
            BEQ   ADONE
            CMPB  #10
            BEQ   ALOOP
            CMPB  #8
            BEQ   ABKSP
            CMPB  #127
            BEQ   ABKSP

            LDD   ,U
            CMPD  2,U
            BEQ   ALOOP

            LDX   4,U
            LEAX  D,X
            TFR   Y,D
            STB   ,X
            LDD   ,U
            ADDD  #1
            STD   ,U

            TFR   Y,D
            CLRA
            PSHU  D
            JSR   EMITW
            BRA   ALOOP

ABKSP:      LDD   ,U
            BEQ   ALOOP
            SUBD  #1
            STD   ,U
            LDD   #8
```

```
            PSHU  D
            JSR   EMITW
            LDD   #32
            PSHU  D
            JSR   EMITW
            LDD   #8
            PSHU  D
            JSR   EMITW
            BRA   ALOOP

ADONE:
            IFEQ  SERIALPOLL        ; >>>>>>>>>>  interrupt-driven build: nothing
                                    ; extra needed, see ACCEPT's entry above
            ELSE                    ; <<<<<<<<<<  polling build: the line is complete - tell
                                    ; the host to pause before INTERPRET runs
                                    ; with KEY never polled at all
            JSR   PUTXOFF
            ENDC                    ; <<<<<<<<<<
            LDD   ,U
            STD   4,U               ; +n2
            LEAU  4,U
            RTS

; ------------------------------------------------------------
; EXPECT  ( c-addr +n -- )
; Read a line as ACCEPT does and store the count in SPAN.
; ------------------------------------------------------------
EXPECTW:    JSR   ACCEPTW
            PULU  D
            STD   SPAN
            RTS

; ------------------------------------------------------------
; QUERY  ( -- )
; Read a line into the terminal input buffer and make it the input
; source: set NTIB and SRCLEN to its length, SRCADDR to TIBBUF, SRCID
; to 0 and >IN to 0.
; ------------------------------------------------------------
QUERYW:     LDX   #TIBBUF
            PSHU  X
            LDD   #TIBBUFL
            PSHU  D
            JSR   ACCEPTW
            PULU  D
            STD   NTIB
            STD   SRCLEN
            LDD   #TIBBUF
            STD   SRCADDR
            LDD   #0
            STD   SRCID
            STD   TOIN
            RTS
```

8.12 CATCH / THROW

```
; ============================================================
; SECTION 11: EXCEPTIONS (CFERR, CATCH, THROW)
; ============================================================
; ------------------------------------------------------------
; Report a control-structure mismatch: throw -22. Called by the
; control-flow words when the tag on the control-flow stack is not the
; one expected.
; CFERR
;    Inputs:
;        none
;    Outputs:
;        does not return (THROW -22)
;    Registers: RegD changed; THROW decides the rest.
; ------------------------------------------------------------
CFERR:      LDD   #-22
```

```
            PSHU  D                 ; -22
            JSR   THROWW
            RTS

; ------------------------------------------------------------
; CATCH  ( i*x xt -- j*x 0 | i*x n )
; Execute xt with an exception frame in place. On normal return push
; 0. If xt (or anything it calls) executes THROW with a non-zero n,
; THROW unwinds to this frame and CATCH returns n.
; Frame on S, top first: >IN, SRCID, SRCLEN, SRCADDR, saved U,
; previous HANDLER. HANDLER points at the >IN cell.
; ------------------------------------------------------------
CATCHW:     PULU  X                 ; xt
            LDD   HANDLER
            PSHS  D
            PSHS  U
            ; Save the input source specification.
            ; See bugfix: CATCHW.1
            LDD   SRCADDR
            PSHS  D
            LDD   SRCLEN
            PSHS  D
            LDD   SRCID
            PSHS  D
            LDD   TOIN
            PSHS  D
            TFR   S,D
            STD   HANDLER

            JSR   ,X

            LEAS  8,S               ; drop saved input source
            LEAS  2,S               ; saved U
            PULS  D
            STD   HANDLER
            LDD   #0
            PSHU  D                 ; 0 = no exception
            RTS

; ------------------------------------------------------------
; THROW  ( k*x n -- k*x | i*x n )
; If n is zero, do nothing. Otherwise restore the state saved by the
; most recent CATCH (S, U, input source, >IN, HANDLER) and make that
; CATCH return n. With no CATCH active, push n and ABORT.
; ------------------------------------------------------------
THROWW:     PULU  D                 ; n
            CMPD  #0
            BEQ   THDONE

            STD   THROWN
            LDX   HANDLER
            BEQ   THUNCAU

            TFR   X,S
            PULS  D                 ; restore input source saved by CATCH
            STD   TOIN
            PULS  D
            STD   SRCID
            PULS  D
            STD   SRCLEN
            PULS  D
            STD   SRCADDR
            PULS  D
            TFR   D,U
            PULS  D
            STD   HANDLER

            LDD   THROWN
            PSHU  D
            RTS
THDONE:     RTS
```

```
            ; No CATCH is active: leave n on the stack and ABORT.
THUNCAU:    LDD   THROWN
            PSHU  D
            JMP   ABORT
```

8.13 Control Flow

```
; ============================================================
; SECTION 12: CONTROL FLOW (IF/THEN/ELSE, BEGIN family,
; DO/LOOP/+LOOP/I/J/LEAVE/UNLOOP/?DO, EXIT, CASE family)
; ============================================================
; ------------------------------------------------------------
; Store a branch displacement: the cell at location receives target -
; location.
; PATCH   ( target location -- )
;    Inputs:
;        target and location on the data stack (location on top)
;    Outputs:
;        the cell at location holds target - location
;    Registers: RegD and RegX changed.
; ------------------------------------------------------------
PATCH:      PULU  X                 ; location
            PULU  D                 ; target
            PSHS  X
            SUBD  ,S++              ; target - location
            STD   ,X
            RTS

; ------------------------------------------------------------
; IF  ( C: -- orig )
; Compile ZBRANCH with a placeholder operand. Leave the operand
; address and TAGFWD on the control-flow stack for ELSE or THEN.
; ------------------------------------------------------------
IFW:        LDD   #ZBRANCH
            PSHU  D
            JSR   CCALL
            LDD   #0
            PSHU  D
            JSR   CODECOMMA
            LDD   CODEHERE
            SUBD  #2
            PSHU  D                 ; orig
            LDD   #TAGFWD
            PSHU  D
            RTS

; ------------------------------------------------------------
; THEN  ( C: orig -- )
; Resolve the forward branch left by IF, ELSE or WHILE: patch its
; operand to jump to the current code address.
; ------------------------------------------------------------
THENW:      PULU  D
            CMPD  #TAGFWD
            BEQ   THOK
            JSR   CFERR
THOK:       PULU  X                 ; orig
            LDD   CODEHERE
            PSHU  D
            PSHU  X
            JSR   PATCH
            RTS

; ------------------------------------------------------------
; ELSE  ( C: orig1 -- orig2 )
; Compile BRANCH with a placeholder operand, resolve orig1 to the
; address just after it, and leave the new operand address and TAGFWD.
; ------------------------------------------------------------
ELSEW:      PULU  D
            CMPD  #TAGFWD
```

```
            BEQ   ELOK
            JSR   CFERR
ELOK:       PULU  D                 ; orig1
            PSHS  D                 ; See bugfix: ELSEW.1 (5k: kept on S)
            LDD   #BRANCH
            PSHU  D
            JSR   CCALL
            LDD   #0
            PSHU  D
            JSR   CODECOMMA
            LDD   CODEHERE
            PSHU  D
            PULS  D                 ; orig1
            PSHU  D
            JSR   PATCH
            LDD   CODEHERE
            SUBD  #2
            PSHU  D                 ; orig2
            LDD   #TAGFWD
            PSHU  D
            RTS

; ---------------------------------------------------------------
; BEGIN  ( C: -- dest )
; Leave the current code address and TAGBACK on the control-flow
; stack.
; ---------------------------------------------------------------
BEGINW:     LDD   CODEHERE
            PSHU  D                 ; dest
            LDD   #TAGBACK
            PSHU  D
            RTS

; ---------------------------------------------------------------
; UNTIL  ( C: dest -- )
; Compile ZBRANCH with a displacement back to dest.
; ---------------------------------------------------------------
UNTILW:     PULU  D
            CMPD  #TAGBACK
            BEQ   UNOK
            JSR   CFERR
UNOK:       PULU  D                 ; dest
            PSHU  D                 ; park dest. See bugfix: UNTILW.1
            LDD   #ZBRANCH
            PSHU  D
            JSR   CCALL
            LDD   #0
            PSHU  D
            JSR   CODECOMMA
            LDD   CODEHERE
            SUBD  #2
            PSHU  D                 ; location; the parked dest is beneath
            JSR   PATCH
            RTS

; ---------------------------------------------------------------
; AGAIN  ( C: dest -- )
; Compile BRANCH with a displacement back to dest.
; ---------------------------------------------------------------
AGAINW:     PULU  D
            CMPD  #TAGBACK
            BEQ   AGOK
            JSR   CFERR
AGOK:       PULU  D                 ; dest
            PSHU  D                 ; park dest. See bugfix: AGAINW.1
            LDD   #BRANCH
            PSHU  D
            JSR   CCALL
            LDD   #0
            PSHU  D
            JSR   CODECOMMA
```

```
            LDD   CODEHERE
            SUBD  #2
            PSHU  D                 ; location; the parked dest is beneath
            JSR   PATCH
            RTS

; ------------------------------------------------------------
; WHILE  ( C: dest -- orig dest )
; Compile ZBRANCH with a placeholder operand. Leave orig (its operand
; address) and TAGFWD on top of the BEGIN frame.
; ------------------------------------------------------------
WHILEW:     LDD   #ZBRANCH
            PSHU  D
            JSR   CCALL
            LDD   #0
            PSHU  D
            JSR   CODECOMMA
            LDD   CODEHERE
            SUBD  #2
            PSHU  D                 ; orig
            LDD   #TAGFWD
            PSHU  D
            RTS

; ------------------------------------------------------------
; REPEAT  ( C: orig dest -- )
; Compile BRANCH back to the BEGIN address and resolve the WHILE
; branch to just after it. A BEGIN frame beneath further open WHILE
; frames is found by scanning, and removed from beneath them.
; ------------------------------------------------------------
REPEATW:    PULU  D
            CMPD  #TAGFWD
            BEQ   RPOK1
            JSR   CFERR
RPOK1:      PULU  X                 ; orig
            PSHS  X                 ; 2,S = orig (5k: S-frame)

            ; Find the BEGIN frame, scanning past any open TAGFWD frames.
            ; See bugfix: REPEATW.1
            LEAX  0,U               ; scan from top of control stack
            STX   MSCR2             ; MSCR2 = base of scan region
            STX   MSCR4             ; MSCR4 = scan pointer
RPSCAN:     LDD   MSCR4
            CMPD  CSP
            BNE   RPSCANOK
            JSR   CFERR             ; no matching BEGIN
RPSCANOK:   LDX   MSCR4
            LDD   ,X
            CMPD  #TAGBACK
            BEQ   RPFOUND
            CMPD  #TAGFWD
            BEQ   RPSKIP
            JSR   CFERR             ; anything else is a genuine mismatch
RPSKIP:     LDD   MSCR4
            ADDD  #4
            STD   MSCR4
            BRA   RPSCAN
RPFOUND:    LDX   MSCR4             ; X = found BEGIN frame
            LDD   2,X               ; dest = BEGIN's address
            PSHS  D                 ; 0,S = dest

            ; Close the gap: move the frames above the BEGIN frame down 4
            ; bytes, highest word first.
            LDD   MSCR4
            SUBD  MSCR2
            STD   MSCR3             ; MSCR3 = bytes to shift
            BEQ   RPSHIFTDONE
RPSHIFT:    LDD   MSCR3
            SUBD  #2
            STD   MSCR3
            LDX   MSCR2
```

```
            LEAX  D,X               ; X = MSCR2 + this pass's offset
            LDD   ,X
            LEAX  4,X
            STD   ,X                ; copy one word 4 bytes higher
            LDD   MSCR3
            BNE   RPSHIFT
RPSHIFTDONE:
            LDX   MSCR2
            LEAX  4,X
            TFR   X,U               ; U now excludes BEGIN frame

            LDD   #BRANCH
            PSHU  D
            JSR   CCALL
            LDD   #0
            PSHU  D
            JSR   CODECOMMA
            LDD   0,S               ; target = BEGIN's address
            PSHU  D
            LDD   CODEHERE
            SUBD  #2                ; location = the operand just compiled
            PSHU  D
            JSR   PATCH
            LDD   CODEHERE
            PSHU  D
            LDD   2,S               ; orig
            PSHU  D
            JSR   PATCH
            LEAS  4,S               ; drop the frame
            RTS

; ------------------------------------------------------------
; RECURSE  ( -- )
; Compile a call to the word being defined (named or :NONAME), using
; the xt held in CURXT.
; ------------------------------------------------------------
RECURSEW:   LDD   CURXT             ; xt of word being compiled
            PSHU  D                 ; See bugfix: RECURSEW.1
            JSR   CCALL
            RTS

; ------------------------------------------------------------
; DO  ( C: -- dest )
; Compile a call to DOSETUP and leave the loop start address and TAGDO
; on the control-flow stack for LOOP or +LOOP.
; ------------------------------------------------------------
DOW:        LDD   #DOSETUP
            PSHU  D
            JSR   CCALL
            LDD   CODEHERE
            PSHU  D                 ; dest
            LDD   #TAGDO
            PSHU  D
            RTS

; ------------------------------------------------------------
; Run-time start of DO and ?DO: build the loop frame on S, beneath the
; return address.
; DOSETUP   ( limit index -- )
;    Inputs:
;        limit and index on the data stack (index on top)
;    Outputs:
;        S holds index, limit and a zero leave flag (top first)
;    Registers: RegD and RegX changed; MSCR and MSCR2 used as scratch.
; ------------------------------------------------------------
DOSETUP:    PULU  D                 ; index
            STD   MSCR
            PULU  D                 ; limit
            STD   MSCR2
            PULS  X
            LDD   #0
```

```
            PSHS  D
            LDD   MSCR2
            PSHS  D
            LDD   MSCR
            PSHS  D
            PSHS  X
            RTS

; ------------------------------------------------------------
; I  ( -- n )
; Push the index of the innermost loop (2,S, beyond the return
; address).
; ------------------------------------------------------------
IW:         LDD   2,S               ; n. See bugfix: IW.1
            PSHU  D
            RTS

; ------------------------------------------------------------
; J  ( -- n )
; Push the index of the next outer loop (8,S, beyond the return
; address and the inner loop frame).
; ------------------------------------------------------------
JW:         LDD   8,S               ; n. See bugfix: JW.1
            PSHU  D
            RTS

; ------------------------------------------------------------
; LEAVE  ( -- )
; Set the leave flag in the innermost loop frame. The loop exits at
; its next LOOP or +LOOP test.
; ------------------------------------------------------------
LEAVEW:     LDD   #TRUEV            ; true
            STD   6,S               ; See bugfix: LEAVEW.1
            RTS

; ------------------------------------------------------------
; LOOP  ( C: dest -- )
; Compile a call to DOTEST followed by a displacement back to dest.
; For a ?DO loop, also resolve its skip branch to the loop exit.
; ------------------------------------------------------------
LOOPW:      PULU  D
            CMPD  #TAGDO
            BEQ   LOOPOK
            JSR   CFERR
LOOPOK:     PULU  D                 ; dest
            PSHU  D                 ; park dest. See bugfix: LOOPW.1
            LDD   #DOTEST
            PSHU  D
            JSR   CCALL
            LDD   #0
            PSHU  D
            JSR   CODECOMMA
            LDD   CODEHERE
            SUBD  #2
            PSHU  D                 ; location; the parked dest is beneath
            JSR   PATCH

            LDD   ,U
            CMPD  #TAGQDO           ; ?DO frame? See bugfix: LOOPW.2
            BNE   LOOPDONE
            PULU  D
            PULU  X                 ; ?DO skip operand
            LDD   CODEHERE
            PSHU  D
            PSHU  X
            JSR   PATCH
LOOPDONE:   RTS

; ------------------------------------------------------------
; Run-time end of LOOP. Called by JSR; the return address points at
; the back-branch operand. Add 1 to the index. If the leave flag is
```

```
; set or the index has reached the limit, discard the loop frame and
; skip the operand; otherwise branch back to the loop start.
; DOTEST
;    Inputs:
;        S holds return address, index, limit and leave flag (top
;        first)
;    Outputs:
;        branch taken, or loop frame removed and operand skipped
;    Registers: RegD and RegX changed.
; ------------------------------------------------------------
DOTEST:     LDD   6,S               ; leave flag. See bugfix: DOTEST.1
            BNE   DTEXIT            ; set: exit
            LDD   2,S               ; index
            ADDD  #1
            STD   2,S
            CMPD  4,S               ; limit
            BEQ   DTEXIT
            PULS  X                 ; X = branch operand address
            LDD   ,X
            LEAX  D,X
            PSHS  X                 ; return to loop start
            RTS
DTEXIT:     PULS  X
            LEAX  2,X               ; skip the branch operand
            LEAS  6,S               ; discard the loop frame
            PSHS  X
            RTS

; ------------------------------------------------------------
; +LOOP  ( C: dest -- )
; Compile a call to DOPLUSTEST followed by a displacement back to
; dest. For a ?DO loop, also resolve its skip branch to the loop exit.
; ------------------------------------------------------------
PLUSLOOPW:  PULU  D
            CMPD  #TAGDO
            BEQ   PLOOPOK
            JSR   CFERR
PLOOPOK:    PULU  D                 ; dest
            PSHU  D                 ; park dest. See bugfix: PLUSLOOPW.1
            LDD   #DOPLUSTEST
            PSHU  D
            JSR   CCALL
            LDD   #0
            PSHU  D
            JSR   CODECOMMA
            LDD   CODEHERE
            SUBD  #2
            PSHU  D                 ; location; the parked dest is beneath
            JSR   PATCH

            LDD   ,U
            CMPD  #TAGQDO           ; ?DO frame? See bugfix: PLUSLOOPW.2
            BNE   PLOOPDONE
            PULU  D
            PULU  X                 ; ?DO skip operand
            LDD   CODEHERE
            PSHU  D
            PSHU  X
            JSR   PATCH
PLOOPDONE:  RTS

; ------------------------------------------------------------
; Run-time end of +LOOP. Called by JSR; the return address points at
; the back-branch operand. Add the step to the index. If the leave
; flag is set or the index crossed the limit, discard the loop frame
; and skip the operand; otherwise branch back to the loop start.
; DOPLUSTEST   ( n -- )
;    Inputs:
;        step n on the data stack; S as for DOTEST
;    Outputs:
;        branch taken, or loop frame removed and operand skipped
```

```
;    Registers: RegD and RegX changed; MSCR, MSCR2 and MSCR3 used as
;        scratch.
; ------------------------------------------------------------
DOPLUSTEST: PULU  D                 ; n = step. See bugfix: DOPLUSTEST.1
            STD   MSCR              ; MSCR = step
            LDD   6,S               ; leave flag. See bugfix: DOPLUSTEST.2
            BNE   DPTEXIT           ; set: exit
            LDD   2,S               ; old index
            SUBD  4,S               ; old_u = old_index - limit, mod 65536
            STD   MSCR2
            LDD   2,S
            ADDD  MSCR              ; new_index = old_index + step
            STD   2,S
            SUBD  4,S               ; new_u = new_index - limit, mod 65536
            STD   MSCR3
            ; Crossing test on the unsigned distances old_u and new_u: with a
            ; positive step the limit was crossed if new_u < old_u, with a
            ; negative step if new_u > old_u. See bugfix: DOPLUSTEST.3
            LDD   MSCR              ; reload the step to test its sign
            BMI   DPNEG
DPPOS:      LDD   MSCR3             ; step > 0: crossed if new_u < old_u
            CMPD  MSCR2
            BLO   DPTEXIT
            BRA   DPCONT
DPNEG:      LDD   MSCR3             ; step < 0: crossed if new_u > old_u
            CMPD  MSCR2
            BHI   DPTEXIT
DPCONT:     PULS  X                 ; X = branch operand address
            LDD   ,X
            LEAX  D,X
            PSHS  X
            RTS
DPTEXIT:    PULS  X
            LEAX  2,X               ; skip the branch operand
            LEAS  6,S               ; discard the loop frame
            PSHS  X
            RTS

; ------------------------------------------------------------
; ?DO  ( C: -- orig dest )
; Compile a call to QDOSETUP with a placeholder skip operand. Leave
; orig (the skip operand address) and TAGQDO, then dest and TAGDO.
; ------------------------------------------------------------
QDOW:       LDD   #QDOSETUP
            PSHU  D
            JSR   CCALL
            LDD   #0
            PSHU  D
            JSR   CODECOMMA
            LDD   CODEHERE
            SUBD  #2
            PSHU  D                 ; orig
            LDD   #TAGQDO           ; See bugfix: QDOW.1
            PSHU  D
            LDD   CODEHERE
            PSHU  D                 ; dest
            LDD   #TAGDO
            PSHU  D
            RTS

; ------------------------------------------------------------
; Run-time start of ?DO. If index equals limit, branch over the whole
; loop; otherwise build the loop frame as DOSETUP does and skip the
; branch operand.
; QDOSETUP   ( limit index -- )
;    Inputs:
;        limit and index on the data stack (index on top)
;    Outputs:
;        loop skipped, or loop frame on S and operand skipped
;    Registers: RegD and RegX changed; MSCR and MSCR2 used as scratch.
; ------------------------------------------------------------
```

```
QDOSETUP:   PULU  D                 ; index
            STD   MSCR
            PULU  D                 ; limit
            STD   MSCR2
            PULS  X
            LDD   MSCR2
            CMPD  MSCR
            BNE   QDBUILD
            LDD   ,X
            LEAX  D,X
            PSHS  X
            RTS
QDBUILD:    LEAX  2,X
            LDD   #0
            PSHS  D
            LDD   MSCR2
            PSHS  D
            LDD   MSCR
            PSHS  D
            PSHS  X
            RTS

; ------------------------------------------------------------
; UNLOOP  ( -- )
; Discard the loop frame (index, limit, leave flag) beneath UNLOOP's
; return address, so that a following I or J sees the enclosing loop.
; ------------------------------------------------------------
UNLOOPW:    PULS  X
            LEAS  6,S
            PSHS  X
            RTS

; ------------------------------------------------------------
; EXIT  ( -- )
; Compile a call to EXITUNLOOP followed by a count of the DO frames
; found on the control-flow stack. The count is not used at run time.
; A loop must be left with UNLOOP before EXIT.
; ------------------------------------------------------------
EXITW:      LDY   #0                ; Y = count of DO frames (5k: no scratch)
            TFR   U,X               ; X = scan pointer
EXSCAN:     CMPX  CSP
            BEQ   EXSCANDONE
            LDD   ,X
            CMPD  #TAGDO
            BNE   EXNOTDO
            LEAY  1,Y
EXNOTDO:    LEAX  4,X
            BRA   EXSCAN
EXSCANDONE:
            PSHS  Y                 ; the count survives CCALL on S
            LDD   #EXITUNLOOP
            PSHU  D
            JSR   CCALL
            PULS  D
            PSHU  D
            JSR   CODECOMMA
            RTS

; ------------------------------------------------------------
; Run-time part of EXIT: drop the address of the inline count, then
; return from the definition. Loop frames are not discarded.
; EXITUNLOOP
;    Inputs:
;        S holds the address of the inline count, then the return
;        address
;    Outputs:
;        returns to the caller of the definition
;    Registers: RegX and RegY changed.
; ------------------------------------------------------------
EXITUNLOOP: PULS  X                 ; See bugfix: EXITUNLOOP.1
            PULS  Y                 ; Y = return address
```

```
            JMP   ,Y

; ------------------------------------------------------------
; CASE  ( C: -- case-sys )
; Start a CASE structure. Leave a filler cell and TAGCASE, a 4-byte
; frame like those of DO and OF, which the EXIT scan relies on.
; ------------------------------------------------------------
CASEW:      LDD   #0                ; filler. See bugfix: CASEW.1
            PSHU  D
            LDD   #TAGCASE
            PSHU  D
            RTS

; ------------------------------------------------------------
; OF  ( C: -- orig )
; Compile OVER = ZBRANCH <operand> DROP. Leave orig (the operand
; address) and TAGOF.
; ------------------------------------------------------------
OFW:        LDD   #OVERW
            PSHU  D
            JSR   CCALL
            LDD   #EQUALW
            PSHU  D
            JSR   CCALL
            LDD   #ZBRANCH
            PSHU  D
            JSR   CCALL
            LDD   #0
            PSHU  D
            JSR   CODECOMMA
            LDD   CODEHERE
            SUBD  #2
            PSHU  D                 ; orig
            LDD   #DROPW
            PSHU  D
            JSR   CCALL
            LDD   #TAGOF
            PSHU  D
            RTS

; ------------------------------------------------------------
; ENDOF  ( C: orig1 -- orig2 )
; Compile BRANCH with a placeholder operand, resolve orig1 to just
; after it, and leave the new operand address and TAGENDOF.
; ------------------------------------------------------------
ENDOFW:     PULU  D
            CMPD  #TAGOF
            BEQ   EOFOK
            JSR   CFERR
EOFOK:      PULU  D                 ; orig1
            PSHS  D                 ; See bugfix: ENDOFW.1 (5k: kept on S)
            LDD   #BRANCH
            PSHU  D
            JSR   CCALL
            LDD   #0
            PSHU  D
            JSR   CODECOMMA
            LDD   CODEHERE
            PSHU  D
            PULS  D                 ; orig1
            PSHU  D
            JSR   PATCH
            LDD   CODEHERE
            SUBD  #2
            PSHU  D                 ; orig2
            LDD   #TAGENDOF
            PSHU  D
            RTS

; ------------------------------------------------------------
; ENDCASE  ( C: case-sys -- )
```

```
; Compile DROP, then resolve every ENDOF branch to the current code
; address and discard the CASE frame.
; ------------------------------------------------------------
ENDCASEW:   LDD   #DROPW
            PSHU  D
            JSR   CCALL
ECLOOP:     PULU  D
            CMPD  #TAGCASE
            BEQ   ECDONE
            CMPD  #TAGENDOF
            BEQ   ECPATCH
            JSR   CFERR
ECPATCH:    PULU  X                 ; orig
            LDD   CODEHERE
            PSHU  D
            PSHU  X
            JSR   PATCH
            BRA   ECLOOP
ECDONE:     PULU  D                 ; drop filler. See bugfix: ENDCASEW.1
            RTS
```

8.14 Compiling Words

```
; ============================================================
; SECTION 13: COMPILING WORDS (IMMEDIATE/[/]/'/COMPILE,/
; LITERAL/[']/POSTPONE/>BODY, SLITERAL, ABORT")
; ============================================================
; ------------------------------------------------------------
; STATE  ( -- a-addr )
; Push the address of the STATE variable.
; ------------------------------------------------------------
STATEW:     LDD   #STATE
            PSHU  D                 ; a-addr
            RTS

; ------------------------------------------------------------
; IMMEDIATE  ( -- )
; Set the immediate flag ($80) in the length byte of the most recent
; definition (LATEST).
; ------------------------------------------------------------
IMMEDIATEW: LDX   LATEST
            LDA   ,X
            ORA   #$80
            STA   ,X
            RTS

; ------------------------------------------------------------
; [  ( -- )
; Enter interpretation state: STATE = 0.
; ------------------------------------------------------------
LBRACKETW:  LDD   #0
            STD   STATE
            RTS

; ------------------------------------------------------------
; ]  ( -- )
; Enter compilation state: STATE = -1.
; ------------------------------------------------------------
RBRACKETW:  LDD   #-1
            STD   STATE
            RTS

; ------------------------------------------------------------
; '  ( "name" -- xt )
; Parse the next blank-delimited name and return its xt. THROW -13 if
; the name is not found.
; ------------------------------------------------------------
TICKW:      LDD   #32
            PSHU  D                 ; BL delimiter
            JSR   WORDW
```

```
            JSR   FINDW
            PULU  D                 ; found flag
            CMPD  #0
            BNE   TICKOK
            PULU  D                 ; drop the name
            LDD   #-13              ; -13
            PSHU  D
            JSR   THROWW
TICKOK:     RTS

; ------------------------------------------------------------
; COMPILE,  ( xt -- )
; Compile a call to xt (see CCALL).
; ------------------------------------------------------------
COMPILECOMMAW:
            JMP   CCALL

; ------------------------------------------------------------
; Append a JSR to xt at the code pointer CODEHERE.
; CCALL   ( xt -- )
;    Inputs:
;        xt on the data stack
;    Outputs:
;        JSR xt appended at CODEHERE; CODEHERE advanced by 3
;    Registers: RegA, RegB, RegD and RegX changed.
; ------------------------------------------------------------
CCALL:      LDX   CODEHERE          ; X = code pointer
            LDA   #OPJSR            ; JSR opcode
            STA   ,X+
            PULU  D                 ; xt
            STD   ,X++              ; See bugfix: CCALL.1
            STX   CODEHERE
            RTS

; ------------------------------------------------------------
; LITERAL  ( x -- )
; Compile LIT followed by x, so that x is pushed when the definition
; runs.
; ------------------------------------------------------------
LITERALW:   LDD   #LIT
            PSHU  D
            JSR   CCALL
            JSR   CODECOMMA         ; x
            RTS

; ------------------------------------------------------------
; 2LITERAL  ( x1 x2 -- )  IMMEDIATE
; Compile two LITs so that x1 x2 are pushed when the definition runs.
; ------------------------------------------------------------
TWOLITERALW:
            PULU  D                 ; x2
            PSHS  D
            JSR   LITERALW          ; x1 first, so it lands below x2
            PULS  D
            PSHU  D
            JSR   LITERALW
            RTS

; ------------------------------------------------------------
; [']  ( "name" -- )
; Compile the xt of the next name as a literal. THROW -14 if not
; compiling, -13 if the name is not found.
; ------------------------------------------------------------
BRACKTICKW: LDD   STATE
            BNE   BTSTOK
            LDD   #-14              ; -14
            PSHU  D
            JSR   THROWW
BTSTOK:     LDD   #32
            PSHU  D
            JSR   WORDW
```

```
            JSR   FINDW
            PULU  D
            CMPD  #0
            BNE   BTOK
            PULU  D
            LDD   #-13              ; -13
            PSHU  D
            JSR   THROWW
BTOK:       JSR   LITERALW
            RTS

; ------------------------------------------------------------
; POSTPONE  ( "name" -- )
; If the name is immediate, compile a call to it now. Otherwise
; compile its xt as a literal followed by a call to COMPILE, so that
; it is compiled when the enclosing definition runs. THROW -14 if not
; compiling, -13 if the name is not found.
; ------------------------------------------------------------
POSTPONEW:  LDD   STATE
            BNE   PPSTOK
            LDD   #-14              ; -14
            PSHU  D
            JSR   THROWW
PPSTOK:     LDD   #32
            PSHU  D
            JSR   WORDW
            JSR   FINDW
            PULU  D
            CMPD  #0
            BNE   PPFOUND
            PULU  D
            LDD   #-13              ; -13
            PSHU  D
            JSR   THROWW
PPFOUND:    CMPD  #1                ; 1 = immediate
            BEQ   PPIMM
            JSR   LITERALW
            LDD   #COMPILECOMMAW
            PSHU  D
            JSR   CCALL
            RTS
PPIMM:      JSR   COMPILECOMMAW
            RTS

; ------------------------------------------------------------
; [COMPILE]  ( "name" -- )
; Compile a call to the next name, whether or not it is immediate.
; Obsolescent: POSTPONE is the general replacement. THROW -14 if not
; compiling, -13 if the name is not found.
; Original comment: shadow BRACKCOMPILEW.0.
; ------------------------------------------------------------
BRACKCOMPILEW:
            LDD   STATE
            BNE   XCSTOK
            LDD   #-14
            PSHU  D
            JSR   THROWW
XCSTOK:     LDD   #32
            PSHU  D
            JSR   WORDW
            JSR   FINDW
            PULU  D
            CMPD  #0
            BNE   XCFOUND
            PULU  D
            LDD   #-13
            PSHU  D
            JSR   THROWW
XCFOUND:    JSR   COMPILECOMMAW
            RTS
```

```
; ------------------------------------------------------------
; >BODY  ( xt -- a-addr )
; Return the data-field address of a CREATEd word: the cell at xt+5 of
; its JSR DODOES trampoline.
; ------------------------------------------------------------
TOBODYW:    PULU  D                 ; xt
            ADDD  #5
            TFR   D,X
            LDD   ,X
            PSHU  D
            RTS

; ------------------------------------------------------------
; SLITERAL  ( c-addr u -- )
; Compile DOSTR followed by a count byte and a copy of the string.
; THROW -14 if not compiling.
; ------------------------------------------------------------
SLITERALW:  LDD   STATE
            BNE   SLSTOK
            LDD   #-14
            PSHU  D
            JSR   THROWW
SLSTOK:     LDD   #DOSTR
            JMP   CSTRCOMP          ; compile DOSTR and the counted string

; ------------------------------------------------------------
; Run-time part of ABORT". The counted string follows the call. If
; flag is non-zero, TYPE the string and THROW -2; otherwise skip over
; the string.
; DOABORTQUOTE   ( flag -- )
;    Inputs:
;        flag on the data stack; return address on S points at the
;        count byte
;    Outputs:
;        execution continues after the string, or THROW -2
;    Registers: RegA, RegB, RegD, RegX, RegY changed.
; ------------------------------------------------------------
DOABORTQUOTE:
            PULS  X
            LDB   ,X
            LEAX  1,X               ; X = text, B = length
            CLRA
            PULU  Y                 ; flag
            CMPY  #0
            BNE   AQTHROW
            LEAX  B,X
            PSHS  X
            RTS
AQTHROW:    PSHU  X
            PSHU  D
            JSR   TYPEW
            LDD   #-2               ; -2
            PSHU  D
            JMP   THROWW

; ------------------------------------------------------------
; ABORT"  ( "ccc<quote>" -- )
; Compile DOABORTQUOTE followed by the string up to the closing quote
; as a counted string. THROW -14 if not compiling.
; ------------------------------------------------------------
ABORTQUOTEW:
            LDD   STATE
            BNE   AQSTOK
            LDD   #-14
            PSHU  D
            JSR   THROWW
AQSTOK:     JSR   PARSEQUOTE        ; See bugfix: SQUOTEW.3
            LDD   #DOABORTQUOTE
            JMP   CSTRCOMP

; ------------------------------------------------------------
```

```
; BL  ( -- char )
; Push 32, the space character.
; ------------------------------------------------------------
BLW:        LDD   #32
            PSHU  D                 ; char
            RTS

; ------------------------------------------------------------
; >IN  ( -- a-addr )
; Push the address of the >IN variable (TOIN).
; ------------------------------------------------------------
TOINW:      LDD   #TOIN
            PSHU  D
            RTS

; ------------------------------------------------------------
; SPAN  ( -- a-addr )
; Push the address of the SPAN variable.
; ------------------------------------------------------------
SPANW:      LDD   #SPAN
            PSHU  D
            RTS

; ------------------------------------------------------------
; TIB  ( -- c-addr )
; Push the address of the terminal input buffer.
; ------------------------------------------------------------
TIBW:       LDD   #TIBBUF
            PSHU  D
            RTS

; ------------------------------------------------------------
; #TIB  ( -- a-addr )
; Push the address of the NTIB variable.
; ------------------------------------------------------------
NUMTIBW:    LDD   #NTIB
            PSHU  D
            RTS
```

8.15 Stack Manipulation

```
; ============================================================
; SECTION 14: STACK MANIPULATION (Core + Core Ext + return stack)
; ============================================================
; ------------------------------------------------------------
; DUP  ( x -- x x )
; Duplicate the top cell.
; ------------------------------------------------------------
DUPW:       LDD   ,U                ; x
            PSHU  D
            RTS

; ------------------------------------------------------------
; DROP  ( x -- )
; Discard the top cell.
; ------------------------------------------------------------
DROPW:      LEAU  2,U               ; x
            RTS

; ------------------------------------------------------------
; SWAP  ( x1 x2 -- x2 x1 )
; Exchange the top two cells.
; ------------------------------------------------------------
SWAPW:      LDD   ,U                ; x2
            LDX   2,U               ; x1
            STX   ,U
            STD   2,U
            RTS

; ------------------------------------------------------------
```

```
; OVER  ( x1 x2 -- x1 x2 x1 )
; Copy the second cell to the top.
; ---------------------------------------------------------------
OVERW:      LDD   2,U               ; x1
            PSHU  D
            RTS

; ---------------------------------------------------------------
; ROT  ( x1 x2 x3 -- x2 x3 x1 )
; Rotate the third cell to the top.
; ---------------------------------------------------------------
ROTW:       LDD   ,U                ; x3
            LDX   2,U               ; x2
            LDY   4,U               ; x1
            STY   ,U
            STD   2,U
            STX   4,U
            RTS

; ---------------------------------------------------------------
; ?DUP  ( x -- 0 | x x )
; Duplicate the top cell if it is non-zero.
; ---------------------------------------------------------------
QDUPW:      LDD   ,U                ; x
            CMPD  #0
            BEQ   QDUPDONE
            PSHU  D
QDUPDONE:   RTS

; ---------------------------------------------------------------
; DEPTH  ( -- +n )
; Push the number of cells on the data stack: (SP0 - U) / 2.
; ---------------------------------------------------------------
DEPTHW:     LDD   #SP0
            PSHS  U
            SUBD  ,S++
            LSRA
            RORB
            PSHU  D                 ; +n
            RTS

; ---------------------------------------------------------------
; 2DUP  ( x1 x2 -- x1 x2 x1 x2 )
; Duplicate the top cell pair.
; ---------------------------------------------------------------
DDUPW:      LDD   2,U               ; x1
            LDX   ,U                ; x2
            PSHU  D
            PSHU  X
            RTS

; ---------------------------------------------------------------
; 2DROP  ( x1 x2 -- )
; Discard the top cell pair.
; ---------------------------------------------------------------
DDROPW:     LEAU  4,U
            RTS

; ---------------------------------------------------------------
; 2SWAP  ( x1 x2 x3 x4 -- x3 x4 x1 x2 )
; Exchange the top two cell pairs.
; ---------------------------------------------------------------
DSWAPW:     LDD   ,U                ; x4
            STD   MSCR
            LDD   2,U               ; x3
            LDX   4,U               ; x2
            LDY   6,U               ; x1
            STD   6,U
            STX   ,U
            STY   2,U
            LDD   MSCR
```

```
            STD   4,U
            RTS

; ------------------------------------------------------------
; 2OVER  ( x1 x2 x3 x4 -- x1 x2 x3 x4 x1 x2 )
; Copy the second cell pair to the top.
; ------------------------------------------------------------
DOVERW:     LDD   6,U               ; x1
            LDX   4,U               ; x2
            PSHU  D
            PSHU  X
            RTS

; ------------------------------------------------------------
; NIP  ( x1 x2 -- x2 )
; Discard the second cell.
; ------------------------------------------------------------
NIPW:       LDD   ,U                ; x2
            STD   2,U
            LEAU  2,U
            RTS

; ------------------------------------------------------------
; TUCK  ( x1 x2 -- x2 x1 x2 )
; Copy the top cell beneath the second cell.
; ------------------------------------------------------------
TUCKW:      LDD   ,U                ; x2
            LDX   2,U               ; x1
            PSHU  D
            STX   2,U
            STD   4,U
            RTS

; ------------------------------------------------------------
; PICK  ( xu ... x0 u -- xu ... x0 xu )
; Copy the cell u deep (0 PICK is DUP) to the top.
; ------------------------------------------------------------
PICKW:      PULU  D                 ; u
            LSLB
            ROLA
            LDD   D,U
            PSHU  D
            RTS

; ------------------------------------------------------------
; ROLL  ( xu xu-1 ... x0 u -- xu-1 ... x0 xu )
; Move the cell u deep (0 ROLL does nothing) to the top, shifting the
; cells above it down one place. RDST holds the byte offset and RVAL
; the cell being moved.
; ------------------------------------------------------------
ROLLW:      PULU  D                 ; u
            BEQ   ROLLDONE
            TFR   D,Y               ; Y = cells to shift
            LSLB
            ROLA
            LEAX  D,U               ; X = xu
            LDD   ,X
            PSHS  D                 ; xu waits on the return stack
RLOOP:      LDD   -2,X
            STD   ,X
            LEAX  -2,X
            LEAY  -1,Y
            BNE   RLOOP
            PULS  D
            STD   ,U
ROLLDONE:   RTS

; ------------------------------------------------------------
; 2ROT  ( x1 x2 x3 x4 x5 x6 -- x3 x4 x5 x6 x1 x2 )
; Rotate the third cell pair to the top. X and Y hold the pair
; while the others move.
```

```
; ------------------------------------------------------------
DROTW:      LDX   10,U              ; x1
            LDY   8,U               ; x2
            LDD   6,U
            STD   10,U
            LDD   4,U
            STD   8,U
            LDD   2,U
            STD   6,U
            LDD   0,U
            STD   4,U
            STY   ,U
            STX   2,U
            RTS

; ------------------------------------------------------------
; >R  ( x -- )  ( R: -- x )
; Move x from the data stack to the return stack. The return address
; of this call is on top of S, so it is lifted off and put back around
; the transfer.
; ------------------------------------------------------------
TORW:       PULU  D                 ; x
            PULS  X
            PSHS  D
            PSHS  X
            RTS

; ------------------------------------------------------------
; R>  ( -- x )  ( R: x -- )
; Move x from the return stack to the data stack. The return address
; of this call is on top of S, so it is lifted off and put back around
; the transfer.
; ------------------------------------------------------------
FROMRW:     PULS  X
            PULS  D                 ; x
            PSHS  X
            PSHU  D
            RTS

; ------------------------------------------------------------
; R@  ( -- x )  ( R: x -- x )
; Copy the top of the return stack to the data stack. The return
; address of this call is on top of S, so it is lifted off and put
; back around the transfer.
; ------------------------------------------------------------
RFETCHW:    PULS  X
            LDD   ,S                ; x
            PSHS  X
            PSHU  D
            RTS

; ------------------------------------------------------------
; 2>R  ( x1 x2 -- )  ( R: -- x1 x2 )
; Move a cell pair to the return stack. R2A and R2B are scratch. The
; return address of this call is on top of S, so it is lifted off and
; put back around the transfer.
; ------------------------------------------------------------
TWOTORW:    PULS  X                 ; return address
            PULU  D                 ; x2
            PULU  Y                 ; x1
            PSHS  Y
            PSHS  D
            PSHS  X
            RTS

; ------------------------------------------------------------
; 2R>  ( -- x1 x2 )  ( R: x1 x2 -- )
; Move a cell pair from the return stack. The
; return address of this call is on top of S, so it is lifted off and
; put back around the transfer.
; ------------------------------------------------------------
```

```
TWOFROMRW:  PULS  X                 ; return address
            PULS  D                 ; x2
            PULS  Y                 ; x1
            PSHS  X
            PSHU  Y
            PSHU  D
            RTS

; ------------------------------------------------------------
; 2R@  ( -- x1 x2 )  ( R: x1 x2 -- x1 x2 )
; Copy the top cell pair of the return stack.
; The return address of this call is on top of S, so it is lifted off
; and put back around the transfer.
; ------------------------------------------------------------
TWORFETCHW: PULS  X                 ; return address
            LDD   ,S                ; x2
            LDY   2,S               ; x1
            PSHS  X
            PSHU  Y
            PSHU  D
            RTS
```

8.16 Arithmetic

```
; ============================================================
; SECTION 15: ARITHMETIC (single + double + mixed precision)
; ============================================================
; ------------------------------------------------------------
; +  ( n1 n2 -- n3 )
; Add n2 to n1.
; ------------------------------------------------------------
PLUSW:      PULU  D                 ; n2
            ADDD  ,U
            STD   ,U
            RTS

; ------------------------------------------------------------
; -  ( n1 n2 -- n3 )
; Subtract n2 from n1.
; ------------------------------------------------------------
MINUSW:     PULU  D                 ; n2
            STD   MSCR
            LDD   ,U
            SUBD  MSCR
            STD   ,U
            RTS

; ------------------------------------------------------------
; NEGATE  ( n1 -- n2 )
; Two's complement negate.
; ------------------------------------------------------------
NEGATEW:    LDD   ,U                ; n1
            COMA
            COMB
            ADDD  #1
            STD   ,U
            RTS

; ------------------------------------------------------------
; ABS  ( n -- u )
; Absolute value.
; ------------------------------------------------------------
ABSW:       LDD   ,U                ; n
            BPL   ABSDONE
            COMA
            COMB
            ADDD  #1
            STD   ,U
ABSDONE:    RTS
```

```
; ------------------------------------------------------------
; MIN  ( n1 n2 -- n3 )
; The lesser of two signed numbers.
; ------------------------------------------------------------
MINW:       PULU  D                 ; n2
            CMPD  ,U
            BLT   MINISN2
            RTS
MINISN2:    STD   ,U
            RTS

; ------------------------------------------------------------
; MAX  ( n1 n2 -- n3 )
; The greater of two signed numbers.
; ------------------------------------------------------------
MAXW:       PULU  D                 ; n2
            CMPD  ,U
            BGT   MAXISN2
            RTS
MAXISN2:    STD   ,U
            RTS

; ------------------------------------------------------------
; 1+  ( n1 -- n2 )
; Add 1.
; ------------------------------------------------------------
ONEPLUSW:   LDD   ,U
            ADDD  #1
            STD   ,U
            RTS

; ------------------------------------------------------------
; 1-  ( n1 -- n2 )
; Subtract 1.
; ------------------------------------------------------------
ONEMINUSW:  LDD   ,U
            SUBD  #1
            STD   ,U
            RTS

; ------------------------------------------------------------
; 2+  ( n1 -- n2 )
; Add 2.
; ------------------------------------------------------------
TWOPLUSW:   LDD   ,U
            ADDD  #2
            STD   ,U
            RTS

; ------------------------------------------------------------
; *  ( n1 n2 -- n3 )
; Signed multiply, keeping the low 16 bits of the product. The
; magnitudes are multiplied byte-wise with MUL and the sign is
; restored.
; ------------------------------------------------------------
STARW:      PULU  D                 ; n2
            STD   MSCR
            LDD   ,U
            CLR   MSIGN
            BPL   SNOFLIP1
            COM   MSIGN
            COMA
            COMB
            ADDD  #1
SNOFLIP1:   STA   MAHI
            STB   MALO
            LDD   MSCR
            BPL   SNOFLIP2
            COM   MSIGN
            COMA
            COMB
```

```
            ADDD  #1
SNOFLIP2:   STA   MBHI
            STB   MBLO
            LDA   MALO
            LDB   MBLO
            MUL
            PSHS  D                 ; partial product on the return stack
            LDA   MAHI
            LDB   MBLO
            MUL
            ADDB  ,S                ; cross term into the high byte
            STB   ,S
            LDA   MALO
            LDB   MBHI
            MUL
            ADDB  ,S
            STB   ,S
            PULS  D                 ; 16-bit result. See shadow MVSCRATCH.1
            TST   MSIGN
            BEQ   SDONE
            COMA
            COMB
            ADDD  #1
SDONE:      STD   ,U
            RTS

; ------------------------------------------------------------
; 2*  ( x1 -- x2 )
; Shift left one bit.
; ------------------------------------------------------------
TWOSTARW:   LDD   ,U
            ASLB
            ROLA
            STD   ,U
            RTS

; ------------------------------------------------------------
; Unsigned 16-bit division by shift and subtract. The divisor is not
; checked for zero.
; UDIV16
;    Inputs:
;        DIVNUM = dividend, DIVDEN = divisor (unsigned)
;    Outputs:
;        DIVNUM = quotient, DIVREM = remainder
;    Registers: RegD changed; DIVCNT used as the loop counter.
; ------------------------------------------------------------
UDIV16:     CLR   DIVREM
            CLR   DIVREM+1
            LDB   #16
            STB   DIVCNT
UD16LOOP:   ASL   DIVNUM+1
            ROL   DIVNUM
            ROL   DIVREM+1
            ROL   DIVREM
            LDD   DIVREM
            SUBD  DIVDEN
            BLO   UDSKIP
            STD   DIVREM
            INC   DIVNUM+1
UDSKIP:     DEC   DIVCNT
            BNE   UD16LOOP
            RTS

; ------------------------------------------------------------
; Common part of /, MOD and /MOD: divide n1 by n2 with the sign
; handling for truncating division. THROW -10 if n2 is zero.
; DIVCOMMON   ( n1 n2 -- )
;    Inputs:
;        n1 and n2 on the data stack (n2 on top)
;    Outputs:
;        DIVNUM = quotient, DIVREM = remainder (data stack popped)
```

```
;    Registers: RegD changed; DIVNUM, DIVDEN, DIVREM, DNSIGN, DVSIGN
;        used.
; ------------------------------------------------------------
DIVCOMMON:  PULU  D                 ; n2
            STD   DIVDEN
            CMPD  #0
            BNE   DCOK
            LDD   #-10              ; -10
            PSHU  D
            JSR   THROWW
DCOK:       PULU  D                 ; n1
            STD   DIVNUM
            CLR   DVSIGN
            CLR   DNSIGN
            TST   DIVNUM
            BPL   DNPOS
            COM   DNSIGN
            COM   DVSIGN
            LDD   DIVNUM
            COMA
            COMB
            ADDD  #1
            STD   DIVNUM
DNPOS:      TST   DIVDEN
            BPL   DVPOS
            COM   DVSIGN
            LDD   DIVDEN
            COMA
            COMB
            ADDD  #1
            STD   DIVDEN
DVPOS:      JSR   UDIV16
            LDD   DIVNUM
            TST   DVSIGN
            BEQ   DQPOS
            COMA
            COMB
            ADDD  #1
            STD   DIVNUM
DQPOS:      LDD   DIVREM
            TST   DNSIGN
            BEQ   DCRPOS
            COMA
            COMB
            ADDD  #1
            STD   DIVREM
DCRPOS:     RTS

; ------------------------------------------------------------
; /  ( n1 n2 -- n3 )
; Quotient of n1 divided by n2. Division truncates toward zero; the
; remainder has the sign of the dividend. THROW -10 if the divisor is
; zero.
; ------------------------------------------------------------
SLASHW:     JSR   DIVCOMMON
            LDD   DIVNUM
            PSHU  D
            RTS

; ------------------------------------------------------------
; MOD  ( n1 n2 -- n3 )
; Remainder of n1 divided by n2. Division truncates toward zero; the
; remainder has the sign of the dividend. THROW -10 if the divisor is
; zero.
; ------------------------------------------------------------
MODW:       JSR   DIVCOMMON
            LDD   DIVREM
            PSHU  D
            RTS

; ------------------------------------------------------------
```

```
; /MOD  ( n1 n2 -- n3 n4 )
; Divide n1 by n2, giving the remainder n3 and the quotient n4.
; Division truncates toward zero; the remainder has the sign of the
; dividend. THROW -10 if the divisor is zero.
; ------------------------------------------------------------
SLASHMODW:  JSR   DIVCOMMON
            LDD   DIVREM
            PSHU  D
            LDD   DIVNUM
            PSHU  D
            RTS

; ------------------------------------------------------------
; 2/  ( x1 -- x2 )
; Arithmetic shift right one bit.
; ------------------------------------------------------------
TWOSLASHW:  LDD   ,U
            ASRA
            RORB
            STD   ,U
            RTS

; ------------------------------------------------------------
; Unsigned 16 x 16 to 32-bit multiply from four partial products.
; UMUL32
;    Inputs:
;        MAHI:MALO and MBHI:MBLO (unsigned factors)
;    Outputs:
;        PRODHI:PRODLO = product
;    Registers: RegA, RegB and RegD changed.
; ------------------------------------------------------------
UMUL32:     LDA   MALO
            LDB   MBLO
            MUL
            STD   PRODLO
            CLR   PRODHI
            CLR   PRODHI+1
            LDA   MAHI
            LDB   MBLO
            MUL
            ADDB  PRODLO
            STB   PRODLO
            ADCA  #0
            ADDA  PRODHI+1
            STA   PRODHI+1
            BCC   UM32A
            INC   PRODHI
UM32A:      LDA   MALO
            LDB   MBHI
            MUL
            ADDB  PRODLO
            STB   PRODLO
            ADCA  #0
            ADDA  PRODHI+1
            STA   PRODHI+1
            BCC   UM32B
            INC   PRODHI
UM32B:      LDA   MAHI
            LDB   MBHI
            MUL
            ADDD  PRODHI
            STD   PRODHI
            RTS

; ------------------------------------------------------------
; Unsigned 32 / 16 division (restoring, 32 iterations). Used by
; UM/MOD, SM/REM, FM/MOD, */ and */MOD.
; UDIV32
;    Inputs:
;        PRODHI:PRODLO = dividend, DIVDEN = divisor (unsigned)
;    Outputs:
```

```
;        PRODLO = quotient, DIVREM = remainder
;    Registers: RegD changed; DIVCNT used as the loop counter.
; Original comment: shadow UDIV32.1.
; ------------------------------------------------------------
UDIV32:     CLR   DIVREM
            CLR   DIVREM+1
            LDB   #32
            STB   DIVCNT
UD32LP:     ASL   PRODLO+1
            ROL   PRODLO
            ROL   PRODHI+1
            ROL   PRODHI
            ROL   DIVREM+1
            ROL   DIVREM
            BCS   UD32FORCE         ; 17th bit set. See bugfix: UDIV32.1
            LDD   DIVREM
            SUBD  DIVDEN
            BLO   UD32SKIP
UD32TAKE:   STD   DIVREM
            INC   PRODLO+1
            BRA   UD32SKIP
UD32FORCE:  LDD   DIVREM
            SUBD  DIVDEN
            BRA   UD32TAKE
UD32SKIP:   DEC   DIVCNT
            BNE   UD32LP
            RTS

; ------------------------------------------------------------
; Negate the 32-bit value in PRODHI:PRODLO.
; MNEG32
;    Inputs:
;        PRODHI:PRODLO
;    Outputs:
;        PRODHI:PRODLO negated
;    Registers: RegD changed.
; ------------------------------------------------------------
MNEG32:     LDD   PRODLO
            COMA
            COMB
            STD   PRODLO
            LDD   PRODHI
            COMA
            COMB
            STD   PRODHI
            LDD   PRODLO
            ADDD  #1
            STD   PRODLO
            BCC   MN32DONE
            LDD   PRODHI
            ADDD  #1
            STD   PRODHI
MN32DONE:   RTS

; ------------------------------------------------------------
; Common part of */ and */MOD: form the 32-bit product n1 * n2, then
; divide it by n3. The quotient is negated if an odd number of the
; three is negative, the remainder if the product is negative. THROW
; -10 if n3 is zero.
; STARSLASHCOMMON   ( n1 n2 n3 -- )
;    Inputs:
;        n1, n2 and n3 on the data stack (n3 on top)
;    Outputs:
;        PRODLO = quotient, DIVREM = remainder (data stack popped)
;    Registers: RegA, RegB and RegD changed; many scratch cells used.
; ------------------------------------------------------------
STARSLASHCOMMON:
            PULU  D                 ; n3
            STD   DIVDEN
            CMPD  #0
            BNE   SSOK
```

```
            LDD   #-10              ; -10
            PSHU  D
            JSR   THROWW
SSOK:       CLR   PSIGN
            TST   DIVDEN
            BPL   SSN3POS
            COM   PSIGN
            LDD   DIVDEN
            COMA
            COMB
            ADDD  #1
            STD   DIVDEN
SSN3POS:    LDA   #0
            STA   PRSIGN
            PULU  D                 ; n2
            STD   MSCR
            TST   MSCR
            BPL   SSN2POS
            COM   PSIGN
            COM   PRSIGN
            LDD   MSCR
            COMA
            COMB
            ADDD  #1
            STD   MSCR
SSN2POS:    LDA   MSCR
            STA   MBHI
            LDA   MSCR+1
            STA   MBLO
            PULU  D                 ; n1
            STD   MSCR
            TST   MSCR
            BPL   SSN1POS
            COM   PSIGN
            COM   PRSIGN
            LDD   MSCR
            COMA
            COMB
            ADDD  #1
            STD   MSCR
SSN1POS:    LDA   MSCR
            STA   MAHI
            LDA   MSCR+1
            STA   MALO
            JSR   UMUL32
            JSR   UDIV32
            LDD   PRODLO
            TST   PSIGN
            BEQ   SSQPOS
            COMA
            COMB
            ADDD  #1
            STD   PRODLO
SSQPOS:     LDD   DIVREM
            TST   PRSIGN
            BEQ   SSRPOS
            COMA
            COMB
            ADDD  #1
            STD   DIVREM
SSRPOS:     RTS

; ------------------------------------------------------------
; */  ( n1 n2 n3 -- n4 )
; Multiply n1 by n2 giving a 32-bit intermediate product, then divide
; by n3. The quotient is n4.
; ------------------------------------------------------------
STARSLASHW: JSR   STARSLASHCOMMON
            LDD   PRODLO
            PSHU  D
            RTS
```

```
; ------------------------------------------------------------
; */MOD  ( n1 n2 n3 -- n4 n5 )
; Multiply n1 by n2 giving a 32-bit intermediate product, then divide
; by n3, giving the remainder n4 and the quotient n5.
; ------------------------------------------------------------
STARSLASHMODW:
            JSR   STARSLASHCOMMON
            LDD   DIVREM
            PSHU  D
            LDD   PRODLO
            PSHU  D
            RTS

; ------------------------------------------------------------
; UM*  ( u1 u2 -- ud )
; Unsigned 16 x 16 to 32-bit multiply.
; ------------------------------------------------------------
UMSTARW:    PULU  D                 ; u2
            STD   MSCR
            PULU  D                 ; u1
            STA   MAHI
            STB   MALO
            LDD   MSCR
            STA   MBHI
            STB   MBLO
            JSR   UMUL32
            LDD   PRODLO
            PSHU  D
            LDD   PRODHI
            PSHU  D
            RTS

; ------------------------------------------------------------
; UM/MOD  ( ud u1 -- u2 u3 )
; Divide the unsigned double ud by u1, giving the remainder u2 and the
; quotient u3. THROW -10 if u1 is zero.
; ------------------------------------------------------------
UMSLASHMODW:
            PULU  D                 ; u1
            STD   DIVDEN
            CMPD  #0
            BNE   UMOK
            LDD   #-10              ; -10
            PSHU  D
            JSR   THROWW
UMOK:       PULU  D                 ; ud
            STD   PRODHI
            PULU  D
            STD   PRODLO
            JSR   UDIV32
            LDD   DIVREM
            PSHU  D
            LDD   PRODLO
            PSHU  D
            RTS

; ------------------------------------------------------------
; M*  ( n1 n2 -- d )
; Signed 16 x 16 to 32-bit multiply.
; ------------------------------------------------------------
MSTARW:     PULU  D                 ; n2
            STD   MSCR
            PULU  D                 ; n1
            CLR   MSIGN
            TSTA                    ; sign of n1. See bugfix: MSTARW.1
            BPL   MSN1POS
            COM   MSIGN
            COMA
            COMB
            ADDD  #1
```

```
MSN1POS:    STA   MAHI
            STB   MALO
            LDD   MSCR
            BPL   MSN2POS
            COM   MSIGN
            COMA
            COMB
            ADDD  #1
MSN2POS:    STA   MBHI
            STB   MBLO
            JSR   UMUL32
            TST   MSIGN
            BEQ   MSDONE
            JSR   MNEG32
MSDONE:     LDD   PRODLO
            PSHU  D
            LDD   PRODHI
            PSHU  D
            RTS

; ------------------------------------------------------------
; SM/REM  ( d n1 -- n2 n3 )
; Symmetric division of the double d by n1, giving the remainder n2
; and the quotient n3. The quotient is truncated toward zero and the
; remainder has the sign of d. THROW -10 if n1 is zero.
; ------------------------------------------------------------
SMSLASHREMW:
            PULU  D                 ; n1
            STD   DIVDEN
            CMPD  #0
            BNE   SMOK
            LDD   #-10              ; -10
            PSHU  D
            JSR   THROWW
SMOK:       PULU  D                 ; d
            STD   PRODHI
            PULU  D
            STD   PRODLO
            CLR   DNSIGN
            CLR   DVSIGN
            TST   PRODHI
            BPL   SMDPOS
            COM   DNSIGN
            COM   DVSIGN
            JSR   MNEG32
SMDPOS:     LDD   DIVDEN
            BPL   SMDVPOS
            COM   DVSIGN
            LDD   DIVDEN
            COMA
            COMB
            ADDD  #1
            STD   DIVDEN
SMDVPOS:    JSR   UDIV32
            LDD   DIVREM
            TST   DNSIGN
            BEQ   SMRPOS
            COMA
            COMB
            ADDD  #1
SMRPOS:     PSHU  D
            LDD   PRODLO
            TST   DVSIGN
            BEQ   SMQPOS
            COMA
            COMB
            ADDD  #1
SMQPOS:     PSHU  D
            RTS

; ------------------------------------------------------------
```

```
; FM/MOD  ( d n1 -- n2 n3 )
; Floored division of the double d by n1, giving the remainder n2 and
; the quotient n3. The quotient is rounded toward negative infinity
; and the remainder has the sign of n1. THROW -10 if n1 is zero.
; ------------------------------------------------------------
FMSLASHMODW:
            PULU  D                 ; n1
            STD   DIVDEN
            CMPD  #0
            BNE   FMOK
            LDD   #-10              ; -10
            PSHU  D
            JSR   THROWW
FMOK:       PULU  D                 ; d
            STD   PRODHI
            PULU  D
            STD   PRODLO
            CLR   DNSIGN
            CLR   DVSIGN
            CLR   DVOWNSIGN
            TST   PRODHI
            BPL   FMDPOS
            COM   DNSIGN
            COM   DVSIGN
            JSR   MNEG32
FMDPOS:     LDD   DIVDEN
            BPL   FMDVPOS
            COM   DVSIGN
            COM   DVOWNSIGN
            LDD   DIVDEN
            COMA
            COMB
            ADDD  #1
            STD   DIVDEN
FMDVPOS:    JSR   UDIV32
            TST   DVSIGN
            BEQ   FMNOFLOOR
            LDD   DIVREM
            BEQ   FMNOFLOOR
            LDD   PRODLO
            ADDD  #1
            STD   PRODLO
            LDD   DIVDEN
            SUBD  DIVREM
            STD   DIVREM
FMNOFLOOR:  LDD   DIVREM
            TST   DVOWNSIGN
            BEQ   FMRPOS
            COMA
            COMB
            ADDD  #1
FMRPOS:     PSHU  D
            LDD   PRODLO
            TST   DVSIGN
            BEQ   FMQPOS
            COMA
            COMB
            ADDD  #1
FMQPOS:     PSHU  D
            RTS

; ------------------------------------------------------------
; D+  ( d1 d2 -- d3 )
; Add two doubles.
; ------------------------------------------------------------
DPLUSW:     PULU  D                 ; d2 high
            STD   MSCR
            PULU  D                 ; d2 low
            STD   MSCR2
            PULU  D                 ; d1 high
            STD   MSCR3
```

```
            PULU  D                ; d1 low
            ADDD  MSCR2
            STD   MSCR4
            BCC   DPNOCY
            LDD   MSCR3
            ADDD  MSCR
            ADDD  #1
            BRA   DPHIDONE
DPNOCY:     LDD   MSCR3
            ADDD  MSCR
DPHIDONE:   STD   MSCR3
            LDD   MSCR4
            PSHU  D
            LDD   MSCR3
            PSHU  D
            RTS

; ---------------------------------------------------------------
; D-  ( d1 d2 -- d3 )
; Subtract d2 from d1.
; ---------------------------------------------------------------
DMINUSW:    PULU  D                ; d2 high
            STD   MSCR
            PULU  D                ; d2 low
            STD   MSCR2
            PULU  D                ; d1 high
            STD   MSCR3
            PULU  D                ; d1 low
            SUBD  MSCR2
            STD   MSCR4
            BCC   DMNOBOR
            LDD   MSCR3
            SUBD  MSCR
            SUBD  #1
            BRA   DMHIDONE
DMNOBOR:    LDD   MSCR3
            SUBD  MSCR
DMHIDONE:   STD   MSCR3
            LDD   MSCR4
            PSHU  D
            LDD   MSCR3
            PSHU  D
            RTS

; ---------------------------------------------------------------
; DNEGATE  ( d1 -- d2 )
; Negate a double.
; ---------------------------------------------------------------
DNEGATEW:   PULU  D                ; d1 high
            STD   PRODHI
            PULU  D                ; d1 low
            STD   PRODLO
            JSR   MNEG32
            LDD   PRODLO
            PSHU  D
            LDD   PRODHI
            PSHU  D
            RTS

; ---------------------------------------------------------------
; DABS  ( d -- ud )
; Absolute value of a double.
; ---------------------------------------------------------------
DABSW:      PULU  D                ; d high
            STD   PRODHI
            PULU  D                ; d low
            STD   PRODLO
            TST   PRODHI
            BPL   DABSDONE
            JSR   MNEG32
DABSDONE:   LDD   PRODLO
```

```
            PSHU  D
            LDD   PRODHI
            PSHU  D
            RTS

; ---------------------------------------------------------------
; M+  ( d1 n -- d2 )
; Add the single n, sign-extended, to the double d1.
; ---------------------------------------------------------------
MPLUSW:     PULU  D                 ; n
            STD   MSCR2
            BPL   MPPOSN
            LDD   #-1
            BRA   MPSIGNED
MPPOSN:     LDD   #0
MPSIGNED:   STD   MSCR
            PULU  D                 ; d1 high
            STD   MSCR3
            PULU  D                 ; d1 low
            ADDD  MSCR2
            STD   MSCR4
            BCC   MPNOCY
            LDD   MSCR3
            ADDD  MSCR
            ADDD  #1
            BRA   MPHIDONE
MPNOCY:     LDD   MSCR3
            ADDD  MSCR
MPHIDONE:   STD   MSCR3
            LDD   MSCR4
            PSHU  D
            LDD   MSCR3
            PSHU  D
            RTS

; ---------------------------------------------------------------
; S>D  ( n -- d )
; Sign-extend a single to a double.
; ---------------------------------------------------------------
STODW:      PULU  D                 ; n
            PSHU  D
            TSTA                    ; sign of n. See bugfix: STODW.1
            BPL   SDPOS
            LDD   #-1
            BRA   SDPUSH
SDPOS:      LDD   #0
SDPUSH:     PSHU  D
            RTS

; ---------------------------------------------------------------
; D>S  ( d -- n )
; Drop the high cell of a double.
; ---------------------------------------------------------------
DTOSW:      PULU  D                 ; drop high cell
            RTS

; ---------------------------------------------------------------
; DMAX  ( d1 d2 -- d3 )
; The greater of two signed doubles.
; ---------------------------------------------------------------
DMAXW:      PULU  D                 ; d2 high
            STD   MSCR
            PULU  D                 ; d2 low
            STD   MSCR2
            PULU  D                 ; d1 high
            STD   MSCR3
            PULU  D                 ; d1 low
            STD   MSCR4
            LDD   MSCR3
            CMPD  MSCR
            BGT   DMXD1
```

```
            BLT   DMXD2
            LDD   MSCR4
            CMPD  MSCR2
            BHS   DMXD1
DMXD2:      LDD   MSCR2
            PSHU  D
            LDD   MSCR
            PSHU  D
            RTS
DMXD1:      LDD   MSCR4
            PSHU  D
            LDD   MSCR3
            PSHU  D
            RTS

; ------------------------------------------------------------
; DMIN  ( d1 d2 -- d3 )
; The lesser of two signed doubles.
; ------------------------------------------------------------
DMINW:      PULU  D                 ; d2 high
            STD   MSCR
            PULU  D                 ; d2 low
            STD   MSCR2
            PULU  D                 ; d1 high
            STD   MSCR3
            PULU  D                 ; d1 low
            STD   MSCR4
            LDD   MSCR3
            CMPD  MSCR
            BLT   DMND1
            BGT   DMND2
            LDD   MSCR4
            CMPD  MSCR2
            BLS   DMND1
DMND2:      LDD   MSCR2
            PSHU  D
            LDD   MSCR
            PSHU  D
            RTS
DMND1:      LDD   MSCR4
            PSHU  D
            LDD   MSCR3
            PSHU  D
            RTS
```

8.17 Logic, Shifts & Address Arithmetic

```
; ============================================================
; SECTION 16: LOGIC / SHIFTS / ADDRESS ARITHMETIC
; ============================================================
; ------------------------------------------------------------
; AND  ( x1 x2 -- x3 )
; Bitwise AND.
; ------------------------------------------------------------
ANDW:       PULU  D                 ; x2
            ANDA  ,U
            ANDB  1,U
            STD   ,U
            RTS

; ------------------------------------------------------------
; OR  ( x1 x2 -- x3 )
; Bitwise OR.
; ------------------------------------------------------------
ORW:        PULU  D                 ; x2
            ORA   ,U
            ORB   1,U
            STD   ,U
            RTS
```

```
; ------------------------------------------------------------
; XOR  ( x1 x2 -- x3 )
; Bitwise exclusive OR.
; ------------------------------------------------------------
XORW:       PULU  D                 ; x2
            EORA  ,U
            EORB  1,U
            STD   ,U
            RTS

; ------------------------------------------------------------
; INVERT  ( x1 -- x2 )
; Ones complement.
; ------------------------------------------------------------
INVERTW:    LDD   ,U
            COMA
            COMB
            STD   ,U
            RTS

; ------------------------------------------------------------
; LSHIFT  ( x1 u -- x2 )
; Shift x1 left u bits (only the low byte of u is used).
; ------------------------------------------------------------
LSHIFTW:    PULU  D                 ; u
            STB   SHCNT
            LDD   ,U
LSLOOP:     LDB   SHCNT
            BEQ   LSDONE
            ASL   1,U
            ROL   ,U
            DEC   SHCNT
            BRA   LSLOOP
LSDONE:     RTS

; ------------------------------------------------------------
; RSHIFT  ( x1 u -- x2 )
; Logical shift of x1 right u bits (only the low byte of u is used).
; ------------------------------------------------------------
RSHIFTW:    PULU  D                 ; u
            STB   SHCNT
RSLOOP:     LDB   SHCNT
            BEQ   RSDONE
            LSR   ,U
            ROR   1,U
            DEC   SHCNT
            BRA   RSLOOP
RSDONE:     RTS

; ------------------------------------------------------------
; CELLS  ( n1 -- n2 )
; Convert a cell count to address units (multiply by 2).
; ------------------------------------------------------------
CELLSW:     LDD   ,U
            ASLB
            ROLA
            STD   ,U
            RTS

; ------------------------------------------------------------
; CELL+  ( a-addr1 -- a-addr2 )
; Add the size of a cell (2).
; ------------------------------------------------------------
CELLPLUSW:  LDD   ,U
            ADDD  #2
            STD   ,U
            RTS

; ------------------------------------------------------------
; CHARS  ( n1 -- n2 )
; Convert a character count to address units. A character is one
```

```
; address unit, so nothing is done.
; ---------------------------------------------------------------
CHARSW:     RTS

; ---------------------------------------------------------------
; CHAR+  ( c-addr1 -- c-addr2 )
; Add the size of a character (1).
; ---------------------------------------------------------------
CHARPLUSW:  LDD   ,U
            ADDD  #1
            STD   ,U
            RTS

; ---------------------------------------------------------------
; ALIGN  ( -- )
; Nothing to do: the 6809 has no alignment restrictions.
; ---------------------------------------------------------------
ALIGNW:     RTS

; ---------------------------------------------------------------
; ALIGNED  ( addr -- a-addr )
; Nothing to do: the 6809 has no alignment restrictions.
; ---------------------------------------------------------------
ALIGNEDW:   RTS
```

8.18 Comparison

```
; ===============================================================
; SECTION 17: COMPARISON
; ===============================================================
; ---------------------------------------------------------------
; =  ( x1 x2 -- flag )
; True if x1 equals x2.
; ---------------------------------------------------------------
EQUALW:     PULU  D                 ; x2
            CMPD  ,U
            BEQ   EQTRUE
            LDD   #FALSEV
            STD   ,U
            RTS
EQTRUE:     LDD   #TRUEV
            STD   ,U
            RTS

; ---------------------------------------------------------------
; <  ( n1 n2 -- flag )
; True if n1 is less than n2 (signed).
; ---------------------------------------------------------------
LESSW:      PULU  D                 ; n2
            STD   MSCR
            LDD   ,U
            CMPD  MSCR
            BLT   LTTRUE
            LDD   #FALSEV
            STD   ,U
            RTS
LTTRUE:     LDD   #TRUEV
            STD   ,U
            RTS

; ---------------------------------------------------------------
; >  ( n1 n2 -- flag )
; True if n1 is greater than n2 (signed).
; ---------------------------------------------------------------
GREATERW:   PULU  D                 ; n2
            STD   MSCR
            LDD   ,U
            CMPD  MSCR
            BGT   GTTRUE
            LDD   #FALSEV
```

```
            STD   ,U
            RTS
GTTRUE:     LDD   #TRUEV
            STD   ,U
            RTS

; ------------------------------------------------------------
; 0=  ( x -- flag )
; True if x is zero.
; ------------------------------------------------------------
ZEROEQW:    LDD   ,U                ; x
            BEQ   ZEQTRUE
            LDD   #FALSEV
            STD   ,U
            RTS
ZEQTRUE:    LDD   #TRUEV
            STD   ,U
            RTS

; ------------------------------------------------------------
; 0<  ( n -- flag )
; True if n is negative.
; ------------------------------------------------------------
ZEROLTW:    LDD   ,U                ; n
            BMI   ZLTTRUE
            LDD   #FALSEV
            STD   ,U
            RTS
ZLTTRUE:    LDD   #TRUEV
            STD   ,U
            RTS

; ------------------------------------------------------------
; U<  ( u1 u2 -- flag )
; True if u1 is less than u2 (unsigned).
; ------------------------------------------------------------
ULESSW:     PULU  D                 ; u2
            STD   MSCR
            LDD   ,U
            CMPD  MSCR
            BLO   ULTRUE
            LDD   #FALSEV
            STD   ,U
            RTS
ULTRUE:     LDD   #TRUEV
            STD   ,U
            RTS

; ------------------------------------------------------------
; <>  ( x1 x2 -- flag )
; True if x1 is not equal to x2 (= then invert).
; ------------------------------------------------------------
NOTEQUALW:  JSR   EQUALW
            LDD   ,U
            COMA
            COMB
            STD   ,U
            RTS

; ------------------------------------------------------------
; 0<>  ( x -- flag )
; True if x is non-zero (0= then invert).
; ------------------------------------------------------------
ZERONEW:    JSR   ZEROEQW
            LDD   ,U
            COMA
            COMB
            STD   ,U
            RTS

; ------------------------------------------------------------
```

```
; 0>  ( n -- flag )
; True if n is greater than zero.
; ------------------------------------------------------------
ZEROGTW:    LDD   ,U                ; n
            BEQ   ZGTFALSE
            BMI   ZGTFALSE
            LDD   #TRUEV
            STD   ,U
            RTS
ZGTFALSE:   LDD   #FALSEV
            STD   ,U
            RTS

; ------------------------------------------------------------
; U>  ( u1 u2 -- flag )
; True if u1 is greater than u2 (unsigned).
; ------------------------------------------------------------
UGREATERW:  PULU  D                 ; u2
            STD   MSCR
            LDD   ,U
            CMPD  MSCR
            BLO   UGFALSE
            BEQ   UGFALSE
            LDD   #TRUEV
            STD   ,U
            RTS
UGFALSE:    LDD   #FALSEV
            STD   ,U
            RTS

; ------------------------------------------------------------
; WITHIN  ( n1 n2 n3 -- flag )
; True if n2 <= n1 < n3, evaluated as the unsigned comparison (n1 -
; n2) U< (n3 - n2), so it also works for ranges that wrap.
; ------------------------------------------------------------
WITHINW:    PULU  D                 ; n3
            STD   MSCR
            PULU  D                 ; n2
            STD   MSCR2
            LDD   ,U
            SUBD  MSCR2
            STD   MSCR3
            LDD   MSCR
            SUBD  MSCR2
            STD   MSCR
            LDD   MSCR3
            CMPD  MSCR
            BLO   WITHTRUE
            LDD   #FALSEV
            STD   ,U
            RTS
WITHTRUE:   LDD   #TRUEV
            STD   ,U
            RTS

; ------------------------------------------------------------
; D=  ( d1 d2 -- flag )
; True if the doubles are equal.
; ------------------------------------------------------------
DEQUALW:    PULU  D                 ; d2 high
            STD   MSCR
            PULU  D                 ; d2 low
            STD   MSCR2
            PULU  D                 ; d1 high
            STD   MSCR3
            PULU  D                 ; d1 low
            CMPD  MSCR2
            BNE   DEQFALSE
            LDD   MSCR3
            CMPD  MSCR
            BNE   DEQFALSE
```

```
            LDD   #TRUEV
            PSHU  D
            RTS
DEQFALSE:   LDD   #FALSEV
            PSHU  D
            RTS

; ------------------------------------------------------------
; D<  ( d1 d2 -- flag )
; True if d1 is less than d2 (signed): compare the high cells signed,
; then the low cells unsigned.
; ------------------------------------------------------------
DLESSW:     PULU  D                 ; d2 high
            STD   MSCR
            PULU  D                 ; d2 low
            STD   MSCR2
            PULU  D                 ; d1 high
            STD   MSCR3
            PULU  D                 ; d1 low
            STD   MSCR4
            LDD   MSCR3
            CMPD  MSCR
            BLT   DLTRUE
            BGT   DLFALSE
            LDD   MSCR4
            CMPD  MSCR2
            BLO   DLTRUE
DLFALSE:    LDD   #FALSEV
            PSHU  D
            RTS
DLTRUE:     LDD   #TRUEV
            PSHU  D
            RTS

; ------------------------------------------------------------
; DU<  ( ud1 ud2 -- flag )
; True if ud1 is less than ud2 (unsigned).
; ------------------------------------------------------------
DULESSW:    PULU  D                 ; ud2 high
            STD   MSCR
            PULU  D                 ; ud2 low
            STD   MSCR2
            PULU  D                 ; ud1 high
            STD   MSCR3
            PULU  D                 ; ud1 low
            STD   MSCR4
            LDD   MSCR3
            CMPD  MSCR
            BLO   DULTRUE
            BHI   DULFALSE
            LDD   MSCR4
            CMPD  MSCR2
            BLO   DULTRUE
DULFALSE:   LDD   #FALSEV
            PSHU  D
            RTS
DULTRUE:    LDD   #TRUEV
            PSHU  D
            RTS

; ------------------------------------------------------------
; D0=  ( xd -- flag )
; True if the double is zero.
; ------------------------------------------------------------
DZEROEQW:   LDD   ,U                ; high cell
            BNE   DZEFALSE
            LDD   2,U               ; low cell
            BNE   DZEFALSE
            LDD   #TRUEV
            BRA   DZEDONE
DZEFALSE:   LDD   #FALSEV
```

```
DZEDONE:    LEAU  2,U
            STD   ,U
            RTS

; ------------------------------------------------------------
; D0<  ( d -- flag )
; True if the double is negative.
; ------------------------------------------------------------
DZEROLTW:   LDD   ,U                ; high cell holds the sign
            BMI   DZLTRUE
            LDD   #FALSEV
            BRA   DZLDONE
DZLTRUE:    LDD   #TRUEV
DZLDONE:    LEAU  2,U
            STD   ,U
            RTS

; ------------------------------------------------------------
; D2*  ( xd1 -- xd2 )
; Shift the double left one bit.
; ------------------------------------------------------------
DTWOSTARW:  LDD   2,U               ; low cell
            ASLB
            ROLA
            STD   2,U               ; STD leaves the carry alone
            LDD   ,U                ; neither does LDD
            ROLB
            ROLA
            STD   ,U
            RTS

; ------------------------------------------------------------
; D2/  ( xd1 -- xd2 )
; Arithmetic shift of the double right one bit; the sign is kept.
; ------------------------------------------------------------
DTWOSLASHW: LDD   ,U                ; high cell
            ASRA
            RORB
            STD   ,U
            LDD   2,U               ; low cell takes the bit shifted out
            RORA
            RORB
            STD   2,U
            RTS

; ------------------------------------------------------------
; M*/  ( d1 n1 +n2 -- d2 )
; Multiply d1 by n1 into a 48-bit intermediate product and divide it
; by n2. The quotient is truncated toward zero (like */ and symmetric
; division in this system) and its low 32 bits are returned. THROW -10
; if n2 is zero. Method: take the magnitudes; form |d1.lo|*|n1| and
; |d1.hi|*|n1| with UMUL32 and add them into three words P2:P1:P0;
; then divide P2, then R:P1, then R:P0 by |n2| with UDIV32, each a
; 32/16 division whose quotient fits in 16 bits. The sign is applied
; to the 32-bit result at the end. See shadow M*/W.1.
; M*/W
;    Inputs:
;        d1 n1 n2 on the data stack (n2 on top)
;    Outputs:
;        d2 on the data stack
;    Registers: RegD, RegX changed; PRODHI/PRODLO, DIVDEN, DIVREM,
;        MAHI-MBLO used. S frame: 0,S sign, 1,S P0, 3,S P1, 5,S P2.
; ------------------------------------------------------------
MSTARSLASHW:
            LDD   ,U
            BNE   MSLOK
            LDD   #-10              ; -10, division by zero
            PSHU  D
            JSR   THROWW
MSLOK:      LEAS  -7,S
            CLR   ,S                ; sign of the result
```

```
            LDD   ,U              ; |n2|
            BPL   MSLN1
            COM   ,S
            COMA
            COMB
            ADDD  #1
            STD   ,U
MSLN1:      LDD   2,U             ; |n1|
            BPL   MSLD1
            COM   ,S
            COMA
            COMB
            ADDD  #1
            STD   2,U
MSLD1:      LDD   4,U             ; |d1|
            BPL   MSLMUL
            COM   ,S
            LDD   6,U
            STD   PRODLO
            LDD   4,U
            STD   PRODHI
            JSR   MNEG32
            LDD   PRODLO
            STD   6,U
            LDD   PRODHI
            STD   4,U
MSLMUL:     LDD   6,U             ; d1 low * n1
            STA   MAHI
            STB   MALO
            LDD   2,U
            STA   MBHI
            STB   MBLO
            JSR   UMUL32
            LDD   PRODLO
            STD   1,S             ; P0
            LDD   PRODHI
            STD   3,S             ; P1 so far
            LDD   4,U             ; d1 high * n1
            STA   MAHI
            STB   MALO
            LDD   2,U
            STA   MBHI
            STB   MBLO
            JSR   UMUL32
            LDD   3,S
            ADDD  PRODLO
            STD   3,S             ; P1
            LDD   PRODHI
            ADCB  #0
            ADCA  #0
            STD   5,S             ; P2
            LDD   ,U              ; divisor
            STD   DIVDEN
            CLR   PRODHI
            CLR   PRODHI+1
            LDD   5,S
            STD   PRODLO
            JSR   UDIV32          ; remainder only; the quotient is above bit 31
            LDD   DIVREM
            STD   PRODHI
            LDD   3,S
            STD   PRODLO
            JSR   UDIV32
            LDD   PRODLO
            STD   3,S             ; q1
            LDD   DIVREM
            STD   PRODHI
            LDD   1,S
            STD   PRODLO
            JSR   UDIV32          ; PRODLO = q0
            LDD   3,S
```

```
            STD   PRODHI            ; PRODHI:PRODLO = q1:q0
            TST   ,S
            BEQ   MSLPOS
            JSR   MNEG32
MSLPOS:     LEAU  4,U               ; drop n2 and n1
            LDD   PRODHI
            STD   ,U
            LDD   PRODLO
            STD   2,U
            LEAS  7,S
            RTS
```

8.19 Memory

```
; ============================================================
; SECTION 18: MEMORY (fetch/store, block ops)
; ============================================================
; ------------------------------------------------------------
; !  ( x a-addr -- )
; Store x at a-addr.
; ------------------------------------------------------------
STOREW:     PULU  X                 ; a-addr
            PULU  D                 ; x
            STD   ,X
            RTS

; ------------------------------------------------------------
; C@  ( c-addr -- char )
; Fetch the byte at c-addr.
; ------------------------------------------------------------
CFETCHW:    PULU  X                 ; c-addr
            LDB   ,X
            CLRA
            PSHU  D
            RTS

; ------------------------------------------------------------
; C!  ( char c-addr -- )
; Store the low byte of char at c-addr.
; ------------------------------------------------------------
CSTOREW:    PULU  X                 ; c-addr
            PULU  D                 ; char
            STB   ,X
            RTS

; ------------------------------------------------------------
; +!  ( n a-addr -- )
; Add n to the cell at a-addr.
; ------------------------------------------------------------
PLUSSTOREW: PULU  X                 ; a-addr
            PULU  D                 ; n
            ADDD  ,X
            STD   ,X
            RTS

; ------------------------------------------------------------
; 2@  ( a-addr -- x1 x2 )
; Fetch the cell pair at a-addr: x2 from a-addr (on top) and x1 from
; a-addr+2.
; ------------------------------------------------------------
TWOFETCHW:  PULU  X                 ; a-addr. See bugfix: TWOFETCHW.1
            LDD   2,X
            PSHU  D
            LDD   ,X
            PSHU  D
            RTS

; ------------------------------------------------------------
; 2!  ( x1 x2 a-addr -- )
; Store the cell pair: x2 at a-addr and x1 at a-addr+2.
```

```
; ------------------------------------------------------------
TWOSTOREW:  PULU  X                 ; a-addr
            PULU  D                 ; x2
            STD   ,X
            PULU  D                 ; x1
            STD   2,X
            RTS

; ------------------------------------------------------------
; CMOVE  ( c-addr1 c-addr2 u -- )
; Copy u bytes from c-addr1 to c-addr2, starting at the low addresses.
; ------------------------------------------------------------
CMOVEW:     LDD   ,U                ; u. Parameters stay on the data stack,
            BEQ   CMDONE            ; addressed relative to U: u at 0, c-addr2
            LDX   4,U               ; at 2, c-addr1 at 4. See shadow CMOVEW.2
            LEAX  D,X               ; end address = c-addr1 + u
            STX   ,U                ; keep it in u's old slot as the loop limit
            LDX   4,U               ; X = source
            LDY   2,U               ; Y = destination
CMVLOOP:    LDA   ,X+
            STA   ,Y+
            CMPX  ,U                ; reached the end address?
            BNE   CMVLOOP
CMDONE:     LEAU  6,U               ; drop the three parameters
            RTS

; ------------------------------------------------------------
; CMOVE>  ( c-addr1 c-addr2 u -- )
; Copy u bytes from c-addr1 to c-addr2, starting at the high
; addresses.
; ------------------------------------------------------------
CMOVEGTW:   LDD   ,U                ; u. Parameters stay on the data stack:
            BEQ   CGDONE            ; u at 0, c-addr2 at 2, c-addr1 at 4
            LDX   4,U
            LEAX  D,X               ; X = one past the end of the source
            LDY   2,U
            LEAY  D,Y               ; Y = one past the end of the destination
CGLOOP:     LDA   ,-X
            STA   ,-Y
            CMPX  4,U               ; back at the start of the source?
            BNE   CGLOOP
CGDONE:     LEAU  6,U               ; drop the three parameters
            RTS

; ------------------------------------------------------------
; MOVE  ( addr1 addr2 u -- )
; Copy u bytes from addr1 to addr2, choosing CMOVE or CMOVE> so that
; overlapping regions are handled correctly.
; ------------------------------------------------------------
MOVEW:      LDD   2,U               ; addr2
            CMPD  4,U               ; addr1
            BHI   CMOVEGTW          ; addr2 above addr1: copy from the top
            JMP   CMOVEW

; ------------------------------------------------------------
; FILL  ( c-addr u char -- )
; Store char in each of u bytes starting at c-addr.
; ------------------------------------------------------------
FILLW:      LDD   2,U               ; u. char is at 0, c-addr at 4
            BEQ   FDONE
            LDX   4,U
            LEAY  D,X               ; end address
            STY   2,U               ; keep it in u's slot as the loop limit
            LDA   1,U               ; fill byte. See bugfix: FILLW.1
FILLOOP:    STA   ,X+
            CMPX  2,U
            BNE   FILLOOP
FDONE:      LEAU  6,U               ; drop the three parameters
            RTS

; ------------------------------------------------------------
```

```
; ERASE  ( c-addr u -- )
; Fill u bytes starting at c-addr with zero.
; ------------------------------------------------------------
ERASEW:     LDD   #0
            PSHU  D
            JMP   FILLW

; ------------------------------------------------------------
; BLANK  ( c-addr u -- )
; Store a space in each of u bytes starting at c-addr.
; ------------------------------------------------------------
BLANKW:     LDD   #32
            PSHU  D
            JMP   FILLW
```

8.20 String Words

```
; ============================================================
; SECTION 19: STRING WORDS
; ============================================================
; ------------------------------------------------------------
; Run-time part of S". The counted string follows the call: push its
; address and length and skip over it.
; DOSTR   ( -- c-addr u )
;    Inputs:
;        return address on S points at the count byte
;    Outputs:
;        c-addr and u on the data stack; execution resumes after the
;        string
;    Registers: RegB, RegD and RegX changed.
; ------------------------------------------------------------
DOSTR:      PULS  X
            LDB   ,X
            LEAX  1,X
            PSHU  X
            CLRA
            PSHU  D
            LEAX  B,X
            PSHS  X
            RTS

; ------------------------------------------------------------
; Parse the text up to the next double quote for S", ." and ABORT".
; WORD cannot be used: it skips leading delimiters, so an empty string
; (S" ") ran on to the next quote in the source. PARSE does not skip.
; PARSEQUOTE
;    Inputs:
;        none (parses from >IN)
;    Outputs:
;        c-addr u on the data stack: the text in the input buffer.
;        THROW -18 if the text is longer than 255 characters.
;    Registers: RegA, RegB, RegX, RegY changed.
; ------------------------------------------------------------
PARSEQUOTE: LDD   #34
            PSHU  D
            JSR   PARSEW
            TST   ,U               ; high byte of the length
            BNE   PQLONG
            RTS
PQLONG:     LDD   #-18
            PSHU  D
            JMP   THROWW

; ------------------------------------------------------------
; Compile a call to a run-time word followed by a counted string.
; CSTRCOMP   ( c-addr u -- )
;    Inputs:
;        RegD = xt of the run-time word (DOSTR, DOTSTR, DOABORTQUOTE)
;        c-addr u on the data stack
;    Outputs:
```

```
;        JSR xt, the count byte and the text compiled at CODEHERE
;    Registers: RegA, RegB, RegX, RegY changed.
; -------------------------------------------------------------
CSTRCOMP:   PSHU  D
            JSR   CCALL
            PULU  D                 ; u
            PULU  Y                 ; c-addr
            LDX   CODEHERE
            STB   ,X+
            TSTB
            BEQ   CSEND
CSCPY:      LDA   ,Y+
            STA   ,X+
            DECB
            BNE   CSCPY
CSEND:      STX   CODEHERE
            RTS

; -------------------------------------------------------------
; S"  ( "ccc<quote>" -- c-addr u )
; Parse up to the closing quote (PARSEQUOTE). When compiling, compile
; DOSTR followed by the counted string. When interpreting, copy the
; string to PAD and return its address and length. An empty string is
; valid. See shadow SQUOTEW.3.
; -------------------------------------------------------------
SQUOTEW:    JSR   PARSEQUOTE        ; See bugfix: SQUOTEW.3
SQSTATE:    LDD   STATE
            BEQ   SQINTERP
            LDD   #DOSTR
            JMP   CSTRCOMP

            ; Interpreting: copy the string to PAD. Original comment: shadow
            ; SQUOTEW.2.
SQINTERP:
            JSR   PADW
            PULU  X                 ; PAD
            PULU  D                 ; u
            PULU  Y                 ; c-addr
            PSHU  X                 ; result c-addr
            PSHU  D                 ; result u
            TSTB
            BEQ   SQIEND
SQICPY:     LDA   ,Y+
            STA   ,X+
            DECB
            BNE   SQICPY
SQIEND:     RTS

; -------------------------------------------------------------
; S\"  ( "ccc<quote>" -- c-addr u )
; As S", but the text may contain escape sequences introduced by a
; backslash, and a backslash-quote does not end the string. The
; translated text is built at PAD and then handled exactly like S" text
; (SQSTATE). The escapes are: \a 7, \b 8, \e 27, \f 12, \l 10, \m the
; pair 13 10, \n 10, \q 34, \r 13, \t 9, \v 11, \z 0, \" 34, \\ 92 and
; \xhh (one or two hex digits). Any other character after a backslash
; stands for itself. An unterminated string ends at the end of the line.
; A result over 255 characters throws -18.
; Frame on S: 0,S input end, 2,S output start, 4,S output pointer.
; -------------------------------------------------------------
SQUOTEESCW: JSR   PADW              ; leaves PAD on the data stack
            LDD   ,U
            PSHS  D                 ; output pointer
            PSHS  D                 ; output start
            LDD   SRCADDR
            ADDD  SRCLEN
            PSHS  D                 ; input end
            LDD   SRCADDR
            ADDD  TOIN
            TFR   D,Y               ; input pointer
SQEL:       CMPY  0,S
```

```
            BHS   SQEND
            LDA   ,Y+
            CMPA  #34
            BEQ   SQEND
            CMPA  #92
            BNE   SQEPUT
            CMPY  0,S
            BHS   SQEND
            LDA   ,Y+               ; the escaped character
            CMPA  #'x'
            BEQ   SQEHEX
            CMPA  #'m'
            BEQ   SQEM
            LDX   #SQETAB
SQEFND:     TST   ,X
            BEQ   SQEPUT            ; not in the table: stands for itself
            CMPA  ,X++
            BNE   SQEFND
            LDA   -1,X
SQEPUT:     LDX   4,S
            STA   ,X+
            STX   4,S
            BRA   SQEL
SQEM:       LDX   4,S
            LDA   #13
            STA   ,X+
            LDA   #10
            STA   ,X+
            STX   4,S
            BRA   SQEL
SQEHEX:     BSR   SQEHD             ; first digit
            BCS   SQEL              ; none: nothing is added
            LEAY  1,Y
            TFR   A,B
            BSR   SQEHD             ; second digit
            BCC   SQEH2
            TFR   B,A               ; one digit only
            BRA   SQEPUT
SQEH2:      LEAY  1,Y
            ASLB
            ASLB
            ASLB
            ASLB
            PSHS  B
            ADDA  ,S+
            BRA   SQEPUT
; Hex digit at ,Y: value in A, carry clear; carry set if not a digit
; or the input is exhausted. Y is not advanced. RegB is preserved.
SQEHD:      CMPY  2,S               ; input end (one return address down)
            BHS   SQEHNO
            LDA   ,Y
            CMPA  #'0'
            BLO   SQEHNO
            CMPA  #'9'
            BLS   SQEHDG
            ANDA  #$DF              ; to upper case
            CMPA  #'A'
            BLO   SQEHNO
            CMPA  #'F'
            BHI   SQEHNO
            SUBA  #'A'-10
            ANDCC #$FE
            RTS
SQEHDG:     SUBA  #'0'
            ANDCC #$FE
            RTS
SQEHNO:     ORCC  #1
            RTS
SQEND:      TFR   Y,D
            SUBD  SRCADDR
            STD   TOIN              ; resume after the closing quote
```

```
            LDD   4,S
            SUBD  2,S               ; length
            LEAS  6,S
            TSTA
            BNE   SQEOVF
            PSHU  D                 ; ( c-addr u )
            JMP   SQSTATE
SQEOVF:     LDD   #-18
            PSHU  D
            JMP   THROWW

; Escape table: character, value. A zero character ends it.
SQETAB:     FCB   'a',7
            FCB   'b',8
            FCB   'e',27
            FCB   'f',12
            FCB   'l',10
            FCB   'n',10
            FCB   'q',34
            FCB   'r',13
            FCB   't',9
            FCB   'v',11
            FCB   'z',0
            FCB   0

; ------------------------------------------------------------
; CONVERT  ( ud1 c-addr1 -- ud2 c-addr2 )
; Obsolescent. Convert the digits after c-addr1 using >NUMBER with no
; length limit; c-addr2 is the first character that is not a digit.
; ------------------------------------------------------------
CONVERTW:   LDD   ,U
            ADDD  #1
            STD   ,U                ; skip the character at c-addr1
            LDD   #$FFFF
            PSHU  D                 ; u
            JSR   TONUMBERW
            LEAU  2,U               ; drop u2
            RTS

; ------------------------------------------------------------
; Run-time part of ."  The counted string follows the call: TYPE it
; and skip over it.
; DOTSTR
;    Inputs:
;        return address on S points at the count byte
;    Outputs:
;        string displayed; execution resumes after the string
;    Registers: RegB, RegD and RegX changed.
; ------------------------------------------------------------
DOTSTR:     PULS  X
            LDB   ,X
            LEAX  1,X
            PSHU  X
            CLRA
            PSHU  D
            LEAX  B,X
            PSHS  X
            JMP   TYPEW

; ------------------------------------------------------------
; ."  ( "ccc<quote>" -- )
; Compile DOTSTR followed by the string up to the closing quote as a
; counted string.
; ------------------------------------------------------------
DOTQUOTEW:  JSR   PARSEQUOTE        ; See bugfix: SQUOTEW.3
            LDD   #DOTSTR
            JMP   CSTRCOMP

; ------------------------------------------------------------
; Run-time part of C"  The counted string follows the call: push its
; address and skip over it.
```

```
; DOCSTR
;    Outputs:
;        ( -- c-addr ) the count byte address; execution resumes after
;        the string
;    Registers: RegB, RegD and RegX changed.
; ------------------------------------------------------------
DOCSTR:     PULS  X
            PSHU  X
            LDB   ,X
            LEAX  1,X
            LEAX  B,X
            PSHS  X
            RTS

; ------------------------------------------------------------
; C"  ( "ccc<quote>" -- )  compile: ( -- c-addr ) run-time
; Compile DOCSTR followed by the string up to the closing quote as a
; counted string. THROW -14 if not compiling.
; ------------------------------------------------------------
CQUOTEW:    LDD   STATE
            BNE   CQOK
            LDD   #-14
            PSHU  D
            JSR   THROWW
CQOK:       JSR   PARSEQUOTE
            LDD   #DOCSTR
            JMP   CSTRCOMP

; ------------------------------------------------------------
; .(  ( "ccc<paren>" -- )
; Immediate. Parse up to the closing parenthesis and display the text
; at once, even while compiling.
; ------------------------------------------------------------
DOTPARENW:  LDD   #41
            PSHU  D
            JSR   PARSEW
            JMP   TYPEW

; ------------------------------------------------------------
; TYPE  ( c-addr u -- )
; Display u characters starting at c-addr, through EMIT.
; ------------------------------------------------------------
TYPEW:      LDD   ,U              ; u. Parameters stay on the data stack:
            BEQ   TYDONE          ; u at 0, c-addr at 2
TYLOOP:     LDX   2,U
            LDA   ,X+
            STX   2,U
            TFR   A,B
            CLRA
            PSHU  D
            JSR   EMITW
            LDD   ,U
            SUBD  #1
            STD   ,U
            BNE   TYLOOP
TYDONE:     LEAU  4,U             ; drop the two parameters
            RTS

; ------------------------------------------------------------
; COUNT  ( c-addr1 -- c-addr2 u )
; Convert a counted string to its address and length.
; ------------------------------------------------------------
COUNTW:     PULU  X               ; c-addr1
            LDB   ,X
            CLRA
            STD   MSCR
            LEAX  1,X
            PSHU  X
            LDD   MSCR
            PSHU  D
            RTS
```

```
; ------------------------------------------------------------
; CHAR  ( "name" -- char )
; Parse the next blank-delimited name and return its first character.
; ------------------------------------------------------------
CHARW:      LDD   #32
            PSHU  D
            JSR   WORDW
            PULU  X
            LDB   1,X
            CLRA
            PSHU  D
            RTS

; ------------------------------------------------------------
; [CHAR]  ( "name" -- )
; Compile the first character of the next name as a literal. THROW -14
; if not compiling.
; ------------------------------------------------------------
BRACKCHARW: LDD   STATE
            BNE   BCSTOK
            LDD   #-14              ; -14
            PSHU  D
            JSR   THROWW
BCSTOK:     LDD   #32
            PSHU  D
            JSR   WORDW
            PULU  X
            LDB   1,X
            CLRA
            PSHU  D
            JSR   LITERALW
            RTS

; ------------------------------------------------------------
; PARSE  ( char "ccc<char>" -- c-addr u )
; Parse the input up to the delimiter char (which is consumed) and
; return the parsed text. A >IN beyond the end of the input is treated
; as an exhausted input (empty result).
; ------------------------------------------------------------
PARSEW:     PULU  D                 ; char
            PSHS  B                 ; 0,S = delimiter (5k: no scratch cells)
            LDD   SRCLEN
            SUBD  TOIN
            LBLO  PEMP              ; See bugfix: PARSEW.1
            TFR   D,Y
            LDD   TOIN
            LDX   SRCADDR
            LEAX  D,X
            PSHU  X                 ; c-addr result; u is computed at the end
PSCAN:      CMPY  #0
            BEQ   PDONE
            LDA   ,X
            CMPA  ,S
            BEQ   PFOUND
            LEAX  1,X
            LEAY  -1,Y
            BRA   PSCAN
PFOUND:     TFR   X,D
            SUBD  ,U                ; length excludes the delimiter
            PSHU  D
            LEAX  1,X               ; consume the delimiter
            BRA   PFIN
PDONE:      TFR   X,D
            SUBD  ,U
            PSHU  D
PFIN:       TFR   X,D
            SUBD  SRCADDR
            STD   TOIN
            LEAS  1,S
            RTS
```

```
PEMP:       LEAS  1,S
            LBRA  PNEMPTY

; ------------------------------------------------------------
; PARSE-NAME  ( "<spaces>name<space>" -- c-addr u )
; Skip leading blanks, then parse a blank-delimited name and return
; it. An empty result is returned at the end of the input.
; ------------------------------------------------------------
PARSENAMEW: LDD   TOIN
            LDX   SRCADDR
            LEAX  D,X
            LDD   SRCLEN
            SUBD  TOIN
            BLO   PNEMPTY           ; See bugfix: PARSENAMEW.1
            TFR   D,Y
PNSKIP:     CMPY  #0
            BEQ   PNEMPTY
            LDA   ,X
            CMPA  #32
            BNE   PNSTART
            LEAX  1,X
            LEAY  -1,Y
            BRA   PNSKIP
PNSTART:    PSHU  X                 ; c-addr result
PNSCAN:     CMPY  #0
            BEQ   PNDONE
            LDA   ,X
            CMPA  #32
            BEQ   PNFOUND
            LEAX  1,X
            LEAY  -1,Y
            BRA   PNSCAN
PNFOUND:    TFR   X,D
            SUBD  ,U
            PSHU  D
            LEAX  1,X
            BRA   PNFIN
PNDONE:     TFR   X,D
            SUBD  ,U
            PSHU  D
PNFIN:      TFR   X,D
            SUBD  SRCADDR
            STD   TOIN
            RTS
PNEMPTY:    LDD   SRCLEN
            LDX   SRCADDR
            LEAX  D,X
            STD   TOIN
            PSHU  X
            LDD   #0
            PSHU  D
            RTS

; ------------------------------------------------------------
; /STRING  ( c-addr1 u1 n -- c-addr2 u2 )
; Remove n characters from the start of the string.
; ------------------------------------------------------------
SLASHSTRINGW:
            PULU  D                 ; n
            STD   MSCR
            PULU  D
            SUBD  MSCR
            STD   MSCR2
            PULU  D
            ADDD  MSCR
            PSHU  D
            LDD   MSCR2
            PSHU  D
            RTS

; ------------------------------------------------------------
```

```
; -TRAILING  ( c-addr u1 -- c-addr u2 )
; Reduce the length to exclude trailing blanks.
; ------------------------------------------------------------
DASHTRAILINGW:
            LDY   ,U
            LDX   2,U
            TFR   Y,D
            LEAX  D,X               ; X = end of the string
DTLOOP:     CMPY  #0
            BEQ   DTDONE
            LDA   ,-X
            CMPA  #32
            BNE   DTDONE
            LEAY  -1,Y
            BRA   DTLOOP
DTDONE:     STY   ,U
            RTS

; ------------------------------------------------------------
; COMPARE  ( c-addr1 u1 c-addr2 u2 -- n )
; Compare two strings byte by byte. n is -1, 0 or 1 as the first
; string is less than, equal to or greater than the second; if one is
; a prefix of the other, the shorter is less.
; ------------------------------------------------------------
COMPAREW:   LDX   6,U                ; Parameters stay on the data stack: u2 at 0,
            LDY   2,U               ; c-addr2 at 2, u1 at 4, c-addr1 at 6
            LDD   4,U
            CMPD  ,U
            BLS   CMMINSET
            LDD   ,U                ; D = the shorter length
CMMINSET:   STD   2,U               ; the c-addr2 slot is free now: use it as
CMPLOOP:    LDD   2,U               ; the remaining count
            BEQ   CMTIEBREAK
            LDA   ,X+
            CMPA  ,Y+
            BLO   CMLT
            BHI   CMGT
            LDD   2,U
            SUBD  #1
            STD   2,U
            BRA   CMPLOOP
CMTIEBREAK: LDD   4,U
            CMPD  ,U
            BLO   CMLT
            BHI   CMGT
            LDD   #0
            BRA   CMOUT
CMLT:       LDD   #-1
            BRA   CMOUT
CMGT:       LDD   #1
CMOUT:      STD   6,U               ; result replaces c-addr1
            LEAU  6,U               ; drop the other three cells
            RTS

; ------------------------------------------------------------
; SEARCH  ( c-addr1 u1 c-addr2 u2 -- c-addr3 u3 flag )
; Search the string c-addr1 u1 for the string c-addr2 u2. If found,
; return the address and remaining length of the match and true;
; otherwise return the original string and false.
; ------------------------------------------------------------
SEARCHW:    LEAU  -6,U              ; locals: 0,U candidate address, 2,U positions
            LDD   6,U               ; left, 4,U end of the needle.  Then u2 at 6,
            BNE   SRCHNZ            ; c-addr2 at 8, u1 at 10, c-addr1 at 12
            LDD   12,U              ; an empty needle matches at the start
            STD   ,U
            BRA   SFOUND
SRCHNZ:     LDD   10,U
            SUBD  6,U
            BLT   SRCHNOTFOUND
            ADDD  #1
            STD   2,U
```

```
            LDD   12,U
            STD   ,U
            LDD   8,U
            ADDD  6,U
            STD   4,U
SPOSLOOP:   LDD   2,U
            BEQ   SRCHNOTFOUND
            LDX   ,U
            LDY   8,U
SMATCH:     LDA   ,X+
            CMPA  ,Y+
            BNE   SNOMATCH
            CMPY  4,U
            BNE   SMATCH
SFOUND:     LDD   12,U              ; u3 = (c-addr1 + u1) - match address,
            ADDD  10,U              ; the characters remaining. See shadow
            SUBD  ,U                ; SEARCHW.2
            STD   10,U
            LDD   ,U
            STD   12,U              ; c-addr3
            LDD   #TRUEV
            BRA   SRCHOUT
SRCHNOTFOUND:
            LDD   #FALSEV           ; c-addr1 and u1 are already in place
SRCHOUT:    STD   8,U               ; flag
            LEAU  8,U               ; drop the locals and u2
            RTS
SNOMATCH:   LDX   ,U
            LEAX  1,X
            STX   ,U
            LDD   2,U
            SUBD  #1
            STD   2,U
            BRA   SPOSLOOP

; ------------------------------------------------------------
; SNAME  ( xt -- c-addr u | 0 0 )
; Find the name of the word whose xt is given by searching the
; dictionary from LATEST. Return the name or two zeros if not found.
; ------------------------------------------------------------
SNAMEW:     PULU  Y                 ; xt (5k: Y = target, X = header, no scratch)
            LDX   LATEST
SNLOOP:     CMPX  #0
            BEQ   SNNOTFOUND
            LDB   ,X
            ANDB  #$1F              ; name length
            CLRA
            LEAX  D,X               ; X = header + length
            CMPY  3,X               ; CFA field follows the name and link
            BEQ   SNFOUND
            LDX   1,X               ; LINK field
            BRA   SNLOOP
SNFOUND:    TFR   D,Y               ; Y = length
            TFR   X,D
            PSHS  Y
            SUBD  ,S++              ; header
            ADDD  #1                ; name address
            PSHU  D
            PSHU  Y
            RTS
SNNOTFOUND: LDD   #0
            PSHU  D
            PSHU  D
            RTS

; ------------------------------------------------------------
; UNESCAPE  ( c-addr1 u1 c-addr2 -- c-addr2 u2 )
; Copy the string c-addr1 u1 to c-addr2, doubling every "%" so that
; SUBSTITUTE turns it back into the original. u2 is the length of the
; result.
; ------------------------------------------------------------
```

```
UNESCAPEW:  LDX   4,U               ; c-addr1. Parameters stay on the data stack:
            LDY   ,U                ; c-addr2 at 0, u1 at 2. See bugfix:
                                    ; UNESCAPEW.1 (still applies) and
                                    ; shadow UNESCAPEW.5

            ; Double every %; copy all other characters unchanged.
            ; See bugfix: UNESCAPEW.2
UELOOP:     LDD   2,U
            BEQ   UEDONE
            LDA   ,X+
            CMPA  #'%'
            BNE   UENOTPCT
            LDB   #'%'              ; extra % written first
            STB   ,Y+
UENOTPCT:   STA   ,Y+               ; write the character
            LDD   2,U
            SUBD  #1
            STD   2,U
            BRA   UELOOP
UEDONE:     TFR   Y,D               ; See bugfix: UNESCAPEW.4
            SUBD  ,U                ; u2 = end - destination
            STD   2,U               ; result: c-addr2 u2
            LDD   ,U
            STD   4,U
            LEAU  2,U
            RTS

; ------------------------------------------------------------
; REPLACES  ( c-addr1 u1 c-addr2 u2 -- )
; Register the substitution text c-addr1 u1 for the name c-addr2 u2.
; Only one registration is kept (a single slot); a new call replaces
; the previous one. The strings are not copied.
; Original comment: shadow REPLACESW.0.
; ------------------------------------------------------------
REPLACESW:  LDD   ,U                ; Parameters stay on the data stack: u2 at
            CMPD  #255              ; 0, c-addr2 at 2, u1 at 4, c-addr1 at 6.
            BHI   REPLERR           ; Names and texts are limited to 255
            LDD   4,U               ; characters. See shadow REPLACES.1
            CMPD  #255
            BHI   REPLERR
            LDD   ,U
            LDX   2,U
            JSR   SUBFIND           ; X = existing entry, or 0
            CMPX  #0
            BEQ   REPLNEW
            LDB   5,U               ; the new text length
            CMPB  4,X               ; does it fit the old entry's space?
            BHI   REPLNEW
            STB   3,X               ; yes: replace the text in place
            LDB   2,X
            CLRA
            LEAY  5,X
            LEAY  D,Y               ; Y = the text field
            BRA   REPLTEXT
REPLNEW:    LDD   SUBFREE           ; no: add a new entry at the head
            BNE   REPLHAVE
            LDD   #SUBPOOL          ; first use: the pool is empty
REPLHAVE:   TFR   D,Y               ; Y = the new entry
            CLRA
            LDB   1,U
            ADDD  #5
            TFR   D,X
            LDB   5,U
            CLRA
            LEAX  D,X               ; X = entry size: 5 + name + text
            PSHS  X
            TFR   Y,D
            ADDD  ,S++              ; D = end of the entry
            CMPD  #SUBPOOL+SUBPOOLSZ
            BHI   REPLFULL
            STD   SUBFREE
```

```
            LDD   SUBHEAD
            STD   ,Y                ; link
            LDB   1,U
            STB   2,Y               ; name length
            LDB   5,U
            STB   3,Y               ; text length
            STB   4,Y               ; text capacity
            PSHS  Y
            LDX   2,U
            LEAY  5,Y
            LDB   1,U
            BEQ   REPLNAMED
REPLNLP:    LDA   ,X+
            STA   ,Y+
            DECB
            BNE   REPLNLP
REPLNAMED:  PULS  X
            STX   SUBHEAD           ; the new entry is now the newest
REPLTEXT:   LDX   6,U               ; copy the text to Y
            LDB   5,U
            BEQ   REPLDONE
REPLVLP:    LDA   ,X+
            STA   ,Y+
            DECB
            BNE   REPLVLP
REPLDONE:   LEAU  8,U
            RTS
REPLERR:    LDD   #-18              ; parsed string overflow: name or text
            BRA   REPLTHROW         ; longer than 255
REPLFULL:   LDD   #-8               ; dictionary overflow: SUBPOOL is full
REPLTHROW:  PSHU  D
            JMP   THROWW

; ------------------------------------------------------------
; CLR-REPLACES ( -- )  Empties the REPLACES table, freeing the whole pool.
; Not a standard word. See shadow CLRREPLACESW.1.
; ------------------------------------------------------------
CLRREPLACESW:
            CLRA
            CLRB
            STD   SUBHEAD
            STD   SUBFREE
            RTS

; ------------------------------------------------------------
; Find the REPLACES entry for a name, newest first.
; SUBFIND
;    Inputs:
;        RegD = name length, RegX = name address
;    Outputs:
;        RegX = entry address, or 0 if there is none. An entry is: 0 link,
;        2 name length, 3 text length, 4 text capacity, 5 name, then text.
;    Registers: RegA, RegB, RegX and RegY changed. A length over 255 is
;        never found.
; ------------------------------------------------------------
SUBFIND:    TSTA
            BNE   SFNONE            ; too long to be a name
            PSHS  D,X               ; 0,S length, 2,S name address
            LDY   SUBHEAD
SFLOOP:     CMPY  #0
            BEQ   SFEND
            LDB   2,Y
            CMPB  1,S
            BNE   SFNEXT
            LDX   2,S
            PSHS  Y
            LEAY  5,Y
SFCMP:      TSTB
            BEQ   SFMATCH
            LDA   ,X+
            CMPA  ,Y+
```

```
            BNE   SFDIFF
            DECB
            BRA   SFCMP
SFMATCH:    PULS  X                 ; the entry
            LEAS  4,S
            RTS
SFDIFF:     PULS  Y
SFNEXT:     LDY   ,Y
            BRA   SFLOOP
SFEND:      LEAS  4,S
SFNONE:     LDX   #0
            RTS

; ------------------------------------------------------------
; Append bytes to the SUBSTITUTE output buffer. THROW -1 if the buffer
; would overflow.
; SUBCOPY
;    Inputs:
;        RegD = byte count, RegX = source address, U = SUBSTITUTE's
;        frame (6,U output length, 8,U write pointer, 10,U capacity)
;    Outputs:
;        bytes appended at the write pointer; length and pointer advanced
;    Registers: RegA, RegD, RegX and RegY changed.
; ------------------------------------------------------------
SUBCOPY:    CMPD  #0
            BEQ   SUBCPDONE
            PSHS  D                 ; bytes still to copy
SUBCPLP:    LDD   6,U               ; SUBSTITUTE's frame (U is unchanged):
            CMPD  10,U              ; output length against capacity
            LBHS  SUBOVERFLOW       ; buffer full: THROW -1
            LDY   8,U               ; write pointer
            LDA   ,X+
            STA   ,Y+
            STY   8,U
            LDD   6,U
            ADDD  #1
            STD   6,U
            LDD   ,S
            SUBD  #1
            STD   ,S
            BNE   SUBCPLP
            LEAS  2,S
SUBCPDONE:  RTS

; ------------------------------------------------------------
; SUBSTITUTE  ( c-addr1 u1 c-addr2 u2 -- c-addr2 u3 n )
; Copy the template c-addr1 u1 to the buffer c-addr2 u2, replacing
; each %name% that matches the name registered by REPLACES with its
; text. "%%" becomes a single "%"; a %name% that does not match, and
; an unpaired "%", are copied unchanged. u3 is the output length and n
; the number of substitutions. THROW -1 if the output does not fit.
; Scratch: MSCR = read position, MSCR2 = substitution count, MSCR3 and
; MSCR4 = temporaries for each %-pair. Original comment: shadow
; SUBSTITUTEW.1.
; ------------------------------------------------------------
SUBSTITUTEW:
            LEAU  -10,U             ; frame: 0 read position, 2 substitution
                                    ; count, 4 closing '%' position, 6 output
                                    ; length, 8 write pointer; then the
                                    ; parameters: 10 u2 (capacity), 12 c-addr2,
                                    ; 14 u1, 16 c-addr1. See shadow SUBSTITUTEW.5
            LDD   16,U              ; the source must not be the destination
            CMPD  12,U
            LBEQ  SUBBAD
            LDD   #0
            STD   ,U
            STD   2,U
            STD   6,U
            LDD   12,U
            STD   8,U
```

```
SUBSCAN:    LDD   ,U
            CMPD  14,U
            LBHS  SUBSDONE
            LDX   16,U
            LEAX  D,X
            LDA   ,X
            CMPA  #$25              ; '%' delimiter
            BEQ   SUBPCT
            LDD   #1
            JSR   SUBCOPY
            LDD   ,U
            ADDD  #1
            STD   ,U
            LBRA  SUBSCAN

SUBPCT:     LDD   ,U
            ADDD  #1
            STD   4,U               ; scan for the closing '%'
SUBFINDCL:  LDD   4,U
            CMPD  14,U
            BLO   SUBFC2
            ; no closing delimiter found - residue passed unchanged
            LDD   ,U
            LDX   16,U
            LEAX  D,X
            LDD   14,U
            SUBD  ,U
            JSR   SUBCOPY
            LDD   14,U
            STD   ,U
            LBRA  SUBSCAN
SUBFC2:     LDD   4,U
            LDX   16,U
            LEAX  D,X
            LDA   ,X
            CMPA  #$25
            BEQ   SUBFOUNDCL
            LDD   4,U
            ADDD  #1
            STD   4,U
            LBRA  SUBFINDCL

SUBFOUNDCL: LDD   4,U
            SUBD  ,U
            SUBD  #1                ; enclosed name length
            LBNE  SUBHASNAME
            ; namelen 0: "%%" -> single '%' to output, count unchanged
            LDD   ,U
            LDX   16,U
            LEAX  D,X
            LDD   #1
            JSR   SUBCOPY
            LDD   4,U
            ADDD  #1
            STD   ,U
            LBRA  SUBSCAN

SUBHASNAME: LDD   ,U
            ADDD  #1
            LDX   16,U
            LEAX  D,X               ; X = the name
            LDD   4,U
            SUBD  ,U
            SUBD  #1                ; D = its length
            JSR   SUBFIND
            CMPX  #0
            BEQ   SUBNOMATCH
            ; valid name - replace the whole %name% span
            LDB   2,X               ; name length
            CLRA
            TFR   D,Y
```

```
            LDB   3,X               ; text length
            PSHS  B
            LEAX  5,X
            TFR   Y,D
            LEAX  D,X               ; X = the text
            PULS  B
            CLRA
            JSR   SUBCOPY
            LDD   2,U
            ADDD  #1
            STD   2,U
            LDD   4,U
            ADDD  #1
            STD   ,U
            LBRA  SUBSCAN

            ; Not a registered name: pass %name% through unchanged.
SUBNOMATCH:
            LDD   ,U
            LDX   16,U
            LEAX  D,X
            LDD   4,U
            SUBD  ,U
            ADDD  #1
            JSR   SUBCOPY
            LDD   4,U
            ADDD  #1
            STD   ,U
            LBRA  SUBSCAN

SUBSDONE:   LDD   12,U              ; result c-addr2 u3 n replaces the frame
            STD   16,U
            LDD   6,U
            STD   14,U
            LDD   2,U
            STD   12,U
            LEAU  12,U
            RTS

            ; The output buffer is too small: THROW -1.
SUBOVERFLOW:
            LEAS  4,S               ; SUBCOPY's count and return address
            LDD   #-18              ; parsed string overflow: output too long
            BRA   SUBFAIL
SUBBAD:     LDD   #-9               ; invalid address: source = destination
SUBFAIL:    TFR   D,X               ; error result c-addr2 0 n. See shadow
            LDD   12,U              ; SUBSTITUTEW.6
            STD   16,U
            LDD   #0
            STD   14,U
            STX   12,U
            LEAU  12,U
            RTS
```

8.21 Numeric Output

```
; ============================================================
; SECTION 20: NUMERIC OUTPUT (pictured + direct)
; ============================================================
; ------------------------------------------------------------
; <#  ( -- )
; Start a pictured numeric output string: HLD is set to the end of
; PAD. The string is built backwards from there.
; ------------------------------------------------------------
LTNUMW:     JSR   PADW
            PULU  D
            STD   HLD
            RTS

; ------------------------------------------------------------
```

```
; HOLD  ( char -- )
; Add char to the front of the pictured numeric output string.
; ------------------------------------------------------------
HOLDW:      PULU  D                 ; char
            LDX   HLD
            LEAX  -1,X
            STX   HLD
            STB   ,X
            RTS

; ------------------------------------------------------------
; HOLDS  ( c-addr u -- )
; Add the string c-addr u to the front of the pictured numeric output
; string.
; ------------------------------------------------------------
HOLDSW:     LDD   ,U                ; u at 0, c-addr at 2
            BEQ   HSDONE
            SUBD  #1
            STD   ,U                ; remaining count
            LDX   2,U
            LDA   D,X               ; the last remaining character
            TFR   A,B
            CLRA
            PSHU  D
            JSR   HOLDW             ; consumes the character
            BRA   HOLDSW
HSDONE:     LEAU  4,U               ; drop the two parameters
            RTS

; ------------------------------------------------------------
; #  ( ud1 -- ud2 )
; Divide ud1 by BASE, add the remainder as a digit to the pictured
; numeric output string, and leave the quotient ud2.
; ------------------------------------------------------------
NUMW:       JSR   UDDIGIT           ; ud1 is divided in place on the data stack
            CMPA  #10
            BLO   NDIGIT
            ADDA  #'A'-10
            BRA   NHOLD
NDIGIT:     ADDA  #'0'
NHOLD:      TFR   A,B
            CLRA
            PSHU  D
            JMP   HOLDW

; ------------------------------------------------------------
; Divide the unsigned double on the data stack (high cell at 0,U, low
; cell at 2,U) by BASE (restoring division, 32 iterations). Only the
; low byte of BASE is used.
; UDDIGIT
;    Inputs:
;        ud on the data stack
;    Outputs:
;        ud replaced by the quotient; RegA = remainder
;    Registers: RegA and RegB changed (RegB is the loop counter).
; ------------------------------------------------------------
UDDIGIT:    CLRA                    ; A = remainder
            LDB   #32
UDDLOOP:    ASL   3,U
            ROL   2,U
            ROL   1,U
            ROL   ,U
            ROLA
            CMPA  BASE+1
            BLO   UDNEXT
            SUBA  BASE+1
            INC   3,U
UDNEXT:     DECB
            BNE   UDDLOOP
            RTS
```

```
; ------------------------------------------------------------
; #S  ( ud -- 0 0 )
; Convert digits with # until the quotient is zero (at least one digit
; is produced).
; ------------------------------------------------------------
NUMSW:      JSR   NUMW
            LDD   ,U
            BNE   NUMSW
            LDD   2,U
            BNE   NUMSW
            RTS

; ------------------------------------------------------------
; SIGN  ( n -- )
; If n is negative, add a minus sign to the pictured numeric output
; string.
; ------------------------------------------------------------
SIGNW:      PULU  D                 ; n
            TSTA                    ; sign of n. See bugfix: SIGNW.1
            BPL   SIGNDONE
            LDD   #'-'
            PSHU  D
            JSR   HOLDW
SIGNDONE:   RTS

; ------------------------------------------------------------
; #>  ( xd -- c-addr u )
; Drop xd and return the pictured numeric output string.
; ------------------------------------------------------------
NUMGTW:     PULU  D                 ; xd high
            PULU  D                 ; xd low
            LDX   HLD
            PSHU  X
            JSR   PADW
            PULU  D
            SUBD  HLD
            PSHU  D
            RTS

; ------------------------------------------------------------
; .  ( n -- )
; Display n in the current base, followed by a space.
; ------------------------------------------------------------
DOTW:       PULU  D                 ; n
            PSHS  D                 ; n kept on S for SIGN (5m)
            TSTA
            BPL   DABSOK
            COMA
            COMB
            ADDD  #1
DABSOK:     PSHU  D
            LDD   #0
            PSHU  D
            JSR   LTNUMW
            JSR   NUMSW
            PULS  D
            PSHU  D
            JSR   SIGNW
            JSR   NUMGTW
            JSR   TYPEW
            LDD   #32
            PSHU  D
            JSR   EMITW
            RTS

; ------------------------------------------------------------
; U.  ( u -- )
; Display u in the current base, followed by a space.
; ------------------------------------------------------------
UDOTW:      LDD   #0                ; ud high cell = 0
            PSHU  D
```

```
            JSR   LTNUMW
            JSR   NUMSW
            JSR   NUMGTW
            JSR   TYPEW
            LDD   #32
            PSHU  D
            JSR   EMITW
            RTS

; ------------------------------------------------------------
; .R  ( n1 n2 -- )
; Display n1 right-justified in a field of n2 characters.
; ------------------------------------------------------------
DOTRW:      PULU  D                 ; n2 (width)
            PSHS  D                 ; width waits on the return stack
            PULU  D                 ; n1
            PSHS  D                 ; kept on S for SIGN, above the width
            TSTA
            BPL   DRABSOK
            COMA
            COMB
            ADDD  #1
DRABSOK:    PSHU  D
            LDD   #0
            PSHU  D
            JSR   LTNUMW
            JSR   NUMSW
            PULS  D
            PSHU  D
            JSR   SIGNW
            JSR   NUMGTW
            PULS  D                 ; width
            LBRA  RTYPE

; ------------------------------------------------------------
; Type the string c-addr u right-justified in a field of D characters.
; RTYPE   ( c-addr u -- )
;    Inputs:
;        RegD = field width; the string on the data stack
;    Outputs:
;        padding spaces, then the string
;    Registers: all changed.
; ------------------------------------------------------------
RTYPE:      SUBD  ,U                ; width - u
            BLE   RTNOPAD
            PSHS  D                 ; spaces still to send
RTPADLP:    LDD   #32
            PSHU  D
            JSR   EMITW
            LDD   ,S
            SUBD  #1
            STD   ,S
            BNE   RTPADLP
            LEAS  2,S
RTNOPAD:    JMP   TYPEW

; ------------------------------------------------------------
; U.R  ( u n -- )
; Display u right-justified in a field of n characters.
; ------------------------------------------------------------
UDOTRW:     PULU  D                 ; n (width)
            PSHS  D
            LDD   #0                ; ud high cell = 0
            PSHU  D
            JSR   LTNUMW
            JSR   NUMSW
            JSR   NUMGTW
            PULS  D
            LBRA  RTYPE

; ------------------------------------------------------------
```

```
; ?  ( a-addr -- )
; Display the cell at a-addr.
; ------------------------------------------------------------
QMARKW:     PULU  X                 ; a-addr
            LDD   ,X
            PSHU  D
            JSR   DOTW
            RTS

; ------------------------------------------------------------
; D.  ( d -- )
; Display the double d in the current base, followed by a space.
; ------------------------------------------------------------
DDOTW:      PULU  D                 ; d high
            STD   PRODHI
            PULU  D                 ; d low
            STD   PRODLO
            LDD   PRODHI
            PSHS  D                 ; sign cell, kept on S
            BPL   DDPOS
            JSR   MNEG32
DDPOS:      LDD   PRODLO
            PSHU  D
            LDD   PRODHI
            PSHU  D
            JSR   LTNUMW
            JSR   NUMSW
            PULS  D
            PSHU  D
            JSR   SIGNW
            JSR   NUMGTW
            JSR   TYPEW
            LDD   #32
            PSHU  D
            JSR   EMITW
            RTS

; ------------------------------------------------------------
; D.R  ( d n -- )
; Display the double d right-justified in a field of n characters.
; ------------------------------------------------------------
DDOTRW:     PULU  D                 ; n (width)
            PSHS  D
            PULU  D                 ; d high
            STD   PRODHI
            PULU  D                 ; d low
            STD   PRODLO
            LDD   PRODHI
            PSHS  D                 ; sign cell, kept on S above the width
            BPL   DDRPOS
            JSR   MNEG32
DDRPOS:     LDD   PRODLO
            PSHU  D
            LDD   PRODHI
            PSHU  D
            JSR   LTNUMW
            JSR   NUMSW
            PULS  D
            PSHU  D
            JSR   SIGNW
            JSR   NUMGTW
            PULS  D
            LBRA  RTYPE
```

8.22 Base / Radix Control

```
; ============================================================
; SECTION 21: BASE / RADIX CONTROL
; ============================================================
; ------------------------------------------------------------
```

```
; BASE  ( -- a-addr )
; Push the address of the BASE variable.
; ------------------------------------------------------------
BASEW:      LDD   #BASE             ; a-addr
            PSHU  D
            RTS

; ------------------------------------------------------------
; DECIMAL  ( -- )
; Set BASE to 10.
; ------------------------------------------------------------
DECIMALW:   LDD   #10
            STD   BASE
            RTS

; ------------------------------------------------------------
; HEX  ( -- )
; Set BASE to 16.
; ------------------------------------------------------------
HEXW:       LDD   #16
            STD   BASE
            RTS

; ------------------------------------------------------------
; BINARY  ( -- )
; Set BASE to 2.
; ------------------------------------------------------------
BINARYW:    LDD   #2
            STD   BASE
            RTS
```

8.23 Output Formatting

```
; ============================================================
; SECTION 22: OUTPUT FORMATTING (CR/SPACE/SPACES)
; ============================================================
; ------------------------------------------------------------
; CR  ( -- )
; Send carriage return and line feed.
; ------------------------------------------------------------
CRW:        LDD   #13
            PSHU  D
            JSR   EMITW
            LDD   #10
            PSHU  D
            JSR   EMITW
            RTS

; ------------------------------------------------------------
; SPACE  ( -- )
; Send one space.
; ------------------------------------------------------------
SPACEW:     LDD   #32
            PSHU  D
            JSR   EMITW
            RTS

; ------------------------------------------------------------
; SPACES  ( n -- )
; Send n spaces (nothing if n is zero or negative).
; ------------------------------------------------------------
SPACESW:    LDD   ,U                ; n stays on the data stack
            BLE   SPDONE
SPLOOP:     LDD   #32
            PSHU  D
            JSR   EMITW
            LDD   ,U
            SUBD  #1
            STD   ,U
            BNE   SPLOOP
```

```
SPDONE:     LEAU  2,U               ; drop n
            RTS
```

8.24 Comment Words

```
; ============================================================
; SECTION 23: COMMENT WORDS
; ============================================================
; ------------------------------------------------------------
; (  ( "ccc<paren>" -- )
; Parse and discard text up to the closing parenthesis.
; ------------------------------------------------------------
LPARENW:    LDD   #')'
            PSHU  D
            JSR   WORDW
            PULU  D                 ; discard c-addr. See bugfix: LPARENW.1
            RTS

; ------------------------------------------------------------
; \  ( "ccc<eol>" -- )
; Discard the rest of the input line by setting >IN to the source
; length.
; ------------------------------------------------------------
BACKSLASHW: LDD   SRCLEN
            STD   TOIN
            RTS
```

8.25 Environmental Query / SOURCE / REFILL / EVALUATE

```
; ============================================================
; SECTION 24: ENVIRONMENTAL QUERY / SOURCE / REFILL / EVALUATE
; ============================================================
; ------------------------------------------------------------
; SOURCE  ( -- c-addr u )
; Push the address and length of the current input source.
; ------------------------------------------------------------
SOURCEW:    LDD   SRCADDR
            PSHU  D
            LDD   SRCLEN
            PSHU  D
            RTS

; ------------------------------------------------------------
; SOURCE-ID  ( -- 0 | -1 )
; Push 0 for the terminal, -1 for a string being EVALUATEd.
; ------------------------------------------------------------
SOURCEIDW:  LDD   SRCID
            PSHU  D
            RTS

; ------------------------------------------------------------
; SAVE-INPUT  ( -- x1 x2 x3 3 )
; Save the source address, source length and >IN.
; ------------------------------------------------------------
SAVEINPUTW: LDD   SRCADDR
            PSHU  D
            LDD   SRCLEN
            PSHU  D
            LDD   TOIN
            PSHU  D
            LDD   #3
            PSHU  D
            RTS

; ------------------------------------------------------------
; RESTORE-INPUT  ( x1 x2 x3 3 -- flag )
; Restore >IN if the input source is the same one that was saved.
; Return false on success, true if the saved state is unusable (wrong
; cell count, or a different source address or length).
; ------------------------------------------------------------
```

```
RESTOREINPUTW:
            PULU  D                 ; n
            CMPD  #3
            BNE   RIBAD
            PULU  Y                 ; saved >IN
            PULU  D                 ; saved source length
            PULU  X                 ; saved source address
            CMPD  SRCLEN
            BNE   RIFAIL
            CMPX  SRCADDR
            BNE   RIFAIL
            STY   TOIN
            LDD   #FALSEV
            PSHU  D
            RTS
RIBAD:      ASLB                    ; drop n cells
            ROLA
            LEAU  D,U
RIFAIL:     LDD   #TRUEV
            PSHU  D
            RTS

; ------------------------------------------------------------
; REFILL  ( -- flag )
; From the terminal, read a new line with QUERY and return true. From
; an EVALUATEd string, return false.
; ------------------------------------------------------------
REFILLW:    LDD   SRCID
            BEQ   RFTERM
            LDD   #FALSEV
            PSHU  D
            RTS
RFTERM:     JSR   QUERYW
            LDD   #TRUEV
            PSHU  D
            RTS

; ------------------------------------------------------------
; EVALUATE  ( c-addr u -- )
; Interpret the string c-addr u as the input source, then restore the
; previous input source (kept on the return stack).
; ------------------------------------------------------------
EVALUATEW:  LDD   SRCADDR           ; save the input source on S
            PSHS  D                 ; See bugfix: EVALUATEW.1
            LDD   SRCLEN
            PSHS  D
            LDD   SRCID
            PSHS  D
            LDD   TOIN
            PSHS  D
            PULU  D                 ; u
            STD   SRCLEN
            PULU  D                 ; c-addr
            STD   SRCADDR
            LDD   #-1
            STD   SRCID
            LDD   #0
            STD   TOIN
            JSR   INTERPRET
            PULS  D
            STD   TOIN
            PULS  D
            STD   SRCID
            PULS  D
            STD   SRCLEN
            PULS  D
            STD   SRCADDR
            RTS

; ------------------------------------------------------------
; ENVIRONMENT?  ( c-addr u -- false | i*x true )
```

```
; Look up the attribute name c-addr u first in ENVTABLE (single-cell
; values), then in ENVTABLE2 (double-cell values). If found, return
; the value and true; otherwise return false. WORDLISTS is
; deliberately absent because the Search-Order word set is not
; implemented. FLOORED is false because /, MOD and /MOD use symmetric
; division.
; Original comment: shadow ENVQW.0.
; ------------------------------------------------------------
ENVQW:      PULU  D                 ; u
            PSHS  D                 ; frame: 2,S = u, 0,S = c-addr (5k)
            PULU  D                 ; c-addr
            PSHS  D
            LDX   #ENVTABLE
ENVLOOP:    LDD   ,X
            CMPD  #0
            BEQ   ENV2START         ; end of table: try ENVTABLE2
            PSHS  X                 ; See bugfix: ENVQW.2 (frame now +2)
            PSHU  D
            LDD   2,X
            PSHU  D
            LDD   2,S               ; c-addr
            PSHU  D
            LDD   4,S               ; u
            PSHU  D
            JSR   COMPAREW
            PULS  X                 ; restore X before using it again
            PULU  D
            CMPD  #0
            BEQ   ENVFOUND
            LEAX  6,X
            BRA   ENVLOOP
ENVFOUND:   LEAS  4,S               ; drop frame
            LDD   4,X
            PSHU  D
            LDD   #TRUEV
            PSHU  D
            RTS

ENV2START:  LDX   #ENVTABLE2        ; double-cell table
ENV2LOOP:   LDD   ,X
            CMPD  #0
            BEQ   ENVNOTFOUND
            PSHS  X                 ; See bugfix: ENVQW.2
            PSHU  D
            LDD   2,X
            PSHU  D
            LDD   2,S               ; c-addr
            PSHU  D
            LDD   4,S               ; u
            PSHU  D
            JSR   COMPAREW
            PULS  X
            PULU  D
            CMPD  #0
            BEQ   ENV2FOUND
            LEAX  8,X               ; 8-byte entries
            BRA   ENV2LOOP
ENV2FOUND:  LEAS  4,S               ; drop frame
            LDD   4,X               ; low cell, pushed first
            PSHU  D
            LDD   6,X               ; high cell - pushed second/top
            PSHU  D
            LDD   #TRUEV
            PSHU  D
            RTS

ENVNOTFOUND:
            LEAS  4,S               ; drop frame
            LDD   #FALSEV
            PSHU  D
            RTS
```

```
; ------------------------------------------------------------
; ENVTABLE2
; Double-cell ENVIRONMENT? entries: name address, name length, low
; cell, high cell. A zero ends the table.
; ------------------------------------------------------------
ENVTABLE2:
            FDB   EN11,EN11L,$FFFF,$7FFF
                                ; MAX-D
            FDB   EN12,EN12L,$FFFF,$FFFF
                                ; MAX-UD
            FDB   0
EN11:       FCC   "MAX-D"
EN11L       EQU   *-EN11
EN12:       FCC   "MAX-UD"
EN12L       EQU   *-EN12

; ------------------------------------------------------------
; ENVTABLE
; Single-cell ENVIRONMENT? entries: name address, name length, value.
; A zero ends the table.
; ------------------------------------------------------------
ENVTABLE:
            FDB   EN1,EN1L,255      ; /COUNTED-STRING. See bugfix: ENVTABLE.1
            FDB   EN2,EN2L,32767    ; MAX-N
            FDB   EN3,EN3L,65535    ; MAX-U
            FDB   EN6,EN6L,8        ; ADDRESS-UNIT-BITS
            FDB   EN7,EN7L,HOLDMINSIZE
                                ; /HOLD
            FDB   EN8,EN8L,PADMINSIZE
                                ; /PAD
            FDB   EN9,EN9L,0        ; FLOORED = false
            FDB   EN10,EN10L,255    ; MAX-CHAR
            FDB   EN13,EN13L,(RSTACK-DSTACK)/2
                                ; RETURN-STACK-CELLS
            FDB   EN14,EN14L,(DSTACK-CODETOP+1)/2
                                ; STACK-CELLS
            FDB   EN15,EN15L,TRUEV  ; CORE (complete)
            FDB   EN16,EN16L,TRUEV  ; DOUBLE (complete)
            FDB   EN17,EN17L,TRUEV  ; DOUBLE-EXT (complete)
            FDB   EN18,EN18L,TRUEV  ; CORE-EXT (complete)
            FDB   0
EN1:        FCC   "/COUNTED-STRING"
EN1L        EQU   *-EN1
EN2:        FCC   "MAX-N"
EN2L        EQU   *-EN2
EN3:        FCC   "MAX-U"
EN3L        EQU   *-EN3
EN6:        FCC   "ADDRESS-UNIT-BITS"
EN6L        EQU   *-EN6
EN7:        FCC   "/HOLD"
EN7L        EQU   *-EN7
EN8:        FCC   "/PAD"
EN8L        EQU   *-EN8
EN9:        FCC   "FLOORED"
EN9L        EQU   *-EN9
EN10:       FCC   "MAX-CHAR"
EN10L       EQU   *-EN10
EN13:       FCC   "RETURN-STACK-CELLS"
EN13L       EQU   *-EN13
EN14:       FCC   "STACK-CELLS"
EN14L       EQU   *-EN14
EN15:       FCC   "CORE"
EN15L       EQU   *-EN15
EN16:       FCC   "DOUBLE"
EN16L       EQU   *-EN16
EN17:       FCC   "DOUBLE-EXT"
EN17L       EQU   *-EN17
EN18:       FCC   "CORE-EXT"
EN18L       EQU   *-EN18
```

8.26 Tools Word Set

```
; ============================================================
; SECTION 25: TOOLS WORD SET (.S / WORDS / DUMP)
; ============================================================
; ------------------------------------------------------------
; .S  ( -- )
; Display every cell on the data stack, top first, each followed by a
; space, using ".". The stack is not changed.
; ------------------------------------------------------------
DOTSW:      PSHS  U                 ; walking pointer lives on the return stack
DSLOOP:     LDX   ,S
            CMPX  #SP0
            BEQ   DSDONE
            LDD   ,X++
            STX   ,S
            PSHU  D
            JSR   DOTW
            BRA   DSLOOP
DSDONE:     LEAS  2,S
            RTS

; ------------------------------------------------------------
; WORDS  ( -- )
; Display the names of all dictionary words, newest first, then a
; carriage return.
; ------------------------------------------------------------
WORDSW:     LDD   LATEST
            PSHS  D                 ; header being shown, on the return stack
WWLOOP:     LDX   ,S
            BEQ   WWDONE
            LDB   ,X
            ANDB  #$1F
            CLRA
            LEAX  1,X
            PSHU  X
            PSHU  D
            JSR   TYPEW
            JSR   SPACEW
            LDX   ,S
            LDB   ,X
            ANDB  #$1F
            CLRA
            LEAX  1,X
            LEAX  D,X
            LDD   ,X                ; LINK to the next header
            STD   ,S
            BRA   WWLOOP
WWDONE:     LEAS  2,S
            JSR   CRW
            RTS

; ------------------------------------------------------------
; Display one hexadecimal digit.
; HEXDIGIT   ( n -- )
;    Inputs:
;        n (0 to 15) on the data stack
;    Outputs:
;        the digit sent with EMIT
;    Registers: RegD changed.
; ------------------------------------------------------------
HEXDIGIT:   PULU  D
            CMPD  #10
            BLO   HDDIGIT
            ADDD  #'A'-10
            BRA   HDEMIT
HDDIGIT:    ADDD  #'0'
HDEMIT:     PSHU  D
            JSR   EMITW
            RTS
```

```
; ------------------------------------------------------------
; Display a byte as two hexadecimal digits.
; HEXBYTE   ( byte -- )
;    Inputs:
;        byte on the data stack
;    Outputs:
;        two digits sent with EMIT
;    Registers: RegD changed; MSCR used as scratch.
; ------------------------------------------------------------
HEXBYTE:    PULU  D
            STB   MSCR              ; See bugfix: HEXBYTE.1
            LDB   MSCR              ; high nibble
            LSRB
            LSRB
            LSRB
            LSRB
            CLRA
            PSHU  D
            JSR   HEXDIGIT
            LDB   MSCR              ; low nibble
            ANDB  #$0F
            CLRA
            PSHU  D
            JSR   HEXDIGIT
            RTS

; ------------------------------------------------------------
; DUMP  ( addr u -- )
; Display u bytes starting at addr, 16 per line: the hex bytes (no
; address), then the same bytes as ASCII with non-printing characters
; shown as ".". A partial last line is padded so the ASCII columns
; line up. A carriage return is sent first and after every line.
; ------------------------------------------------------------
DUMPW:      JSR   CRW               ; leading CR
            LEAS  -4,S              ; locals: 0,S byte pointer, 2,S bytes on
                                    ; this line, 3,S column. u stays at 0,U
                                    ; and addr at 2,U. See shadow DUMPW.5
DULINE:     LDD   ,U
            LBEQ  DUDONE            ; no bytes left
            CMPD  #16
            BLS   DUN
            LDD   #16
DUN:        STB   2,S               ; n = bytes on this line
            LDD   2,U
            STD   ,S
            CLR   3,S
DUHEX:      LDB   3,S
            CMPB  2,S
            BEQ   DUHEXPAD
            LDX   ,S
            LDB   ,X+
            STX   ,S
            CLRA
            PSHU  D
            JSR   HEXBYTE
            LDD   #32
            PSHU  D
            JSR   EMITW
            INC   3,S
            BRA   DUHEX
DUHEXPAD:   LDB   3,S
            CMPB  #16
            BEQ   DUASCII
            LDD   #32               ; three spaces for each missing byte
            PSHU  D
            JSR   EMITW
            LDD   #32
            PSHU  D
            JSR   EMITW
            LDD   #32
            PSHU  D
```

```
            JSR   EMITW
            INC   3,S
            BRA   DUHEXPAD
DUASCII:    LDD   #32
            PSHU  D
            JSR   EMITW
            CLR   3,S
DUACHAR:    LDB   3,S
            CMPB  #16
            BEQ   DULEND
            CMPB  2,S
            BHS   DUABLANK
            LDX   2,U
            ABX                       ; X = line start + column
            LDA   ,X
            CMPA  #32
            BLO   DUDOT
            CMPA  #127
            BHS   DUDOT
            BRA   DUPRINT
DUDOT:      LDA   #'.'
DUPRINT:    TFR   A,B
            CLRA
            PSHU  D
            JSR   EMITW
            INC   3,S
            BRA   DUACHAR
DUABLANK:   LDD   #32
            PSHU  D
            JSR   EMITW
            INC   3,S
            BRA   DUACHAR
DULEND:     JSR   CRW
            LDX   2,U                 ; advance addr and reduce u by n
            LDB   2,S
            ABX
            STX   2,U
            LDD   ,U
            SUBB  2,S
            SBCA  #0
            STD   ,U
            LBRA  DULINE
DUDONE:     LEAS  4,S
            LEAU  4,U                 ; drop addr and u
            RTS
```

8.27 Constant-Value Words (TRUE, FALSE, 1, -1, 2, -2)

```
; ============================================================
; SECTION 26: CONSTANT-VALUE WORDS (TRUE, FALSE, 1, -1, 2, -2)
; ============================================================
; ------------------------------------------------------------
; TRUE  ( -- flag )
; Push true ($FFFF), the value of TRUEV. A direct LDD/PSHU/RTS: no
; DODOES trampoline and no value cell.
; Original comment: shadow TRUEW.0.
; ------------------------------------------------------------
TRUEW:      LDD   #$FFFF
            PSHU  D
            RTS

; ------------------------------------------------------------
; FALSE  ( -- flag )
; Push false ($0000), the value of FALSEV.
; ------------------------------------------------------------
FALSEW:     LDD   #$0000
            PSHU  D
            RTS

; ------------------------------------------------------------
```

```
; 1  ( -- 1 )
; Push the constant 1. The words -1, 2 and -2 below work the same way.
; Their headers are H_POSONE, H_NEGONE, H_POSTWO and H_NEGTWO (Section
; 27), since "1" and "-1" are not valid assembler labels.
; Original comment: shadow POSONEW.0.
; ------------------------------------------------------------
POSONEW:    LDD   #1
            PSHU  D
            RTS

; ------------------------------------------------------------
; -1  ( -- -1 )
; Push the constant -1.
; ------------------------------------------------------------
NEGONEW:    LDD   #-1
            PSHU  D
            RTS

; ------------------------------------------------------------
; 2  ( -- 2 )
; Push the constant 2.
; ------------------------------------------------------------
POSTWOW:    LDD   #2
            PSHU  D
            RTS

; ------------------------------------------------------------
; -2  ( -- -2 )
; Push the constant -2.
; ------------------------------------------------------------
NEGTWOW:    LDD   #-2
            PSHU  D
            RTS

; Verify no collision with init code,
; value should match ORG INITCODE.
BASECODEEND EQU   *
BASECODESIZE EQU   BASECODEEND-BASECODE

; Prevent the assembler from extinguishing the gap between the
; BASECODE block and the INITCODE block when it generates the
; .bin file.
BASEND:
            FILL  $FF,INITCODE-BASEND
```

8.28 Forth Dictionary (ROM Base Dictionary Headers)

```
; ============================================================
; SECTION 27: FORTH DICTIONARY (ROM base dictionary headers)
;
; One header per word, chained by LINK, placed at BASEDICT. Each
; header is: length|flags, name, LINK, CFA. Every CFA is the word's
; own code label (raw code entries, no trampoline), including TRUE
; and FALSE. DOES> is next to CREATE; its code label is DOESGTW
; because ">" is not valid in a label. ABORT and QUIT are
; hand-built at the end of the section.
;
; Names containing a double-quote (S", ." and ABORT") have it split
; out of the FCC into a separate FCB $22.
; The original header comment is shadow SECTION27.0.
; ============================================================

            ORG   BASEDICT

H_KEY:
            FCB   $03
            FCC   "KEY"
            FDB   0
            FDB   KEYW
H_KEYQ:
```

```
            FCB   $04
            FCC   "KEY?"
            FDB   H_KEY
            FDB   KEYQW
H_EMIT:
            FCB   $04
            FCC   "EMIT"
            FDB   H_KEYQ
            FDB   EMITW
H_ACCEPT:
            FCB   $06
            FCC   "ACCEPT"
            FDB   H_EMIT
            FDB   ACCEPTW
H_EXPECT:
            FCB   $06
            FCC   "EXPECT"
            FDB   H_ACCEPT
            FDB   EXPECTW
H_QUERY:
            FCB   $05
            FCC   "QUERY"
            FDB   H_EXPECT
            FDB   QUERYW
H_TYPE:
            FCB   $04
            FCC   "TYPE"
            FDB   H_QUERY
            FDB   TYPEW
H_CR:
            FCB   $02
            FCC   "CR"
            FDB   H_TYPE
            FDB   CRW
H_SPACE:
            FCB   $05
            FCC   "SPACE"
            FDB   H_CR
            FDB   SPACEW
H_SPACES:
            FCB   $06
            FCC   "SPACES"
            FDB   H_SPACE
            FDB   SPACESW
H_DUP:
            FCB   $03
            FCC   "DUP"
            FDB   H_SPACES
            FDB   DUPW
H_DROP:
            FCB   $04
            FCC   "DROP"
            FDB   H_DUP
            FDB   DROPW
H_SWAP:
            FCB   $04
            FCC   "SWAP"
            FDB   H_DROP
            FDB   SWAPW
H_OVER:
            FCB   $04
            FCC   "OVER"
            FDB   H_SWAP
            FDB   OVERW
H_ROT:
            FCB   $03
            FCC   "ROT"
            FDB   H_OVER
            FDB   ROTW
H_QDUP:
            FCB   $04
```

```
            FCC   "?DUP"
            FDB   H_ROT
            FDB   QDUPW
H_DEPTH:
            FCB   $05
            FCC   "DEPTH"
            FDB   H_QDUP
            FDB   DEPTHW
H_DDUP:
            FCB   $04
            FCC   "2DUP"
            FDB   H_DEPTH
            FDB   DDUPW
H_DDROP:
            FCB   $05
            FCC   "2DROP"
            FDB   H_DDUP
            FDB   DDROPW
H_DSWAP:
            FCB   $05
            FCC   "2SWAP"
            FDB   H_DDROP
            FDB   DSWAPW
H_DOVER:
            FCB   $05
            FCC   "2OVER"
            FDB   H_DSWAP
            FDB   DOVERW
H_NIP:
            FCB   $03
            FCC   "NIP"
            FDB   H_DOVER
            FDB   NIPW
H_TUCK:
            FCB   $04
            FCC   "TUCK"
            FDB   H_NIP
            FDB   TUCKW
H_PICK:
            FCB   $04
            FCC   "PICK"
            FDB   H_TUCK
            FDB   PICKW
H_ROLL:
            FCB   $04
            FCC   "ROLL"
            FDB   H_PICK
            FDB   ROLLW
H_DROT:
            FCB   $04
            FCC   "2ROT"
            FDB   H_ROLL
            FDB   DROTW
H_TOR:
            FCB   $02
            FCC   ">R"
            FDB   H_DROT
            FDB   TORW
H_FROMR:
            FCB   $02
            FCC   "R>"
            FDB   H_TOR
            FDB   FROMRW
H_RFETCH:
            FCB   $02
            FCC   "R@"
            FDB   H_FROMR
            FDB   RFETCHW
H_TWOTOR:
            FCB   $03
            FCC   "2>R"
```

```
            FDB   H_RFETCH
            FDB   TWOTORW
H_TWOFROMR:
            FCB   $03
            FCC   "2R>"
            FDB   H_TWOTOR
            FDB   TWOFROMRW
H_TWORFETCH:
            FCB   $03
            FCC   "2R@"
            FDB   H_TWOFROMR
            FDB   TWORFETCHW
H_PLUS:
            FCB   $01
            FCC   "+"
            FDB   H_TWORFETCH
            FDB   PLUSW
H_MINUS:
            FCB   $01
            FCC   "-"
            FDB   H_PLUS
            FDB   MINUSW
H_STAR:
            FCB   $01
            FCC   "*"
            FDB   H_MINUS
            FDB   STARW
H_SLASH:
            FCB   $01
            FCC   "/"
            FDB   H_STAR
            FDB   SLASHW
H_MOD:
            FCB   $03
            FCC   "MOD"
            FDB   H_SLASH
            FDB   MODW
H_SLASHMOD:
            FCB   $04
            FCC   "/MOD"
            FDB   H_MOD
            FDB   SLASHMODW
H_NEGATE:
            FCB   $06
            FCC   "NEGATE"
            FDB   H_SLASHMOD
            FDB   NEGATEW
H_ABS:
            FCB   $03
            FCC   "ABS"
            FDB   H_NEGATE
            FDB   ABSW
H_MIN:
            FCB   $03
            FCC   "MIN"
            FDB   H_ABS
            FDB   MINW
H_MAX:
            FCB   $03
            FCC   "MAX"
            FDB   H_MIN
            FDB   MAXW
H_ONEPLUS:
            FCB   $02
            FCC   "1+"
            FDB   H_MAX
            FDB   ONEPLUSW
H_ONEMINUS:
            FCB   $02
            FCC   "1-"
            FDB   H_ONEPLUS
```

```
            FDB   ONEMINUSW
H_TWOPLUS:
            FCB   $02
            FCC   "2+"
            FDB   H_ONEMINUS
            FDB   TWOPLUSW
H_TWOSTAR:
            FCB   $02
            FCC   "2*"
            FDB   H_TWOPLUS
            FDB   TWOSTARW
H_TWOSLASH:
            FCB   $02
            FCC   "2/"
            FDB   H_TWOSTAR
            FDB   TWOSLASHW
H_STARSLASH:
            FCB   $02
            FCC   "*/"
            FDB   H_TWOSLASH
            FDB   STARSLASHW
H_STARSLASHMOD:
            FCB   $05
            FCC   "*/MOD"
            FDB   H_STARSLASH
            FDB   STARSLASHMODW
H_UMSTAR:
            FCB   $03
            FCC   "UM*"
            FDB   H_STARSLASHMOD
            FDB   UMSTARW
H_UMSLASHMOD:
            FCB   $06
            FCC   "UM/MOD"
            FDB   H_UMSTAR
            FDB   UMSLASHMODW
H_MSTAR:
            FCB   $02
            FCC   "M*"
            FDB   H_UMSLASHMOD
            FDB   MSTARW
H_FMSLASHMOD:
            FCB   $06
            FCC   "FM/MOD"
            FDB   H_MSTAR
            FDB   FMSLASHMODW
H_SMSLASHREM:
            FCB   $06
            FCC   "SM/REM"
            FDB   H_FMSLASHMOD
            FDB   SMSLASHREMW
H_DPLUS:
            FCB   $02
            FCC   "D+"
            FDB   H_SMSLASHREM
            FDB   DPLUSW
H_DMINUS:
            FCB   $02
            FCC   "D-"
            FDB   H_DPLUS
            FDB   DMINUSW
H_DNEGATE:
            FCB   $07
            FCC   "DNEGATE"
            FDB   H_DMINUS
            FDB   DNEGATEW
H_DABS:
            FCB   $04
            FCC   "DABS"
            FDB   H_DNEGATE
            FDB   DABSW
```

```
H_MPLUS:
            FCB   $02
            FCC   "M+"
            FDB   H_DABS
            FDB   MPLUSW
H_STOD:
            FCB   $03
            FCC   "S>D"
            FDB   H_MPLUS
            FDB   STODW
H_DTOS:
            FCB   $03
            FCC   "D>S"
            FDB   H_STOD
            FDB   DTOSW
H_DMAX:
            FCB   $04
            FCC   "DMAX"
            FDB   H_DTOS
            FDB   DMAXW
H_DMIN:
            FCB   $04
            FCC   "DMIN"
            FDB   H_DMAX
            FDB   DMINW
H_AND:
            FCB   $03
            FCC   "AND"
            FDB   H_DMIN
            FDB   ANDW
H_OR:
            FCB   $02
            FCC   "OR"
            FDB   H_AND
            FDB   ORW
H_XOR:
            FCB   $03
            FCC   "XOR"
            FDB   H_OR
            FDB   XORW
H_INVERT:
            FCB   $06
            FCC   "INVERT"
            FDB   H_XOR
            FDB   INVERTW
H_LSHIFT:
            FCB   $06
            FCC   "LSHIFT"
            FDB   H_INVERT
            FDB   LSHIFTW
H_RSHIFT:
            FCB   $06
            FCC   "RSHIFT"
            FDB   H_LSHIFT
            FDB   RSHIFTW
H_CELLS:
            FCB   $05
            FCC   "CELLS"
            FDB   H_RSHIFT
            FDB   CELLSW
H_CELLPLUS:
            FCB   $05
            FCC   "CELL+"
            FDB   H_CELLS
            FDB   CELLPLUSW
H_CHARS:
            FCB   $05
            FCC   "CHARS"
            FDB   H_CELLPLUS
            FDB   CHARSW
H_CHARPLUS:
```

```
            FCB   $05
            FCC   "CHAR+"
            FDB   H_CHARS
            FDB   CHARPLUSW
H_ALIGN:
            FCB   $05
            FCC   "ALIGN"
            FDB   H_CHARPLUS
            FDB   ALIGNW
H_ALIGNED:
            FCB   $07
            FCC   "ALIGNED"
            FDB   H_ALIGN
            FDB   ALIGNEDW
H_EQUAL:
            FCB   $01
            FCC   "="
            FDB   H_ALIGNED
            FDB   EQUALW
H_LESS:
            FCB   $01
            FCC   "<"
            FDB   H_EQUAL
            FDB   LESSW
H_GREATER:
            FCB   $01
            FCC   ">"
            FDB   H_LESS
            FDB   GREATERW
H_ZEROEQ:
            FCB   $02
            FCC   "0="
            FDB   H_GREATER
            FDB   ZEROEQW
H_ZEROLT:
            FCB   $02
            FCC   "0<"
            FDB   H_ZEROEQ
            FDB   ZEROLTW
H_ULESS:
            FCB   $02
            FCC   "U<"
            FDB   H_ZEROLT
            FDB   ULESSW
H_NOTEQUAL:
            FCB   $02
            FCC   "<>"
            FDB   H_ULESS
            FDB   NOTEQUALW
H_ZERONE:
            FCB   $03
            FCC   "0<>"
            FDB   H_NOTEQUAL
            FDB   ZERONEW
H_ZEROGT:
            FCB   $02
            FCC   "0>"
            FDB   H_ZERONE
            FDB   ZEROGTW
H_UGREATER:
            FCB   $02
            FCC   "U>"
            FDB   H_ZEROGT
            FDB   UGREATERW
H_WITHIN:
            FCB   $06
            FCC   "WITHIN"
            FDB   H_UGREATER
            FDB   WITHINW
H_DEQUAL:
            FCB   $02
```

```
            FCC   "D="
            FDB   H_WITHIN
            FDB   DEQUALW
H_DLESS:
            FCB   $02
            FCC   "D<"
            FDB   H_DEQUAL
            FDB   DLESSW
H_DULESS:
            FCB   $03
            FCC   "DU<"
            FDB   H_DLESS
            FDB   DULESSW
H_IF:
            FCB   $82
            FCC   "IF"
            FDB   H_DULESS
            FDB   IFW
H_THEN:
            FCB   $84
            FCC   "THEN"
            FDB   H_IF
            FDB   THENW
H_ELSE:
            FCB   $84
            FCC   "ELSE"
            FDB   H_THEN
            FDB   ELSEW
H_BEGIN:
            FCB   $85
            FCC   "BEGIN"
            FDB   H_ELSE
            FDB   BEGINW
H_UNTIL:
            FCB   $85
            FCC   "UNTIL"
            FDB   H_BEGIN
            FDB   UNTILW
H_AGAIN:
            FCB   $85
            FCC   "AGAIN"
            FDB   H_UNTIL
            FDB   AGAINW
H_WHILE:
            FCB   $85
            FCC   "WHILE"
            FDB   H_AGAIN
            FDB   WHILEW
H_REPEAT:
            FCB   $86
            FCC   "REPEAT"
            FDB   H_WHILE
            FDB   REPEATW
H_RECURSE:
            FCB   $87
            FCC   "RECURSE"
            FDB   H_REPEAT
            FDB   RECURSEW
H_DO:
            FCB   $82
            FCC   "DO"
            FDB   H_RECURSE
            FDB   DOW
H_QDO:
            FCB   $83
            FCC   "?DO"
            FDB   H_DO
            FDB   QDOW
H_LOOP:
            FCB   $84
            FCC   "LOOP"
```

```
            FDB   H_QDO
            FDB   LOOPW
H_PLUSLOOP:
            FCB   $85
            FCC   "+LOOP"
            FDB   H_LOOP
            FDB   PLUSLOOPW
H_I:
            FCB   $01
            FCC   "I"
            FDB   H_PLUSLOOP
            FDB   IW
H_J:
            FCB   $01
            FCC   "J"
            FDB   H_I
            FDB   JW
H_LEAVE:
            FCB   $05
            FCC   "LEAVE"
            FDB   H_J
            FDB   LEAVEW
H_UNLOOP:
            FCB   $06
            FCC   "UNLOOP"
            FDB   H_LEAVE
            FDB   UNLOOPW
H_EXIT:
            FCB   $84
            FCC   "EXIT"
            FDB   H_UNLOOP
            FDB   EXITW
H_CASE:
            FCB   $84
            FCC   "CASE"
            FDB   H_EXIT
            FDB   CASEW
H_OF:
            FCB   $82
            FCC   "OF"
            FDB   H_CASE
            FDB   OFW
H_ENDOF:
            FCB   $85
            FCC   "ENDOF"
            FDB   H_OF
            FDB   ENDOFW
H_ENDCASE:
            FCB   $87
            FCC   "ENDCASE"
            FDB   H_ENDOF
            FDB   ENDCASEW
H_COLON:
            FCB   $01
            FCC   ":"
            FDB   H_ENDCASE
            FDB   COLONW
H_SEMI:
            FCB   $81
            FCC   ";"
            FDB   H_COLON
            FDB   SEMIW
H_NONAME:
            FCB   $07               ; not IMMEDIATE (runs only in INTERPRET)
            FCC   ":NONAME"
            FDB   H_SEMI
            FDB   NONAMEW
H_CREATE:
            FCB   $06
            FCC   "CREATE"
            FDB   H_NONAME
```

```
            FDB   CREATEW
H_DOESGT:
            FCB   $85               ; $80 IMMEDIATE | 5 (length of "DOES>")
            FCC   "DOES>"
            FDB   H_CREATE
            FDB   DOESGTW
H_VARIABLE:
            FCB   $08
            FCC   "VARIABLE"
            FDB   H_DOESGT
            FDB   VARIABLEW
H_CONSTANT:
            FCB   $08
            FCC   "CONSTANT"
            FDB   H_VARIABLE
            FDB   CONSTANTW
H_VALUE:
            FCB   $05
            FCC   "VALUE"
            FDB   H_CONSTANT
            FDB   VALUEW
H_TO:
            FCB   $82
            FCC   "TO"
            FDB   H_VALUE
            FDB   TOW
H_TWOVARIABLE:
            FCB   $09
            FCC   "2VARIABLE"
            FDB   H_TO
            FDB   TWOVARIABLEW
H_TWOCONSTANT:
            FCB   $09
            FCC   "2CONSTANT"
            FDB   H_TWOVARIABLE
            FDB   TWOCONSTANTW
H_BUFFERCOLON:
            FCB   $07
            FCC   "BUFFER:"
            FDB   H_TWOCONSTANT
            FDB   BUFFERCOLONW
H_DEFER:
            FCB   $05
            FCC   "DEFER"
            FDB   H_BUFFERCOLON
            FDB   DEFERW
H_DEFERFETCH:
            FCB   $06
            FCC   "DEFER@"
            FDB   H_DEFER
            FDB   DEFERFETCHW
H_DEFERSTORE:
            FCB   $06
            FCC   "DEFER!"
            FDB   H_DEFERFETCH
            FDB   DEFERSTOREW
H_IS:
            FCB   $82
            FCC   "IS"
            FDB   H_DEFERSTORE
            FDB   ISW
H_ACTIONOF:
            FCB   $89
            FCC   "ACTION-OF"
            FDB   H_IS
            FDB   ACTIONOFW
H_MARKER:
            FCB   $06
            FCC   "MARKER"
            FDB   H_ACTIONOF
            FDB   MARKERW
```

```
H_IMMEDIATE:
            FCB   $09
            FCC   "IMMEDIATE"
            FDB   H_MARKER
            FDB   IMMEDIATEW
H_STATE:
            FCB   $05
            FCC   "STATE"
            FDB   H_IMMEDIATE
            FDB   STATEW
H_LBRACKET:
            FCB   $81
            FCC   "["
            FDB   H_STATE
            FDB   LBRACKETW
H_RBRACKET:
            FCB   $81
            FCC   "]"
            FDB   H_LBRACKET
            FDB   RBRACKETW
H_TICK:
            FCB   $01
            FCC   "'"
            FDB   H_RBRACKET
            FDB   TICKW
H_COMPILECOMMA:
            FCB   $08
            FCC   "COMPILE,"
            FDB   H_TICK
            FDB   COMPILECOMMAW
H_LITERAL:
            FCB   $87
            FCC   "LITERAL"
            FDB   H_COMPILECOMMA
            FDB   LITERALW
H_BRACKTICK:
            FCB   $83
            FCC   "[']"
            FDB   H_LITERAL
            FDB   BRACKTICKW
H_POSTPONE:
            FCB   $88
            FCC   "POSTPONE"
            FDB   H_BRACKTICK
            FDB   POSTPONEW
H_BRACKCOMPILE:
            FCB   $89               ; $80 IMMEDIATE | 9 (length of "[COMPILE]")
            FCC   "[COMPILE]"
            FDB   H_POSTPONE
            FDB   BRACKCOMPILEW
H_TOBODY:
            FCB   $05
            FCC   ">BODY"
            FDB   H_BRACKCOMPILE
            FDB   TOBODYW
H_EXECUTE:
            FCB   $07
            FCC   "EXECUTE"
            FDB   H_TOBODY
            FDB   EXECUTEW
H_SLITERAL:
            FCB   $88
            FCC   "SLITERAL"
            FDB   H_EXECUTE
            FDB   SLITERALW
H_ABORTQUOTE:
            FCB   $86
            FCC   "ABORT"
            FCB   $22               ; '"' split out of the FCC string
            FDB   H_SLITERAL
            FDB   ABORTQUOTEW
```

```
H_FETCH:
            FCB   $01
            FCC   "@"
            FDB   H_ABORTQUOTE
            FDB   FETCHW
H_STORE:
            FCB   $01
            FCC   "!"
            FDB   H_FETCH
            FDB   STOREW
H_CFETCH:
            FCB   $02
            FCC   "C@"
            FDB   H_STORE
            FDB   CFETCHW
H_CSTORE:
            FCB   $02
            FCC   "C!"
            FDB   H_CFETCH
            FDB   CSTOREW
H_PLUSSTORE:
            FCB   $02
            FCC   "+!"
            FDB   H_CSTORE
            FDB   PLUSSTOREW
H_TWOFETCH:
            FCB   $02
            FCC   "2@"
            FDB   H_PLUSSTORE
            FDB   TWOFETCHW
H_TWOSTORE:
            FCB   $02
            FCC   "2!"
            FDB   H_TWOFETCH
            FDB   TWOSTOREW
H_COMMA:
            FCB   $01
            FCC   ","
            FDB   H_TWOSTORE
            FDB   COMMAW
H_CCOMMA:
            FCB   $02
            FCC   "C,"
            FDB   H_COMMA
            FDB   CCOMMAW
H_ALLOT:
            FCB   $05
            FCC   "ALLOT"
            FDB   H_CCOMMA
            FDB   ALLOTW
H_HERE:
            FCB   $04
            FCC   "HERE"
            FDB   H_ALLOT
            FDB   HEREW
H_VCOMMA:
            FCB   $02
            FCC   "V,"
            FDB   H_HERE
            FDB   VCOMMAW
H_VCCOMMA:
            FCB   $03
            FCC   "VC,"
            FDB   H_VCOMMA
            FDB   VCCOMMAW
H_VALLOT:
            FCB   $06
            FCC   "VALLOT"
            FDB   H_VCCOMMA
            FDB   VALLOTW
H_VHERE:
```

```
            FCB   $05
            FCC   "VHERE"
            FDB   H_VALLOT
            FDB   VHEREW
H_PAD:
            FCB   $03
            FCC   "PAD"
            FDB   H_VHERE
            FDB   PADW
H_UNUSED:
            FCB   $06
            FCC   "UNUSED"
            FDB   H_PAD
            FDB   UNUSEDW
H_VUNUSED:
            FCB   $07
            FCC   "VUNUSED"
            FDB   H_UNUSED
            FDB   VUNUSEDW
H_MOVE:
            FCB   $04
            FCC   "MOVE"
            FDB   H_VUNUSED
            FDB   MOVEW
H_FILL:
            FCB   $04
            FCC   "FILL"
            FDB   H_MOVE
            FDB   FILLW
H_ERASE:
            FCB   $05
            FCC   "ERASE"
            FDB   H_FILL
            FDB   ERASEW
H_BLANK:
            FCB   $05
            FCC   "BLANK"
            FDB   H_ERASE
            FDB   BLANKW
H_CMOVE:
            FCB   $05
            FCC   "CMOVE"
            FDB   H_BLANK
            FDB   CMOVEW
H_CMOVEGT:
            FCB   $06
            FCC   "CMOVE>"
            FDB   H_CMOVE
            FDB   CMOVEGTW
H_COUNT:
            FCB   $05
            FCC   "COUNT"
            FDB   H_CMOVEGT
            FDB   COUNTW
H_WORD:
            FCB   $04
            FCC   "WORD"
            FDB   H_COUNT
            FDB   WORDW
H_CHAR:
            FCB   $04
            FCC   "CHAR"
            FDB   H_WORD
            FDB   CHARW
H_BRACKCHAR:
            FCB   $86
            FCC   "[CHAR]"
            FDB   H_CHAR
            FDB   BRACKCHARW
H_PARSE:
            FCB   $05
```

```
            FCC   "PARSE"
            FDB   H_BRACKCHAR
            FDB   PARSEW
H_PARSENAME:
            FCB   $0A
            FCC   "PARSE-NAME"
            FDB   H_PARSE
            FDB   PARSENAMEW
H_SQUOTE:
            FCB   $82
            FCC   "S"
            FCB   $22               ; '"' split out of the FCC string
            FDB   H_PARSENAME
            FDB   SQUOTEW
H_DOTQUOTE:
            FCB   $82
            FCC   "."
            FCB   $22               ; '"' split out of the FCC string
            FDB   H_SQUOTE
            FDB   DOTQUOTEW
H_COMPARE:
            FCB   $07
            FCC   "COMPARE"
            FDB   H_DOTQUOTE
            FDB   COMPAREW
H_SEARCH:
            FCB   $06
            FCC   "SEARCH"
            FDB   H_COMPARE
            FDB   SEARCHW
H_DASHTRAILING:
            FCB   $09
            FCC   "-TRAILING"
            FDB   H_SEARCH
            FDB   DASHTRAILINGW
H_SLASHSTRING:
            FCB   $07
            FCC   "/STRING"
            FDB   H_DASHTRAILING
            FDB   SLASHSTRINGW
H_REPLACES:
            FCB   $08
            FCC   "REPLACES"
            FDB   H_SLASHSTRING
            FDB   REPLACESW
H_CLRREPLACES:
            FCB   $0C
            FCC   "CLR-REPLACES"
            FDB   H_REPLACES
            FDB   CLRREPLACESW
H_SUBSTITUTE:
            FCB   $0A
            FCC   "SUBSTITUTE"
            FDB   H_CLRREPLACES
            FDB   SUBSTITUTEW
H_SNAME:
            FCB   $05
            FCC   "SNAME"
            FDB   H_SUBSTITUTE
            FDB   SNAMEW
H_UNESCAPE:
            FCB   $08
            FCC   "UNESCAPE"
            FDB   H_SNAME
            FDB   UNESCAPEW
H_LTNUM:
            FCB   $02
            FCC   "<#"
            FDB   H_UNESCAPE
            FDB   LTNUMW
H_NUM:
```

```
            FCB   $01
            FCC   "#"
            FDB   H_LTNUM
            FDB   NUMW
H_NUMS:
            FCB   $02
            FCC   "#S"
            FDB   H_NUM
            FDB   NUMSW
H_NUMGT:
            FCB   $02
            FCC   "#>"
            FDB   H_NUMS
            FDB   NUMGTW
H_HOLD:
            FCB   $04
            FCC   "HOLD"
            FDB   H_NUMGT
            FDB   HOLDW
H_HOLDS:
            FCB   $05
            FCC   "HOLDS"
            FDB   H_HOLD
            FDB   HOLDSW
H_SIGN:
            FCB   $04
            FCC   "SIGN"
            FDB   H_HOLDS
            FDB   SIGNW
H_TONUMBER:
            FCB   $07
            FCC   ">NUMBER"
            FDB   H_SIGN
            FDB   TONUMBERW
H_DOT:
            FCB   $01
            FCC   "."
            FDB   H_TONUMBER
            FDB   DOTW
H_UDOT:
            FCB   $02
            FCC   "U."
            FDB   H_DOT
            FDB   UDOTW
H_DOTR:
            FCB   $02
            FCC   ".R"
            FDB   H_UDOT
            FDB   DOTRW
H_UDOTR:
            FCB   $03
            FCC   "U.R"
            FDB   H_DOTR
            FDB   UDOTRW
H_QMARK:
            FCB   $01
            FCC   "?"
            FDB   H_UDOTR
            FDB   QMARKW
H_DDOT:
            FCB   $02
            FCC   "D."
            FDB   H_QMARK
            FDB   DDOTW
H_DDOTR:
            FCB   $03
            FCC   "D.R"
            FDB   H_DDOT
            FDB   DDOTRW
H_BASE:
            FCB   $04
```

```
            FCC   "BASE"
            FDB   H_DDOTR
            FDB   BASEW
H_DECIMAL:
            FCB   $07
            FCC   "DECIMAL"
            FDB   H_BASE
            FDB   DECIMALW
H_HEX:
            FCB   $03
            FCC   "HEX"
            FDB   H_DECIMAL
            FDB   HEXW
H_BINARY:
            FCB   $06
            FCC   "BINARY"
            FDB   H_HEX
            FDB   BINARYW
H_CATCH:
            FCB   $05
            FCC   "CATCH"
            FDB   H_BINARY
            FDB   CATCHW
H_THROW:
            FCB   $05
            FCC   "THROW"
            FDB   H_CATCH
            FDB   THROWW
H_LPAREN:
            FCB   $81
            FCC   "("
            FDB   H_THROW
            FDB   LPARENW
H_BACKSLASH:
            FCB   $81
            FCC   "\"
            FDB   H_LPAREN
            FDB   BACKSLASHW
H_ENVQ:
            FCB   $0C
            FCC   "ENVIRONMENT?"
            FDB   H_BACKSLASH
            FDB   ENVQW
H_SOURCE:
            FCB   $06
            FCC   "SOURCE"
            FDB   H_ENVQ
            FDB   SOURCEW
H_SOURCEID:
            FCB   $09
            FCC   "SOURCE-ID"
            FDB   H_SOURCE
            FDB   SOURCEIDW
H_REFILL:
            FCB   $06
            FCC   "REFILL"
            FDB   H_SOURCEID
            FDB   REFILLW
H_EVALUATE:
            FCB   $08
            FCC   "EVALUATE"
            FDB   H_REFILL
            FDB   EVALUATEW
H_TIB:
            FCB   $03
            FCC   "TIB"
            FDB   H_EVALUATE
            FDB   TIBW
H_NUMTIB:
            FCB   $04
            FCC   "#TIB"
```

```
            FDB   H_TIB
            FDB   NUMTIBW
H_TOIN:
            FCB   $03
            FCC   ">IN"
            FDB   H_NUMTIB
            FDB   TOINW
H_SPAN:
            FCB   $04
            FCC   "SPAN"
            FDB   H_TOIN
            FDB   SPANW
H_BL:
            FCB   $02
            FCC   "BL"
            FDB   H_SPAN
            FDB   BLW
H_DOTS:
            FCB   $02
            FCC   ".S"
            FDB   H_BL
            FDB   DOTSW
H_WORDS:
            FCB   $05
            FCC   "WORDS"
            FDB   H_DOTS
            FDB   WORDSW
H_DUMP:
            FCB   $04
            FCC   "DUMP"
            FDB   H_WORDS
            FDB   DUMPW

H_TRUE:
            FCB   $04
            FCC   "TRUE"
            FDB   H_DUMP
            FDB   TRUEW

H_FALSE:
            FCB   $05
            FCC   "FALSE"
            FDB   H_TRUE
            FDB   FALSEW

; H_ABORT and H_QUIT are hand-built (HEADER/CREATE could not be used
; for them). Chain, newest first: H_NEGTWO, H_POSTWO, H_NEGONE,
; H_POSONE, H_FIND, H_QUIT, H_ABORT, H_FALSE ... H_KEY, then 0.
; Original comment: shadow H_ABORT.0.
H_ABORT:    FCB   5
            FCC   "ABORT"
            FDB   H_FALSE           ; previous newest entry
            FDB   ABORTW            ; See bugfix: H_ABORT.2

H_QUIT:     FCB   4
            FCC   "QUIT"
            FDB   H_ABORT
            FDB   QUITW

; FIND: pushes xt and 1 (immediate) or -1 (normal) on success; on
; failure pushes the original c-addr and 0.
H_FIND:     FCB   4
            FCC   "FIND"
            FDB   H_QUIT
            FDB   FINDW

H_POSONE:   FCB   1
            FCC   "1"
            FDB   H_FIND
            FDB   POSONEW
```

```
H_NEGONE:   FCB   2
            FCC   "-1"
            FDB   H_POSONE
            FDB   NEGONEW

H_POSTWO:   FCB   1
            FCC   "2"
            FDB   H_NEGONE
            FDB   POSTWOW

H_NEGTWO:   FCB   2
            FCC   "-2"
            FDB   H_POSTWO
            FDB   NEGTWOW

H_DZEROEQ:  FCB   3
            FCC   "D0="
            FDB   H_NEGTWO
            FDB   DZEROEQW

H_DZEROLT:  FCB   3
            FCC   "D0<"
            FDB   H_DZEROEQ
            FDB   DZEROLTW

H_DTWOSTAR: FCB   3
            FCC   "D2*"
            FDB   H_DZEROLT
            FDB   DTWOSTARW

H_DTWOSLASH: FCB  3
            FCC   "D2/"
            FDB   H_DTWOSTAR
            FDB   DTWOSLASHW

H_MSTARSLASH: FCB 3
            FCC   "M*/"
            FDB   H_DTWOSLASH
            FDB   MSTARSLASHW

H_TWOLITERAL: FCB $88
            FCC   "2LITERAL"
            FDB   H_MSTARSLASH
            FDB   TWOLITERALW

H_TWOVALUE: FCB   6
            FCC   "2VALUE"
            FDB   H_TWOLITERAL
            FDB   TWOVALUEW

H_DOTPAREN: FCB   $82
            FCC   ".("
            FDB   H_TWOVALUE
            FDB   DOTPARENW

H_CQUOTE:   FCB   $82
            FCC   "C"
            FCB   $22               ; '"' split out of the FCC string
            FDB   H_DOTPAREN
            FDB   CQUOTEW

H_SAVEINPUT: FCB  10
            FCC   "SAVE-INPUT"
            FDB   H_CQUOTE
            FDB   SAVEINPUTW

H_RESTOREINPUT: FCB 13
            FCC   "RESTORE-INPUT"
            FDB   H_SAVEINPUT
            FDB   RESTOREINPUTW
```

```
H_SQUOTEESC: FCB  $83
             FCC   "S\"
             FCB   $22               ; '"' split out of the FCC string
             FDB   H_RESTOREINPUT
             FDB   SQUOTEESCW

H_CONVERT:  FCB   7
             FCC   "CONVERT"
             FDB   H_SQUOTEESC
             FDB   CONVERTW

BASELATEST  EQU   H_CONVERT        ; head of the ROM dictionary (see COLD)

; Verify no collision with base code.
; Value should match ORG BASECODE
BASEDICTEND EQU   *
BASEDICTSIZE EQU   BASEDICTEND-BASEDICT
```

9. Alphabetical Index

Every word implemented, alphabetically, with the page in the Glossary (Section 3) where its full entry appears. Page numbers are live Word fields — update fields (Ctrl+A, F9) if they show as blank or "?" after edits.

- 15
-1 17
-2 17
-TRAILING 31
, ... 28
; 24
: 24
:NONAME 24
! 27
? 33
?DO 22
?DUP 13
. 33
." 31
.(..... 34
.R 33
.S 36
' 26
(..... 34
[..... 26
[COMPILE] 26
['] 26
[CHAR] 30
] 26
@ 27
* 15
*/ 17
*/MOD 17
/ 15
/MOD 16
/STRING 31
\ 34
..... 32
#> 32
#S 32
#TIB 35

+ 15
+! 27
+LOOP 23
< 20
<# 32
<> 21
= 20
> 20
>BODY 26
>IN 35
>NUMBER 30
>R 15
0< 20
0<> 21
0= 20
0> 21
1 17
1- 16
1+ 16
2 17
2! 27
2@ 27
2* 16
2/ 16
2+ 16
2>R 15
2CONSTANT 25
2DROP 14
2DUP 14
2LITERAL 26
2OVER 14
2R@ 15
2R> 15
2ROT 14
2SWAP 14
2VALUE 24
2VARIABLE 25
ABORT 13
ABORT" 27
ABS 16
ACCEPT 12

ACTION-OF ... 25
AGAIN ... 22
ALIGN ... 20
ALIGNED ... 20
ALLOT ... 28
AND ... 19
BASE ... 33
BEGIN ... 22
BINARY ... 34
BL ... 35
BLANK ... 29
BUFFER: ... 25
C, ... 28
C! ... 27
C@ ... 27
C" ... 31
CASE ... 23
CATCH ... 34
CELL+ ... 20
CELLS ... 19
CHAR ... 30
CHAR+ ... 20
CHARS ... 20
CLR-REPLACES ... 31
CMOVE ... 29
CMOVE> ... 29
COMPARE ... 31
COMPILE, ... 26
CONSTANT ... 24
CONVERT ... 30
COUNT ... 29
CR ... 12
CREATE ... 24
D- ... 18
D. ... 33
D.R ... 33
D+ ... 18
D0< ... 18
D0= ... 18
D2* ... 19
D2/ ... 19

D< ... 21
D= ... 21
D>S ... 18
DABS ... 18
DECIMAL ... 33
DEFER ... 25
DEFER! ... 25
DEFER@ ... 25
DEPTH ... 13
DMAX ... 18
DMIN ... 18
DNEGATE ... 18
DO ... 22
DOES> ... 24
DROP ... 13
DU< ... 21
DUMP ... 36
DUP ... 13
ELSE ... 22
EMIT ... 12
ENDCASE ... 23
ENDOF ... 23
ENVIRONMENT? ... 34
ERASE ... 29
EVALUATE ... 35
EXECUTE ... 27
EXIT ... 23
EXPECT ... 12
FALSE ... 21
FILL ... 29
FIND ... 30
FM/MOD ... 17
HERE ... 28
HEX ... 34
HOLD ... 32
HOLDS ... 33
I ... 23
IF ... 21
IMMEDIATE ... 25
INVERT ... 19
IS ... 25

J ... 23
KEY ... 12
KEY? ... 12
LEAVE ... 23
LITERAL ... 26
LOOP ... 22
LSHIFT ... 19
M* ... 17
M*/ ... 19
M+ ... 18
MARKER ... 25
MAX ... 16
MIN ... 16
MOD ... 16
MOVE ... 29
NEGATE ... 16
NIP ... 14
OF ... 23
OR ... 19
OVER ... 13
PAD ... 28
PARSE ... 30
PARSE-NAME ... 30
PICK ... 14
POSTPONE ... 26
QUERY ... 12
QUIT ... 13
R@ ... 15
R> ... 15
RECURSE ... 22
REFILL ... 35
REPEAT ... 22
REPLACES ... 31
RESTORE-INPUT ... 35
ROLL ... 14
ROT ... 13
RSHIFT ... 19
S" ... 30
S>D ... 18
SAVE-INPUT ... 35
S\" ... 30

SEARCH ... 31
SIGN ... 33
SLITERAL ... 27
SM/REM ... 17
SNAME ... 32
SOURCE ... 35
SOURCE-ID ... 35
SPACE ... 13
SPACES ... 13
SPAN ... 35
STATE ... 26
SUBSTITUTE ... 32
SWAP ... 13
THEN ... 22
THROW ... 34
TIB ... 35
TO ... 24
TRUE ... 21
TUCK ... 14
TYPE ... 12
U. ... 33
U.R ... 33
U< ... 20
U> ... 21
UM* ... 17
UM/MOD ... 17
UNESCAPE ... 32
UNLOOP ... 23
UNTIL ... 22
UNUSED ... 28
V, ... 28
VALLOT ... 28
VALUE ... 24
VARIABLE ... 24
VC, ... 28
VHERE ... 28
VUNUSED ... 28
WHILE ... 22
WITHIN ... 21
WORD ... 29
WORDS ... 36

XOR..... 19